

Optimization Concepts for Data Science and Artificial Intelligence

Course Notes

Optimization Concepts for Data Science and Artificial Intelligence Course Notes was developed by Robert Blanchard, Mark Hartmann, Chip Wells, and Ari Zitin. Additional contributions were made by Manoj Chari, Tao Huang, Ed Hughes, Trevor Kearney, Jay Laramore, Bob Lucas, Rob Pratt, and Jonathan Stephenson. Instructional design, editing, and production support was provided by the Learning Design and Development team.

SAS and all other SAS Institute Inc. product or service names are registered trademarks or trademarks of SAS Institute Inc. in the USA and other countries. ® indicates USA registration. Other brand and product names are trademarks of their respective companies.

Optimization Concepts for Data Science and Artificial Intelligence Course Notes

Copyright © 2021 SAS Institute Inc. Cary, NC, USA. All rights reserved. Printed in the United States of America. No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, or otherwise, without the prior written permission of the publisher, SAS Institute Inc.

Book code E78825, course code LWORVY01/ORVY01, prepared date 21Sep2021. LWORVY01_001

Table of Contents

Lesson 1	Introduction to Mathematical Optimization	1-1
1.1	Introduction.....	1-3
	Practice	1-14
1.2	A Simple Example	1-17
1.3	OPTMODEL Procedure.....	1-21
	Demonstration: Solving the Simple Polynomial Example Using PROC OPTMODEL	1-22
	Practice	1-25
1.4	Solutions	1-26
	Solutions to Practices.....	1-26
	Solutions to Activities and Questions	1-29
Lesson 2	Linear Programming.....	2-1
2.1	Introduction to Linear Programming	2-3
	Demonstration: Solving a Linear Programming Problem Using PROC OPTMODEL	2-14
	Practice	2-20
	Demonstration: Solving the Dual Linear Programming Problem Using PROC OPTMODEL (Self-Study).....	2-22
2.2	Formulating and Solving Linear Programming Problems Using the OPTMODEL Procedure.....	2-24
	Demonstration: Using PROC OPTMODEL to Solve the Furniture-Making Problem	2-33
	Practice	2-42
2.3	Using Index Sets and Arrays in the OPTMODEL Procedure	2-43
	Demonstration: Using Arrays and Index Sets in PROC OPTMODEL to Solve the Furniture-Making Problem	2-69
	Practice	2-76
2.4	Dual Values and Reduced Costs in the Simplex Method (Self-Study)	2-77

Demonstration: Dual Values and Reduced Profits in the Furniture-Making Problem (Self-Study)	2-89
2.5 Reading and Writing Data in the OPTMODEL Procedure	2-91
Demonstration: Reading the In-Memory Class Data Set in PROC OPTMODEL	2-97
Demonstration: Reading Problem Data in PROC OPTMODEL for the Furniture-Making Problem	2-112
Demonstration: Writing Solution Data from PROC OPTMODEL for the Furniture-Making Problem	2-117
2.6 Solutions	2-118
Solutions to Practices	2-118
Solutions to Activities and Questions	2-125
Lesson 3 Nonlinear Programming	3-1
3.1 Introduction to Nonlinear Programming	3-3
Demonstration: Minimizing the Banana Function Using PROC OPTMODEL	3-20
Demonstration: Solving the Sinc Problem Using PROC OPTMODEL	3-30
Demonstration: Solving the Sinc Problem Using PROC OPTMODEL MULTISTART Options	3-36
3.2 Solving Nonlinear Programming Problems Using the OPTMODEL Procedure	3-39
Demonstration: Solving a Portfolio Optimization Problem Using PROC OPTMODEL	3-42
Practice	3-51
3.3 Solutions	3-55
Solutions to Practices	3-55
Solutions to Activities and Questions	3-57
Lesson 4 Integer and Mixed Integer Linear Programming	4-1
4.1 Introduction to Integer and Mixed Integer Linear Programming	4-3
Demonstration: Solving a Mixed Integer Linear Programming Problem Using the Branch-and-Bound Algorithm	4-15
Demonstration: Solving a Mixed Integer Linear Programming Problem Step-by-Step (Self-Study)	4-17

Practice	4-23
4.2 Solving Integer and Mixed Integer Linear Programming Problems Using PROC OPTMODEL	4-24
Demonstration: Solving an Integer Linear Diet Problem Using PROC OPTMODEL	4-27
4.3 Solutions	4-35
Solutions to Practices.....	4-35
Solutions to Activities and Questions	4-37
Lesson 5 Open Source Interactivity	5-1
5.1 SAS Viya and Open Source Integration	5-3
5.2 SAS Viya Python APIs	5-11
Demonstration: Executing a PROC OPTMODEL Program from Python Using CAS Actions	5-16
Demonstration: Maximizing Distance Traveled by a Ball Thrown from a Cliff	5-22
Demonstration: Ordinary Least Squares Optimization	5-26
Practice	5-31
5.3 Solutions	5-35
Solutions to Practices.....	5-35
Appendix A A Note about Distributed Processing in the OPTMODEL Procedure	A-1
A.1 Distributed Processing in the OPTMODEL Procedure	A-3
Appendix B References	B-1
B.1 References	B-3

To learn more...



For information about other courses in the curriculum, contact the SAS Education Division at 1-800-333-7660, or send e-mail to training@sas.com. You can also find this information on the web at <http://support.sas.com/training/> as well as in the Training Course Catalog.

For a list of SAS books (including e-books) that relate to the topics covered in this course notes, visit <https://www.sas.com/sas/books.html> or call 1-800-727-0025. US customers receive free shipping to US addresses.

Lesson 1 Introduction to Mathematical Optimization

1.1	Introduction	1-3
	Practice.....	1-14
1.2	A Simple Example.....	1-17
1.3	OPTMODEL Procedure	1-21
	Demonstration: Solving the Simple Polynomial Example Using PROC OPTMODEL	1-22
	Practice.....	1-25
1.4	Solutions	1-26
	Solutions to Practices	1-26
	Solutions to Activities and Questions.....	1-29

1.1 Introduction

Objectives

- Explain the role of optimization in the decision-making process.
- Define the basic terminology and classify mathematical optimization problems.

3

Copyright © SAS Institute Inc. All rights reserved.



What is optimization?



operations research
industrial engineering
management science



available actions that
produce the best results



prescriptive analytics on top of
predictive and descriptive analytics



What happened?
What will happen?

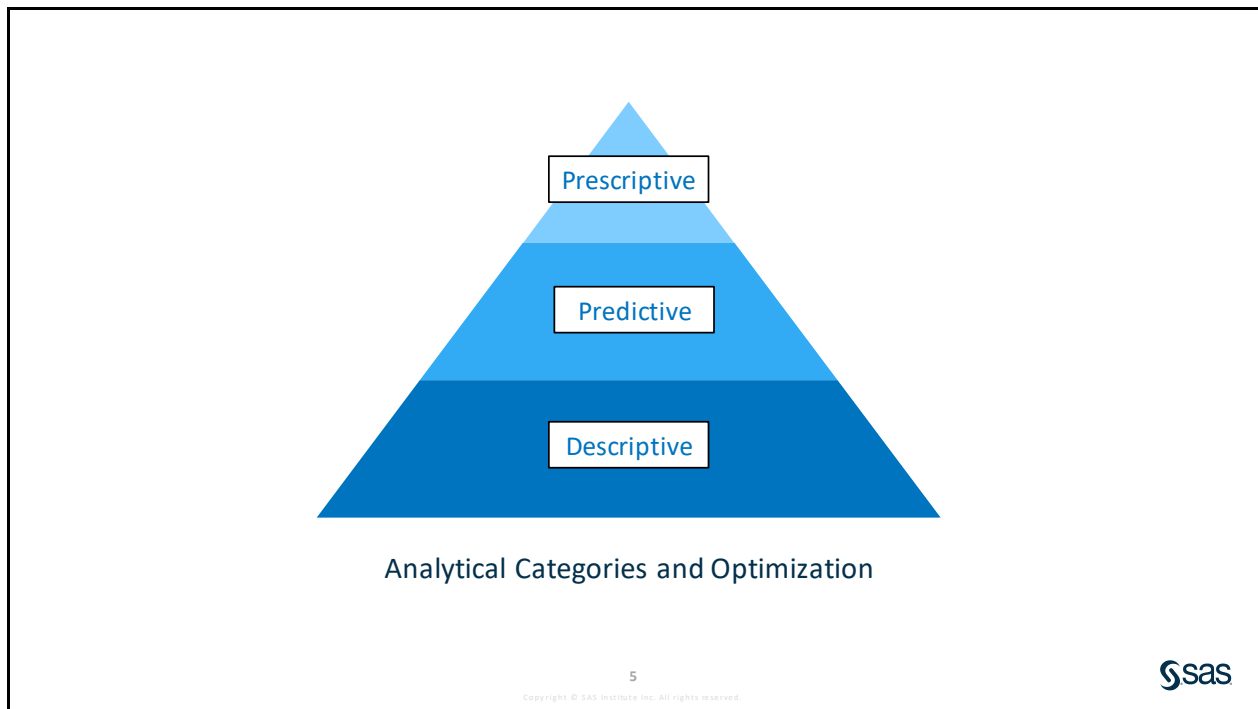
What should you
do?

4

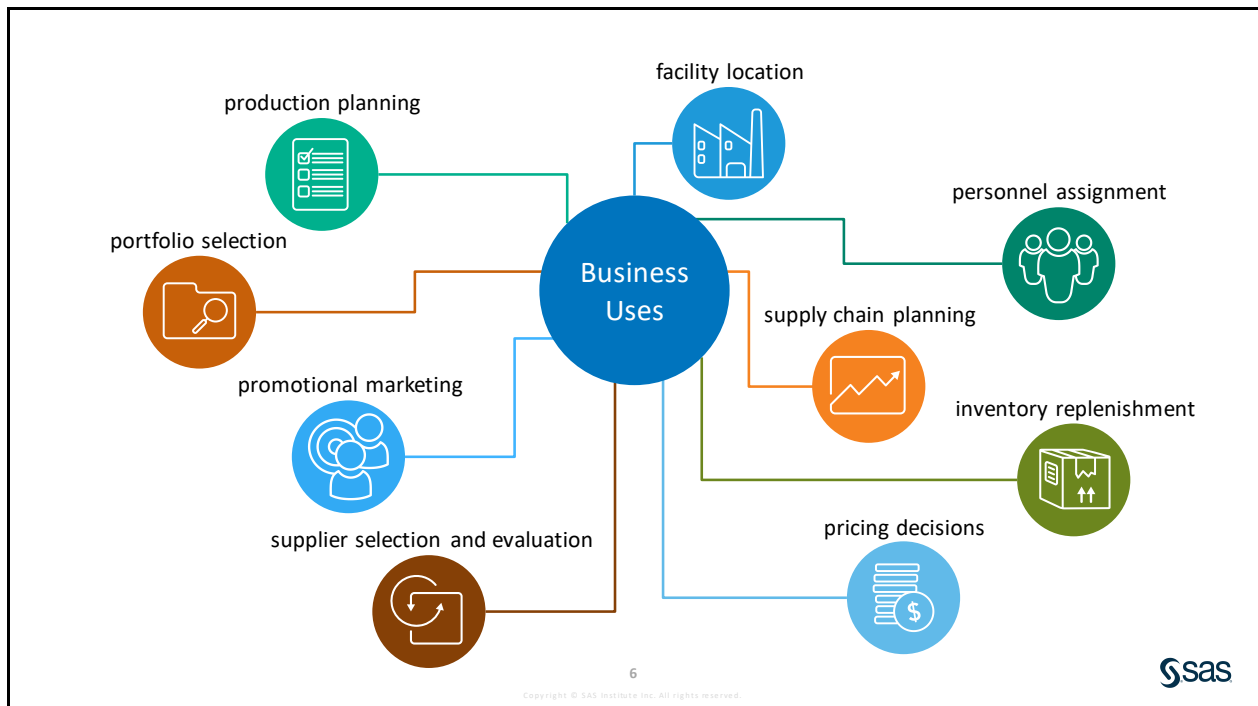
Copyright © SAS Institute Inc. All rights reserved.



Optimization is a major component of operations research, industrial engineering, and management science. Simply put, optimization is the process of choosing the available actions that produce the best results.



Mathematical optimization moves from “What happened?” (descriptive analytics) and “What will happen?” (predictive analytics) to “What should you do?” (prescriptive analytics). It can also inform “What if?” analysis by making logical supporting decisions, that is, “What should you do if ...?”



Mathematical optimization is prevalent in business settings.

Production Planning: Determine which of several possible mixes of products should be produced to achieve the highest profit.

Facility Location: Find the “best” site (for example, for a new factory), in relation to the location of the materials suppliers, distribution centers, and so on.

Personnel Assignment: Match personnel to work requirements so that you can meet current needs and anticipated changes, subject to budget and HR requirements.

Supply Chain Planning: Find the lowest-cost way to move product from factories to distribution centers to stores, and plan for possible disruptions or expansion.

Inventory Replenishment: Set inventory policies (reorder levels and maximum stock levels) to meet customer service goals and minimize costs.

Pricing Decisions: Establish and maintain optimal everyday prices based on costs, regional demand patterns, and competitive price information.

Supplier Selection and Evaluation: Choose which suppliers to deal with so that you can satisfy requirements and maximize leverage. Use a variety of criteria simultaneously to rate suppliers.

Promotional Marketing: Determine the best combination of promotional offers, delivery channels, and customers to maximize the overall return on marketing investment.

Portfolio Selection: Maximize return on investment (ROI) while balancing return versus risk.

An interesting feature of optimization is that applications can usually be grouped into a standard or template problem type that guides the analyst in the formulation and solution of the problem. One of the standard problem types solved in the next lesson is the furniture-making problem. The idea is to maximize the profit of producing items of furniture subject to constraints and parameters like the amount of labor available and the price that the furniture can be sold for. The transshipment problem is another standard type illustrated in the next lesson.

$$\begin{array}{ll} \min | \max & f(x) \\ \text{subject to} & c_i(x) \{ \leq, =, \geq \} b_i \quad (i=1,2,\dots,m) \\ & l_j \leq x_j \leq u_j \quad (j=1,2,\dots,n) \end{array}$$

Where:

x	Array of <i>decision variables</i>
$f(x)$	<i>Objective function</i>
$c_1(x), \dots, c_m(x)$	Functions that determine the constraints (together with the limits and bounds on x)
<i>feasible solution x^*</i>	Set of values that satisfy the constraints
<i>optimal solution x^*</i>	Feasible solution with min or max objective function value

The decision variables x_j correspond to decisions or choices in the underlying business problem. These choices, about which optimal values are to be determined, are under the control of the decision maker.

For some types of mathematical optimization problems, the decision variables are further restricted to be *integers* (...-2,-1,0,1,2, ...).

A *feasible solution* $\mathbf{x}$ is a set of decision variable values that satisfy all of the constraints.

Minimizing $f(\mathbf{x})$ or maximizing $f(\mathbf{x})$ is referred to as the *objective* of the mathematical optimization problem.

The bounds on $\mathbf{x}$ can be $\pm\infty$, so that x_j can be restricted to be nonnegative ($l_j = 0$ and $u_j = +\infty$), or for that matter, free ($l_j = -\infty$ and $u_j = +\infty$). Constraints can be structural: contracts, resource availabilities, legal restrictions, and so on.

Next you see the structure of a PROC OPTMODEL program that builds and solves a mathematical optimization problem.

Declaration statements are used to establish model elements (variables, constraints, and objective) and sets and parameters that help determine their defining coefficients, functions, and limits.

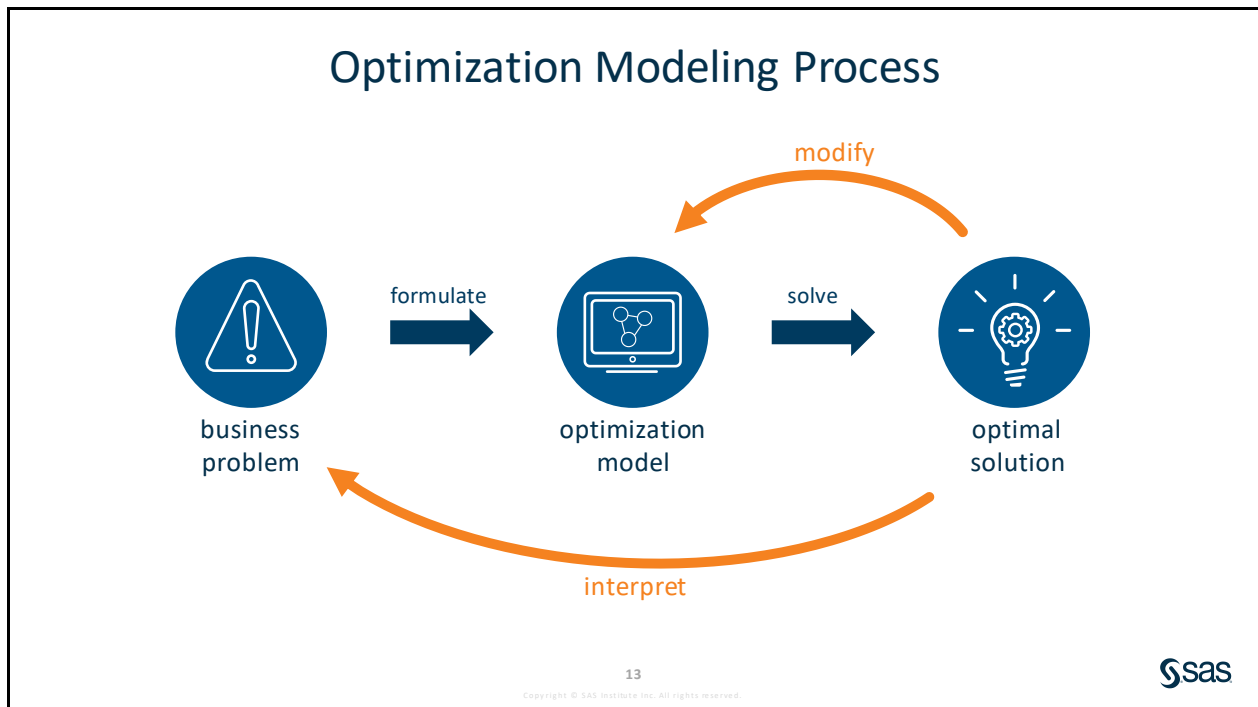
```
proc optmodel;
  /* declare sets and parameters */
  /* declare model elements */
  solve;
  /* print summary statistics */
quit;
```

Initially, sets and parameters are declared. Sets commonly consist of the decision variables for which we are going to choose values. Parameter examples include prices that we can charge for the products that we produce or the amount of resources available. Following this, other model elements like the objective function and constraints are declared. When the problem is solved, optimized values for decision variables and other solution information can be printed and output. Note that PROC OPTMODEL can be run interactively. A QUIT statement is used to get out of the OPTMODEL procedure.

```
proc optmodel;
  /* declare s
  /* declare
  solve;
  /* print s
quit;
```

Solution Summary	
Solver	NLP
Algorithm	Interior Point
Objective Function	Profit
Solution Status	Optimal
Objective Value	171969463.28
Optimality Error	7.340374E-9
Infeasibility	1.9542568E-9
Iterations	64
Presolve Time	0.00
Solution Time	0.27

The SOLVE statement causes the mathematical optimization model to be built from the declaration statements and solved. It produces a solution summary that includes the optimal objective value and solution time.



Formulating a business problem means making the assumptions and approximations that are necessary to represent it as an optimization model that can (ideally) be solved in a reasonable amount of time. If the optimal solution to the optimization problem cannot be interpreted in the context of the business problem, model modifications might be necessary.

The OPTMODEL procedure makes accommodating modifications to the problem formulation straightforward. Instead of reworking the entire problem, many changes and refinements to the formulation can be implemented with small changes to the syntax. For example, after the initial formulation, you might realize that fractional values for solutions are not useful. Producing a half of a table or a desk might not be profitable in applications of the furniture-making problem. Solution values need to be integer valued in this case. This restriction can be implemented with the INTEGER option in the variable declaration statement of PROC OPTMODEL.



The formulation of an optimization problem provides a structure for organizing the decision-making process. It helps with consistency, objectivity (persistence), and repeatability in driving all the decisions that influence optimal outcomes toward the same goals in each iteration of the problem. Adaptability allows modification of parts of the optimization program but preserves the original principles as problems scale to larger and more complex applications.

$$\begin{array}{ll} \min | \max & f(\mathbf{x}) \\ \text{subject to} & c_i(\mathbf{x}) \in \{\leq, =, \geq\} b_i \quad (i = 1, 2, \dots, m) \\ & l_j \leq x_j \leq u_j \quad (j = 1, 2, \dots, n) \end{array}$$

Linear Programming (LP)	$f(\mathbf{x})$ and $c_1(\mathbf{x}), \dots, c_m(\mathbf{x})$ are linear functions.
Nonlinear Programming (NLP)	$f(\mathbf{x})$ and $c_1(\mathbf{x}), \dots, c_m(\mathbf{x})$ are continuous but not all linear functions.
Integer Linear Programming (ILP)	$f(\mathbf{x})$ is linear, $\mathbf{x}$ must be integer, and $c_1(\mathbf{x}), \dots, c_m(\mathbf{x})$ are linear functions.
Mixed Integer Linear Programming (MILP)	$f(\mathbf{x}, \mathbf{y})$ is linear, $\mathbf{x}$ must be integer, and $c_1(\mathbf{x}, \mathbf{y}), \dots, c_m(\mathbf{x}, \mathbf{y})$ are linear functions, but $\mathbf{y}$ can be fractional.

15

Copyright © SAS Institute Inc. All rights reserved.



This course provides an introduction to solving linear, mixed integer linear, and nonlinear programming problems using the OPTMODEL procedure in SAS.

(Graphs of a linear, continuous, and discontinuous functions appear on the following page.)

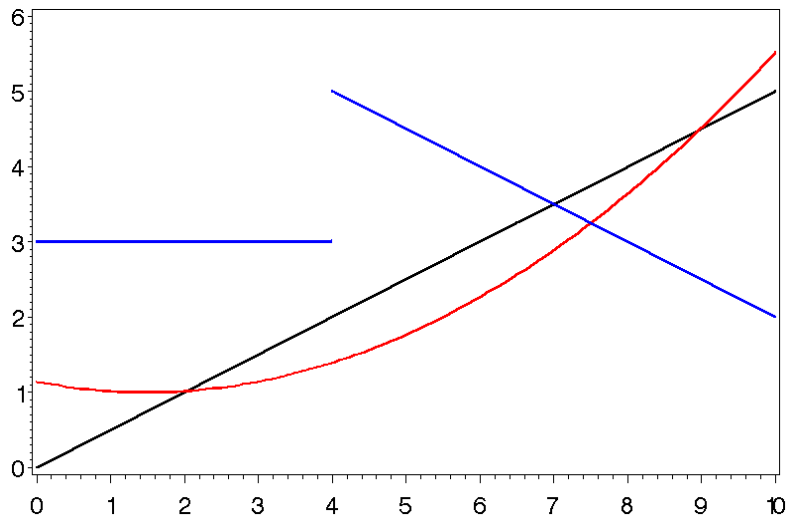
- If $f(\mathbf{x})$ and $c_1(\mathbf{x}), \dots, c_m(\mathbf{x})$ are linear functions, then the mathematical optimization problem is a *linear programming problem*, or *linear program (LP)*.
- If $f(\mathbf{x})$ is linear and the decision variables are required to take on integer values, but otherwise $c_1(\mathbf{x}), \dots, c_m(\mathbf{x})$ are linear functions, then the mathematical optimization problem is an *integer linear programming problem*, or *integer linear program (ILP)*. If only **some** of the decision variables are required to take on integer values, then the problem is a *mixed integer linear programming problem*, or *mixed integer linear program (MILP)*.
- If $f(\mathbf{x})$ and $c_1(\mathbf{x}), \dots, c_m(\mathbf{x})$ are continuous but not **all** linear functions, then the mathematical optimization problem is a *nonlinear programming problem*, or *nonlinear program (NLP)*.

This is not an exhaustive classification. However, problems beyond this set, such as those that combine integer decision variables and nonlinear functions, are solvable only when there is sufficient structure to allow for special-purpose solvers.

The simplex method for solving linear programming problems was designed by G. B. Dantzig in 1947 to solve U.S. Air Force planning problems. Current methods for solving mixed integer and nonlinear programming problems are more recent, and research is ongoing.

1.02 Activity

Which of the three functions below is a linear function?



sas

Recall that $f(\mathbf{x})$ is a linear function if $f(\mathbf{x}) = f_1x_1 + f_2x_2 + \dots + f_nx_n$ for some numbers $f_1, f_2, \dots, f_n$.



Going Up or Going Down?

LP: The default settings work almost every time. You need only to push the correct button.

18

sas

In linear programming problems, the default settings work almost every time. There are usually unique, optimal values for decision variables, and these optimal values are quick and easy to find in this problem type.

Although the number of iterations (steps) that are required to solve an LP varies with the structure of the problem, it is typically on the order of $m \cdot \log(n)$ for an LP with m constraints and n variables.



Going Up or Going Down?

LP: The default settings work almost every time. You need only to push the correct button.

ILP/MILP: Some more difficult problems might require different solver options. What happens when you push the red button?

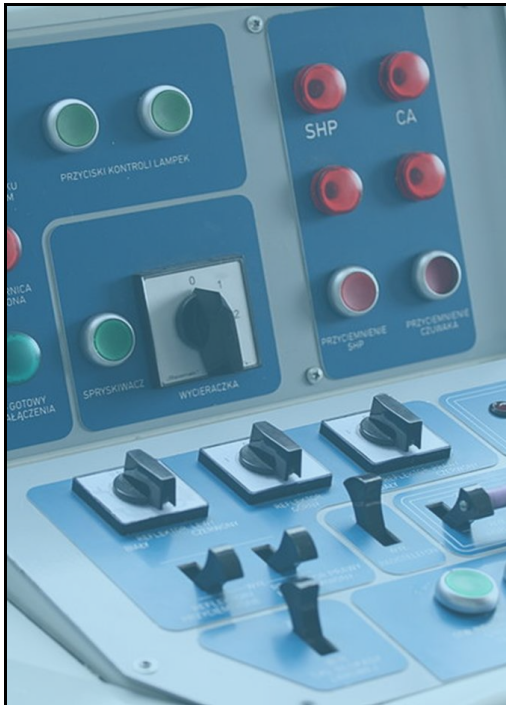
19

Copyright © SAS Institute Inc. All rights reserved.



For mixed integer and integer linear programming problems, some or all of the optimized values for the decision variables are constrained to be integer valued. We discussed an example of this earlier in the context of the furniture-making problem. (That is, it might not make sense for us to produce half a desk or half a table.) In this case, some more difficult problems might require different solver options.

The number of iterations that are required to solve an ILP/MILP is difficult to predict. Some are as easy to solve as LPs. For others, the “solution” process approaches a complete enumeration of the possible integer values (whose number can grow exponentially in the number of constraints and variables).



Going Up or Going Down?

LP: The default settings work almost every time. You need only to push the correct button.

ILP/MILP: Some more difficult problems might require different solver options. What happens when you push the red button?

NLP: Your starting point might determine where you end up. "I've a feeling we're not in Kansas anymore."

20

Copyright © SAS Institute Inc. All rights reserved.



In nonlinear programming problems, you are usually not guaranteed to have a unique, optimal solution for the values of decision variables. Where you start looking for the solution of optimization problems can influence where you wind up.

When you solve an LP, ILP, or MILP, the solvers provide a bound on the relative or absolute objective gap. (The *objective gap* is the gap between the true optimal objective value and the objective value of the solution that is found.) There is only a bound on the relative or absolute objective gap for special classes of NLPs, such as quadratic programming problems. In addition, the solution that is found might depend on the algorithm that is used and the starting solution that is supplied by the decision maker.

	All continuous variables	Some integer variables	All integer variables
All linear functions	Linear Programming (LP)	Mixed-Integer Linear Programming (MILP)	Integer Linear Programming (ILP)
Some nonlinear functions	Nonlinear Programming (NLP)		



Practice

1. Classifying Mathematical Optimization Problems

Use the table above to match the four mathematical optimization problems **a** through **d** with decision variables x , y , and z to the four problem classes: LP (linear programming), ILP (integer linear programming), MILP (mixed integer linear programming), and NLP (nonlinear programming).

- a.** minimize $4x + 5y + 2z$
 subject to $7x + 8y + 3z \geq 20$
 $x \geq 0, y \geq 0, z \geq 0$
 x, y, z integers
- b.** maximize $3x + 5y - 4z$
 subject to $6x^2 + 2y^2 + z^2 \leq 17$
 $x + y + z = 3$
 $-1 \leq x \leq 1$
- c.** maximize $12x + 19y - 4z$
 subject to $x + 3y + z = 225$
 $x + y - z \leq 117$
 $x \geq 0, y \geq 0, z \geq 0$
- d.** maximize $4x + 5y$
 subject to $x + y + 2z \leq 10$
 $2x + 3y - 3z \leq 5$
 $x \geq 0, y \geq 0$
 x, y integers

End of Practices

Setup for the Question

Classify the mathematical optimization problem.

minimize $2xy + 2yz + 2xz$

subject to $xyz = 1$

$x \geq 0, y \geq 0, z \geq 0$

21

Copyright © SAS Institute Inc. All rights reserved.



1.02 Multiple Choice Question

Classify the mathematical optimization problem.

- a. LP
- b. ILP
- c. MILP
- d. NLP

22

Copyright © SAS Institute Inc. All rights reserved.



Setup for the Question

Classify the mathematical optimization problem.

minimize z

subject to $x + y - z \leq 10$

$2x + 3y - z \leq 5$

$x + z \geq 0$

$y + z \geq 0$

24

Copyright © SAS Institute Inc. All rights reserved.



1.03 Multiple Choice Question

Classify the mathematical optimization problem.

- a. LP
- b. ILP
- c. MILP
- d. NLP

25

Copyright © SAS Institute Inc. All rights reserved.



1.2 A Simple Example

Objectives

- Explain local properties of functions that are used to solve mathematical optimization problems.

30

Copyright © SAS Institute Inc. All rights reserved.



Now consider a two-dimensional example. This example is two dimensional because we have two decision variables, x and y . Notice that we do not have any constraints or bounds listed for the decision variables. We just have an objective function that we want to minimize. One method for finding optimal values is to take the derivatives of the objective with respect to the decision variables and then set these derivatives equal to 0.

Problem



minimize $z = x^2 - x - 2y - xy + y^2$

- two decision variables, x and y
- no bounds or other constraints

Vector Calculus



$\nabla z = [\partial z / \partial x, \partial z / \partial y] = [2x - 1 - y, -2 - x + 2y] = [0, 0]$
 $\Rightarrow [x, y] = [4/3, 5/3]$

Hessian Matrix



$H(z) = \begin{bmatrix} \frac{\partial^2 z}{\partial x^2} & \frac{\partial^2 z}{\partial x \partial y} \\ \frac{\partial^2 z}{\partial y \partial x} & \frac{\partial^2 z}{\partial y^2} \end{bmatrix} = \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}$ (positive definite)

33

Copyright © SAS Institute Inc. All rights reserved.



Solving the partial derivatives leads to the solution values $4/3$ and $5/3$. These are the values of the decision variables that minimize the objective function z . Intuitively, we can think about the partial derivative as the slope of the z function in the direction of either x or y . If the slopes of the function in both the x and y direction are equal to 0, we are at the bottom or minimum point of the function. However, this is also the condition that corresponds to being at a saddle point, or at the maximum of a function.

You can think of the second derivatives as the *slope of the slope*. These values are contained in the Hessian matrix, and they give us sufficient information to pin down whether the solution yields a minimum or a maximum. The matrix is positive definite, which indicates that as we move a little bit away from the solution values in either direction, we are traveling uphill. This can happen only if we are at a minimum on this objective function.

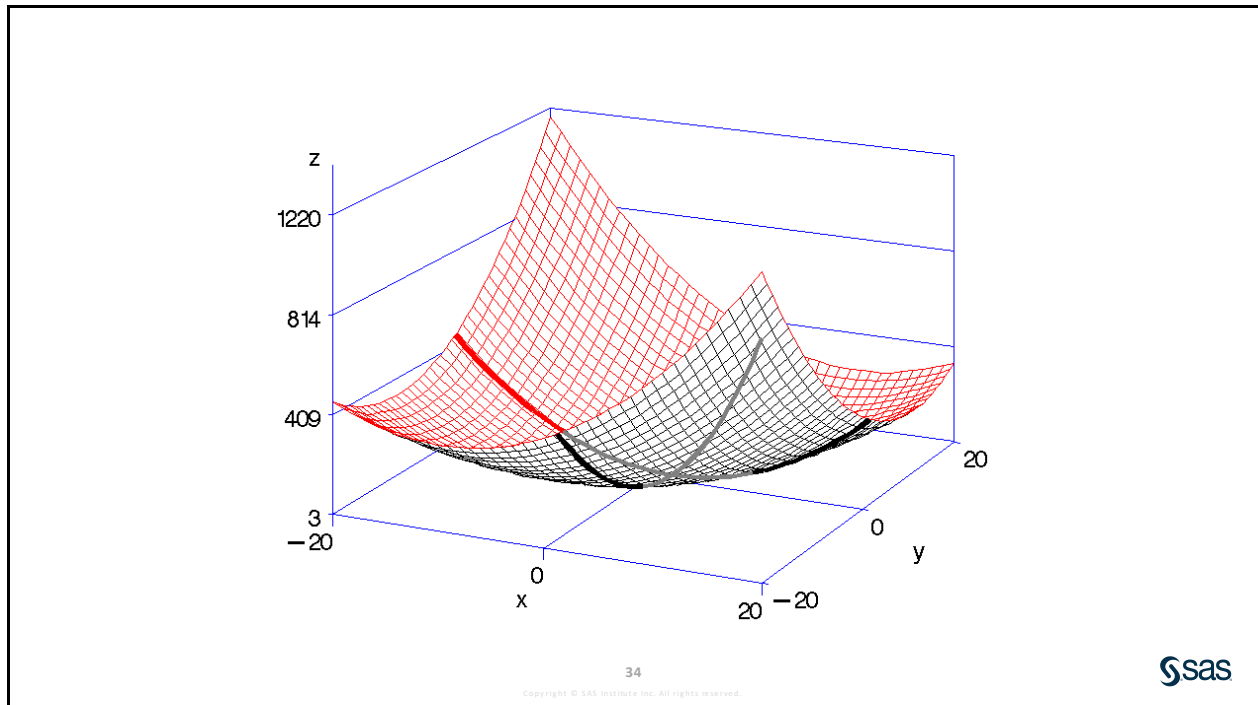
Recall the following two-dimensional Taylor series:

$$z(x+dx, y+dy) \approx z(x, y) + \nabla z^T [dx, dy] + \frac{1}{2} [dx, dy]^T H(z) [dx, dy]$$

which is exact for a quadratic function.

If the gradient $\nabla z(x, y) \neq [0, 0]$, then either increasing or decreasing x or y slightly decreases $z(x, y)$, so the minimum must occur at a point $[x, y]$ where the gradient is zero.

The Hessian $H(z)$ is positive definite (which means $u^T H(z) u > 0$ for all $u \neq 0$), so $[4/3, 5/3]$ is a minimum, not a maximum or a saddle point.



From the graph, you can see that the situation would be more complicated if you added the bounds $x \geq 0$ and $y \leq 0$, which require the solution to lie in the nearest quadrant.

You can generate this plot by running the **polynomial_graph.sas** program. Note the slope of the decision variable axes at the solution values outlined here.

1.04 Question

Is $[x^*, y^*] = [4/3, 5/3]$ a feasible solution to the problem of minimizing $z(x, y) = x^2 - x - 2y - xy + y^2$ subject to the bound constraints $x \geq 0$ and $y \leq 0$?

- ☐ Yes
- ☐ No

1.05 Question

Will the optimal solution $[x^*, y^*]$ to the problem of minimizing $z(x,y) = x^2 - x - 2y - xy + y^2$ subject to the bound constraints $x \geq 0$ and $y \leq 0$ have $x^* > 0$ and $y^* < 0$?

- ☐ Yes
☐ No

37

Copyright © SAS Institute Inc. All rights reserved.



How can you find an optimal solution in this case?

The previous graph (without the annotations) can be produced by the following SAS program:

```
data polynomial;
  do x=-20 to 20 by 1;
    do y=-20 to 20 by 1;
      z=x**2-x-2*y-x*y+y**2;
      output;
    end;
  end;
run;

proc g3d data=polynomial;
  plot y*x=z / grid rotate=-25 ctop=red cbottom=black
           yticknum=3 xticknum=3 zmin=3 zmax=1220;
run;
```

1.3 OPTMODEL Procedure

Objectives

- Use the OPTMODEL procedure to solve a simple mathematical optimization problem.

40

Copyright © SAS Institute Inc. All rights reserved.



Solving a mathematical optimization problem means finding an optimal solution or determining that there is no optimal solution to the mathematical optimization problem. The OPTMODEL procedure in SAS Optimization software is used in this course. SAS Optimization takes advantage of the SAS Viya distributed, in-memory engine to deliver optimization results at breakthrough speeds. In-memory data persistence eliminates the need to load data multiple times during iterative analyses.

The OPTMODEL procedure is a full-fledged interactive algebraic modeling language. The ability to solve mathematical optimization problems is a powerful feature of PROC OPTMODEL. It can be used as a building block in the design of integrated solutions to complex business problems.

In the demonstration that follows, PROC OPTMODEL is used to solve the mathematical optimization problem of minimizing the two-dimensional polynomial. An additional PROC OPTMODEL command prints the values of the decision variables.



Solving the Simple Polynomial Example Using PROC OPTMODEL

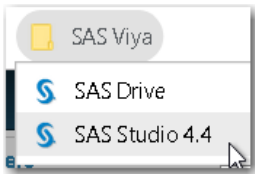
This demonstration illustrates the solution of a simple mathematical optimization problem using PROC OPTMODEL.

Here is how you access the software and demonstration programs used in this course.

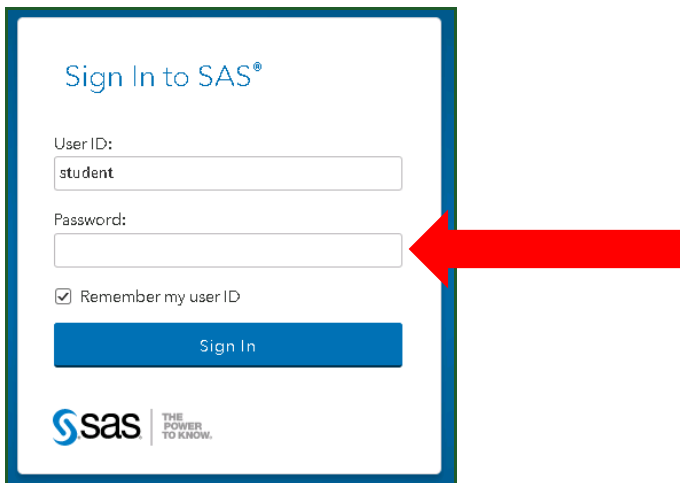
1. Follow the instructions that were provided in order to log on to the course server. From the desktop, click the **Chrome** icon to open a web browser.



2. On the browser's bookmarks bar, select **SAS Studio** from the **SAS Viya** folder.



3. Enter **Metadata0** for the password in the Sign In to SAS window.



4. Expand **Folder Shortcuts** and the **ORVY** folder.
5. Open and run the program titled **ORVY01D01.sas**.

```
proc optmodel;
  var x, y;
  min z = x**2 - x - 2*y - x*y + y**2;
  solve;
  print x y;
quit;
```

The first and last terms in the objective function can also be entered using the caret symbol (^) for exponentiation. However, this symbol is also a logical NOT operator in Base SAS.

PROC OPTMODEL Output

The OPTMODEL Procedure		
Problem Summary		
Objective Sense	Minimization	
Objective Function	z	
Objective Type	Quadratic	
Number of Variables	2	
Bounded Above	0	
Bounded Below	0	
Bounded Below and Above	0	
Free	2	
Fixed	0	
Number of Constraints	0	
Constraint Coefficients	0	
Performance Information		
Execution Mode	Single-Machine	
Number of Threads	4	
Solution Summary		
Solver	QP	
Algorithm	Interior Point	
Objective Function	z	
Solution Status	Optimal	
Objective Value	-2.333333333	
Primal Infeasibility	0	
Dual Infeasibility	6.861556E-17	
Bound Infeasibility	0	
Duality Gap	0	
Complementarity	0	
Iterations	0	
Presolve Time	0.00	
Solution Time	0.00	
	x	y
	1.3333	1.6667

PROC OPTMODEL selects an optimization algorithm based on problem characteristics (in this example, an interior point algorithm for quadratic programming problems). No iterations of the interior point algorithm were required to obtain an optimal solution. The performance information is omitted from subsequent PROC OPTMODEL output.

6. When you add the bounds $x \geq 0$ and $y \leq 0$, you can see that PROC OPTMODEL selects the same algorithm but finds a different optimal solution.

```
proc optmodel;
  var x >= 0, y <= 0;
  min z = x**2 - x - 2*y - x*y + y**2;
  solve;
  print x y;
quit;
```

PROC OPTMODEL Output

The OPTMODEL Procedure

Problem Summary

Objective Sense	Minimization
Objective Function	z
Objective Type	Quadratic
Number of Variables	2
Bounded Above	1
Bounded Below	1
Bounded Below and Above	0
Free	0
Fixed	0
Number of Constraints	0
Constraint Coefficients	0

Solution Summary

Solver	QP
Algorithm	Interior Point
Objective Function	z
Solution Status	Optimal
Objective Value	-0.250000022
Primal Infeasibility	0
Dual Infeasibility	1.9167889E-9
Bound Infeasibility	8.7045896E-9
Duality Gap	1.9890322E-8
Complementarity	0
Iterations	4
Presolve Time	0.00
Solution Time	0.02

x	y
---	---

0.5	8.7046E-09
-----	------------

End of Demonstration



Practice

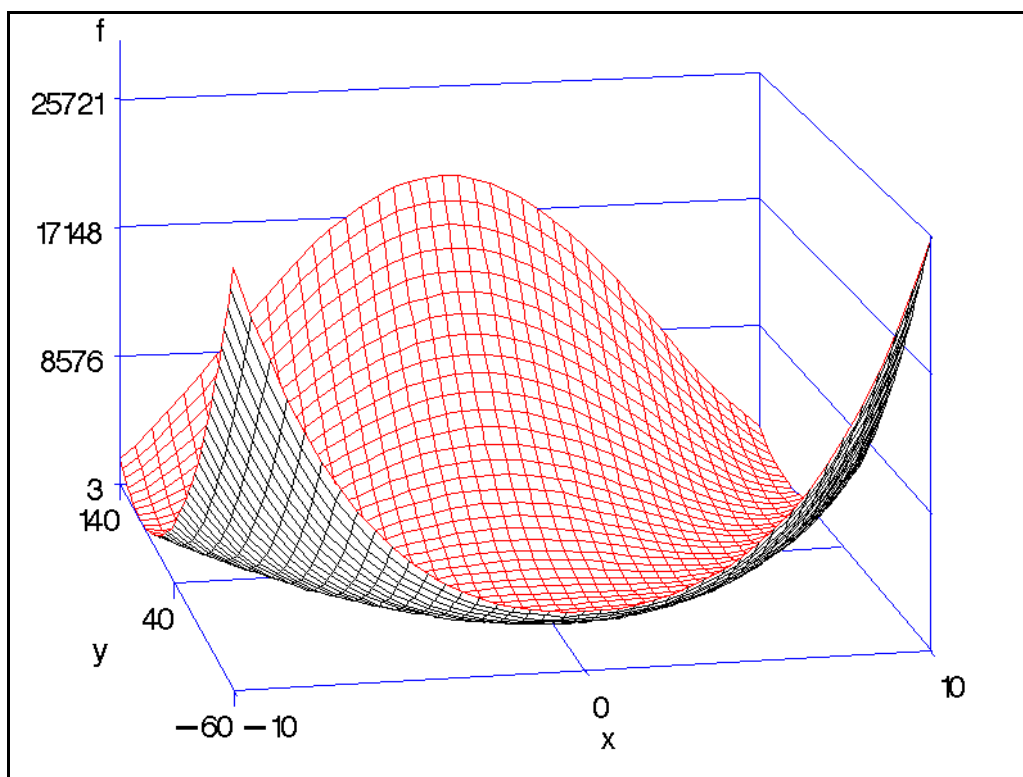
2. Using the OPTMODEL Procedure

Use PROC OPTMODEL to minimize the Rosenbrock banana function.

$$f = (y - x^2)^2 + (1 - x)^2$$

(This is a standard example to show that methods that use only the gradient can fail by repeatedly crossing back and forth over the “valley,” whereas methods that use information about curvature can converge in a few iterations.)

Why do you think that the objective function is called a *banana function*?



Note: This graph can be produced by the SAS program **banana_graph.sas**.

End of Practices

1.4 Solutions

Solutions to Practices

1. Classifying Mathematical Optimization Problems

- a. ILP: All variables are required to be integers. This ILP has two optimal solutions: $x = 3, y = 0, z = 0$, and $x = 2, y = 0, z = 2$. (Both solutions are feasible and have objective value 12.) The first constraint can be used to show that the objective value is at least $20(4/7) = 11\frac{3}{7}$.
- b. NLP: Although the objective function is linear, the inequality constraint contains nonlinear (quadratic) terms. This NLP has a unique optimal solution: $x = 0.67689, y = 2.6588, z = -0.33567$. (Apart from the bound on x , the feasible region is a cross section of an ellipsoid and hence convex.)
- c. LP: Linear constraints can be equations or inequalities. This LP has the unique optimal solution: $x = 171, y = 0, z = 54$.
- d. MILP: Only some of the variables (x and y , but not z) are required to be integer, but otherwise the constraints are linear. Every point with $x = 3, y = 2$, and $2\frac{1}{3} \leq z \leq 2\frac{1}{2}$ is an optimal solution to the MILP with objective value 22. You can think of z as a parameter that enables you to exchange one resource for another.

2. Using the OPTMODEL Procedure

```
proc optmodel;
  var x, y;
  min f = (y-x**2)**2 + (1-x)**2;
  solve;
  print x y;
quit;
```

The objective function can also be entered using the caret symbol (^) for exponentiation, but this symbol is also a logical NOT operator in Base SAS.

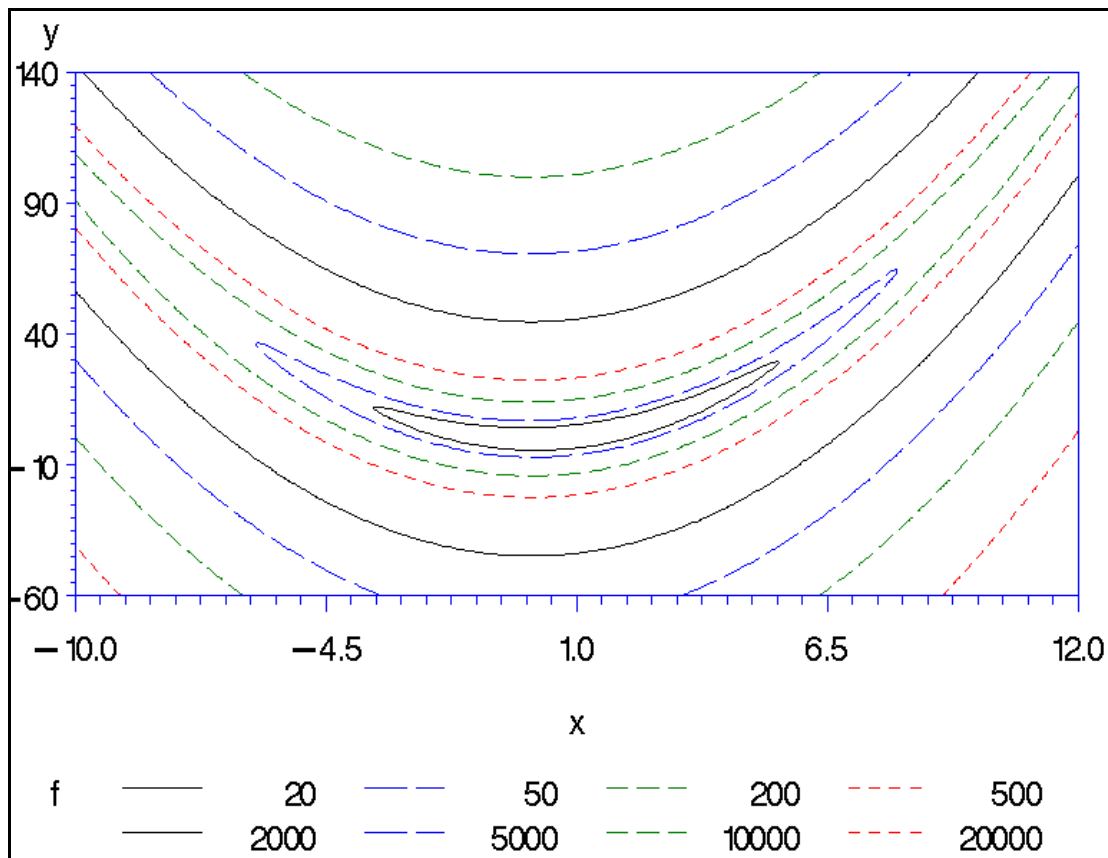
PROC OPTMODEL Output

The OPTMODEL Procedure		
Problem Summary		
Objective Sense	Minimization	
Objective Function	f	
Objective Type	Nonlinear	
Number of Variables	2	
Bounded Above	0	
Bounded Below	0	
Bounded Below and Above	0	
Free	2	
Fixed	0	
Number of Constraints	0	
Solution Summary		
Solver	NLP	
Algorithm	Interior Point	
Objective Function	f	
Solution Status	Optimal	
Objective Value	2.962209E-15	
Optimality Error	8.9089584E-8	
Infeasibility	0	
Iterations	5	
Presolve Time	0.00	
Solution Time	0.03	
x y		
1	1	

PROC OPTMODEL selected a different algorithm than in the demonstration. In this example, five iterations were required.

The solution $x = y = 1$ has objective value zero, but the reported value is positive. The default number of significant digits for the PRINT command (PDIGITS=5) is less than the number for the objective value (BEST12.), so the actual values of x and y found by the solver are not both exactly 1.

The objective function is called a banana function because the level sets $\{[x,y]: f(x,y) = z\}$ for $z > 0$ have a banana shape. (See the contour plot below.)

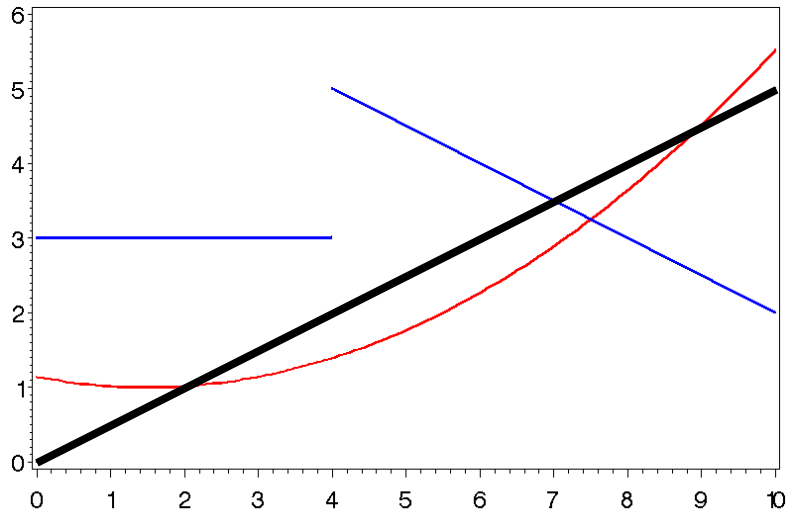


End of Solutions

Solutions to Activities and Questions

1.01 Activity – Correct Answer

Which of the three functions below is a linear function?



The figure shows graphs of a linear function in black ($y = 0.5x$), a continuous nonlinear function in red, and a discontinuous function in blue ($y = 3$ for $x \leq 4$ and $y = 7 - 0.5x$ for $x > 4$) of a single variable x .

1.02 Multiple Choice Question – Correct Answer

Classify the mathematical optimization problem.

- a. LP
- b. ILP
- c. MILP
- ☒ d. NLP

23

Copyright © SAS Institute Inc. All rights reserved.



This NLP has the unique optimal solution $x = y = z = 1$. (A cube has the smallest surface area among rectangular prisms with a given volume.)

1.03 Multiple Choice Question – Correct Answer

Classify the mathematical optimization problem.

- ☒ a. LP
- b. ILP
- c. MILP
- d. NLP

26

Copyright © SAS Institute Inc. All rights reserved.



This LP has the unique optimal solution $x = y = 5/6$ and $z = -5/6$. This problem can be interpreted as finding the point $[x,y]$ that is farthest from the boundary of the region determined by setting $z = 0$.

1.04 Question – Correct Answer

Is $[x^*, y^*] = [4/3, 5/3]$ a feasible solution to the problem of minimizing $z(x,y) = x^2 - x - 2y - xy + y^2$ subject to the bound constraints $x \geq 0$ and $y \leq 0$?

- ☐ Yes
☒ No

36

Copyright © SAS Institute Inc. All rights reserved.



The solution has $y^* > 0$.

1.05 Question – Correct Answer

Will the optimal solution $[x^*, y^*]$ to the problem of minimizing $z(x,y) = x^2 - x - 2y - xy + y^2$ subject to the bound constraints $x \geq 0$ and $y \leq 0$ have $x^* > 0$ and $y^* < 0$?

- ☐ Yes
☒ No

38

Copyright © SAS Institute Inc. All rights reserved.



The gradient $\nabla z(x,y) = [2x - 1 - y, -2 - x + 2y] \neq [0, 0]$ unless $[x,y] = [4/3, 5/3]$. Therefore, no point $[x^*, y^*]$ with $x^* > 0$ and $y^* < 0$ can be an optimal solution. The optimal solution must lie on the boundary $x = 0$ and $y \leq 0$ or $x \geq 0$ and $y = 0$.

Lesson 2 Linear Programming

2.1	Introduction to Linear Programming	2-3
	Demonstration: Solving a Linear Programming Problem Using PROC OPTMODEL.....	2-14
	Practice.....	2-20
	Demonstration: Solving the Dual Linear Programming Problem Using PROC OPTMODEL (Self-Study).....	2-22
2.2	Formulating and Solving Linear Programming Problems Using the OPTMODEL Procedure	2-24
	Demonstration: Using PROC OPTMODEL to Solve the Furniture-Making Problem.....	2-33
	Practice.....	2-42
2.3	Using Index Sets and Arrays in the OPTMODEL Procedure	2-43
	Demonstration: Using Arrays and Index Sets in PROC OPTMODEL to Solve the Furniture-Making Problem.....	2-69
	Practice.....	2-76
2.4	Dual Values and Reduced Costs in the Simplex Method (Self-Study)	2-77
	Demonstration: Dual Values and Reduced Profits in the Furniture-Making Problem (Self-Study).....	2-89
2.5	Reading and Writing Data in the OPTMODEL Procedure	2-91
	Demonstration: Reading the In-Memory Class Data Set in PROC OPTMODEL.....	2-97
	Demonstration: Reading Problem Data in PROC OPTMODEL for the Furniture- Making Problem	2-112
	Demonstration: Writing Solution Data from PROC OPTMODEL for the Furniture- Making Problem	2-117
2.6	Solutions	2-118
	Solutions to Practices	2-118
	Solutions to Activities and Questions.....	2-125

2.1 Introduction to Linear Programming

Objectives

- Introduce the basic ideas in the primal and dual simplex approaches to solving linear programming problems.
- Describe how, geometrically, the primal simplex and dual simplex algorithms solve linear programming problems.
- Introduce the Lagrangian function.
- Use the OPTMODEL procedure to enter and solve simple linear programming problems.

3

Copyright © SAS Institute Inc. All rights reserved.



The General Framework of Linear Programming

Linear Programming Problem

$$\begin{array}{ll} \min | \max & f_1 x_1 + \dots + f_n x_n \\ \text{subject to} & \mathbf{Ax} \{ \leq, =, \geq \} \mathbf{b} \\ & l_j \leq x_j \leq u_j \quad (j = 1, 2, \dots, n) \end{array}$$



Each linear constraint can be either an inequality or an equation.



Bounds can be $\pm\infty$, so x_j can be restricted to be nonnegative ($l_j = 0$ and $u_j = +\infty$) or free ($l_j = -\infty$ and $u_j = +\infty$).

4

Copyright © SAS Institute Inc. All rights reserved.



Using the restrictions associated with linear programming, discussed earlier, the objective function from the general optimization problem can be written as a linear combination of the decision variables. The constraints written above are in matrix notation. The product of the matrix **A** and a (n x 1) vector of decision variables has some relationship with a set of bounds, **b**. If there are m constraints and n decision variables, **A** will be a matrix with m rows and n columns.

Linear Programming Problem

$$\begin{array}{ll} \min | \max & f_1x_1 + \dots + f_nx_n \\ \text{subject to} & \mathbf{Ax} \{ \leq, =, \geq \} \mathbf{b} \\ & l_j \leq x_j \leq u_j \quad (j = 1, 2, \dots, n) \end{array}$$



Proportionality: No economies of scale.



Additivity: No synergy or interaction terms.



Divisibility: Variables can assume any fractional value.



Certainty: All parameters are known constants.

5
Copyright © SAS Institute Inc. All rights reserved.



Proportionality: All returns and resource usages change proportionally with the variables, and there are no economies or diseconomies of scale.

Additivity: The total effect of any two variables is the sum of their individual effects without synergy or interaction (that is, no cross-product terms in the constraints or the objective).

Divisibility: Variables can assume any fractional values.

Certainty: All parameters in the model are known constants, not random variables.

The Lagrangian function provides a framework in which values for decision variables that simultaneously optimize the objective function and that satisfy the constraints can be derived.

Lagrangian Function

$$L(\mathbf{x}, \lambda) = f(\mathbf{x}) + \lambda[\mathbf{b} - \mathbf{Ax}]$$

6

Copyright © SAS Institute Inc. All rights reserved.



The Lagrangian function, shown here, provides a general framework for solving constrained optimization problems. The generic linear programming problem is specified to fit into this framework. Decision variables are represented by the array $\mathbf{x}$. The constraints and the objective function are still linear functions of the decision variables. The constraint multiplier, λ , is also known as a *shadow price*. Its role in optimization is explained later in this lesson.

Lagrangian Function

$$L(\mathbf{x}, \lambda) = f(\mathbf{x}) + \lambda[\mathbf{b} - \mathbf{Ax}]$$

optimize $\rightarrow$

$$\frac{dL}{dx_i} = 0$$

$$\frac{dL}{d\lambda_i} \{\leq, =, \geq\} 0$$

7

Copyright © SAS Institute Inc. All rights reserved.



The Lagrangian function is optimized by taking its derivatives with respect to the decision variables and the constraint multipliers, the lambdas. This leads to the first-order conditions for optimization, shown above. The relationship between the derivative of the Lagrangian with respect to lambda and zero is taken from the specification of the constraint.

Lagrangian Function	
$L(\mathbf{x}, \lambda) = f(\mathbf{x}) + \lambda[\mathbf{b} - \mathbf{Ax}]$	hours of labor used: h
optimize $\rightarrow$	≤ 100 $2h$
$\frac{dL}{dx_i} = 0$	$2h \leq 100 \rightarrow 0 \leq 100 - 2h$
$\frac{dL}{d\lambda_i} \{\leq, =, \geq\} 0$	$100 - 2h \geq 0$
	$\partial L / \partial \lambda_h \geq 0$

8

sas

Copyright © SAS Institute Inc. All rights reserved.

For example, consider a resource constraint on the hours of labor used, h . Suppose that the amount used must be less than or equal to 100, and that two units of labor are required for each unit of output: $2h \leq 100 \rightarrow 0 \leq 100 - 2h$. So the first-order condition is $100 - 2h \geq 0$, or $\partial L / \partial \lambda_h \geq 0$.

A Two-Dimensional Linear Programming Example with Geometric Interpretation

Two-Dimensional Example

maximize $12x + 19y$

subject to

$$\begin{cases} x + 3y \leq 225 \\ x + y \leq 117 \\ 3x + 4y \leq 420 \\ x \geq 0, y \geq 0 \end{cases}$$

Linear programming
(LP) problem with two
decision variables

LP constraints determine a *feasible region* in two dimensions.



9
Copyright © SAS Institute Inc. All rights reserved.

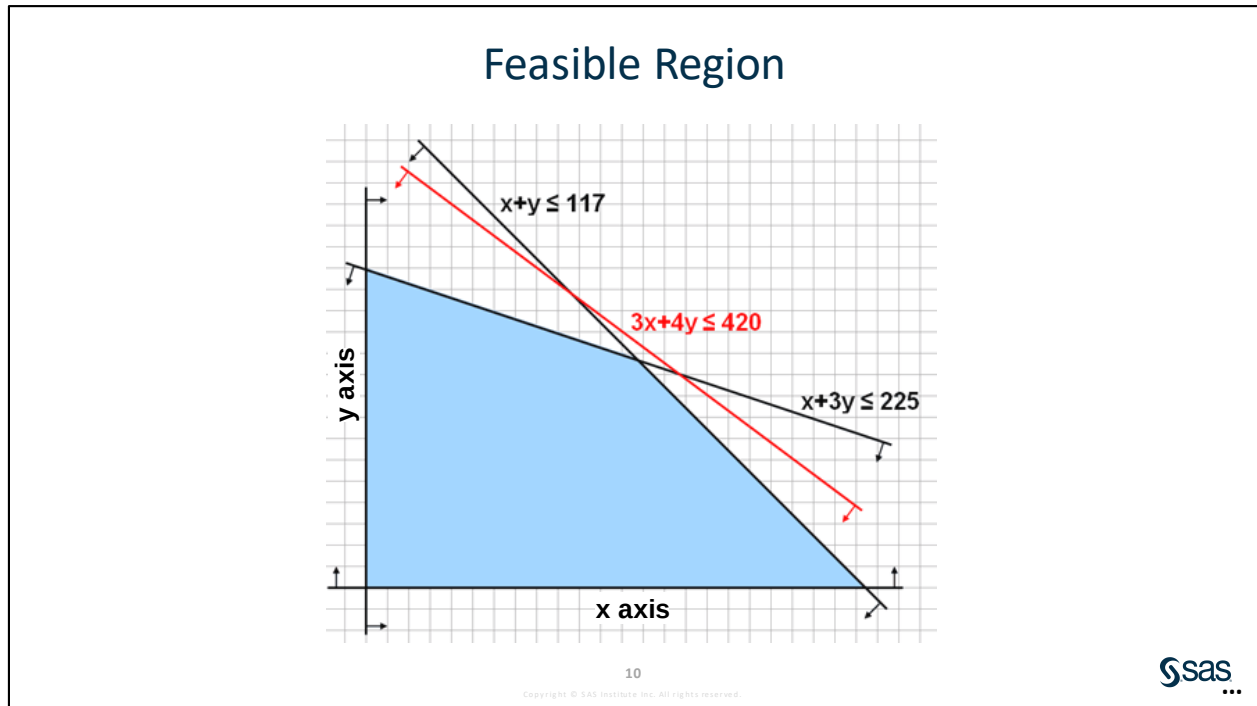


Consider the linear programming problem above. The object is to maximize an objective function. It could represent profit associated with producing goods x and y . The constraints could be related to resource usage, production technology, budgets, and so on. It also does not make sense to consider negative values of the decision variables for this problem, so they are restricted to be nonnegative. The constraints define the feasible region, and that is where the solution for the decision variable values must live.

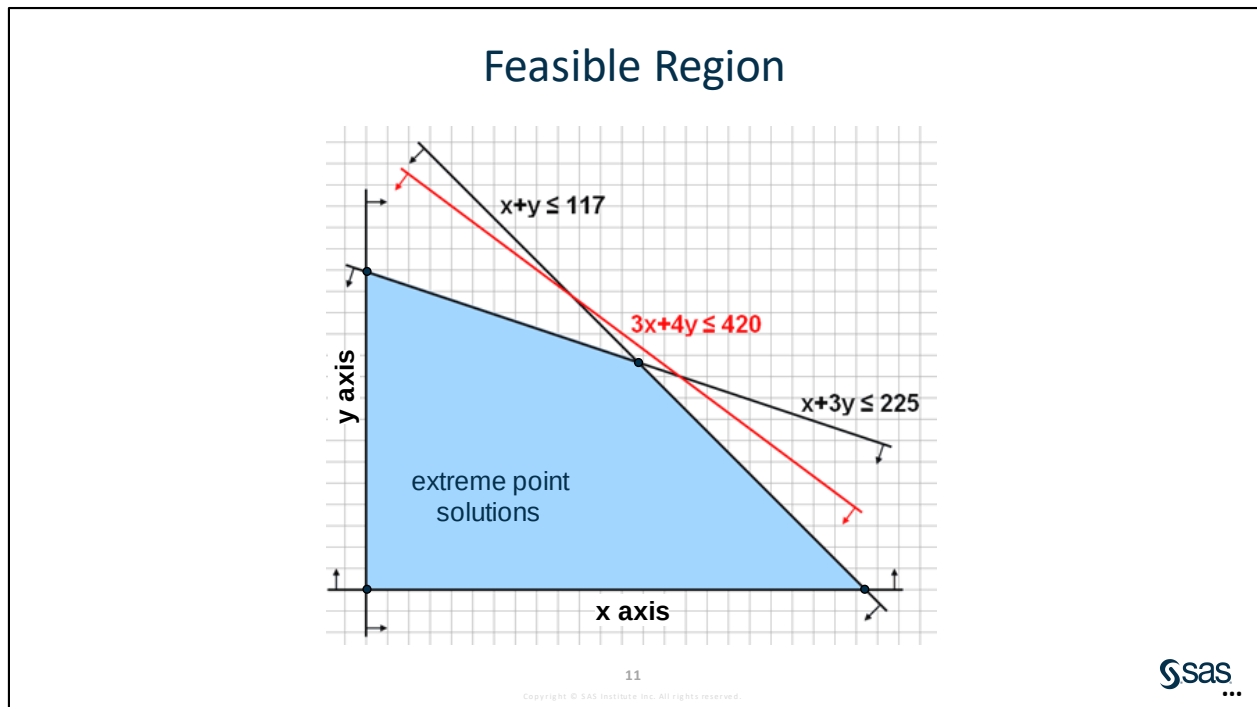
In what follows, the solution of this problem is considered in two ways. First, the focus is on the geometry of the problem. Then, the problem is specified and solved in the Lagrangian framework.

Note: A solution is *feasible* if it satisfies all the constraints.

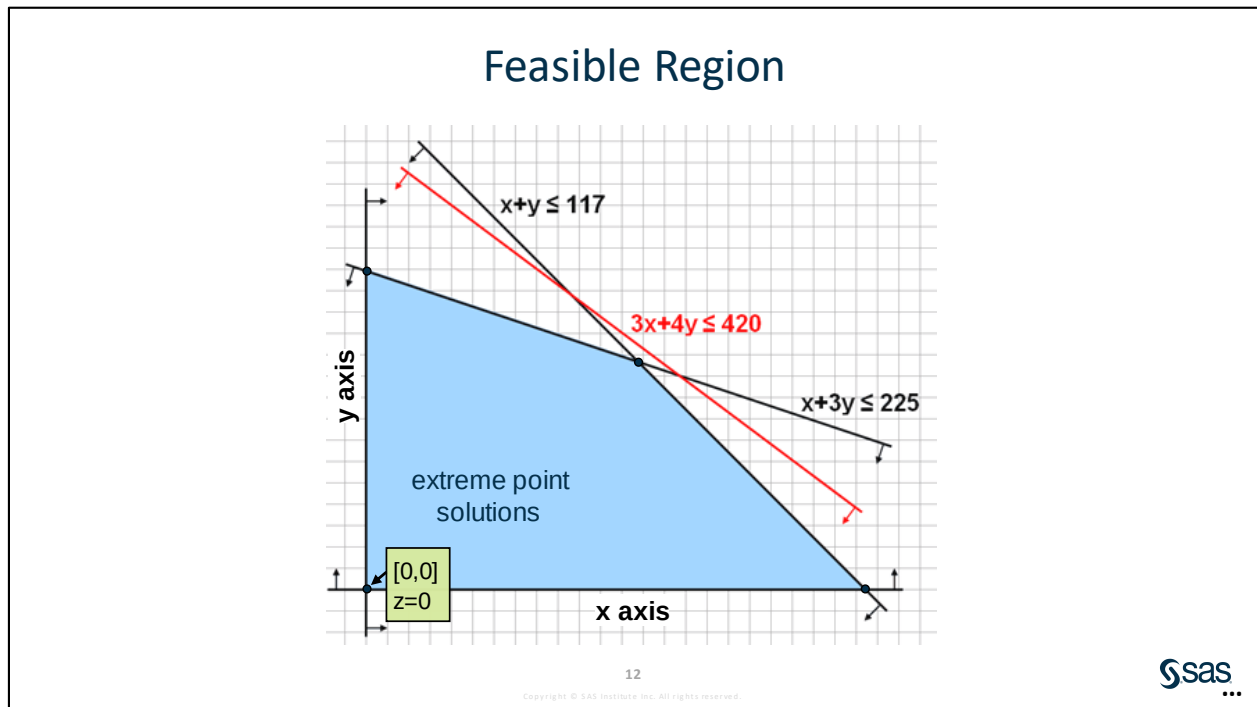
In the figure below, the scale is 1 square = 5 units.



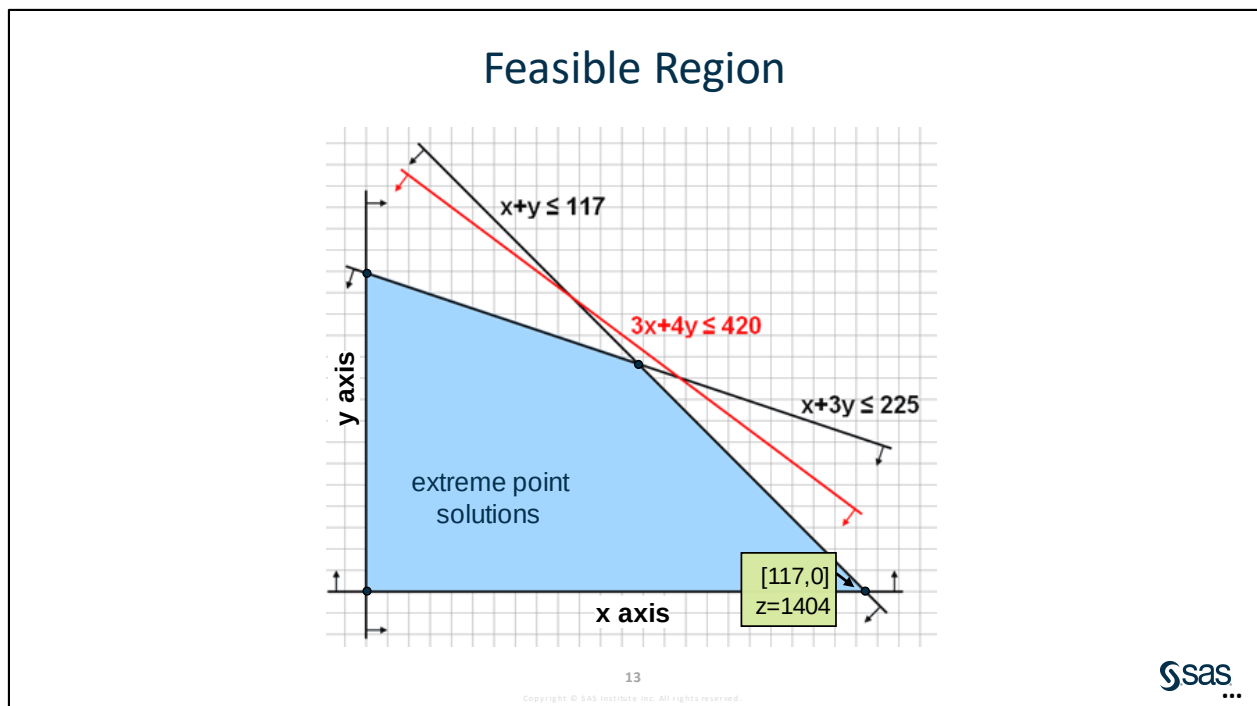
The feasible region defined by the constraints is the blue area in this picture. Note that the constraint listed in red does not help determine the feasible region. That is because the other two constraints will restrict values for the decision variables such that $3x + 4y < 420$.



A characteristic of linear programming problems is that the solution occurs at one of the corners of the feasible region. There are four corners here, counting the origin. One way to solve the problem would be to plug values of the decision variables that correspond to the corners in the blue region into the objective function, and then choose the pair that yields the largest value.

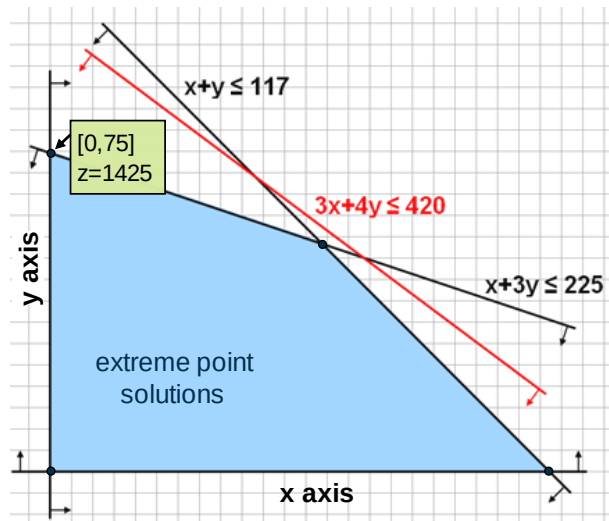


Evaluating the first corner, the origin, yields an objective function value of 0.



Evaluating the second corner gives better results. The objective function value here is 1404.

Feasible Region



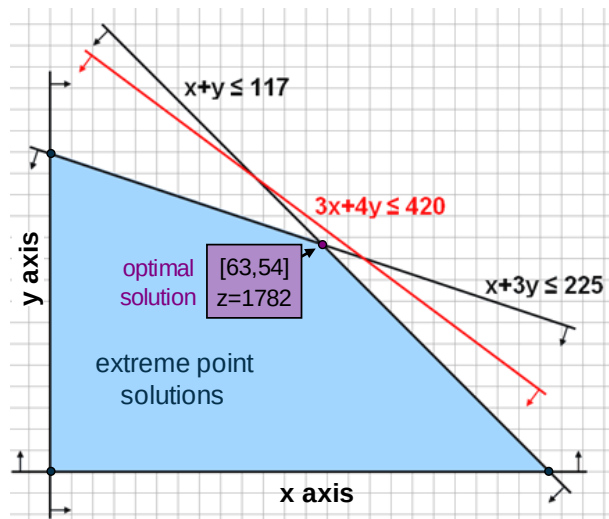
14

Copyright © SAS Institute Inc. All rights reserved.



The third corner is the best so far. It generates an objective function value of 1425.

Feasible Region



15

Copyright © SAS Institute Inc. All rights reserved.



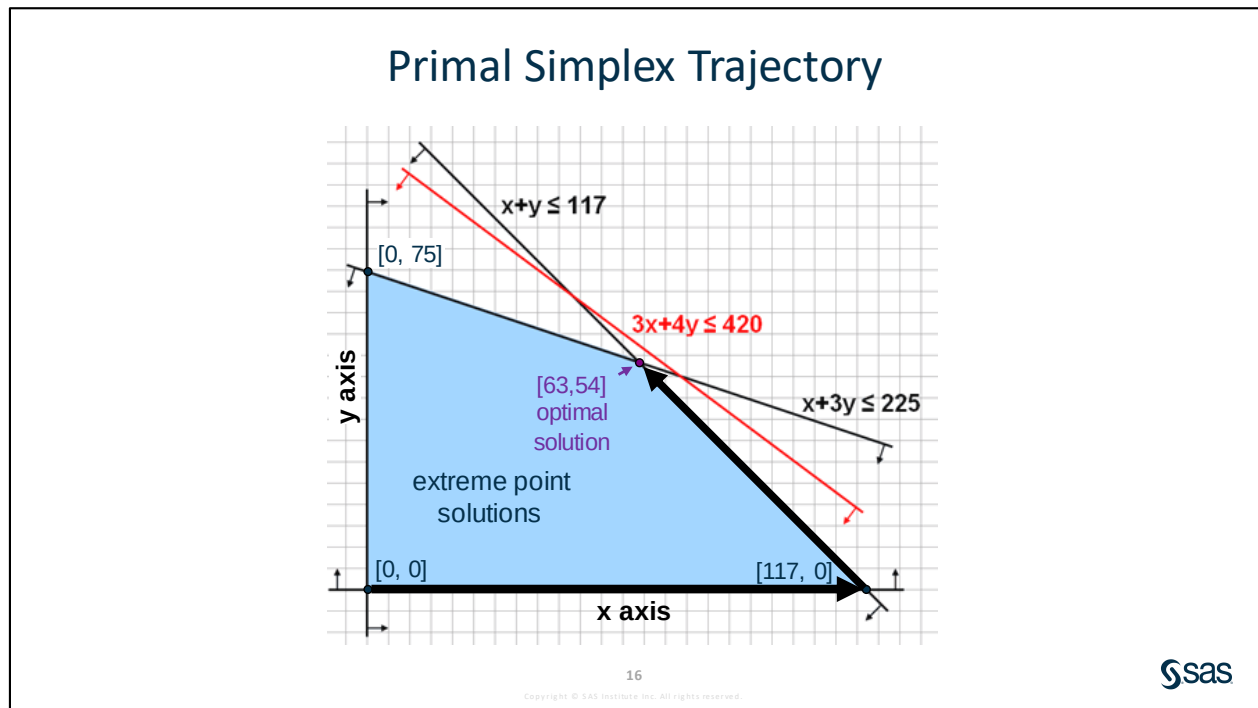
The optimal solution occurs at the fourth corner. The objective function has a value of 1782.

To summarize:

- An *extreme point* is a “corner” of the feasible region.
- An *optimal solution* is a feasible solution that maximizes (or minimizes) the objective function. Because the gradient of a linear function is constant, there is always an extreme point optimal solution if there are any solutions.
- If there are no feasible solutions, the linear programming problem is said to be *infeasible*. For example, if the constraint $3x + 4y \leq 420$ is replaced by the constraint $3x + 4y \geq 420$, the problem is infeasible.
- If the objective can be made arbitrarily large (or small in a minimization problem), the linear programming problem is said to be *unbounded*.

Note: If the linear programming problem is unbounded, then the feasible region must also be unbounded, but the converse is not always true. (Consider this two-dimensional linear programming problem without the nonnegativity constraints.)

The trajectory of the primal simplex algorithm is intuitive and straightforward.



The solution method described illustrates how the primal simplex (PS) algorithm solves linear programming problems. The slide above shows a typical trajectory for the solution of a linear programming problem that starts at an extreme point in the feasible region. All candidate solutions that the primal simplex algorithm tries are feasible.

2.01 Multiple Answer Question

How many optimal solutions can a linear programming problem have?

- a. no optimal solutions
- b. a unique optimal solution
- c. more than one, but a finite number of optimal solutions
- d. an infinite number of optimal solutions

17

Copyright © SAS Institute Inc. All rights reserved.



Lagrangian Function

$$L = 12x + 19y + \lambda_1 [225 - x - 3y] + \lambda_2 [117 - x - y] + \lambda_3 [420 - 3x - 4y]$$

19

Copyright © SAS Institute Inc. All rights reserved.



Now consider how the two-dimensional problem, introduced earlier, can be written in the Lagrangian framework.

Optimizing the Lagrangian function is a clever way to optimize the objective function. In linear programming problems, the product of a constraint and its multiplier (lambda) parameter is zero. If a constraint is tight or equal to its bound, then the multiplier can be nonzero. The multiplier quantifies how much the objective function changes with respect to a unit increase in the bound or right side of the constraint. If the constraint is not tight, then the multiplier or lambda is zero.

For example, assume that the second constraint represents the availability of metal in the process of producing x and y . If the constraint is tight, $x^* + y^* = 117$, then all the available metal is used in production. In this case, assume that $\lambda_2^* = 8.5$. This indicates that increasing the availability of metal by one unit increases the objective (let's assume its profit) by \$8.50. If, on the other hand, $x^* + y^* < 117$, then the constraint is not tight and $\lambda_2^* = 0$. If all of the available metal is not used in production, then getting an additional unit of metal, or increasing the right-hand side to 118, does not impact profit.

Lagrangian Function

$$L = 12x + 19y + \lambda_1 [225 - x - 3y] + \lambda_2 [117 - x - y] + \lambda_3 [420 - 3x - 4y]$$

First-Order Conditions

$$L_x = 12 - \lambda_1 - \lambda_2 - 3\lambda_3 = 0$$

$$L_y = 19 - 3\lambda_1 - \lambda_2 - 4\lambda_3 = 0$$

$$L_{\lambda_1} = 225 - x - 3y \geq 0$$

$$L_{\lambda_2} = 117 - x - y \geq 0$$

$$L_{\lambda_3} = 420 - 3x - 4y \geq 0$$

20

Copyright © SAS Institute Inc. All rights reserved.



Taking derivatives of the Lagrangian with respect to the decision variables and the constraint multipliers gives the first-order conditions that need to be satisfied for optimization. In this case, the first-order conditions are a system of five equations in five unknowns. Optimal values for the decision variables could be derived quickly with a pencil and paper in this instance. However, there is a better tool than these.



Solving a Linear Programming Problem Using PROC OPTMODEL

orvy02d01.sas

```
proc optmodel;
  /* declare variables */
  var x >= 0, y >= 0;

  /* declare objective */
  max z = 12*x + 19*y;

  /* declare constraints */
  con x + 3*y <= 225;
  con x + y <= 117;
  con 3*x + 4*y <= 420;

  solve;

  print x y;

quit;
```

The VAR statement is a (decision) variable declaration statement. It can include both upper and lower bounds.

```
var x >= 0 <= 100;
```

Variable bounds cannot be declared using GE or LE, but these can be used in conditional expressions.

The CON statement (short for CONSTRAINT) is a constraint declaration statement. A constraint can be either an equation or an inequality, and can include both upper and lower bounds (range constraints).

```
con 100 <= x + 3*y <= 225;
```

Note: When you declare a range constraint, the upper and lower bounds must be numeric expressions.

PROC OPTMODEL Output

The OPTMODEL Procedure

Problem Summary	
Objective Sense	Maximization
Objective Function	z
Objective Type	Linear
Number of Variables	2
Bounded Above	0
Bounded Below	2
Bounded Below and Above	0
Free	0
Fixed	0
Number of Constraints	3
Linear LE ($\leq$)	3
Linear EQ ($=$)	0
Linear GE ($\geq$)	0
Linear Range	0
Constraint Coefficients	6

Solution Summary	
Solver	LP
Algorithm	Dual Simplex
Objective Function	z
Solution Status	Optimal
Objective Value	1782
Primal Infeasibility	5.684342E-14
Dual Infeasibility	1.776357E-15
Bound Infeasibility	0
Iterations	4
Presolve Time	0.00
Solution Time	0.00

x	y
63	54

Note: For a problem with linear constraints, the problem summary includes information about the number and type of constraints.

PROC OPTMODEL used the dual simplex algorithm to solve the linear programming problem.

Note: The dual simplex algorithm is the default algorithm for all linear programming problems.

The SOLVE WITH LP statement enables the ALGORITHM= solver option to select a different algorithm such as the primal simplex (PS). This algorithm moves from an extreme point solution to an adjacent extreme point solution until it reaches an optimal solution.

```
proc optmodel;
  /* declare variables */
  var x >= 0, y >= 0;

  /* declare objective */
  max z = 12*x + 19*y;

  /* declare constraints */
  con x + 3*y <= 225;
  con x + y <= 117;
  con 3*x + 4*y <= 420;

  solve with lp / algorithm=ps;

  print x y;
  print _con_.dual;

quit;
```

The `_CON_.DUAL` option in the PRINT statement produces the estimates of the shadow prices to the Solution Summary.

PROC OPTMODEL Output (omitting the problem summary)

Solution Summary	
Solver	LP
Algorithm	Primal Simplex
Objective Function	z
Solution Status	Optimal
Objective Value	1782
Primal Infeasibility	0
Dual Infeasibility	0
Bound Infeasibility	0
Iterations	3
Presolve Time	0.00
Solution Time	0.00

x	y
63	54

[1]	_CON_DUAL
1	3.5
2	8.5
3	0.0

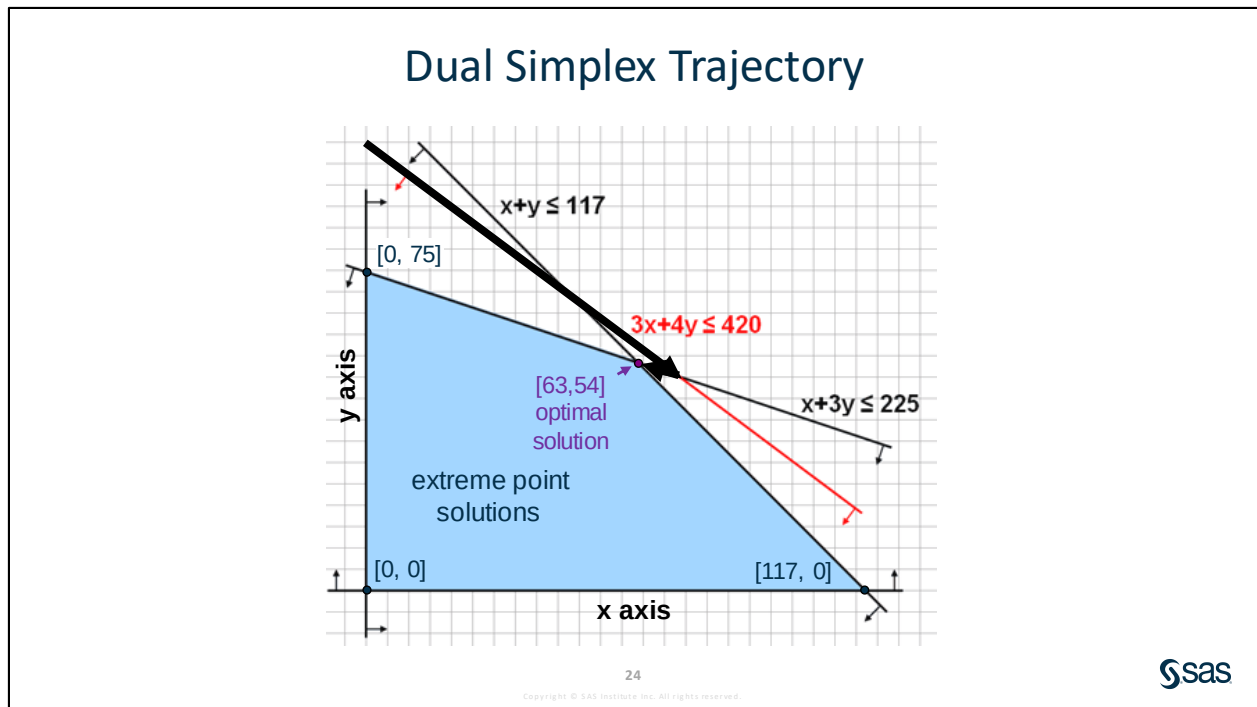
The shadow price on the third constraint is zero. This implies that the third constraint was not tight, or $3x + 4y < 425$.

End of Demonstration

2.02 Multiple Choice Question

What is the default algorithm for linear programming problems?

- a. primal simplex algorithm
- b. dual simplex algorithm
- c. interior point algorithm
- d. depends on problem characteristics



The graph above illustrates solving the two-dimensional problem using the dual simplex algorithm. Iterates of the primal simplex algorithm are feasible, but as the plot illustrates, iterates of the dual simplex algorithm are not feasible until an optimal solution is reached.

You can think of the algorithm starting at some large but infeasible value of the objective. As the algorithm iterates, the objective value diminishes until the point that feasibility is attained. Each dual extreme point solution corresponds to the intersection of two inequalities (or bounds) in the two-dimensional linear programming problem.



Practice

1. Changing the Limit of a Tight Constraint

The optimal solution to the two-dimensional linear programming problem has integer values for the decision variables. This does not need to be the case. Change the limit of the first constraint in the demonstration program above. Re-solve the problem. The optimal solution should change. Why?

2. Detecting Infeasible Linear Programming Problems

Change the sense of the constraint $3x + 4y \leq 420$ from less-than-or-equal ($\leq$) to greater-than-or-equal ($\geq$). Re-solve the problem. How does PROC OPTMODEL indicate that the resulting problem has no feasible solutions?

End of Practices

The Dual Formulation (Self-Study)

Three-Dimensional Example

minimize $225a + 117b + 420c$
 subject to $a + b + 3c \geq 12$
 $3a + b + 4c \geq 19$
 $a \geq 0, b \geq 0, c \geq 0$

← decision variables
a, b, and c

26

Copyright © SAS Institute Inc. All rights reserved.



Primal and dual *formulations* of the problem are related to the primal and dual simplex solvers. The problem shown above is the counterpoint or dual problem to the two-dimensional (primal) maximization problem shown earlier in this lesson. The dual problem has a variable for each constraint in the primal problem and a constraint for each variable in the primal problem.

Every maximization problem has a counterpoint minimization problem. The furniture-making problem presented previously is described in the context of profit maximization. Because the optimal objective values in the primal and dual formulations are the same, the primal, maximization problem could be described equivalently as a minimization problem. An alternative but equivalent formulation can be useful to applied analysts. The alternative might represent the problem in a way that is more relevant to business users, and also be easier to formulate and solve.



Solving the Dual Linear Programming Problem Using PROC OPTMODEL (Self-Study)

orvy02d02.sas

In this context, the two-dimensional maximization problem represents the primal formulation. The dual formulation is a three-dimensional minimization problem.

```
proc optmodel;
  /* declare variables */
  var a >= 0, b >= 0, c >= 0;

  /* declare objective */
  min d = 225*a + 117*b + 420*c;

  /* declare constraints */
  con a + b + 3*c >= 12;
  con 3*a + b + 4*c >= 19;

  solve with lp / algorithm=ps;
  print a b c;
  print _con_.dual;

quit;
```

PROC OPTMODEL Output

The OPTMODEL Procedure

Problem Summary	
Objective Sense	Minimization
Objective Function	d
Objective Type	Linear
Number of Variables	3
Bounded Above	0
Bounded Below	3
Bounded Below and Above	0
Free	0
Fixed	0
Number of Constraints	2
Linear LE (<=)	0
Linear EQ (=)	0
Linear GE (>=)	2
Linear Range	0
Constraint Coefficients	6

Solution Summary	
Solver	LP
Algorithm	Primal Simplex
Objective Function	d
Solution Status	Optimal
Objective Value	1782
Primal Infeasibility	0
Dual Infeasibility	0
Bound Infeasibility	0
Iterations	6
Presolve Time	0.00
Solution Time	0.00

a	b	c
3.5	8.5	0

[1]	_CON_DUAL
1	63
2	54

Notice that the optimized value for the objective function is identical to the objective value from the primal formulation. Also note that values for decision variables that optimize the objective in the dual formulation are identical to the shadow price estimates from the primal formulation.

End of Demonstration

2.2 Formulating and Solving Linear Programming Problems Using the OPTMODEL Procedure

Objectives

- Identify model elements (variables, constraints, and objective) from a description of the business problem and its associated data.
- Use the EXPAND statement to verify that a formulation is correct.

30

Copyright © SAS Institute Inc. All rights reserved.



Decision Variables

Constraints

Objective

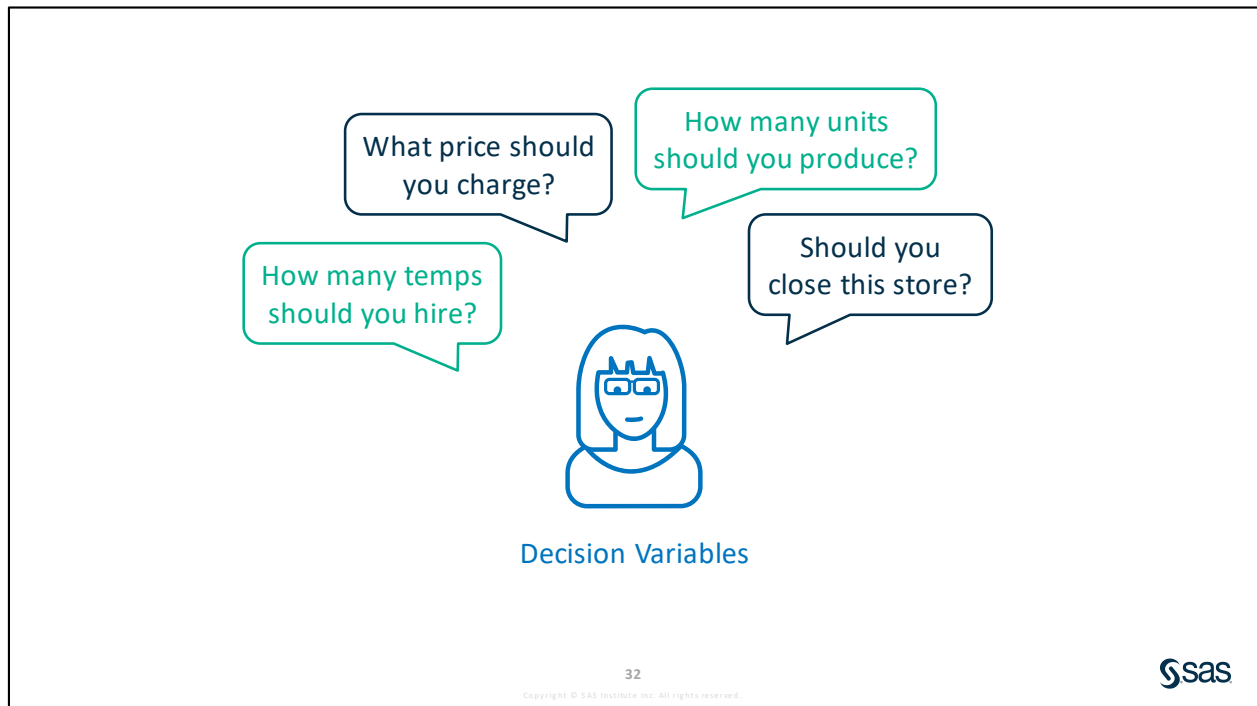


31

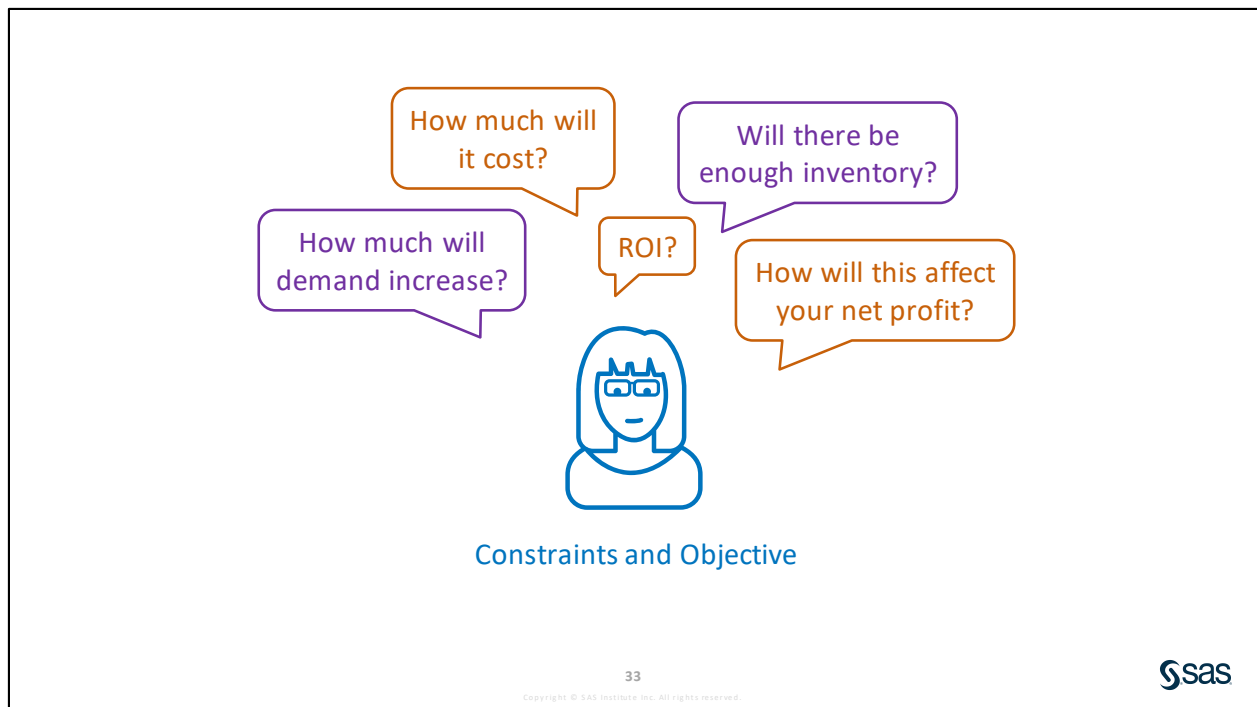
Copyright © SAS Institute Inc. All rights reserved.



To formulate a linear programming problem, you must first identify the decision variables, the constraints, and the objective. Because the constraints and objective involve functions of the decision variables, the decision variables are, in some sense, the most basic model elements.



Some decision variables are (or can be assumed to be) continuous. Others must necessarily be binary or integers.



The next step is identifying and specifying the constraints and the objective function. To do this, you might need information from a variety of sources. Demand planners might need to provide information about the relationship between price and the quantity sold for products. You might need accounting information from finance to determine reasonable budget constraints, production parameters might need to be derived from engineers to specify how resources are transformed into the outputs that you sell, and so on. For the formulation to be solved as a mathematical optimization problem, data must be available at the granularity of the decision variables. This can limit the choice of optimization models.

	Labor (hrs)	Metal (lbs)	Wood (ft ³)	Selling Price (\$)
Desks	2	1	3	94
Chairs	1	1	3	79
Bookcases	3	1	4	125
Bedframes	2	1	4	109



The process of identifying the decision variables and then specifying the objective function and constraints will be illustrated in the context of the furniture-making problem. Furniture items produced are desks, chairs, bookcases, and bed frames. All of these require various person-hours of labor and units of metal and wood. This information is given in the table above.

	Labor (hrs)	Metal (lbs)	Wood (ft ³)
Cost (\$)	14	20	11
Availability	225	117	420



35

Copyright © SAS Institute Inc. All rights reserved.



There are also costs and availabilities associated with the resources that we need. Assume that everything produced can be sold. How many desks, chairs, bookcases, and bed frames should be produced per day so that profit is as large as possible?

To formulate the furniture-making problem, the decision variables need to be identified.



	Labor (hrs)	Metal (lbs)	Wood (ft ³)
Cost (\$)	14	20	11
Availability	225	117	420



36

Copyright © SAS Institute Inc. All rights reserved.





Decision Variables

	Labor (hrs)	Metal (lbs)	Wood (ft ³)
Cost (\$)	14	20	11
Availability	225	117	420



desks



chairs



bookcases



bedframes



37
Copyright © SAS Institute Inc. All rights reserved.

The amount of each item produced seems like a good choice. Note that two words for “bed frames” are not used, because decision variables must be valid SAS variable names with no spaces.

Note: Decision variables follow the usual SAS naming conventions and are not case sensitive.

	Labor (hrs)	Metal (lbs)	Wood (ft ³)	Selling Price (\$)
Desks	2	1	3	94
Chairs	1	1	3	79
Bookcases	3	1	4	125
Bedframes	2	1	4	109
Cost (\$)	14	20	11	
Availability	225	117	420	




Objective Function



38
Copyright © SAS Institute Inc. All rights reserved.

	Labor (hrs)	Metal (lbs)	Wood (ft ³)	Selling Price (\$)
Desks	2	1	3	94
Chairs	1	1	3	79
Bookcases	3	1	4	125
Bedframes	2	1	4	109
Cost (\$)	14	20	11	
Availability	225	117	420	

Maximize **NetProfit**
= **Revenue** – **Cost**



Objective Function

39

Copyright © SAS Institute Inc. All rights reserved.

sas

The goal is to choose the number of desks, chairs, bookcases, and bed frames to produce that maximizes net profit. Net profit is defined as the difference between the revenue that we get from selling products and the cost to produce them. The data table can be used to help formulate the objective function and the constraints.

	Labor (hrs)	Metal (lbs)	Wood (ft ³)	Selling Price (\$)
Desks	2*Desks	Desks	3*Desks	94*Desks
Chairs	Chairs	Chairs	3*Chairs	79*Chairs
Bookcases	3	1	4	125
Bedframes	2	1	4	109
Cost (\$)	14	20	11	
Availability	225	117	420	

Maximize **NetProfit**
= **Revenue** – **Cost**



Objective Function

40

Copyright © SAS Institute Inc. All rights reserved.

sas

Mathematical Optimization Formulations

Definitions for decision variables,
sets, and parameters

min|max objective function
subject to constraints
variable bounds

41

Copyright © SAS Institute Inc. All rights reserved.



The order of the problem elements is shown above. Decision variables, sets, and parameters are defined first, and then the objective function and constraints are specified.

The corresponding PROC OPTMODEL program uses this structure but presents its elements in reverse order. PROC OPTMODEL programs have more flexibility than this structure indicates. For example, the objective can be declared before the constraints. Not all programs have all elements.

PROC OPTMODEL

```
proc optmodel;
  /* declare sets and parameters */
  /* declare variables */
  /* declare constraints */
  /* declare objective */
  solve;
  /* print solution */
quit;
```

42

Copyright © SAS Institute Inc. All rights reserved.



PROC OPTMODEL

```
proc optmodel;
  /* declare variables */
  desks, chairs, bookcases, bedframes
  /* declare constraints */
  /* declare objective */
  solve;
  /* print solution */
quit;
```

43

Copyright © SAS Institute Inc. All rights reserved.



First, the decision variables are specified. Note that we are listing only the steps required for this formulation of the furniture-making problem.

PROC OPTMODEL

```
proc optmodel;
  /* declare variables */
  desks, chairs, bookcases, bedframes
  /* declare constraints */
  availability of labor, metal, wood
  /* declare objective */
  solve;
  /* print solution */
quit;
```

44

Copyright © SAS Institute Inc. All rights reserved.



Next, the constraints are specified. The constraints in this problem are tied to the availability of inputs or resources.

PROC OPTMODEL

```
proc optmodel;
  /* declare variables */
  desks, chairs, bookcases, bedframes
  /* declare constraints */
  availability of labor, metal, wood
  /* declare objective */
  maximize NetProfit=Revenue-Cost
  solve;
  /* print solution */
quit;
```

The objective depends on the generation of revenue and cost.



Using PROC OPTMODEL to Solve the Furniture-Making Problem

orvy02d03.sas

```
proc optmodel;
  /* declare variables */
  var Desks >= 0, Chairs >= 0, Bookcases >= 0, Bedframes >= 0;

  /* declare constraints */
  con Labor: 2*Desks + Chairs + 3*Bookcases + 2*Bedframes <= 225;
  con Metal: Desks + Chairs + Bookcases + Bedframes <= 117;
  con Wood: 3*Desks + 3*Chairs + 4*Bookcases + 4*Bedframes <= 420;

  /* declare objective */
  max NetProfit =
    94*Desks + 79*Chairs + 125*Bookcases + 109*Bedframes
    - 14 * (2*Desks + Chairs + 3*Bookcases + 2*Bedframes)
    - 20 * (Desks + Chairs + Bookcases + Bedframes)
    - 11 * (3*Desks + 3*Chairs + 4*Bookcases + 4*Bedframes);

  expand;

  solve;

  print Desks Chairs Bookcases Bedframes;

quit;
```

The names follow the usual SAS naming conventions. That is, they can be a maximum of 32 characters, are **not** case sensitive, and cannot begin with a number. Avoid names that begin with an underscore. Notice that, in addition to the objective, the constraints are also named.

Model elements must be declared before they are used in subsequent statements. (PROC OPTMODEL executes interactively, unlike many other SAS procedures, and has no RUN statement.)

The EXPAND statement causes PROC OPTMODEL to write a listing of the formulation.

The %LET macro statement can be used for recurring expressions in the constraints and objective.

```
%let Hrs = 2*Desks + Chairs + 3*Bookcases + 2*Bedframes;
%let Lbs = Desks + Chairs + Bookcases + Bedframes;
%let Ft3 = 3*Desks + 3*Chairs + 4*Bookcases + 4*Bedframes;
```

PROC OPTMODEL Output

The OPTMODEL Procedure

```

Var Desks >= 0
Var Chairs >= 0
Var Bookcases >= 0
Var Bedframes >= 0
Maximize NetProfit=13*Desks + 12*Chairs + 19*Bookcases + 17*Bedframes
Constraint Labor: 2*Desks + Chairs + 3*Bookcases + 2*Bedframes <= 225
Constraint Metal: Desks + Chairs + Bookcases + Bedframes <= 117
Constraint Wood: 3*Desks + 3*Chairs + 4*Bookcases + 4*Bedframes <= 420

```

Problem Summary	
Objective Sense	Maximization
Objective Function	NetProfit
Objective Type	Linear
Number of Variables	4
Bounded Above	0
Bounded Below	4
Bounded Below and Above	0
Free	0
Fixed	0
Number of Constraints	3
Linear LE (<=)	3
Linear EQ (=)	0
Linear GE (>=)	0
Linear Range	0
Constraint Coefficients	12

Solution Summary	
Solver	LP
Algorithm	Dual Simplex
Objective Function	NetProfit
Solution Status	Optimal
Objective Value	1827
Primal Infeasibility	5.684342E-14
Dual Infeasibility	1.776357E-15
Bound Infeasibility	0
Iterations	6
Presolve Time	0.00
Solution Time	0.00

Desks	Chairs	Bookcases	Bedframes
0	48	39	30

Notice that the EXPAND statement simplified the objective function. The displayed coefficients are the net profits for the four activities. For example, the net profit for making a single desk is as follows:

$$\begin{aligned} & (\text{Selling Price}) - (\text{Cost of Labor}) - (\text{Cost of Wood}) - (\text{Cost of Metal}) \\ &= \$94 - (\$14 \times 2) - (\$20) - (\$11 \times 3) = \$94 - \$81 = \$13 \end{aligned}$$

PROC OPTMODEL also enables you to declare *implicit* variables for expressions that are used repeatedly in a model. These expressions function much the same as the expressions that are defined using the %LET macro statement.

```
proc optmodel;
  /* declare variables */
  var Desks >= 0, Chairs >= 0, Bookcases >= 0, Bedframes >= 0;

  impvar Hrs = 2*Desks + Chairs + 3*Bookcases + 2*Bedframes;
  impvar Lbs = Desks + Chairs + Bookcases + Bedframes;
  impvar Ft3 = 3*Desks + 3*Chairs + 4*Bookcases + 4*Bedframes;

  /* declare constraints */
  con Labor: Hrs <= 225, Metal: Lbs <= 117, Wood: Ft3 <= 420;

  /* declare objective */
  max NetProfit =
    94*Desks + 79*Chairs + 125*Bookcases + 109*Bedframes
    - 14*Hrs - 20*Lbs - 11*Ft3;

  expand;

  solve;

  print Desks Chairs Bookcases Bedframes;

  print Hrs Lbs Ft3;

quit;
```

PROC OPTMODEL Output (omitting the solution summary)

The OPTMODEL Procedure

```

Var Desks >= 0
Var Chairs >= 0
Var Bookcases >= 0
Var Bedframes >= 0
Impvar Hrs = 2*Desks + Chairs + 3*Bookcases + 2*Bedframes
Impvar Lbs = Desks + Chairs + Bookcases + Bedframes
Impvar Ft3 = 3*Desks + 3*Chairs + 4*Bookcases + 4*Bedframes
Maximize NetProfit=94*Desks + 79*Chairs + 125*Bookcases + 109*Bedframes - 14*Hrs - 20*Lbs - 11*Ft3
Constraint Labor: Hrs <= 225
Constraint Metal: Lbs <= 117
Constraint Wood: Ft3 <= 420

```

Problem Summary	
Objective Sense	Maximization
Objective Function	NetProfit
Objective Type	Linear
Number of Variables	4
Bounded Above	0
Bounded Below	4
Bounded Below and Above	0
Free	0
Fixed	0
Number of Constraints	3
Linear LE (<=)	3
Linear EQ (=)	0
Linear GE (>=)	0
Linear Range	0
Constraint Coefficients	12

Desks	Chairs	Bookcases	Bedframes
0	48	39	30

Hrs	Lbs	Ft3
225	117	420

The problem submitted to the solver is the same with or without implicit variables, because the implicit variables are removed from the problem by substitution before it is solved. Printing the values of the implicit variables (above) shows that all of the available resources are used in the optimal solution.

The SAS log gives additional information about the solution of the linear programming problem.

```

NOTE: Problem generation will use 4 threads.
NOTE: The problem has 4 variables (0 free, 0 fixed).
NOTE: The problem uses 3 implicit variables.
NOTE: The problem has 3 linear constraints (3 LE, 0 EQ, 0 GE, 0 range).
NOTE: The problem has 12 linear constraint coefficients.
NOTE: The problem has 0 nonlinear constraints (0 LE, 0 EQ, 0 GE, 0 range).
94
95      solve;
NOTE: The OPTMODEL presolver is disabled for linear problems.
NOTE: The LP presolver value AUTOMATIC is applied.
NOTE: The LP presolver time is 0.00 seconds.
NOTE: The LP presolver removed 0 variables and 0 constraints.
NOTE: The LP presolver removed 0 constraint coefficients.
NOTE: The presolved problem has 4 variables, 3 constraints, and 12 constraint coefficients.
NOTE: The LP solver is called.
NOTE: The Dual Simplex algorithm is used.
      Objective
Phase Iteration      Value      Time
   D 2           1  7.917750E+03      0
   D 2           6  1.827000E+03      0
NOTE: Optimal.
NOTE: Objective = 1827.
NOTE: The Dual Simplex solve time is 0.00 seconds.

```

The LOGFREQ= k solver option controls the frequency of printing to the SAS log. A line is printed in the first iteration of each phase, every k iterations, and the last iteration. The default for the interior point algorithm is $k=1$ (every iteration). The default for the primal and dual simplex algorithms is determined dynamically by problem size.

Setting LOGFREQ=1 also yields additional detailed information about the solution process.

```

NOTE: Problem generation will use 4 threads.
NOTE: The problem has 4 variables (0 free, 0 fixed).
NOTE: The problem uses 3 implicit variables.
NOTE: The problem has 3 linear constraints (3 LE, 0 EQ, 0 GE, 0 range).
NOTE: The problem has 12 linear constraint coefficients.
NOTE: The problem has 0 nonlinear constraints (0 LE, 0 EQ, 0 GE, 0 range).
93
94      solve with LP / logfreq=1;
NOTE: The LP presolver value AUTOMATIC is applied.
NOTE: The LP presolver time is 0.00 seconds.
NOTE: The LP presolver removed 0 variables and 0 constraints.
NOTE: The LP presolver removed 0 constraint coefficients.
NOTE: The presolved problem has 4 variables, 3 constraints, and 12 constraint coefficients.
NOTE: The LP solver is called.
NOTE: The Dual Simplex algorithm is used.

```

Phase	Iteration	Objective Value	Time	Entering Variable	Leaving Variable
D 2	1	7.917750E+03	0	Desks	Wood (S)
D 2	2	1.983000E+03	0	Bedframes	Labor (S)
D 2	3	1.939200E+03	0	Bookcases	Desks
D 2	4	1.853100E+03	0	Chairs	Bedframes
D 2	5	1.833000E+03	0	Bedframes	Metal (S)
D 2	6	1.827000E+03	0		

```

NOTE: Optimal.
NOTE: Objective = 1827.
NOTE: The Dual Simplex solve time is 0.00 seconds.

```

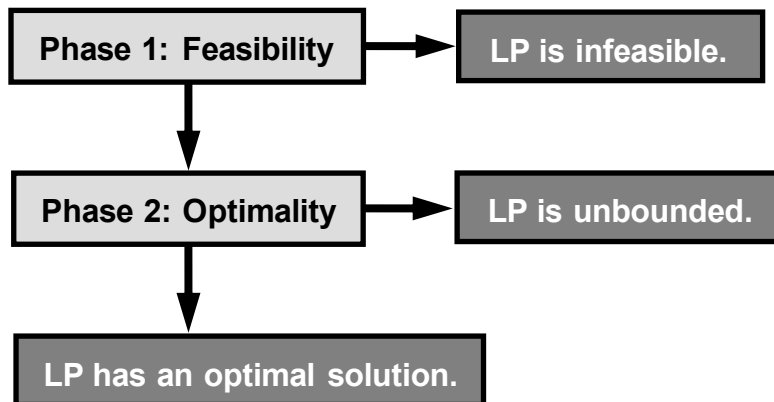
In the SAS log, the (S) denotes a slack variable. These are added in the primal and dual simplex algorithms to convert inequalities to equations.

Note: You can specify solver options directly only when you use the WITH clause. Here is an example:

```
Solve WITH LP / logfreq=1.
```

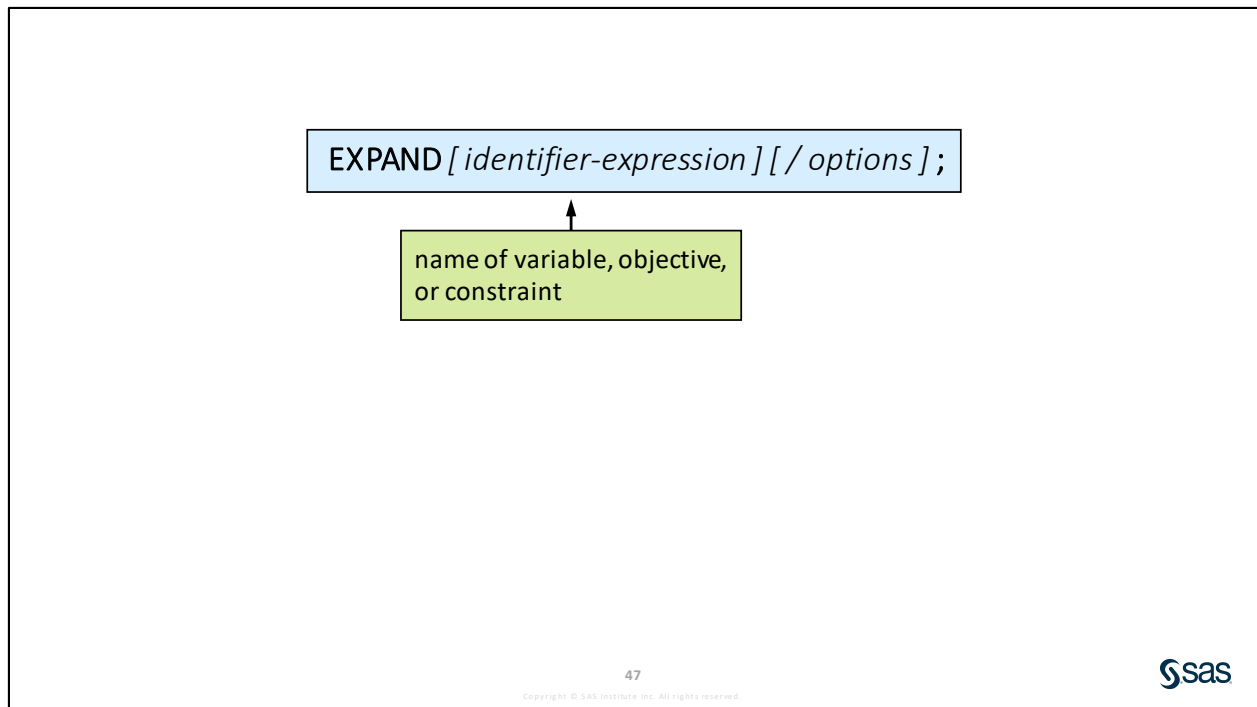
Entering Variable and **Leaving Variable** refer to the set of *basic variables* that are maintained in both the primal and dual simplex algorithms. In the two-dimensional, linear programming problem, basic variables indicate which constraints determine (intersect to form) the extreme point solution in the current iteration.

Phase refers to the two-phase simplex method. Its conceptual flow chart is pictured below for the primal simplex algorithm.

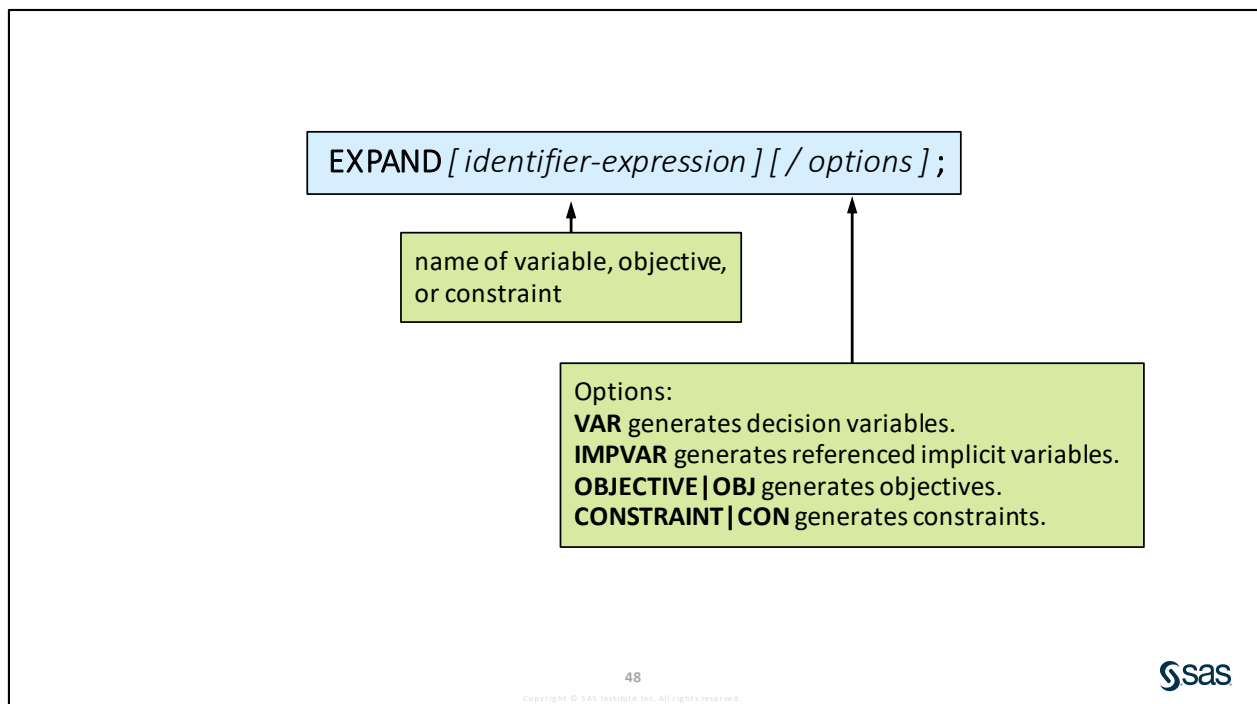


For the dual simplex algorithm, Phase 1 attempts to find a solution that satisfies the optimality conditions (together with the corresponding dual variables). Phase 2 attempts to find a solution that is also feasible (and hence optimal).

End of Demonstration



The `EXPAND` statement causes the `OPTMODEL` procedure to list the formulation. It can produce simplified versions of the objective and constraints, and it is useful for error checking and for documenting your work.



By default, the `EXPAND` statement lists the decision variables, implicit variables defined in the problem, the objective function, and the constraints. However, it has options to list only a subset of these components or individual elements of the formulation.

2.03 Multiple Answer Question

Which of the following statements are true of SAS names?

- a. SAS names can be up to 32 characters long.
- b. SAS names cannot start with a number.
- c. SAS names are case sensitive.



Practice

3. Adding a Budget Constraint to the Furniture-Making Problem

Add a budget constraint to the previous demonstration program. Specify that the total cost of all the resources that are used in the furniture-making problem is at most \$10,000. Expand only this new constraint. Does the optimal objective value change?

End of Practices

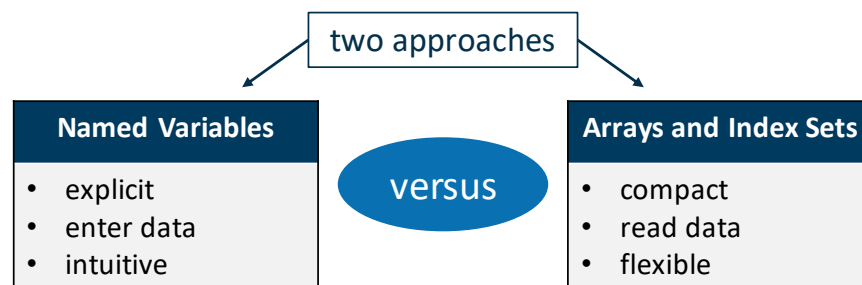
2.3 Using Index Sets and Arrays in the OPTMODEL Procedure

Objectives

- Formulate linear programming problems using index sets and parameter arrays.

53

Copyright © SAS Institute Inc. All rights reserved.



54

Copyright © SAS Institute Inc. All rights reserved.



Arrays and index sets provide an alternative approach to formulating optimization problems in PROC OPTMODEL. It is not as straightforward and intuitive as the approach using named variables, described above. But it is the preferred method to use for solving larger and more complex problems than the ones shown up to this point in the course.

The array and index set approach has distinct advantages for formulating problems that you are likely to run into as an analyst. One main advantage is the ability to read the problem data from tables into parameter arrays.

	Labor (hrs)	Metal (lbs)	Wood (ft ³)	Selling Price (\$)
Desks	2	1	3	94
Chairs	1	1	3	79
Bookcases	3	1	4	125
Bedframes	2	1	4	109
Cost (\$)	14	20	11	
Availability	225	117	420	

Focus on getting the data into PROC OPTMODEL.

Data → Parameter Arrays → Formulation

In this section, values for problem elements like requirements or availabilities are entered directly into parameter arrays in the OPTMODEL procedure syntax. Parameters and parameter arrays are ubiquitous in PROC OPTMODEL programs.

Parameters and Parameter Arrays



Values can be assigned or read from a SAS data set.

```
print values;
```

Values can be listed using the PRINT statement.



values

Values are not changed by the solver.

56

Copyright © SAS Institute Inc. All rights reserved.



Parameters can be used in expressions defining implicit variables, constraints, and objectives. Parameters in the current problem are values like selling prices and requirements. These values are not changed by the algorithm that solves for optimized values of the decision variables.

Code Standards

Expression	Capitalization
Variable names	First letter of each word
Objective names	First letter of each word
Constraint names	First letter
Parameter names	All lowercase
Set names	All uppercase

57

Copyright © SAS Institute Inc. All rights reserved.



Standardizing code by following naming conventions helps with debugging and in making it more production ready. It is considered a best practice, and the table lists the conventions that are followed in this course. By convention, the names of parameters and parameter arrays are lowercase with underscores to separate words.

2.04 Multiple Choice Question

Which potential index set for the parameter array **selling_price** is most indicative of the problem context?

	Selling Price (\$)	
Desks	94	} Index Set
Chairs	79	
Bookcases	125	
Bedframes	109	

- a. {1,2,3,4}
- b. {'P1','P2','P3','P4'}
- c. {'desks','chairs','bookcases','bedframes'}

58

Copyright © SAS Institute Inc. All rights reserved.



Note: Indexing in PROC OPTMODEL is more flexible than array indexing in the SAS DATA step. Index sets can be sets of numbers or strings, and are not required to fit into a rigid sequence.

Index Sets

Index Sets				
	Labor (hrs)	Metal (lbs)	Wood (ft ³)	Selling Price (\$)
Desks	2	1	3	94
Chairs	1	1	3	79
Bookcases	3	1	4	125
Bedframes	2	1	4	109
Cost (\$)	14	20	11	
Availability	225	117	420	

60

sas

Copyright © SAS Institute Inc. All rights reserved.

Certain values organize and structure the important aspects of optimization problems. Pretty much everything happens in relation to them. For the current problem, all of the pieces can be defined in terms of resources and products. In the arrays and index set approach, they are the indexes by which the problem is organized, and it makes sense to declare them first.

Declaration statements for index sets require some new syntax. This will be illustrated first for the furniture-making problems and then defined more generally. Note that the sets of products and resources are declared before the parameter arrays.

Index Sets

	Labor (hrs)	Metal (lbs)	Wood (ft ³)	Selling Price (\$)
Desks	2	1	3	94
Chairs	1	1	3	79
Bookcases	3	1	4	125
Bedframes	2	1	4	109
Cost (\$)	14	20	11	
Availability	225	117	420	

resources

products

```
set RESOURCES = /labor metal wood/;
set PRODUCTS = /desks chairs bookcases bedframes/;
```

61

Copyright © SAS Institute Inc. All rights reserved.



The SET statement is used to declare the index sets for the formulation of the furniture-making problem. The slashes enclose the hardcoded names that are the elements of the declared sets. These are set literal expressions. Unlike arrays, described next, the members of a set cannot be referenced by location. For example, there is no first or last element of a set.

Parameters and Parameter Arrays

Parameters

	Labor (hrs)	Metal (lbs)	Wood (ft ³)	Selling Price (\$)
Desks	2	1	3	94
Chairs	1	1	3	79
Bookcases	3	1	4	125
Bedframes	2	1	4	109
Cost (\$)	14	20	11	
Availability	225	117	420	

resources

```

set PRODUCTS = /desks chairs bookcases bedframes/;
num selling_price {PRODUCTS} = [94 79 125 109];

```

sas

Now that the index sets are defined, arrays of parameters are created that are necessary to define problem elements like costs and revenues. The parameter array, **selling_price**, can be described as being indexed by **PRODUCTS**. This is another way of saying that there is a selling price for each product.

The square bracket is used to initialize the parameter array **selling_price** at the listed values. Individual elements of the parameter array can also be referenced. For example, the selling price of desks is referenced as **selling_price['desks']**.

```
print selling_price;
```

```
[1]
bedframes    109
bookcases    125
chairs       79
desks        94
```

selling_price

63

Copyright © SAS Institute Inc. All rights reserved.



It can be useful to print arrays to check your formulation. Note that the number 1 in brackets refers to the first index set (in this case, **PRODUCTS**). Also note that the order of indices is different from the order in the set literal expression.

Parameters

	Labor (hrs)	Metal (lbs)	Wood (ft ³)	Selling Price (\$)
Desks	2	1	3	94
Chairs	1	1	3	79
Bookcases	3	1	4	125
Bedframes	2	1	4	109
Cost (\$)	14	20	11	
Availability	225	117	420	

products

resources

```
set RESOURCES = /labor metal wood/;
num cost {RESOURCES} = [14 20 11];
```

64

Copyright © SAS Institute Inc. All rights reserved.



The parameter array **cost** is indexed by **RESOURCES**. This is another way of saying that there is a cost for (or associated with) each resource. The set **RESOURCES** is declared only once in the program.

Parameters

	Labor (hrs)	Metal (lbs)	Wood (ft ³)	Selling Price (\$)
Desks	2	1	3	94
Chairs	1	1	3	79
Bookcases	3	1	4	125
Bedframes	2	1	4	109
Cost (\$)	14	20	11	
Availability	225	117	420	

resources

```
set RESOURCES = /labor metal wood/;
num availability {RESOURCES} = [225 117 420];
```

65

Copyright © SAS Institute Inc. All rights reserved.



availability is another parameter array that is indexed by **RESOURCES**. It represents how much labor, metal, and wood are on hand.

Note that the declaration statement for the set **RESOURCES** would not be repeated in the SAS program. It is shown here to reinforce the connection.

Parameters

	Labor (hrs)	Metal (lbs)	Wood (ft ³)
Desks	2	1	3
Chairs	1	1	3
Bookcases	3	1	4
Bedframes	2	1	4

} products

resources

```

set RESOURCES = /labor metal wood/;
set PRODUCTS = /desks chairs bookcases bedframes/;
num required {PRODUCTS, RESOURCES} =
  [2 1 3 1 1 3 3 1 4 2 1 4];

```

66
Copyright © SAS Institute Inc. All rights reserved.

Declaring the resource requirements for producing products is a little trickier. This is because there might be a different amount of each resource required to produce each product. For example, desks require two hours of labor to produce but chairs require only one hour. A unique requirement for each product and resource pair needs to be accommodated. Another way of saying this is that requirements are indexed by **PRODUCTS** and **RESOURCES**.

It is natural to associate the first index set with rows and the second index set with columns, but the order of the index sets could be reversed, as long as it is done consistently throughout the OPTMODEL program.

Declaring Parameters More Generally in PROC OPTMODEL

Declaring Parameters

NUMBER|NUM

```
num n;  
num pi = constant('pi');  
num mat {1..n, 1..n} init 0;
```

Parameters must be declared in PROC OPTMODEL *before* they are used.

67

Copyright © SAS Institute Inc. All rights reserved.



Now the focus switches to how parameters are declared in a more general context. The term *parameter* is used here to distinguish values, like those in the slide, from variables. The term *variables* refers to decision variables in an optimization problem. **Parameters must be declared in PROC OPTMODEL before they are used**, and the values of parameters are not allowed to vary during the solution of the optimization problem.

Three numeric parameter arrays are declared in the slide.

- The index set (note the braces in the declaration) for the two-dimensional array **mat** uses range expressions. The expression **1..n** is read as a set of consecutive integers. For example, if **n** equals 3, then the expression is read 1 2 3.
- **mat** is an n-by-n parameter array or matrix of zeros.
- The **init** keyword is one way of initializing parameters, and all except the first parameter, **n**, are initialized.

Note: A value that is assigned using an equal sign *in a declaration statement* cannot be modified later in the program. A value that is assigned using the **init** keyword or using an equal sign in a statement *after* the declaration statement can be modified later in the program.

Assignment Statements

identifier-expression = expression ;

```
num n;
num pi = constant('pi');
num mat {1..n, 1..n} init 0;

n = 1;
mat[1,1] = pi;
n = n - 1;
```

Assignment statements can be used to assign values to decision variables and parameters.

68

Copyright © SAS Institute Inc. All rights reserved.



Note: Parameters that are declared and initialized with an equal sign (=) in the same statement must not be reassigned a value in a subsequent step, or an error is reported. The syntax in the slide shows **n** being initialized with an equal sign **after** it is declared, so this syntax is okay.

NUMBER|NUM

← declares numeric parameters

```
num n;
num pi = constant('pi');
num mat {1..n, 1..n} init 0;
```

STRING|STR

← declares character-value parameters

```
str name;
str day {1..5} = [Mo Tu We Th Fr];
str type {RESOURCES};
```

69

Copyright © SAS Institute Inc. All rights reserved.



The STRING or STR statement is used to declare character-valued parameters.

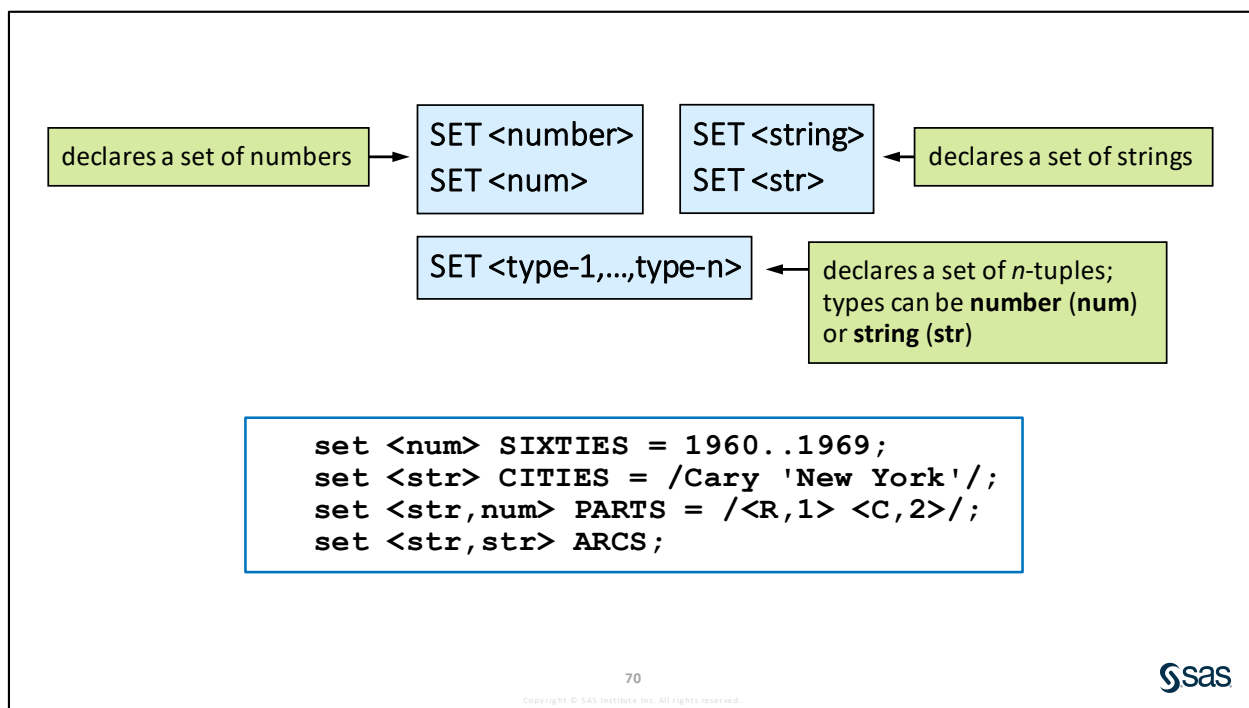
- The parameter array **day** is declared and initialized with the five-character values shown in the square brackets. **day** is indexed by the integer sequence 1 2 3 4 5.

- Note that both **pi** and **day** are assigned values that are unlikely to be changed, except maybe in another language in the case of **day**.
- Quotation marks are not necessary because the strings have no spaces.
- The arrays **name** and **type** are declared but not initialized.

The NUM and STR statements do more than identify whether parameters are numeric or character valued. These statements signal PROC OPTMODEL that a parameter array is being declared. For sets, described next, identifying the type of elements contained in a set can be optional.

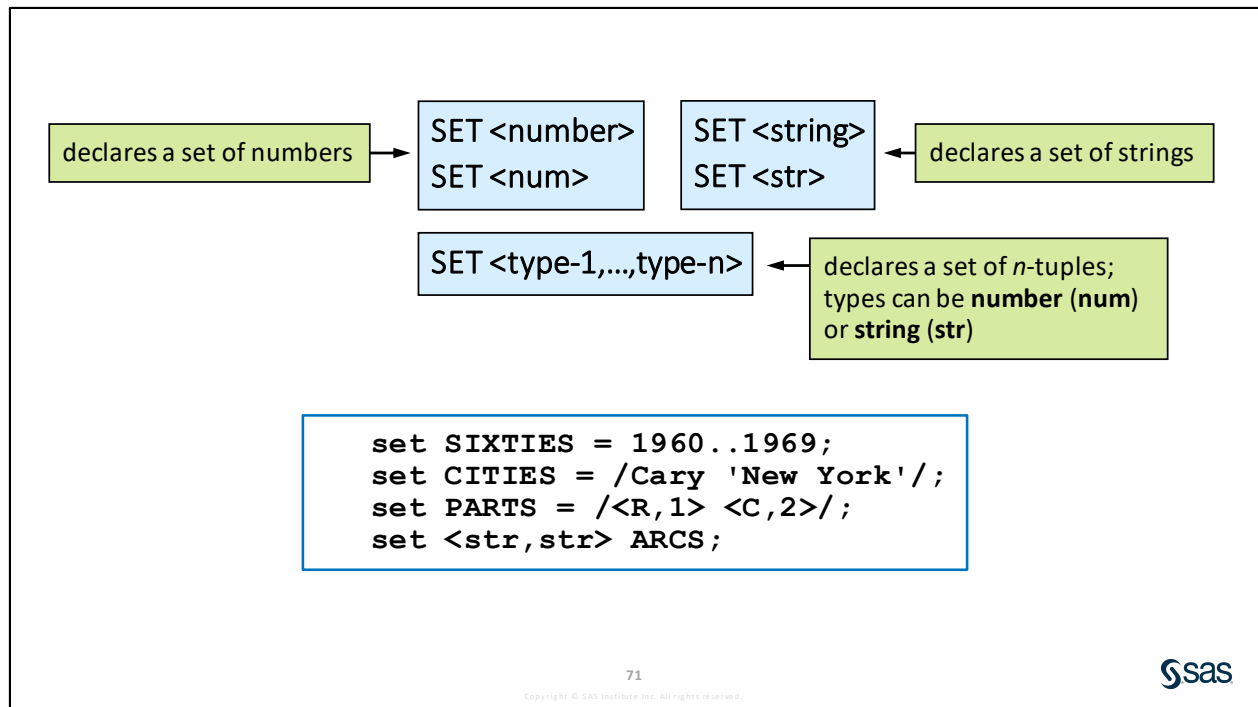
Note: A set declaration begins with **set**, whereas parameter array declaration begins with **num** or **str**. If the type of element contained in the set is obvious from, say, initialization, then we do not need to specify it.

Note: **Braces** are used to enclose a set, and **brackets** are used to denote array locations.



Now consider how to create sets of parameters that are numbers or strings (or both).

- The dots are a range operator. (A BY can be added for a general arithmetic progression.)
- The forward slashes indicate an explicit listing of set elements.
- In some situations, commas and quotation marks are optional. Commas could be included in the set literal expression or omitted from the tuple expressions.
- Angle brackets (or “duck lips”) around **set elements** are necessary only for tuples.



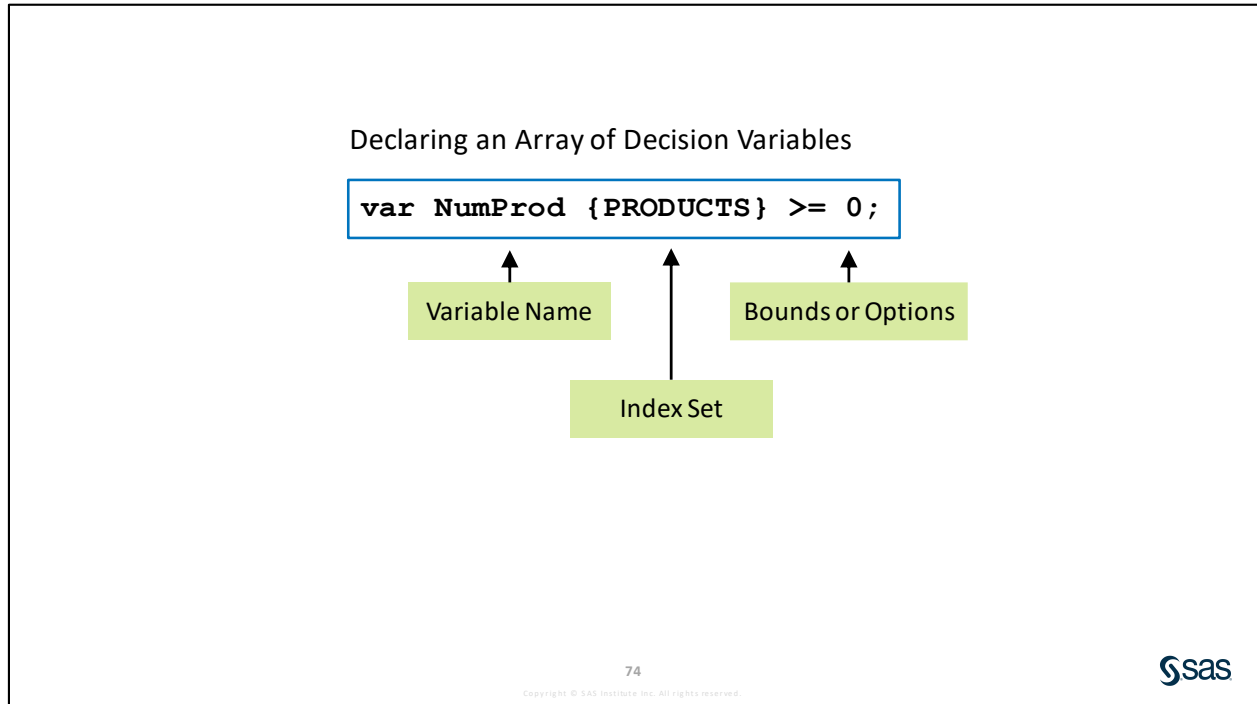
The *type* of a set does not need to be declared if the set is initialized. The default type for sets is **<num>**. Arrays of sets can also be declared in PROC OPTMODEL.

2.05 Multiple Answer Question

Which of the following statements are true of parameters and parameter arrays in PROC OPTMODEL?

- a. Parameters must be declared before they are used.
- b. The type of a set must be declared.
- c. Parameters must be initialized when they are declared.
- d. All parameter values can be modified after they are declared.
- e. Parameter values cannot be modified by the solver.

Decision Variables



The syntax for declaring an array of decision variables follows the same pattern as declaring a parameter array, but there are some important differences. Decision variables have a default initial value of zero. Recall that parameters have no default initial value.

The index set is enclosed in braces. The **Numprod** declaration can be interpreted as “We will choose some nonnegative number of each product to produce.” The nonnegativity is applied to the entire array of decision variables.

In the formulation that uses arrays and index sets, the decision variable **Desks** becomes **NumProd['desks']**.

Note: a decision variable is declared for every element in the set **PRODUCTS**. If a new product is added later to the **PRODUCTS** set, then the corresponding decision variable is declared by this statement. This is convenient for column generation methods.

Declaring Decision Variables

```
VAR var-declaration [ , ... , var-declaration ] ;
```

The syntax for *var-declaration* is

```
name [ { index-set } ] [ var-option(s) ]
```

```
>= expression
<= expression
INIT expression
INTEGER
BINARY
```

75

Copyright © SAS Institute Inc. All rights reserved.



More generally, the VAR statement is used to declare decision variables. Decision variables must be declared before they are used.

Formulation Using Arrays and Index Sets

```
proc optmodel;
  /* declare sets and parameters */
  RESOURCES, PRODUCTS, cost ...
  /* declare variables */
  NumProd {PRODUCTS}
  /* declare constraints */
  /* declare objective */
  solve;
  /* print solution */
quit;
```

76

Copyright © SAS Institute Inc. All rights reserved.



Other declaration statements, including implicit variables, require sums. For example, to calculate total cost for the furniture-making problem, we need to sum over the cost associated with the amount of each resource used. Of course, PROC OPTMODEL has an aggregation operator for this!

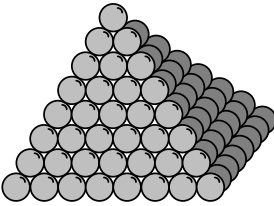
Implicit Variable Details, Summation, and Local Dummy Parameters

SUM Aggregation Operator

```
print ( sum {k in 1..24} k**2 );
```

PROC OPTMODEL Output

4900



77

Copyright © SAS Institute Inc. All rights reserved.



The SUM aggregation operator can be used to add numeric or variable expressions. This operator is illustrated here in the context of the cannonball problem. This problem counts the number of cannonballs that are stacked in a pyramid with a four-sided square base. Above, the base side length is 24.

There are two new features presented in the syntax that solves the cannonball problem: the SUM operator and the **k**. The **k** is a local dummy parameter. The syntax can be read as follows: sum the square of each number, **k**, in the sequence or index of numbers 1 to 24.

Note: In PROC OPTMODEL, parentheses generally indicate that the expression within is to be evaluated.

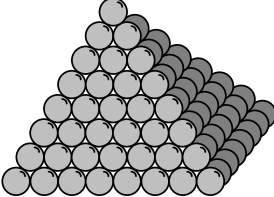
SUM Aggregation Operator

```
print ( sum {k in 1..24} k**2 );
```

PROC OPTMODEL Output

4900

local dummy parameter



78

Copyright © SAS Institute Inc. All rights reserved.

sas

Local dummy parameters can be used “on the fly” in formulations, because they do *not* need to be declared.

SUM Aggregation Operator

```
print ( sum {k in 1..24} k**2 );
```

```
impvar Revenue = sum {p in PRODUCTS}
    selling_price[p]*NumProd[p];
```

Parentheses,
Exponents,
Multiplication and
Division,
Addition and
Subtraction

The SUM aggregation operator goes here in the order of operations.

79

Copyright © SAS Institute Inc. All rights reserved.

sas

If the index set is a set of tuples, the number of local dummy parameters must match the number of terms in the tuple (each local dummy parameter is either a number or a string).

Here are some acronyms for remembering this:

Please Excuse My Dear Aunt Sally

Purple Elephants May Destroy A School

and Pink Elephants Make Dandy Apple Sauce

SUM Aggregation Operator

```
print ( sum {k in 1..24} k**2 );
impvar Revenue = sum {p in PRODUCTS}
    selling_price[p]*NumProd[p];
```

local dummy parameters

80

Copyright © SAS Institute Inc. All rights reserved.



Now consider another example of the SUM operator. In these expressions, **k** and **p** are local dummy parameters.

Note: The expression for **Revenue** is simply the dot product of the two vectors (arrays) **selling_price** and **NumProd**, but PROC OPTMODEL uses the SUM operator because it is not a matrix-based language. The declaration of the implicit variable **Revenue** can be interpreted as follows: for each product, **p**, multiply the selling price and the number produced for each product. Then add these values up across products. The implicit variable **Revenue** will have one value or element that represents the total revenue of all products produced.

2.06 Multiple Choice Question

Which of the following does not need to be declared in PROC OPTMODEL?

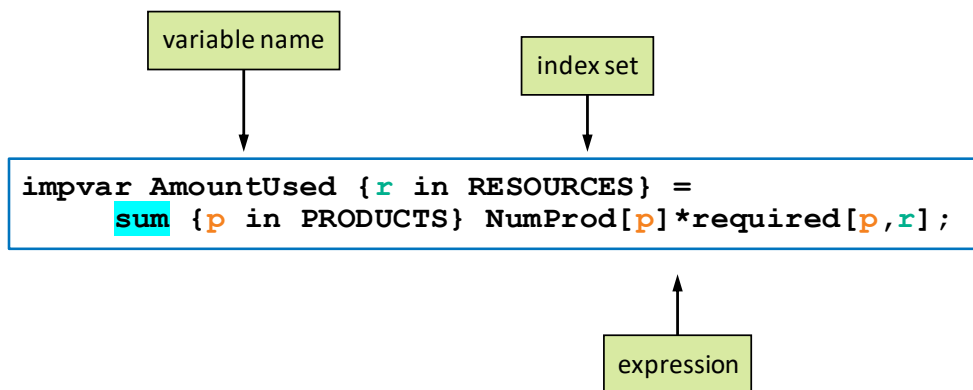
- a. parameter arrays
- b. decision variables
- c. implicit variables
- d. local dummy parameters

81

Copyright © SAS Institute Inc. All rights reserved.



An array of implicit variables can also be declared.

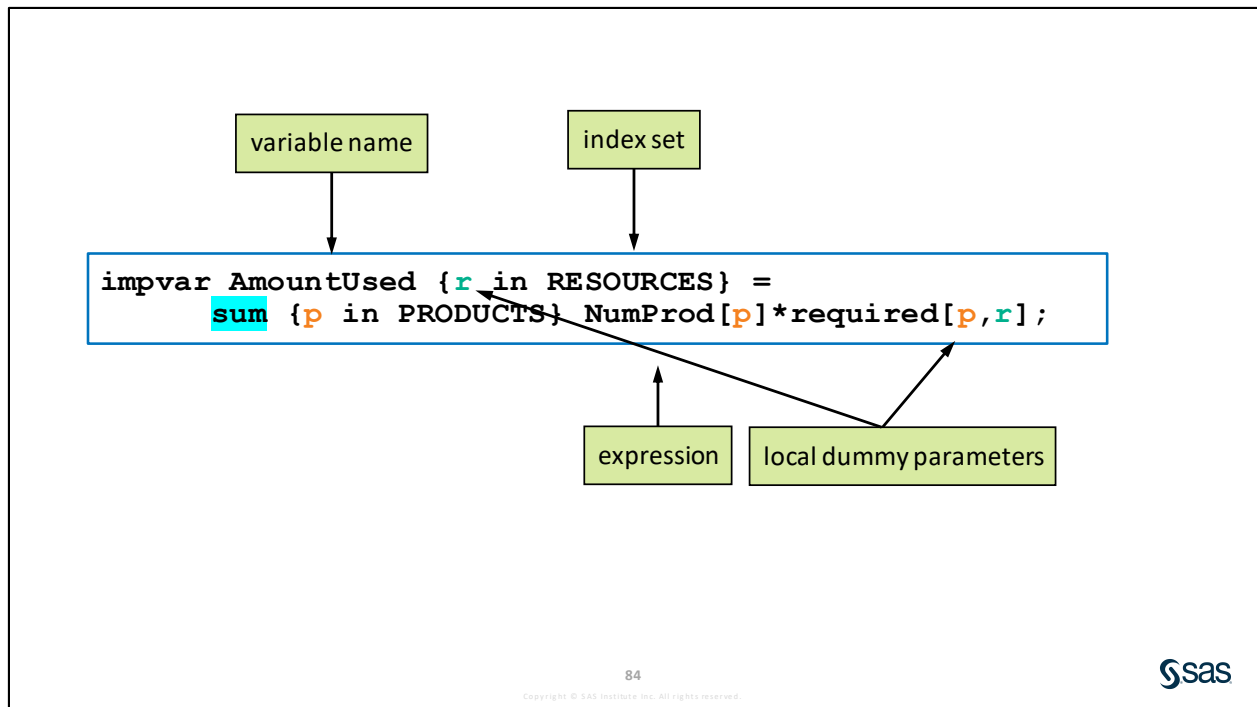


83

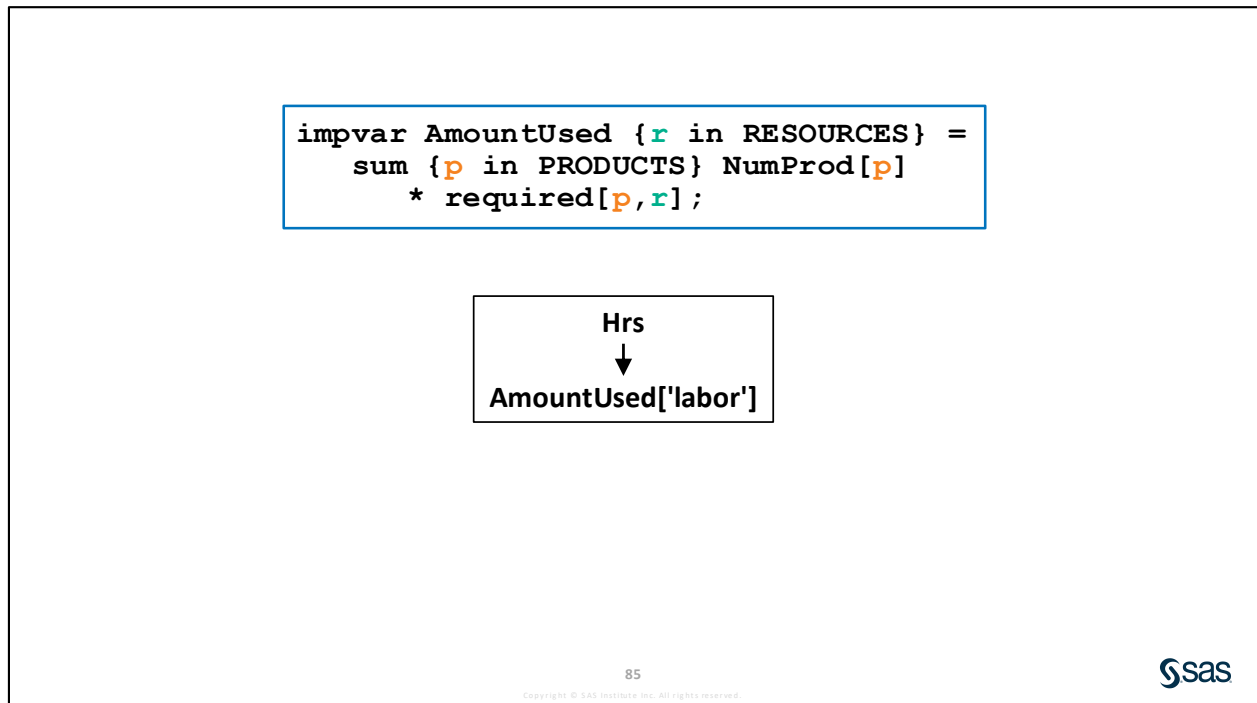
Copyright © SAS Institute Inc. All rights reserved.



In this example, the SUM operator is used in declaring an array of Implicit variables. There is one element in the array for each resource—or, more precisely for the amount of each resource used.



The syntax can be read as follows: For each resource, **r**, add up the amount of that resource used in production across all products. The amount of a resource, **r**, used in the production of an individual product is derived by multiplying the number of the product, **p**, produced by the requirement of resource **r** for that product.



In the previous demonstration, the implicit variable **Hrs** was declared to be the amount of labor required. In the formulation using arrays and index sets, the implicit variable **Hrs** becomes **AmountUsed['labor']**.

```
impvar AmountUsed {r in RESOURCES} =
    sum {p in PRODUCTS} NumProd[p]*required[p,r];
```

```
AmountUsed['labor'] =
    sum {p in PRODUCTS} NumProd[p]*required[p,'labor']

AmountUsed['metal'] =
    sum {p in PRODUCTS} NumProd[p]*required[p,'metal']

AmountUsed['wood'] =
    sum {p in PRODUCTS} NumProd[p]*required[p,'wood']
```

RESOURCES

86

Copyright © SAS Institute Inc. All rights reserved.



In parsing the syntax, it can help to think of unpacking the array of implicit variables into its constituent parts. An implicit variable is declared for every element of the set **RESOURCES**. If an element is later added to the set **RESOURCES**, the corresponding implicit variable is declared by this statement.

```
impvar AmountUsed {r in RESOURCES} =
    sum {p in PRODUCTS} NumProd[p]*required[p,r];
```

```
IMPVAR name [ { index-set } ] = expression ;
```

87

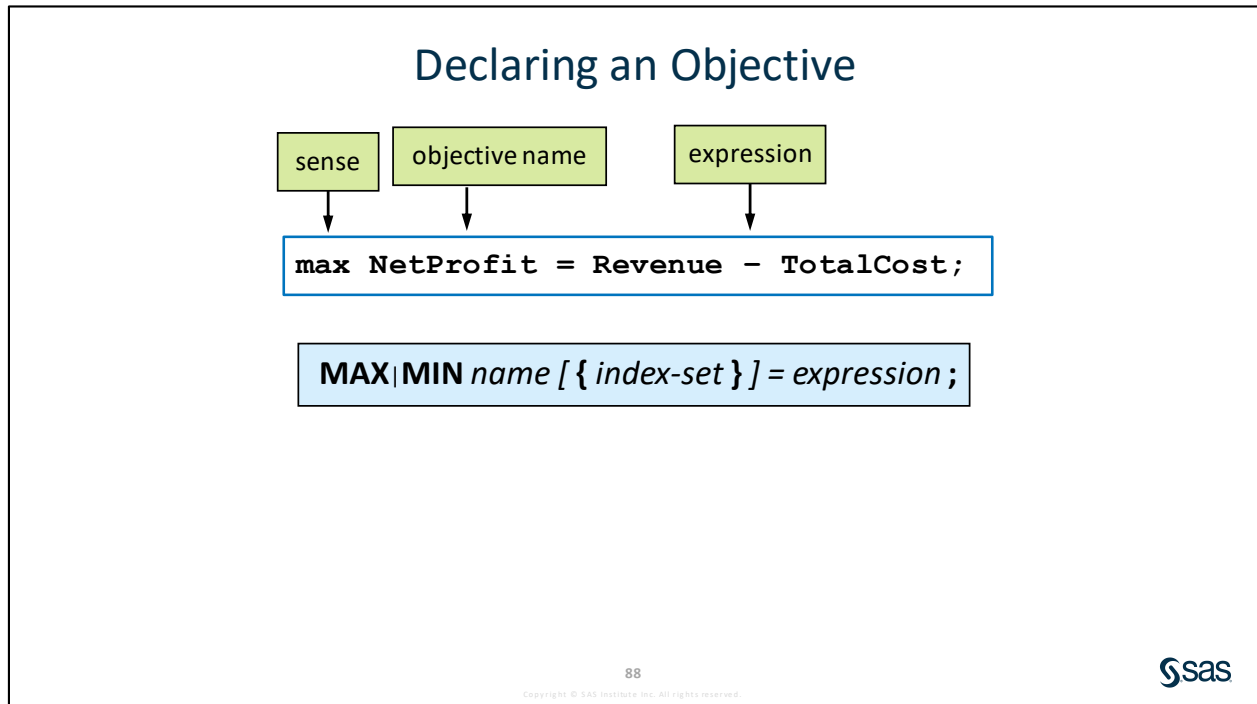
Copyright © SAS Institute Inc. All rights reserved.



The expression can involve decision variables and other (previously declared) implicit variables. The expression for an array of implicit variables can be defined recursively, as long as the recursion is not circular. Implicit variables must be declared before they are used.

Declaring the Objective Function

The implicit variables declared for the furniture-making problem are used to help declare the objective and constraints.



The objective function for the furniture-making problem can be declared as the difference between the implicit variables **Revenue** and **TotalCost**. Note that the sense of the problem is either maximize (max) or minimize (min).

Note: An array of objectives can be declared, and multiple objective functions are permitted. The most recent objective is used unless an objective is specified using the OBJECTIVE (abbreviated OBJ) option. Multiple objectives can be useful in formulations that require conditional processing, for example.

Note: Without implicit variables, the variable expression in this objective would be more complicated. It would involve summations, parameters, and decision variables.

```

max NetProfit = sum {p in PRODUCTS} ( selling_price[p] -
    sum {r in RESOURCES} required[p,r] * cost[r] ) * NumProd[p];

```

The linear programming problem solved is the same with or without implicit variables.

```
graph TD; A["MAX|MIN name [{ index-set } ] = expression ;"] --> B["SOLVE OBJECTIVE|OBJ name ;"]
```

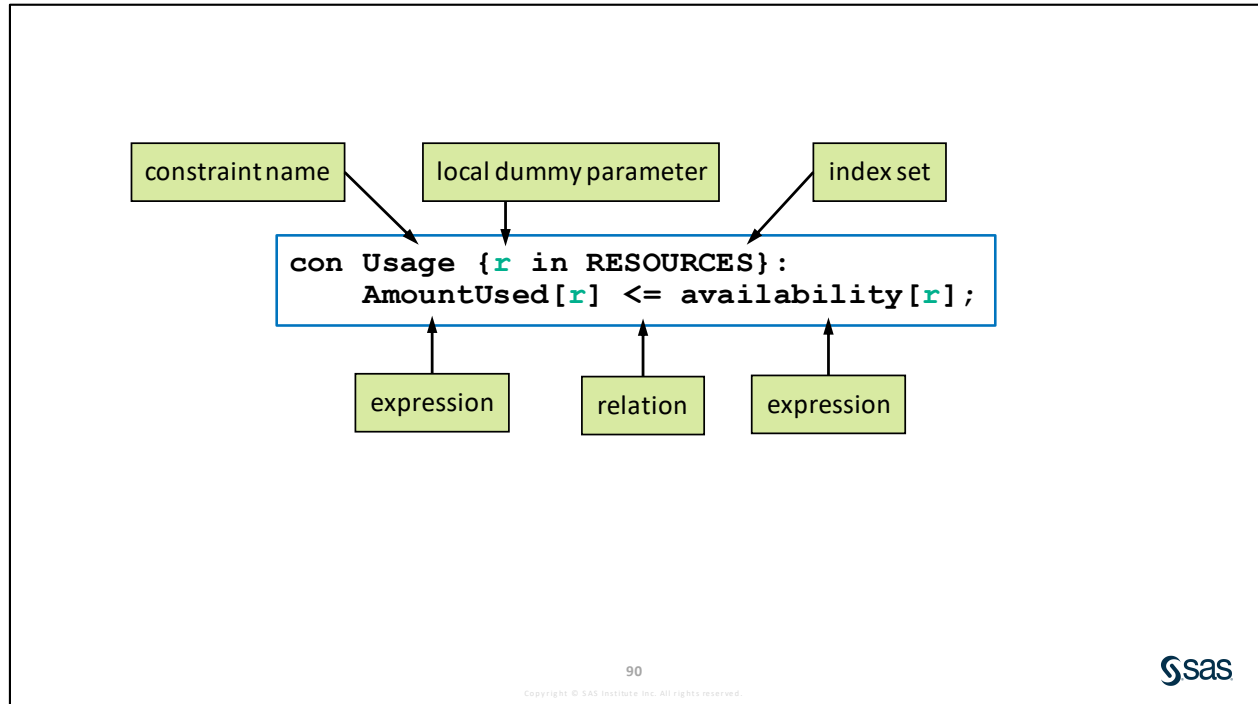
MAX|MIN *name* [{ *index-set* }] = *expression* ;

SOLVE OBJECTIVE|OBJ *name* ;

More generally, you can reference objective names in the same way as implicit variables. Objective functions **must** be named and must be declared before they are used.

Note: Multiple objective functions are allowed. The most recent objective is used unless an OBJECTIVE (OBJ) option is specified in the SOLVE statement. Arrays of objectives can arise in “What-if?” analysis.

Declaring Constraints



Constraints can be declared similarly to implicit variables. This syntax declares a family of three constraints using the CON statement. It can be interpreted as follows: For each **RESOURCE**, *r*, the **AmountUsed** of that resource must be less than or equal to the amount of that resource available.

In the formulation using arrays and index sets, the constraint **Labor** becomes **Usage['labor']**.



It can be helpful to think of “unpacking” the family of constraints. The three unpacked lines illustrate how each element of the Usage constraint family is created.

Note: A constraint is declared for every element of the set **RESOURCES**. If an element is later added to the set **RESOURCES**, then the corresponding constraint is declared by this statement. This is a convenient way to implement methods that involve constraint generation.

```
con Usage {r in RESOURCES}:
    AmountUsed[r] <= availability[r];
```

```
CONSTRAINT constraint [, ... , constraint ] ;
```

allowed forms

```
[ name [ { index-set } ] : ] expression = expression
[ name [ { index-set } ] : ] expression relation expression
[ name [ { index-set } ] : ] bound relation body relation bound
```

CONSTRAINT can be abbreviated as CON, and constraints must be declared before they are used. The allowed forms of *constraint* are shown above. For range constraints, *relation* must be either <= or >= and the same in both positions. If no name is provided, a default name of the form **_ACON_*n*** is assigned.

Note: PROC OPTMODEL automatically declares symbols that are aliases for model components. All decision variables and constraints in a model can be referred to anonymously, as **_VAR_*n*** for *n* in {1..**NVAR**} and **_CON_*n*** for *n* in {1..**NCON**}. The current objective can also be referred to as **_OBJ_**.



Using Arrays and Index Sets in PROC OPTMODEL to Solve the Furniture-Making Problem

orvy02d04.sas

```
proc optmodel;
  /* declare sets and parameters */
  set RESOURCES = /labor metal wood/;
  set PRODUCTS = /desks chairs bookcases bedframes/;
  num selling_price {PRODUCTS} = [94 79 125 109];
  num cost {RESOURCES} = [14 20 11];
  num availability {RESOURCES} = [225 117 420];
  num required {PRODUCTS, RESOURCES} =
    [2 1 3 1 1 3 3 1 4 2 1 4];

  /* declare variables */
  var NumProd {PRODUCTS} >= 0;

  impvar Revenue = sum {p in PRODUCTS}
    selling_price[p] * NumProd[p];

  impvar AmountUsed {r in RESOURCES} =
    sum {p in PRODUCTS} NumProd[p] * required[p,r];

  impvar TotalCost = sum {r in RESOURCES}
    cost[r] * AmountUsed[r];

  /* declare constraints */
  con Usage {r in RESOURCES}:
    AmountUsed[r] <= availability[r];

  /* declare objective */
  max NetProfit = Revenue - TotalCost;

  expand / var impvar;
  expand NetProfit;
  expand Usage;

  solve;

  print NumProd;
  print AmountUsed;

quit;
```

Notice the index set in the constraint declaration and its absence in the last EXPAND statement. The local dummy parameter in the constraint declaration statement is necessary to make each constraint in the family refer to a specific index (in this case, a specific resource).

Note: In the output from the EXPAND statements, the indices are not enclosed in quotation marks. Inside the PROC OPTMODEL program, the quotation marks are necessary to distinguish between a name expression (no quotation marks) and a character string (quotation marks).

PROC OPTMODEL Output

The OPTMODEL Procedure

```

Var NumProd[desks] >= 0
Var NumProd[chairs] >= 0
Var NumProd[bookcases] >= 0
Var NumProd[bedframes] >= 0
Impvar Revenue = 94*NumProd[desks] + 79*NumProd[chairs] + 125*NumProd[bookcases] + 109*NumProd[bedframes]
Impvar AmountUsed[labor] = 2*NumProd[desks] + NumProd[chairs] + 3*NumProd[bookcases] + 2*NumProd[bedframes]
Impvar AmountUsed[metal] = NumProd[desks] + NumProd[chairs] + NumProd[bookcases] + NumProd[bedframes]
Impvar AmountUsed[wood] = 3*NumProd[desks] + 3*NumProd[chairs] + 4*NumProd[bookcases] + 4*NumProd[bedframes]
Impvar TotalCost = 14*AmountUsed[labor] + 20*AmountUsed[metal] + 11*AmountUsed[wood]

```

```
Maximize NetProfit=- TotalCost + Revenue
```

```

Constraint Usage[labor]: AmountUsed[labor] <= 225
Constraint Usage[metal]: AmountUsed[metal] <= 117
Constraint Usage[wood]: AmountUsed[wood] <= 420

```

Problem Summary	
Objective Sense	Maximization
Objective Function	NetProfit
Objective Type	Linear
Number of Variables	4
Bounded Above	0
Bounded Below	4
Bounded Below and Above	0
Free	0
Fixed	0
Number of Constraints	3
Linear LE (<=)	3
Linear EQ (=)	0
Linear GE (>=)	0
Linear Range	0
Constraint Coefficients	12

Solution Summary	
Solver	LP
Algorithm	Dual Simplex
Objective Function	NetProfit
Solution Status	Optimal
Objective Value	1827
Primal Infeasibility	5.684342E-14
Dual Infeasibility	1.776357E-15
Bound Infeasibility	0
Iterations	6
Presolve Time	0.00
Solution Time	0.00

[1]	NumProd
bedframes	30
bookcases	39
chairs	48
desks	0

[1]	AmountUsed
labor	225
metal	117
wood	420

Compare this to the output from the named variables formulation in the demonstration above.

PROC OPTMODEL Output (omitting the problem and solution summaries)

The OPTMODEL Procedure

```

Var Desks >= 0
Var Chairs >= 0
Var Bookcases >= 0
Var Bedframes >= 0
Impvar Hrs = 2*Desks + Chairs + 3*Bookcases + 2*Bedframes
Impvar Lbs = Desks + Chairs + Bookcases + Bedframes
Impvar Ft3 = 3*Desks + 3*Chairs + 4*Bookcases + 4*Bedframes
Maximize NetProfit=94*Desks + 79*Chairs + 125*Bookcases + 109*Bedframes - 14*Hrs - 20*Lbs - 11*Ft3
Constraint Labor: Hrs <= 225
Constraint Metal: Lbs <= 117
Constraint Wood: Ft3 <= 420

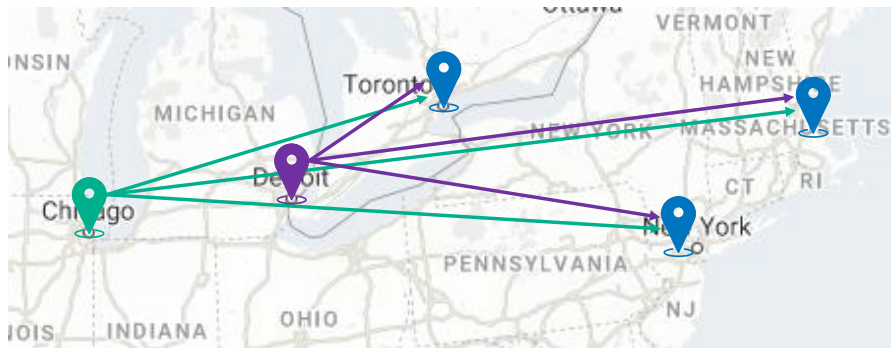
```

Desks	Chairs	Bookcases	Bedframes
0	48	39	30

Hrs	Lbs	Ft3
225	117	420

End of Demonstration

The TRANSPORTATION Problem



Determine the amount to transport from each origin to each destination to meet demand at minimum cost.

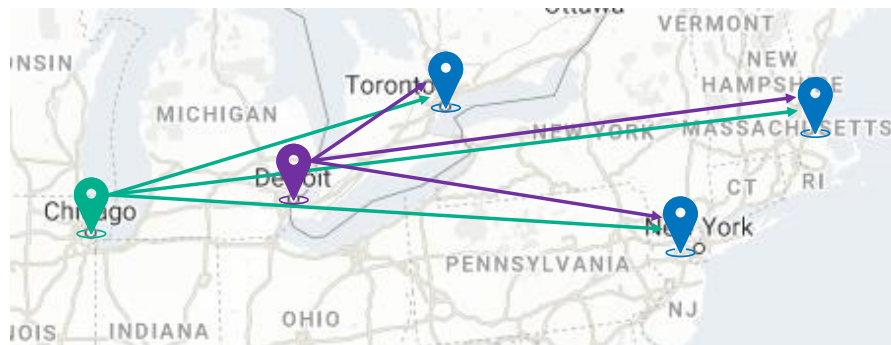
96

Copyright © SAS Institute Inc. All rights reserved.



Now a different optimization problem is considered: the transportation problem. Given supply values at a set of origins, demand values at a set of destinations, and unit shipping costs, the problem to solve is how much to ship from each origin to each destination to meet demand at a minimum cost.

Note: The information given here prepares students to work through the next practice.



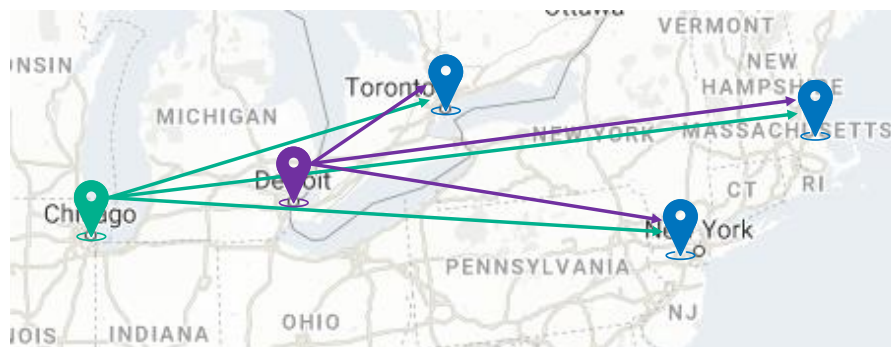
	Boston	Newark	Toronto	Supply
Chicago	40	25	15	200
Detroit	30	15	10	150
Demand	50	150	100	

97

Copyright © SAS Institute Inc. All rights reserved.



Per-unit shipping costs between each origin/destination pair, the amount each origin can supply, and the demands for each destination are given in the table. Note that total supply is greater than total demand. Here, an approach using index sets and arrays is more natural than one using named variables.



	Boston	Newark	Toronto	Supply
Chicago	40	25	15	200
Detroit	30	15	10	150
Demand	50	150	100	

origins

destinations

Copyright © SAS Institute Inc. All rights reserved.



Chicago and Detroit are our origins, or where we ship **from**. Boston, Newark, and Toronto are our destinations, or where we ship **to**. Notice that everything in the problem can be described in relation to origins and destinations.

```
set ORIGINS = /Chicago Detroit/;  
set DESTINATIONS = /Boston Newark Toronto/;
```

	Boston	Newark	Toronto	Supply	
Chicago	40	25	15	200	origins
Detroit	30	15	10	150	
Demand	50	150	100		
destinations					



Index sets are declared first, using set literal expressions to initialize them.


```

set ORIGINS = /Chicago Detroit/;
set DESTINATIONS = /Boston Newark Toronto/;
num supply {ORIGINS} = [200 150];
num demand {DESTINATIONS} = [50 150 100];
num unit_cost {ORIGINS, DESTINATIONS} =
    [40 25 15 30 15 10];

```

	Boston	Newark	Toronto	Supply
Chicago	40	25	15	200
Detroit	30	15	10	150
Demand	50	150	100	

origins

destinations



The parameter arrays needed to formulate the problem are declared.

The index set and arrays approach to formulating problems in PROC OPTMODEL has been described, and the furniture-making problem will be formulated using this approach in the next demonstration. However, there are a few more tools that need to be illustrated before the arrays and index set approach can be implemented effectively. The following slides describe how to create additional, necessary problem elements like decision variables, constraints, and the objective function using the index set and arrays approach.



Practice

4. Formulating a Transportation Problem

Consider the transportation problem with unit shipping costs, supply, and demand in the table below.

		DESTINATIONS		
		Boston	Newark	Toronto
ORIGINS	Chicago	40	25	15
	Detroit	30	15	10
	Demand	50	150	100
		Supply		
		200	150	

- Complete the parameter declaration statements in part **4a** of the program **orvy02e01.sas**. Replace each *INDEX-SET* with the appropriate (declared) index set and *INITIALIZERS* with the appropriate data items from the table. This assigns values to a two-dimensional array **unit_cost** of unit costs, an array **supply** of supplies, and an array **demand** of demands. How does PROC OPTMODEL format the output of the PRINT statement in the SAS program?
- Complete the variable declaration statements in part **4b** of the program **orvy02e01.sas**. Replace each *INDEX-SET* with the appropriate (declared) index set and *EXPRESSION* with the appropriate variable expression. (The decision variables are initialized to 50 in order to print values for the implicit variables.)
- Complete the constraint and objective declaration statements in part **4c** of the program **orvy02e01.sas** and solve the transportation problem using PROC OPTMODEL. Use the EXPAND statement to check your formulation.

Hint: The optimal objective value is \$5,750.

End of Practices

2.4 Dual Values and Reduced Costs in the Simplex Method (Self-Study)

Objectives

- Explain the interpretation of dual values in linear programming.
- Describe how dual values are used in the primal simplex algorithm and how pricing options influence the behavior of the simplex algorithms.

$$\text{Dual Value} = \frac{\text{Change in optimal objective}}{\text{Unit increase in constraint limit}}$$

103

Copyright © SAS Institute Inc. All rights reserved.

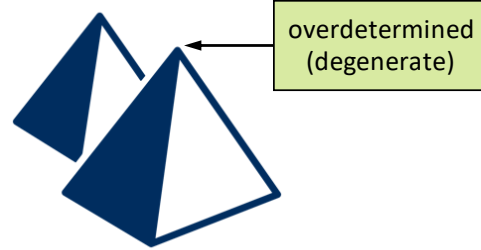


Interpretation of Dual Values: The *dual value* of a constraint is defined as change in optimal objective over unit increase in constraint limit.

Note: Dual values are analogous to Lagrange multipliers in nonlinear programming problems, as described in the next lesson.

$$\text{Dual Value} = \frac{\text{Change in optimal objective}}{\text{Unit increase in constraint limit}}$$

Assumes that the extreme point determining the optimal solution is not overdetermined.



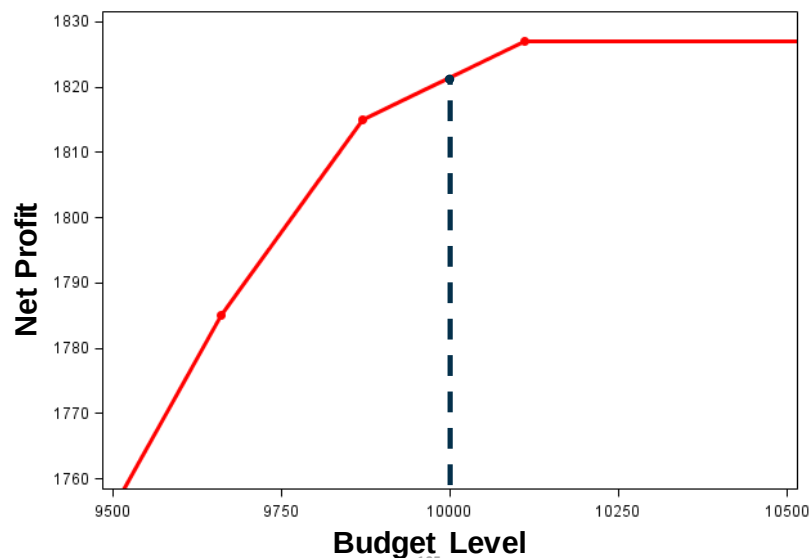
104

Copyright © SAS Institute Inc. All rights reserved.

sas

At the apex of the pyramid, four planes intersect in a single point (one more than is necessary). In three dimensions, three planes determine a point. At the apex of the pyramid, four planes meet. If one of them is moved out, the point determined will not change. Degenerate, in this and other contexts, is considered to be a bad thing.

Net Profit as a Function of Budget Level



105

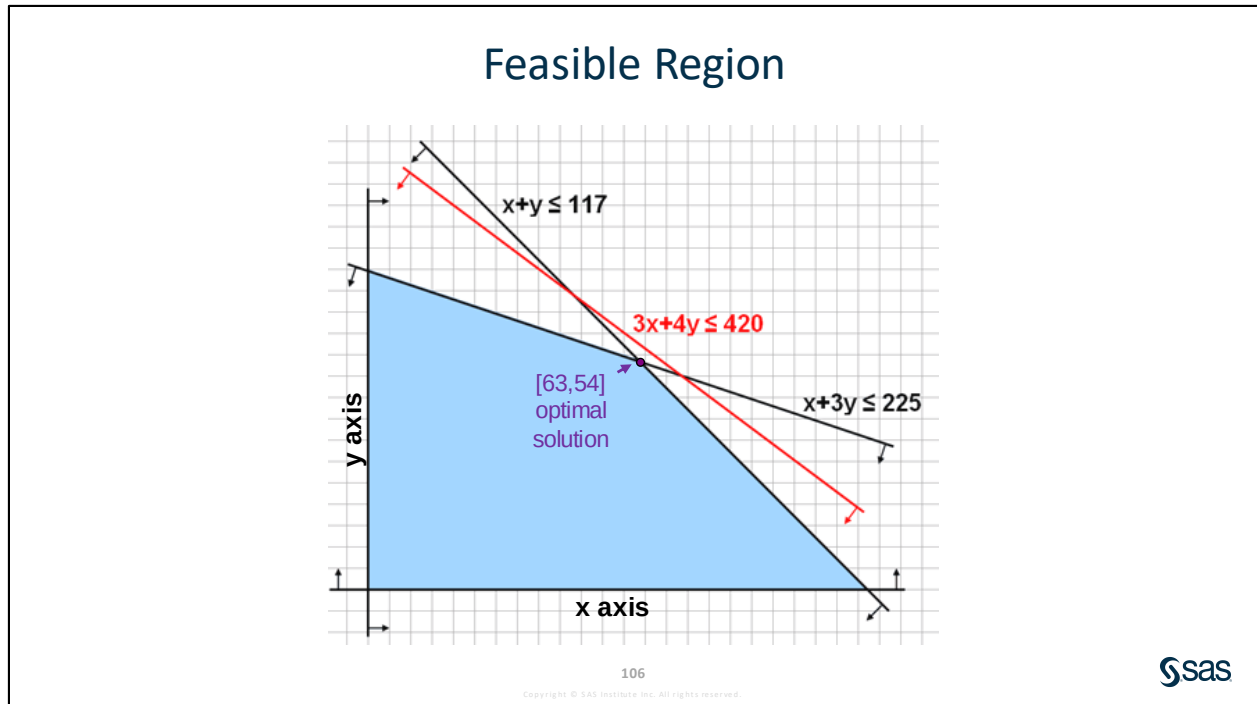
Copyright © SAS Institute Inc. All rights reserved.

sas

The dual value of the budget constraint gives the slope of the graph at the current value of 10,000.

Note: Finding the interval over which the current set of basic variables remains optimal is called *ranging*. Finding the graph of objective value as a function of a right hand-side value is parametric programming. Finding these in PROC OPTMODEL is a good programming exercise.

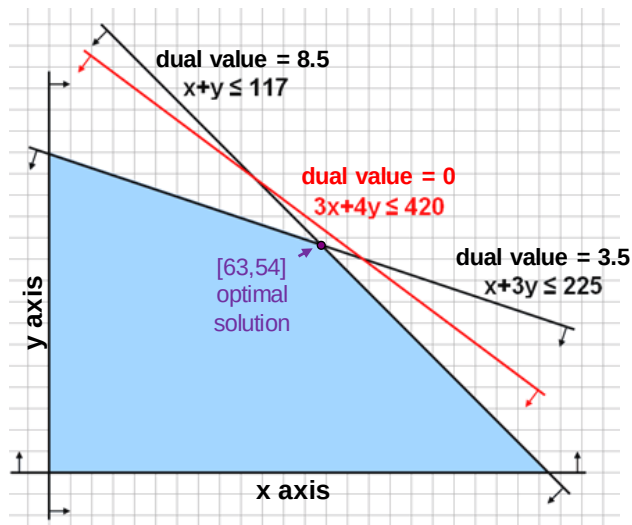
Recall the geometry of the two-dimensional problem, described earlier.



The red constraint is redundant and does not contribute to the definition of the feasible region. It will not be tight at the optimal solution.

The dual values for the two-dimensional linear programming problem are pictured below.

Feasible Region



107

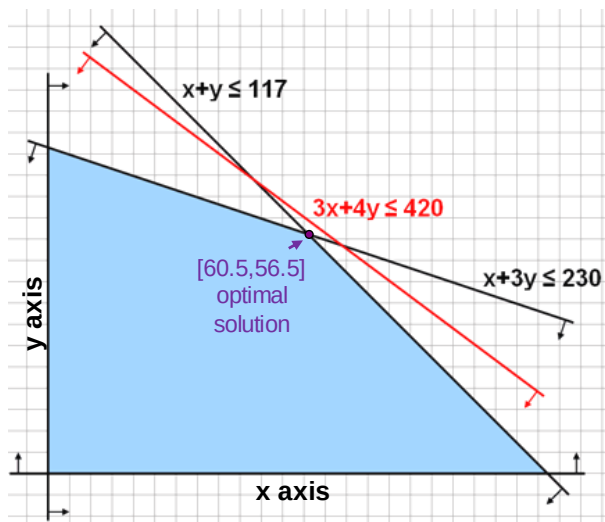
Copyright © SAS Institute Inc. All rights reserved.



The constraint that is not tight (satisfied with equality) at the optimal solution has a dual value of zero.

How does the dual value of the constraint $x + 3y \leq 225$ predict the change in the optimal objective value if the limit of the constraint is increased to 230? The predicted change is $\$3.5 \times 5 = \16.5 .

Feasible Region

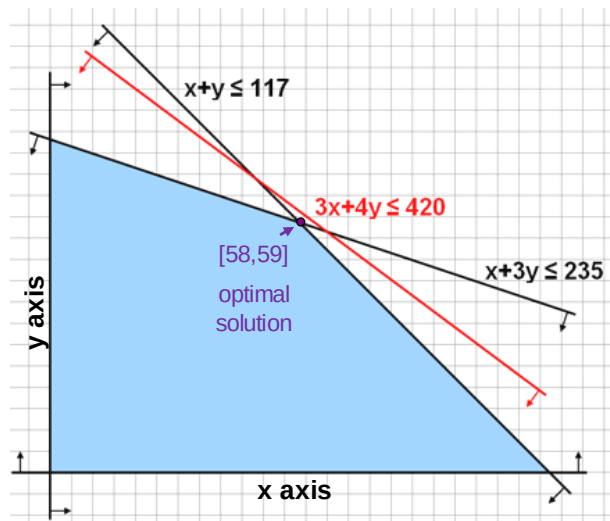


108

Copyright © SAS Institute Inc. All rights reserved.



Feasible Region



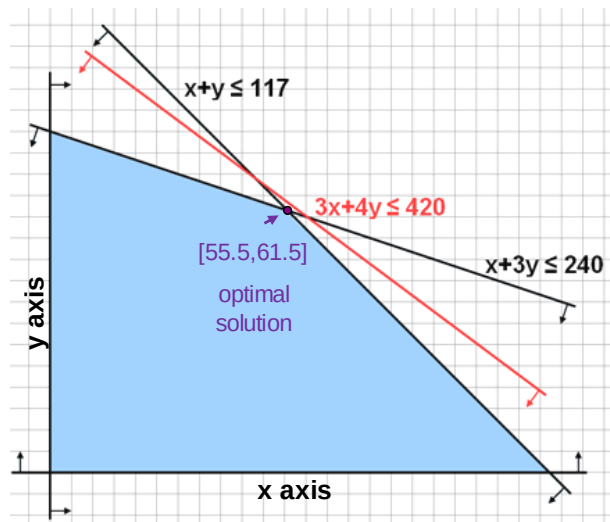
109

Copyright © SAS Institute Inc. All rights reserved.



Here, the predicted change is $\$3.5 \times 10 = \35 .

Feasible Region



110

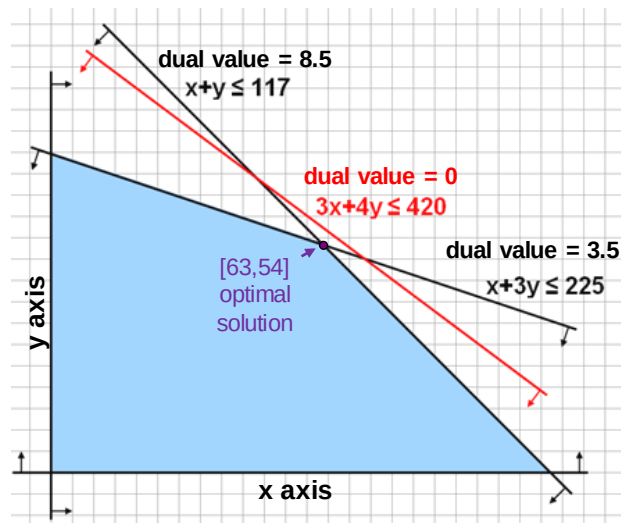
Copyright © SAS Institute Inc. All rights reserved.



The predicted change is $\$3.5 \times 15 = \51.5 .

Next, consider increasing the limit of the other constraint from 117 to 122.

Feasible Region

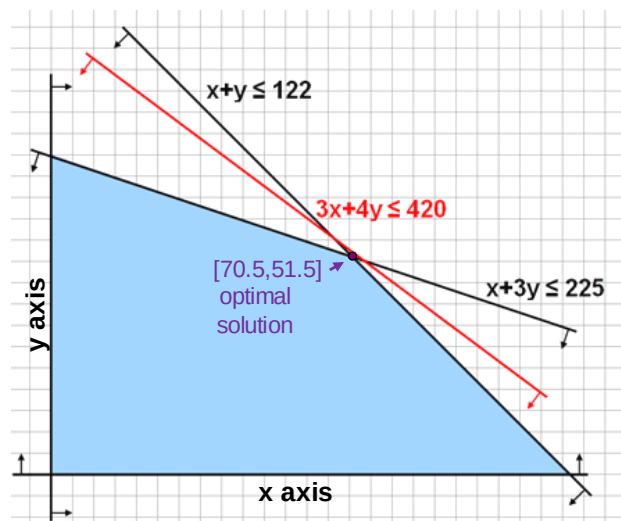


111

Copyright © SAS Institute Inc. All rights reserved.



Feasible Region

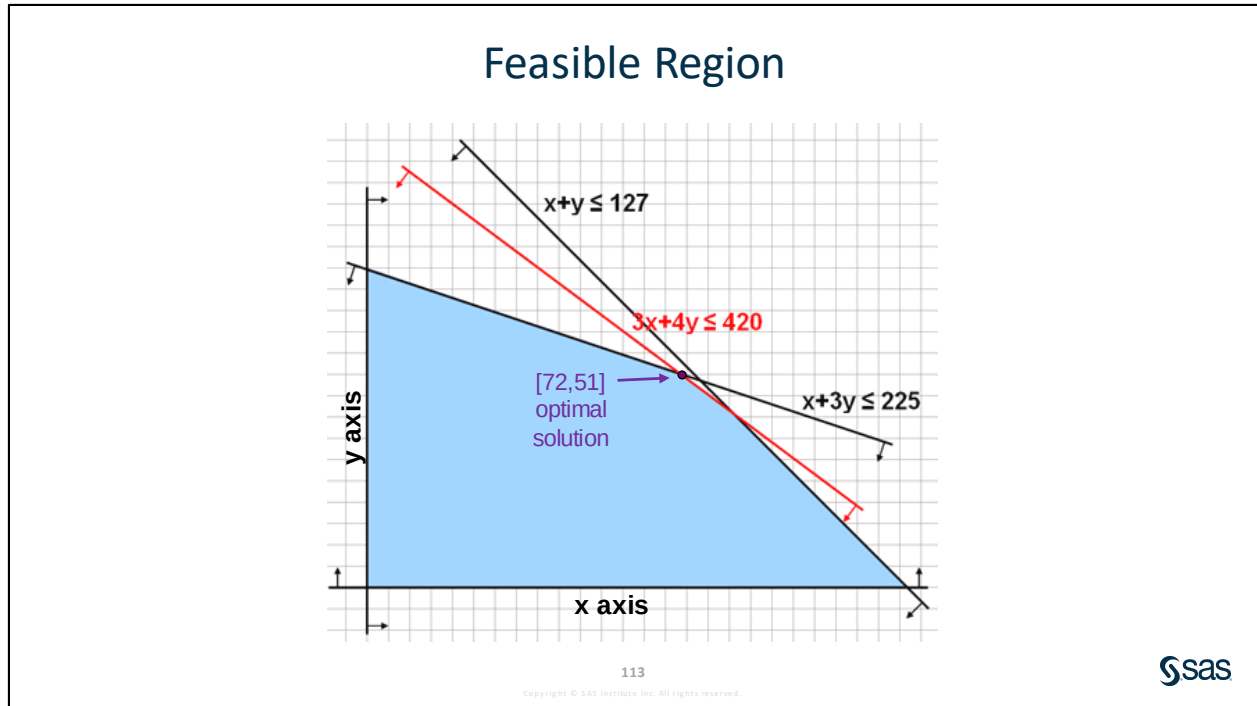


112

Copyright © SAS Institute Inc. All rights reserved.



The predicted change is $\$8.5 \times 5 = \42.5 .



The predicted change is $\$8.5 \times 10 = \85 . However, the actual change is only \$51.

Note: The predicted change in the objective value is accurate only in a neighborhood of the optimal solution. However, it does **bound** the change in the objective value:

$$12(72) + 19(51) = 1833 \leq 1782 + 8.5(10) = 1867$$

The latter quantity is the objective value of the point $[x, y] = [78, 49]$, which is the intersection of the lines $x + y = 127$ and $x + 3y = 225$.

```
print Usage.dual;
print Budget.dual;
```

114

Copyright © SAS Institute Inc. All rights reserved.



Consider dual values for the furniture-making problem. Dual values can be listed using the PRINT statement.

```
print Usage.dual;
print Budget.dual;
```

PROC OPTMODEL Output

	Usage. DUAL
[1]	
labor	1.30
metal	0.00
wood	2.45
	Budget. DUAL
	0.05

115

Copyright © SAS Institute Inc. All rights reserved.



Interpreting dual values can be tricky. Here, increasing the limit of the resource usage constraints increases only the **availability** of the resources at their current cost.



If additional overtime hours are available for \$21 (time-and-a-half), would they be used?

```
print Usage.dual;
print Budget.dual;
```

PROC OPTMODEL Output

	Usage. DUAL
[1] labor	1.30
metal	0.00
wood	2.45

	Budget. DUAL
	0.05

116

Copyright © SAS Institute Inc. All rights reserved.



Let's consider whether overtime hours will be used.



If additional overtime hours are available for \$21 (time-and-a-half), would they be used?

```
print Usage.dual;
print Budget.dual;
```

PROC OPTMODEL Output

	Usage. DUAL
[1] labor	1.30
metal	0.00
wood	2.45

	Budget. DUAL
	0.05

117

Copyright © SAS Institute Inc. All rights reserved.



No. For an additional hour at \$21 = \$14 + \$7, the optimal objective value changes by (at most) \$1.30 – \$7 = –\$5.70.



A more correct analysis would also account for the effect of the incremental cost of overtime on the budget constraint.

If the incremental cost is Δ , then overtime hours are profitable only if $\Delta + 0.05\Delta < 1.30$, or $\Delta < 1.30/1.05 = 1.2381$. This is an example of pricing an activity, the very next topic.

Tables



3 hours labor
1lb metal
2cu ft wood
Sell for **\$98**

Net profit for a table

$$\begin{aligned} & \$98 - 3(\$14) - 1(\$20) - 2(\$11) \\ &= \$98 - \$84 \\ &= \mathbf{\$14} \end{aligned}$$

?

Should any be produced?

118


Copyright © SAS Institute Inc. All rights reserved.

Pricing an Activity: Let's consider whether tables will be produced.

Tables



3 hours labor
1lb metal
2cu ft wood
Sell for **\$98**

Net profit for a table

$$\begin{aligned} & \$98 - 3(\$14) - 1(\$20) - 2(\$11) \\ &= \$98 - \$84 \\ &= \mathbf{\$14} \end{aligned}$$

Cost of reduced availability

$$\begin{aligned} & 3(\$1.30) + 1(\$0) + 2(\$2.45) + 84(\$0.05) \\ &= \mathbf{\$13} \end{aligned}$$

?

Should any be produced?

119


Copyright © SAS Institute Inc. All rights reserved.

Tables



3 hours labor
 1lb metal
 2cu ft wood
 Sell for **\$98**

?

Should any be produced?

Net profit for a table

$$\begin{aligned} & \$98 - 3(\$14) - 1(\$20) - 2(\$11) \\ & = \$98 - \$84 \\ & = \mathbf{\$14} \end{aligned}$$

Cost of reduced availability

$$\begin{aligned} & 3(\$1.30) + 1(\$0) + 2(\$2.45) + 84(\$0.05) \\ & = \mathbf{\$13} \end{aligned}$$

Reduced profit

$$\begin{aligned} & \$14 - \$13 \\ & = \mathbf{\$1} \end{aligned}$$

120

Copyright © SAS Institute Inc. All rights reserved.



The numbers in parentheses that are used to compute the cost of reduced availability are the dual values for the availability of labor, metal, and wood, and the budget constraint. For labor, the dual value \$1.30 is multiplied by 3 because each table requires three hours of labor.

Making tables appears to be a profitable activity. However, the dual values themselves cannot inform the decision about the most profitable mix of tables and other products.

Due to a series of unfortunate events, the canonical form of a linear programming problem **minimizes** the objective function, so the computed quantity is also referred to as the *reduced cost* of making tables. Here, it is **profit** that is reduced.

Note: For a variable in the model that is at its upper or lower bound, the reduced profit or cost is the amount that the variable's objective function coefficient must **decrease** before the variable can change from its current value. For example, in the furniture-making problem, the reduced profit of **NumProd['desks']** is -1 , so the selling price of desks must increase by \$1 (decrease by $-\$1$) before any desks are produced.



Dual Values and Reduced Profits in the Furniture-Making Problem (Self-Study)

orvy02d05.sas

```
%let budget_limit = 10000;

proc optmodel;
  /* declare sets and parameters */
  set RESOURCES = /labor metal wood/;
  set PRODUCTS = /desks chairs bookcases bedframes/;
  num selling_price {PRODUCTS} = [94 79 125 109];
  num cost {RESOURCES} = [14 20 11];
  num availability {RESOURCES} = [225 117 420];
  num required {PRODUCTS, RESOURCES} =
    [2 1 3 1 1 3 3 1 4 2 1 4];

  /* declare variables */
  var NumProd {PRODUCTS} >= 0;

  impvar Revenue = sum {p in PRODUCTS}
    selling_price[p] * NumProd[p];

  impvar AmountUsed {r in RESOURCES} =
    sum {p in PRODUCTS} NumProd[p] * required[p,r];

  impvar TotalCost = sum {r in RESOURCES}
    cost[r] * AmountUsed[r];

  /* declare constraints */
  con Usage {r in RESOURCES}:
    AmountUsed[r] <= availability[r];
  /* Note, Budget constraint from the exercise is added */
  con Budget: TotalCost <= &budget_limit;

  /* declare objective */
  max NetProfit = Revenue - TotalCost;

  solve;

  print NumProd NumProd.dual;
  print AmountUsed availability Usage.dual;
  print TotalCost Budget.ub Budget.dual;
  print _VAR_.name _VAR_.dual;
  print _CON_.name _CON_.body _CON_.ub _CON_.dual;
quit;
```

PROC OPTMODEL Output

Solution Summary	
Solver	LP
Algorithm	Dual Simplex
Objective Function	NetProfit
Solution Status	Optimal
Objective Value	1821.5
Primal Infeasibility	3.637979E-12
Dual Infeasibility	1.776357E-15
Bound Infeasibility	0
Iterations	5
Presolve Time	0.00
Solution Time	0.00

[1]	NumProd	NumProd.DUAL
bedframes	57.5	-8.8818E-16
bookcases	28.0	-1.7764E-15
chairs	26.0	-1.3323E-15
desks	0.0	-1.0000E+00

[1]	AmountUsed	availability	Usage.DUAL
labor	225.0	225	1.30
metal	111.5	117	0.00
wood	420.0	420	2.45

TotalCost	Budget.UB	Budget.DUAL
10000	10000	0.05

[1]	_VAR_NAME	_VAR_DUAL
1	NumProd[desks]	-1.0000E+00
2	NumProd[chairs]	-1.3323E-15
3	NumProd[bookcases]	-1.7764E-15
4	NumProd[bedframes]	-8.8818E-16

[1]	_CON_NAME	_CON_BODY	_CON_UB	_CON_DUAL
1	Usage[labor]	225.0	225	1.30
2	Usage[metal]	111.5	117	0.00
3	Usage[wood]	420.0	420	2.45
4	Budget	10000.0	10000	0.05

End of Demonstration

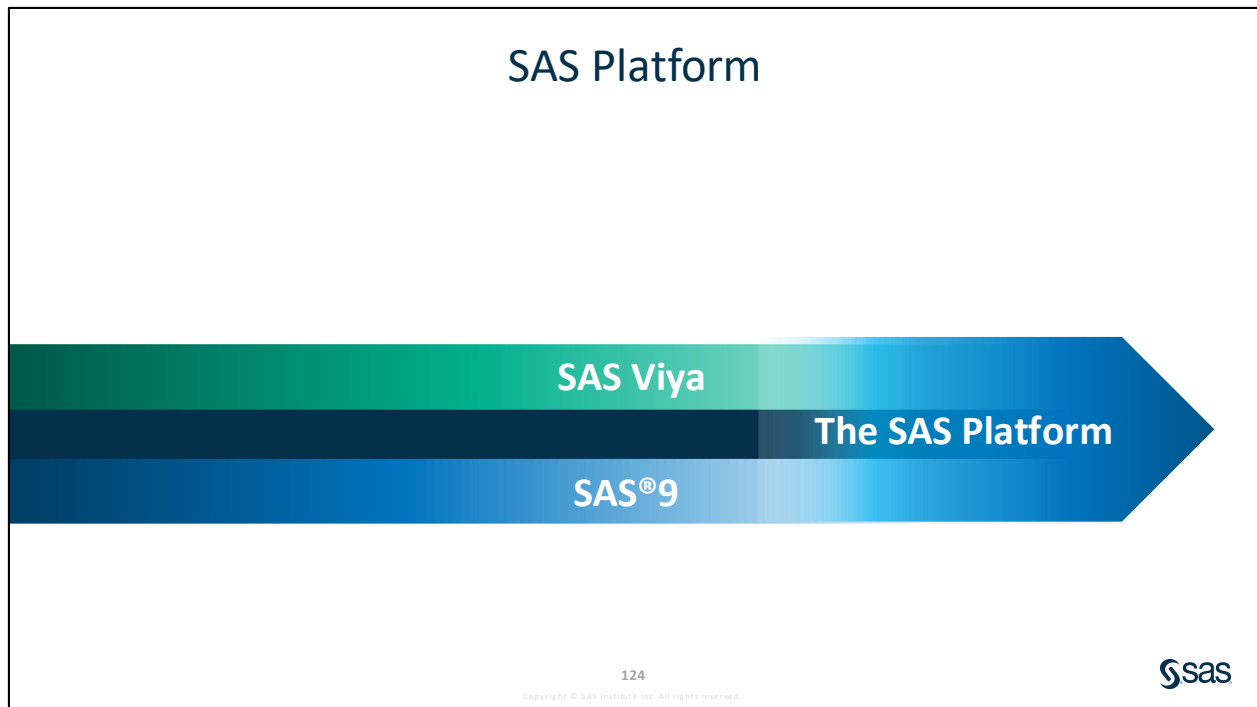
2.5 Reading and Writing Data in the OPTMODEL Procedure

Objectives

- Overview SAS Viya and In-Memory Analytics.
- Read problem data from In-Memory locations to formulate linear programming problems.
- Write problem solution data.

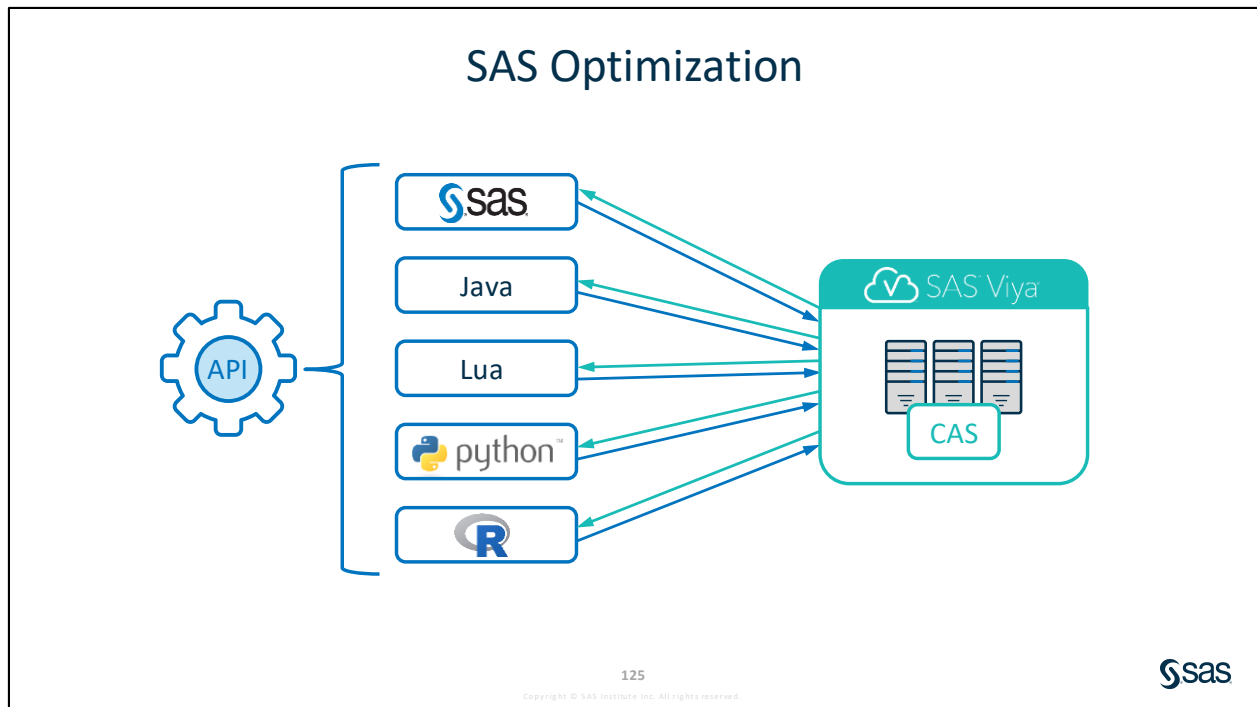
123

Copyright © SAS Institute Inc. All rights reserved.



In describing the arrays and index sets approach to formulation, it was mentioned that one of its main advantages is the ability to read values from data tables into formulation elements like parameter arrays. Getting familiar with reading problem data using PROC OPTMODEL is the focus of this section. PROC OPTMODEL is a part of SAS Optimization, which is a SAS Viya product. Before proceeding to the details of reading data, a brief overview of the architecture of SAS Viya is given. More details about SAS Viya are given later in this course.

SAS Optimization provides the tools that we use in this course, primarily the OPTMODEL procedure. SAS Optimization runs on SAS Viya, which is part of the SAS Platform. SAS Viya provides the tools that you need to solve today's analytical problems.



SAS Viya is an ***in-memory*** analytics engine that is designed to run distributed processes. SAS Viya runs its calculations in SAS Cloud Analytics Services, or CAS. In this lesson, the data is read into PROC OPTMODEL from in-memory locations using a CAS library (or *caslib*) reference.

As you can see, SAS Viya products such as SAS Optimization are not the only way to interact with CAS. The final lesson in this course discusses using open source interactivity to leverage SAS Optimization functionality through open source packages.

The following links provide good background for working with CAS and in-memory data.

<https://blogs.sas.com/content/sqf/2018/08/13/intro-cas-sas-viya/>

<https://support.sas.com/resources/papers/proceedings17/SAS0309-2017.pdf>

The documentation for CAS actions is here: <https://developer.sas.com/apis/cas/actions.html>

Reading Data

SAS Data Set: **casuser.class** (first four rows)

	Name	Height	Weight	Age
1	Alfred	69	112.5	14
2	Alice	56.5	84	13
3	Barbara	65.3	98	13
4	Carol	62.8	102.5	14

126

Copyright © SAS Institute Inc. All rights reserved.



Everything in this problem happens in relation to students, and each student has a height, weight, and age. The READ DATA statement saves the student names as the elements of the set **STUDENTS**, which must already be declared (as must the parameter arrays **height**, **weight**, and **age**).

An important step in reading problem data into a formulation in PROC OPTMODEL is establishing the link between the variable or column that contains index set values and the index set defined in the formulation. In this problem, our job is to map the Name column to a **STUDENTS** index set that has already been declared in PROC OPTMODEL. The variable **Name** is the *read-key-column*. Next, we will read **Height**, **Weight**, and **Age** into arrays that are already declared, and that are indexed by **STUDENTS**.

Reading Data

SAS Data Set: **casuser.class** (first four rows)

	Name	Height	Weight	Age
1	Alfred	69	112.5	14
2	Alice	56.5	84	13
3	Barbara	65.3	98	13
4	Carol	62.8	102.5	14

```
read data casuser.class into STUDENTS=[Name]
height weight age;
```

127

Copyright © SAS Institute Inc. All rights reserved.



In this syntax, the read key column **Name** populates the index set **STUDENTS**. Values from the variables **Height**, **Weight**, and **Age** populate the array's height, weight, and age. This syntax works, because the variable names match the pre-declared array names. Non-matching of names is considered later in this lesson.

Reading Data

SAS Data Set: **casuser.class** (Age = 13)

	Name	Height	Weight	Age
2	Alice	56.5	84	13
3	Barbara	65.3	98	13
4	Jeffery	62.5	84	13



Can you read only rows (observations) with specific characteristics?

128

Copyright © SAS Institute Inc. All rights reserved.



Can you read only rows with certain characteristics? Yes, subsetting the rows to only those with specific characteristics can be useful in data preprocessing and in checking your READ DATA syntax.

Reading Data

SAS Data Set: **casuser.class** (Age = 13)

	Name	Height	Weight	Age
2	Alice	56.5	84	13
3	Barbara	65.3	98	13
4	Jeffery	62.5	84	13

```
read data casuser.class (where= (Age=13) )
into STUDENTS=[Name] height weight age;
```

129

Copyright © SAS Institute Inc. All rights reserved.



Here is an example of reading only the rows of the class table with **Age = 13** into the PROC OPTMODEL formulation.

These other data set options might be used when you read or create data sets from PROC OPTMODEL:

FIRSTOBS=	To skip observations at the start of the data table
PW=	To access a password-protected data table
READ=	To assign a password to a data table
RENAME=	To change the name of data table variables
WHERE=	To read or create a data table that contains only observations meeting certain criteria



Reading the In-Memory Class Data Set in PROC OPTMODEL

orvy02d06.sas

```
libname opt '/home/student/orvy34';
libname casuser cas;

data casuser.class;
    set opt.class;
run;

proc optmodel;
    /* declare variables */
    set <str> STUDENTS;
    num height {STUDENTS}, weight {STUDENTS}, age {STUDENTS};

    /* read data from SAS data sets */
    read data casuser.class into STUDENTS=[Name] height weight age;

    print height weight age;
quit;
```

1. The first part of the program creates two libraries. First, the libref, **opt**, points to a directory containing the SAS table **class.sas7bdat**. Next, a caslib is created to accommodate the in-memory version of the table to be read. The DATA step reads the SAS table into memory.

PROC OPTMODEL Output

[1]	height	weight	age
Alfred	69.0	112.5	14
Alice	56.5	84.0	13
Barbara	65.3	98.0	13
Carol	62.8	102.5	14
Henry	63.5	102.5	14
James	57.3	83.0	12
Jane	59.8	84.5	12
Janet	62.5	112.5	15
Jeffrey	62.5	84.0	13
John	59.0	99.5	12
Joyce	51.3	50.5	11
Judy	64.3	90.0	14
Louise	56.3	77.0	12
Mary	66.5	112.0	15
Philip	72.0	150.0	16
Robert	64.8	128.0	12
Ronald	67.0	133.0	15
Thomas	57.5	85.0	11
William	66.5	112.0	15

SAS Log

NOTE: There were 19 observations read from the data set OPT.CLASS.
--

2. In the second part of the program, the WHERE option is used to subset the table to only students who are 13 years old.

```
/* read data from SAS data sets */
read data casuser.class (where= (Age=13)) into STUDENTS=[Name] height
weight age;
print height weight age;
```

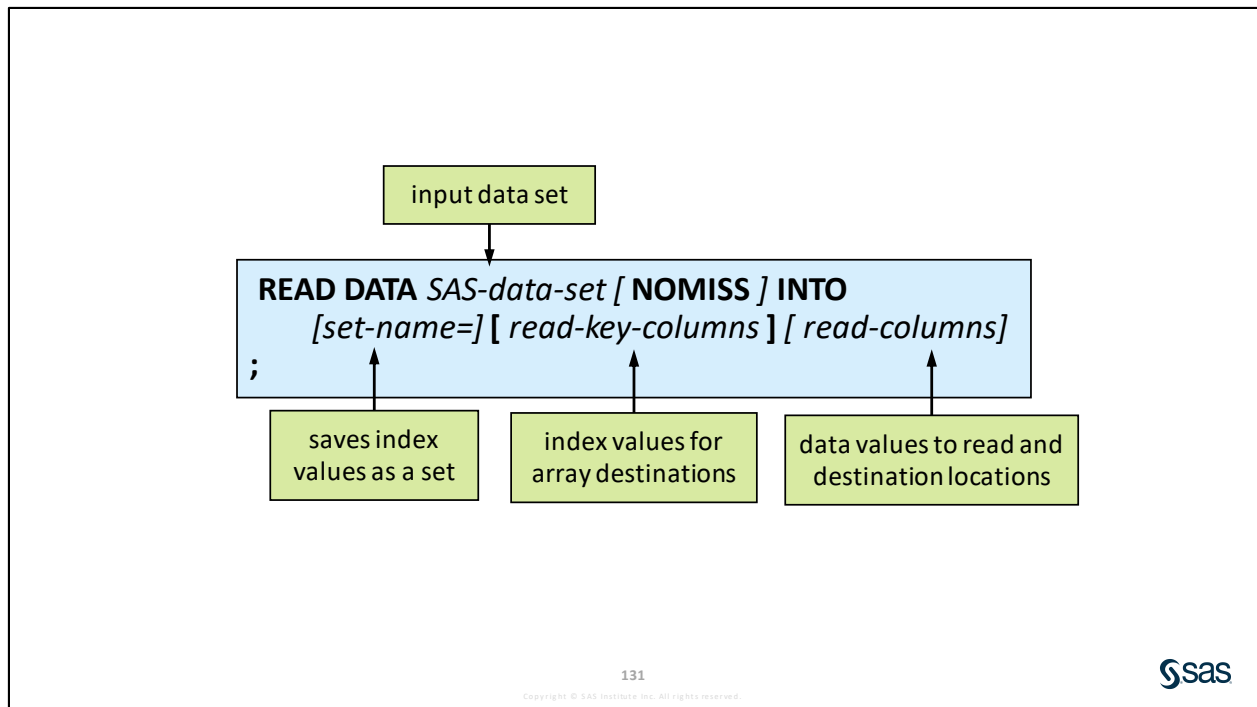
PROC OPTMODEL Output

[1]	height	weight	age
Alice	56.5	84	13
Barbara	65.3	98	13
Jeffrey	62.5	84	13

SAS Log

NOTE: There were 3 observations read from the data set OPT.CLASS. WHERE AGE=13;

End of Demonstration



SAS-data-set specifies the input data set being read.

The **read-key-columns** provide the index values for array destinations.

The optional **set-name** saves index values as a set.

read-columns specifies the variable values to be read from the data and their array destinations.

The syntax for a read-column parameter has several forms, including an iterated form that we describe in detail shortly. An identifier expression for the variable that is read from the data by **read-columns** can be specified. If an identifier expression for the variable is not specified, it is assumed to be the same as the listed parameter array name.

The **NOMISS** keyword is optional. It assigns a missing value place holder in destination locations corresponding to missing data values. An example of its usefulness is illustrated in the Diet Formulation demonstration in a subsequent lesson.

Furniture-Making Problem

```

data casuser.resource_data;
  input Resource $ Cost Amount_Available;
  datalines;
labor 14 225
metal 20 117
wood 11 420
run;

data casuser.product_data;
  length Item $9;
  input Item $ Selling_Price labor metal wood;
  datalines;
desks      94 2 1 3
chairs     79 1 1 3
bookcases 125 3 1 4
bedframes 109 2 1 4
run;

```

133

Copyright © SAS Institute Inc. All rights reserved.



Now consider how the furniture-making problem that was formulated in the previous section can be modified so that index set and array values can be read into the problem. These DATA steps will appear in the program. The in-memory library, **CASUSER**, has already been created.

Furniture-Making Problem

casuser.resource_data

	Resource	Cost	Amount_Available
1	labor	14	225
2	metal	20	117
3	wood	11	420

casuser.product_data

	Item	Selling_Price	labor	metal	wood
1	desks	94	2	1	3
2	chairs	79	1	1	3
3	bookcases	125	3	1	4
4	bedframes	109	2	1	4

134

Copyright © SAS Institute Inc. All rights reserved.



The tables that contain the values to be read are shown above. Recall the index set and arrays approach to formulating the furniture-making problem. Which columns contain the index set values? Which columns contain the parameter arrays?

Note: There are other ways that the same data can be stored in SAS data sets.

```

proc optmodel;
  /* declare sets and parameters */
  set <str> PRODUCTS, RESOURCES;
  num cost {RESOURCES}, availability {RESOURCES};
  num selling_price {PRODUCTS};
  num required {PRODUCTS, RESOURCES};

  read data casuser.resource_data into RESOURCES=[Resource]
    cost availability=Amount_Available;

  read data casuser.product_data into PRODUCTS=[Item]
    {r in RESOURCES} <required[Item,r]=col(r)>
    selling_price;

```

135

Copyright © SAS Institute Inc. All rights reserved.



In PROC OPTMODEL, the tables, **casuser.resource_data** and **casuser.product_data**, appear in the READ DATA statements. These READ DATA statements are examined step by step.

```

proc optmodel;
  /* declare sets and parameters */
  set <str> PRODUCTS, RESOURCES;
  num cost {RESOURCES}, availability {RESOURCES};
  num selling_price {PRODUCTS};
  num required {PRODUCTS, RESOURCES};

  read data casuser.resource_data into RESOURCES=[Resource]
    cost availability=Amount_Available;

  read data casuser.product_data into PRODUCTS=[Item]
    {r in RESOURCES} <required[Item,r]=col(r)>
    selling_price;

```

136

Copyright © SAS Institute Inc. All rights reserved.



In this slide, the index sets are declared with no initialization. They are populated in the READ DATA statements. Note that the SET type needs to be declared as string because the default type is numeric.

```

proc optmodel;
  /* declare sets and parameters */
  set <str> PRODUCTS, RESOURCES;
  num cost {RESOURCES}, availability {RESOURCES};
  num selling_price {PRODUCTS};
  num required {PRODUCTS, RESOURCES};

  read data casuser.resource_data into RESOURCES=[Resource]
    cost availability=Amount_Available;

  read data casuser.product_data into PRODUCTS=[Item]
    {r in RESOURCES} <required[Item,r]=col(r)>
    selling_price;

```

137

Copyright © SAS Institute Inc. All rights reserved.



Next, parameter arrays are declared as being indexed by the currently unpopulated index sets **PRODUCTS** and **RESOURCES**.

Note: Most information in PROC OPTMODEL is stored symbolically and resolved only when necessary.

```

set <str> PRODUCTS, RESOURCES;
num cost {RESOURCES}, availability {RESOURCES};

read data casuser.resource_data into RESOURCES=[Resource]
  cost availability=Amount_Available;

```

read key column
(SAS data set variable name)

casuser.resource_data

	Resource	Cost	Amount_Available
1	labor	14	225
2	metal	20	117
3	wood	11	420

138

Copyright © SAS Institute Inc. All rights reserved.



The read key column variable in the first READ DATA statement is **Resource**. The values in a read key column should be unique. If they are not, a warning message is printed to the log. Subsequent observations overwrite earlier observations with the same value in the read key column.

```

set <str> PRODUCTS, RESOURCES;
num cost {RESOURCES}, availability {RESOURCES};

read data casuser.resource_data into RESOURCES=[Resource]
  cost availability=Amount_Available;

```

set name
(OPTMODEL index set)

casuser.resource_data

	Resource	Cost	Amount_Available
1	labor	14	225
2	metal	20	117
3	wood	11	420

139

Copyright © SAS Institute Inc. All rights reserved.



The SET name just happens to be the plural form of the read key column variable name in this example. The set name is a PROC OPTMODEL name, and the read key column is a SAS data set variable name. They can be the same or completely unrelated.

```

set <str> PRODUCTS, RESOURCES;
num cost {RESOURCES}, availability {RESOURCES};

read data casuser.resource_data into RESOURCES=[Resource]
  cost availability=Amount_Available;

```

read columns
(OPTMODEL array name [=SAS data set variable name])

casuser.resource_data

	Resource	Cost	Amount_Available
1	labor	14	225
2	metal	20	117
3	wood	11	420

140

Copyright © SAS Institute Inc. All rights reserved.



The first array populated is **cost**. Because no data set variable name is provided, its corresponding data set variable is assumed to be **Cost**. The second array populated is **availability**. Its values are contained in the data set variable **Amount_Available**.

Typically, the array name is a parameter array name, but it could also be a suffixed array (for example, bounds for decision variables or constraints).

```
set <str> PRODUCTS, RESOURCES;
num cost {RESOURCES}, availability {RESOURCES};

read data casuser.resource_data into RESOURCES=[Resource]
    cost availability=Amount_Available;
```



```
set <str> PRODUCTS, RESOURCES;
num cost {RESOURCES}, availability {RESOURCES};

read data casuser.resource_data into RESOURCES=[Resource]
    cost[Resource]=Cost
    availability[Resource]=Amount_Available;
```

141

Copyright © SAS Institute Inc. All rights reserved.



The read columns portion of the READ DATA statement can be seen as an abbreviated version of the read columns portion of the equivalent READ DATA statement in the bottom syntax. The bottom version more closely resembles SAS DATA step programming. Note that the name of the read key column functions as a local dummy parameter, which can be used to refer to array locations in the read columns portion of the syntax.

```
set <str> PRODUCTS, RESOURCES;
num selling_price {PRODUCTS};
num required {PRODUCTS, RESOURCES};

read data casuser.product_data into PRODUCTS=[Item]
    {r in RESOURCES} <required[Item,r]=col(r)>
    selling_price;
```

read key column
 (SAS data set variable name)
 casuser.product_data

	Item	Selling_Price	labor	metal	wood
1	desks	94	2	1	3
2	chairs	79	1	1	3
3	bookcases	125	3	1	4
4	bedframes	109	2	1	4

142

Copyright © SAS Institute Inc. All rights reserved.



Using the read key column to refer to array locations in the read columns portion of the syntax is necessary in the READ DATA statement for the table **product_data**. Here, the read key column name **Item** is used to specify array locations in an iterated form of read columns.

```
set <str> PRODUCTS, RESOURCES;
num selling_price {PRODUCTS};
num required {PRODUCTS, RESOURCES};

read data casuser.product_data into PRODUCTS=[Item]
{r in RESOURCES} <required[Item,r]=col(r)>
selling_price;
```

set name
(OPTMODEL index set)
casuser.product_data

	Item	Selling_Price	labor	metal	wood
1	desks	94	2	1	3
2	chairs	79	1	1	3
3	bookcases	125	3	1	4
4	bedframes	109	2	1	4



Copyright © SAS Institute Inc. All rights reserved.

The variable **Item** populates the index set **PRODUCTS**.

```
set <str> PRODUCTS, RESOURCES;
num selling_price {PRODUCTS};
num required {PRODUCTS, RESOURCES};

read data casuser.product_data into PRODUCTS=[Item]
{r in RESOURCES} <required[Item,r]=col(r)>
selling_price;
```

read column
(OPTMODEL array name)
casuser.product_data

	Item	Selling_Price	labor	metal	wood
1	desks	94	2	1	3
2	chairs	79	1	1	3
3	bookcases	125	3	1	4
4	bedframes	109	2	1	4



Copyright © SAS Institute Inc. All rights reserved.

No variable name is supplied for the array **selling_price**, so it is assumed to be **Selling_Price**. Note that the read column **selling_price** comes after the iterated read columns. The order in which the columns are read from a SAS data set is not important.


```

set <str> PRODUCTS, RESOURCES;
num selling_price {PRODUCTS};
num required {PRODUCTS, RESOURCES};

read data casuser.product_data into PRODUCTS=[Item]
  {r in RESOURCES} <required[Item,r]=col(r)>
  selling_price;

```

iterated read column
 (array destination=COL expression)
 work.product_data

	Item	Selling_Price	labor	metal	wood
1	desks	94	2	1	3
2	chairs	79	1	1	3
3	bookcases	125	3	1	4
4	bedframes	109	2	1	4



Copyright © SAS Institute Inc. All rights reserved.

The COL option assigns values to the array **required** from the variables or columns of the **casuser.product_data** table. The values assigned correspond to the elements of **RESOURCES**, which was populated in the first READ DATA statement.

If the SAS data set did not contain a column for one of the resources, an error message would be generated in the SAS log, and the READ DATA statement would not be executed.

```

read data casuser.product_data into PRODUCTS=[Item]
  {r in RESOURCES} <required[Item,r]=col(r)>;

read data casuser.product_data into PRODUCTS=[Item]
  required[Item,'labor']=labor
  required[Item,'metal']=metal
  required[Item,'wood']=wood;

```

SAS Data Set (last columns)

	labor	metal	wood
1	2	1	3
2	1	1	3
3	3	1	4
4	2	1	4

2-D Parameter

required		

} PRODUCTS

RESOURCES={'labor','metal','wood'}



146

Copyright © SAS Institute Inc. All rights reserved.

This slide shows how the same data could be read without using an iterated read column statement, and also helps explain why the COL expression is required. The step-by-step explanations below provide more detail about how column iteration works.

```
read data casuser.product_data into PRODUCTS=[Item]
  {r in RESOURCES} <required[Item,r]=col(r)>
  selling_price;
```

required		selling_price
2		

} PRODUCTS

RESOURCES

casuser.product_data (first two rows)

	Item	Selling_Price	labor	metal	wood
1	desks	94	2	1	3
2	chairs	79	1	1	3

In the first step of the READ DATA statement, the first element of the **PRODUCTS** set is populated from the first row of the data. The first element of the array **required** is populated from the **Item** value in the first row of the table and the column corresponding to the first resource, **labor**.

```
read data casuser.product_data into PRODUCTS=[Item]
{r in RESOURCES} <required[Item,r]=col(r)>
selling_price;
```

required		selling_price	} PRODUCTS
2	1		

RESOURCES

casuser.product_data (first two rows)

	Item	Selling_Price	labor	metal	wood
1	desks	94	2	1	3
2	chairs	79	1	1	3

149

Copyright © SAS Institute Inc. All rights reserved.



In the second step, the read columns portion of the syntax iterates across variables or columns corresponding to elements of the set of **RESOURCES**. The second element of the array **required** is populated from the **Item** value in the first row of the table and the column corresponding to the next resource, **metal**.

```
read data casuser.product_data into PRODUCTS=[Item]
{r in RESOURCES} <required[Item,r]=col(r)>
selling_price;
```

required		selling_price	} PRODUCTS
2	1	3	

RESOURCES

casuser.product_data (first two rows)

	Item	Selling_Price	labor	metal	wood
1	desks	94	2	1	3
2	chairs	79	1	1	3

150

Copyright © SAS Institute Inc. All rights reserved.



We are still on the first row of the **product_data** table. The third element of the array **required** is populated from the **Item** value in the first row of the table and the column corresponding to the last resource, **wood**.

```
read data casuser.product_data into PRODUCTS=[Item]
{r in RESOURCES} <required[Item,r]=col(r)>
selling_price;
```

required			selling_price
2	1	3	94

RESOURCES

PRODUCTS

casuser.product_data (first two rows)

	Item	Selling_Price	labor	metal	wood
1	desks	94	2	1	3
2	chairs	79	1	1	3

151

Copyright © SAS Institute Inc. All rights reserved.



In the final operation on the first row of the **product_data** table, the first element of the array **selling_price** is populated from the column **Selling_Price**.

```
read data casuser.product_data into PRODUCTS=[Item]
{r in RESOURCES} <required[Item,r]=col(r)>
selling_price;
```

required			selling_price
2	1	3	94
1			

RESOURCES

PRODUCTS

casuser.product_data (first two rows)

	Item	Selling_Price	labor	metal	wood
1	desks	94	2	1	3
2	chairs	79	1	1	3

152

Copyright © SAS Institute Inc. All rights reserved.



We begin iterating across column names corresponding to **RESOURCE** set elements on the second row of the data to continue populating the array **required**.

```
read data casuser.product_data into PRODUCTS=[Item]
  {r in RESOURCES} <required[Item,r]=col(r)>
  selling_price;
```

required			selling_price
2	1	3	94
1	1		

} PRODUCTS

RESOURCES

casuser.product_data (first two rows)

	Item	Selling_Price	labor	metal	wood
1	desks	94	2	1	3
2	chairs	79	1	1	3

153

Copyright © SAS Institute Inc. All rights reserved.



The declared arrays have been populated for the **PRODUCT** value in the first row, *desks*. Next, we move to the **PRODUCT** value in the second row, *chairs*. The steps for filling in array values are repeated.



Reading Problem Data in PROC OPTMODEL for the Furniture-Making Problem

orvy02d07.sas

```
data casuser.resource_data;
    input Resource $ Cost Amount_Available;
    datalines;
labor 14 225
metal 20 117
wood 11 420
run;

data casuser.product_data;
    length Item $9;
    input Item $ Selling_Price labor metal wood;
    datalines;
desks      94 2 1 3
chairs     79 1 1 3
bookcases 125 3 1 4
bedframes 109 2 1 4
run;

/*
Leading spaces in the data values following the DATALINES
statements above can cause issues with reading the character
strings into the OPTMODEL Procedure. That is, there's a reason for
not indenting the values.
*/

%let budget_limit = 10000;

proc optmodel;
    /* declare sets and parameters */
    set <str> PRODUCTS, RESOURCES;
    num cost {RESOURCES}, availability {RESOURCES};
    num selling_price {PRODUCTS};
    num required {PRODUCTS, RESOURCES};

    /* read data from SAS data sets */
    read data resource_data into RESOURCES=[Resource]
        cost availability=Amount_Available;

    read data product_data into PRODUCTS=[Item]
        {r in RESOURCES} <required[Item,r]=col(r)>
        selling_price;
```

```

/* print parameter arrays */
print cost dollar. availability;
print selling_price dollar. required;

/* declare variables */
var NumProd {PRODUCTS} >= 0;

impvar Revenue = sum {p in PRODUCTS}
    selling_price[p] * NumProd[p];

impvar AmountUsed {r in RESOURCES} =
    sum {p in PRODUCTS} NumProd[p] * required[p,r];
impvar TotalCost = sum {r in RESOURCES}
    cost[r] * AmountUsed[r];

/* declare constraints */
con Usage {r in RESOURCES}:
    AmountUsed[r] <= availability[r];

con Budget: TotalCost <= &budget_limit;

/* declare objective */
max NetProfit = Revenue - TotalCost;

solve;

print NumProd;
print AmountUsed;

quit;

```

Note: The SAS macro language can be used to input the values of individual parameters, with one macro variable per parameter.

Notice the format applied to the **cost** and **selling_price** parameter arrays in the PRINT statement.

PROC OPTMODEL Output (only the first two PRINT statements)

The OPTMODEL Procedure

[1]	cost	availability
labor	\$14	225
metal	\$20	117
wood	\$11	420

[1]	selling_price
bedframes	\$109
bookcases	\$125
chairs	\$79
desks	\$94

	required		
	labor	metal	wood
bedframes	2	1	4
bookcases	3	1	4
chairs	1	1	3
desks	2	1	3

The problem summary, solution summary, and the output of the last PRINT statements are omitted, because they are identical to the output of the previous furniture-making demonstration program from a previous section.

Notice that, except for the SAS DATA steps, nothing in the PROC OPTMODEL program is specific to the furniture-making problem. In fact, the model is generic and can be used to solve a class of problems called *product mix problems*.

End of Demonstration

Writing Problem Data

This brief sub-section summarizes some of the CREATE DATA functionality in the OPTMODEL procedure.

CREATE DATA Statement

```
CREATE DATA table FROM  
[ key-columns ] [ = key-set ] columns ;
```

158

Copyright © SAS Institute Inc. All rights reserved.



- *table* specifies the table to be output.
- *key-columns* specify variable names in the output table whose values index array locations in *columns*.
- *key-set* (optional) specifies a set of index values for *key-columns*.
- *columns* specify table variable names and PROC OPTMODEL source data.

Writing Output from the Furniture-Making LP

```
create data work.opt_solution from
  [Item]=PRODUCTS Number_Produced=
  NumProd;
```

SAS Data Set: **work.opt_solution**

	Item	Number_Produced
1	desks	0
2	chairs	26
3	bookcases	28
4	bedframes	57.5

159

Copyright © SAS Institute Inc. All rights reserved.



This slide previews the CREATE DATA functionality that will be illustrated in the next demonstration. Recall that **PRODUCTS** is an index set and **NumProd** is an array of decision variables.

- The syntax maps the set values in **PRODUCTS** to the column Item in the output table.
- The **Number_Produced** column in the data is created from optimized values for the decision variables contained in the array **NumProd**.

More details about the CREATE DATA syntax are presented in the Operations Research with SAS Optimization course and in the SAS Optimization documentation;

https://go.documentation.sas.com/?cdcId=pgmsascdc&cdcVersion=9.4_3.5&docsetId=casmopt&docsetTarget=casmopt_optmodel_syntax11.htm&locale=en#casmopt.optmodel.npxcreatestmt



Writing Solution Data from PROC OPTMODEL for the Furniture-Making Problem

orvy02d08.sas

1. A CREATE DATA statement is added to the syntax from the previous demonstration.

```
...
solve;

    print NumProd;
    print AmountUsed;

    create data work.opt_solution from [Item]=PRODUCTS
        Number_Produced=NumProd;

quit;
```

2. The **opt_solution** table in the **Work** library is shown below.

Item	Number_Produced
desks	0
chairs	26
bookcases	28
bedframes	57.5

End of Demonstration

2.6 Solutions

Solutions to Practices

1. Changing the Limit of a Tight Constraint

```
con x + 3*y <= 224 ;
```

PROC OPTMODEL Output

Problem Summary	
Objective Sense	Maximization
Objective Function	z
Objective Type	Linear
Number of Variables	2
Bounded Above	0
Bounded Below	2
Bounded Below and Above	0
Free	0
Fixed	0
Number of Constraints	3
Linear LE (<=)	3
Linear EQ (=)	0
Linear GE (>=)	0
Linear Range	0
Constraint Coefficients	6

Solution Summary	
Solver	LP
Algorithm	Dual Simplex
Objective Function	z
Solution Status	Optimal
Objective Value	1778.5
Primal Infeasibility	2.842171E-14
Dual Infeasibility	1.776357E-15
Bound Infeasibility	0
Iterations	4
Presolve Time	0.00
Solution Time	0.00

x	y
63.5	53.5

The previous optimal solution with $[x,y] = [63,54]$ has $x + 3y = 63 + 162 = 225$, so it is no longer feasible. An inequality that is satisfied with equality by an optimal solution is said to be *tight*.

2. Detecting Infeasible Linear Programming Problems

```
con 3*x + 4*y >= 420 ;
```

PROC OPTMODEL Output

Solution Summary	
Solver	LP
Algorithm	Dual Simplex
Objective Function	z
Solution Status	Infeasible
Iterations	3
Presolve Time	0.00
Solution Time	0.00

x	y
0	0

The solution status indicates that the problem is infeasible.

3. Adding a Budget Constraint to the Furniture-Making Problem

```

proc optmodel;
  /* declare variables */
  var Desks >= 0, Chairs >= 0, Bookcases >= 0, Bedframes >= 0;

  impvar Hrs = 2*Desks + Chairs + 3*Bookcases + 2*Bedframes;
  impvar Lbs = Desks + Chairs + Bookcases + Bedframes;
  impvar Ft3 = 3*Desks + 3*Chairs + 4*Bookcases + 4*Bedframes;

  /* declare constraints */
  con Labor: Hrs <= 225, Metal: Lbs <= 117, Wood: Ft3 <= 420,
    Budget: 14*Hrs + 20*Lbs + 11*Ft3 <= 10000 ;

  /* declare objective */
  max NetProfit =
    94*Desks + 79*Chairs + 125*Bookcases + 109*Bedframes
    - 14*Hrs - 20*Lbs - 11*Ft3;

  expand Budget;

  solve;

  print Desks Chairs Bookcases Bedframes;

  print Hrs Lbs Ft3;

quit;

```

The OPTMODEL Procedure

Solution Summary	
Solver	LP
Algorithm	Dual Simplex
Objective Function	NetProfit
Solution Status	Optimal
Objective Value	1821.5
Primal Infeasibility	3.637979E-12
Dual Infeasibility	1.776357E-15
Bound Infeasibility	0
Iterations	5
Presolve Time	0.00
Solution Time	0.00

Desks	Chairs	Bookcases	Bedframes
0	26	28	57.5

The objective value in the problem without the budget constraint was 1827.

4. Formulating a Transportation Problem

a. Here are the declaration statements:

```
proc optmodel;
  /* declare sets and parameters */
  set ORIGINS = /Chicago Detroit/;
  set DESTINATIONS = /Boston Newark Toronto/;
  num supply {ORIGINS} = [200 150];
  num demand {DESTINATIONS} = [50 150 100];
  num unit_cost {ORIGINS, DESTINATIONS} =
    [40 25 15 30 15 10];

  /* print parameter arrays */
  print unit_cost supply demand;

quit;
```

In the declaration statement for **unit_cost**, the values can be divided into two lines to make it easier to parse the initializers.

PROC OPTMODEL Output

The OPTMODEL Procedure

unit_cost			
	Boston	Newark	Toronto
Chicago	40	25	15
Detroit	30	15	10

[1]	supply	demand
Boston		50
Chicago	200	
Detroit	150	
Newark		150
Toronto		100

The PRINT statement combines **supply** and **demand** as a listing, whereas **unit_cost** is formatted as a separate table because it is a two-dimensional array.

b. The declaration statements are as follows:

```
proc optmodel;
  /* declare sets and parameters */
  set ORIGINS = /Chicago Detroit/;
  set DESTINATIONS = /Boston Newark Toronto/;
  num supply {ORIGINS} = [200 150];
  num demand {DESTINATIONS} = [50 150 100];
  num unit_cost {ORIGINS, DESTINATIONS} =
```

```

[40 25 15 30 15 10];

/* declare variables */
var NumShip {ORIGINS, DESTINATIONS} >= 0 init 50;

impvar FlowOut {i in ORIGINS} =
    sum {j in DESTINATIONS} NumShip[i,j];

impvar FlowIn {j in DESTINATIONS} =
    sum {i in ORIGINS} NumShip[i,j];

expand FlowOut['Chicago'];
expand FlowIn['Boston'];

expand / impvar;

print FlowOut;
print FlowIn;

quit;

```

PROC OPTMODEL Output

The OPTMODEL Procedure

```
Impvar FlowOut[Chicago] = NumShip[Chicago,Boston] + NumShip[Chicago,Newark] + NumShip[Chicago,Toronto]
```

```
Impvar FlowIn[Boston] = NumShip[Chicago,Boston] + NumShip[Detroit,Boston]
```

[1]	FlowOut
Chicago	150
Detroit	150

[1]	FlowIn
Boston	100
Newark	100
Toronto	100

The (individual) implicit variables are expanded, but the EXPAND/IMPVAR statement triggers the current model to be built. It does not produce any output because the implicit variables do not enter into the constraints or objective function of the optimization problem.

- c. The formulation using index sets is as follows:

```

proc optmodel;
    /* declare sets and parameters */
    set ORIGINS = /Chicago Detroit/;
    set DESTINATIONS = /Boston Newark Toronto/;
    num supply {ORIGINS} = [200 150];
    num demand {DESTINATIONS} = [50 150 100];

```



```

num unit_cost {ORIGINS, DESTINATIONS} =
    [40 25 15 30 15 10];

/* declare variables */
var NumShip {ORIGINS, DESTINATIONS} >= 0;

impvar FlowOut {i in ORIGINS} =
    sum {j in DESTINATIONS} NumShip[i,j];

impvar FlowIn {j in DESTINATIONS} =
    sum {i in ORIGINS} NumShip[i,j];

/* declare constraints */
con Supply_con {i in ORIGINS}:
    FlowOut[i] <= supply[i];

con Demand_con {j in DESTINATIONS}:
    FlowIn[j] >= demand[j];

```

```

/* declare objective */
min TotalCost = sum {i in ORIGINS, j in DESTINATIONS}
    unit_cost[i,j] * NumShip[i,j];

expand;

solve;

print NumShip;

quit;

```

PROC OPTMODEL Output

The OPTMODEL Procedure

```

Var NumShip[Chicago,Boston] >= 0
Var NumShip[Chicago,Newark] >= 0
Var NumShip[Chicago,Toronto] >= 0
Var NumShip[Detroit,Boston] >= 0
Var NumShip[Detroit,Newark] >= 0
Var NumShip[Detroit,Toronto] >= 0
Impvar FlowOut[Chicago] = NumShip[Chicago,Boston] + NumShip[Chicago,Newark] + NumShip[Chicago,Toronto]
Impvar FlowOut[Detroit] = NumShip[Detroit,Boston] + NumShip[Detroit,Newark] + NumShip[Detroit,Toronto]
Impvar FlowIn[Boston] = NumShip[Chicago,Boston] + NumShip[Detroit,Boston]
Impvar FlowIn[Newark] = NumShip[Chicago,Newark] + NumShip[Detroit,Newark]
Impvar FlowIn[Toronto] = NumShip[Chicago,Toronto] + NumShip[Detroit,Toronto]
Minimize TotalCost=40*NumShip[Chicago,Boston] + 25*NumShip[Chicago,Newark] + 15*NumShip[Chicago,Toronto] + 30*
NumShip[Detroit,Boston] + 15*NumShip[Detroit,Newark] + 10*NumShip[Detroit,Toronto]
Constraint Supply_con[Chicago]: FlowOut[Chicago] <= 200
Constraint Supply_con[Detroit]: FlowOut[Detroit] <= 150
Constraint Demand_con[Boston]: FlowIn[Boston] >= 50
Constraint Demand_con[Newark]: FlowIn[Newark] >= 150
Constraint Demand_con[Toronto]: FlowIn[Toronto] >= 100

```

Problem Summary	
Objective Sense	Minimization
Objective Function	TotalCost
Objective Type	Linear
Number of Variables	6
Bounded Above	0
Bounded Below	6
Bounded Below and Above	0
Free	0
Fixed	0
Number of Constraints	5
Linear LE ($\leq$)	2
Linear EQ ($=$)	0
Linear GE ($\geq$)	3
Linear Range	0
Constraint Coefficients	12

Solution Summary	
Solver	LP
Algorithm	Dual Simplex
Objective Function	TotalCost
Solution Status	Optimal
Objective Value	5750
Primal Infeasibility	0
Dual Infeasibility	0
Bound Infeasibility	0
Iterations	6
Presolve Time	0.00
Solution Time	0.00

NumShip			
	Boston	Newark	Toronto
Chicago	50	0	100
Detroit	0	150	0

The output of the PRINT statement for **NumShip** is formatted as a table because it is a two-dimensional array. PROC OPTMODEL formats sparse two-dimensional arrays as lists. (The PMATRIX option controls the tolerance level.)

End of Solutions

Solutions to Activities and Questions

2.01 Multiple Answer Question – Correct Answers

How many optimal solutions can a linear programming problem have?

- ☒ a. no optimal solutions
- ☒ b. a unique optimal solution
- ☐ c. more than one, but a finite number of optimal solutions
- ☒ d. an infinite number of optimal solutions

18

Copyright © SAS Institute Inc. All rights reserved.



A linear programming problem has a finite number of extreme point solutions, but these are not the only feasible solutions. Any point on the line between two optimal solutions is an optimal solution.

2.02 Multiple Choice Question – Correct Answer

What is the default algorithm for linear programming problems?

- ☐ a. primal simplex algorithm
- ☒ b. dual simplex algorithm
- ☐ c. interior point algorithm
- ☐ d. depends on problem characteristics

23

Copyright © SAS Institute Inc. All rights reserved.



For linear programming problems, the default algorithm does not depend on problem characteristics. The LP solver must be specified so that you can set solver options.

2.03 Multiple Answer Question – Correct Answers

Which of the following statements are true of SAS names?

- ☒ a. SAS names can be up to 32 characters long.
- ☒ b. SAS names cannot start with a number.
- ☐ c. SAS names are case sensitive.

50

Copyright © SAS Institute Inc. All rights reserved.



SAS names are **not** case sensitive.

2.04 Multiple Choice Question – Correct Answer

Which potential index set for the parameter array **selling_price** is most indicative of the problem context?

	Selling Price (\$)	
Desks	94	} Index Set
Chairs	79	
Bookcases	125	
Bedframes	109	

- a. {1,2,3,4}
- b. {'P1','P2','P3','P4'}
- ☒ c. {'desks','chairs','bookcases','bedframes'}

59

Copyright © SAS Institute Inc. All rights reserved.



The index set **PRODUCTS** can have the product names as elements. (In this case, it is a set of strings.)

2.05 Multiple Answer Question – Correct Answers

Which of the following statements are true of parameters and parameter arrays in PROC OPTMODEL?

- ☒ a. Parameters must be declared before they are used.
- ☐ b. The type of a set must be declared.
- ☐ c. Parameters must be initialized when they are declared.
- ☐ d. All parameter values can be modified after they are declared.
- ☒ e. Parameter values cannot be modified by the solver.

73

Copyright © SAS Institute Inc. All rights reserved.



There is a default type for sets if none is declared, and there are no initializers. Even parameters that are used in defining index sets for other parameters, such as the parameter **n** in the examples, need only to be declared, and not assigned values.

2.06 Multiple Choice Question – Correct Answer

Which of the following does not need to be declared in PROC OPTMODEL?

- ☐ a. parameter arrays
- ☐ b. decision variables
- ☐ c. implicit variables
- ☒ d. local dummy parameters

82

Copyright © SAS Institute Inc. All rights reserved.



Lesson 3 Nonlinear Programming

3.1	Introduction to Nonlinear Programming.....	3-3
	Demonstration: Minimizing the Banana Function Using PROC OPTMODEL.....	3-20
	Demonstration: Solving the Sinc Problem Using PROC OPTMODEL.....	3-30
	Demonstration: Solving the Sinc Problem Using PROC OPTMODEL MULTISTART Options.....	3-36
3.2	Solving Nonlinear Programming Problems Using the OPTMODEL Procedure.....	3-39
	Demonstration: Solving a Portfolio Optimization Problem Using PROC OPTMODEL.....	3-42
	Practice.....	3-51
3.3	Solutions	3-55
	Solutions to Practices	3-55
	Solutions to Activities and Questions.....	3-57

3.1 Introduction to Nonlinear Programming

Objectives

- Describe how, conceptually and geometrically, iterative improvement algorithms solve nonlinear programming problems.
- Explain which types of nonlinear programming problems can be solved in general.
- Explain the optimality conditions for nonlinear programming problems.

Nonlinear Programming Problem

$$\begin{aligned} &\min|\max \quad f(x) \\ &\text{subject to} \quad c_i(x) \{\leq, =, \geq\} b_i \quad (i=1, 2, \dots, m) \\ &\quad \quad \quad l_j \leq x_j \leq u_j \quad (j=1, 2, \dots, n) \\ &\text{where } f(x) \text{ and } c_1(x), \dots, c_m(x) \text{ are continuous functions} \end{aligned}$$

The general form of the nonlinear programming problem, or NLP, is shown here. The objective function and constraints are continuous functions, and one or more are nonlinear functions of the decision variables.

Nonlinear Programming Problem

$$\begin{aligned} &\min|\max \quad f(x) \\ &\text{subject to} \quad c_i(x) \{\leq, =, \geq\} b_i \quad (i=1, 2, \dots, m) \\ &\quad \quad \quad l_j \leq x_j \leq u_j \quad (j=1, 2, \dots, n) \\ &\text{where } f(x) \text{ and } c_1(x), \dots, c_m(x) \text{ are continuous functions} \end{aligned}$$
Unconstrained:

- No constraints or bounds.

Linearly Constrained:

- Constraints are linear functions.

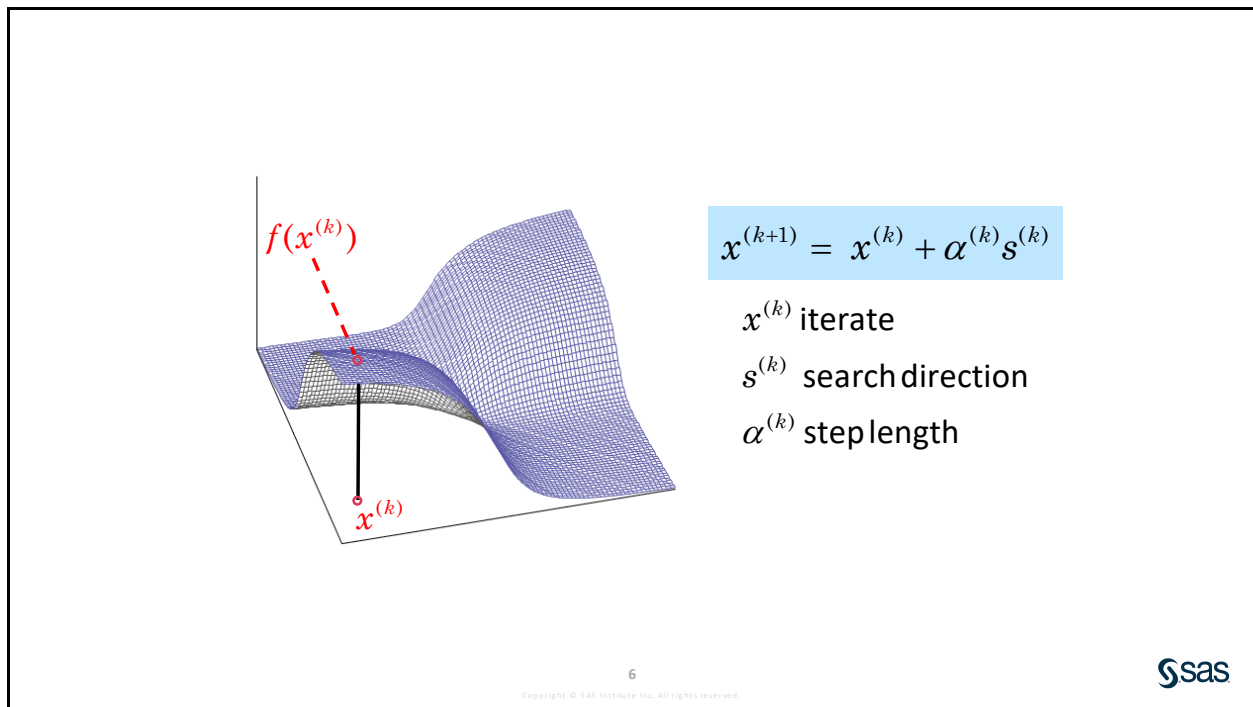
Bound Constrained:

- No constraints other than bounds.

Nonlinearly Constrained:

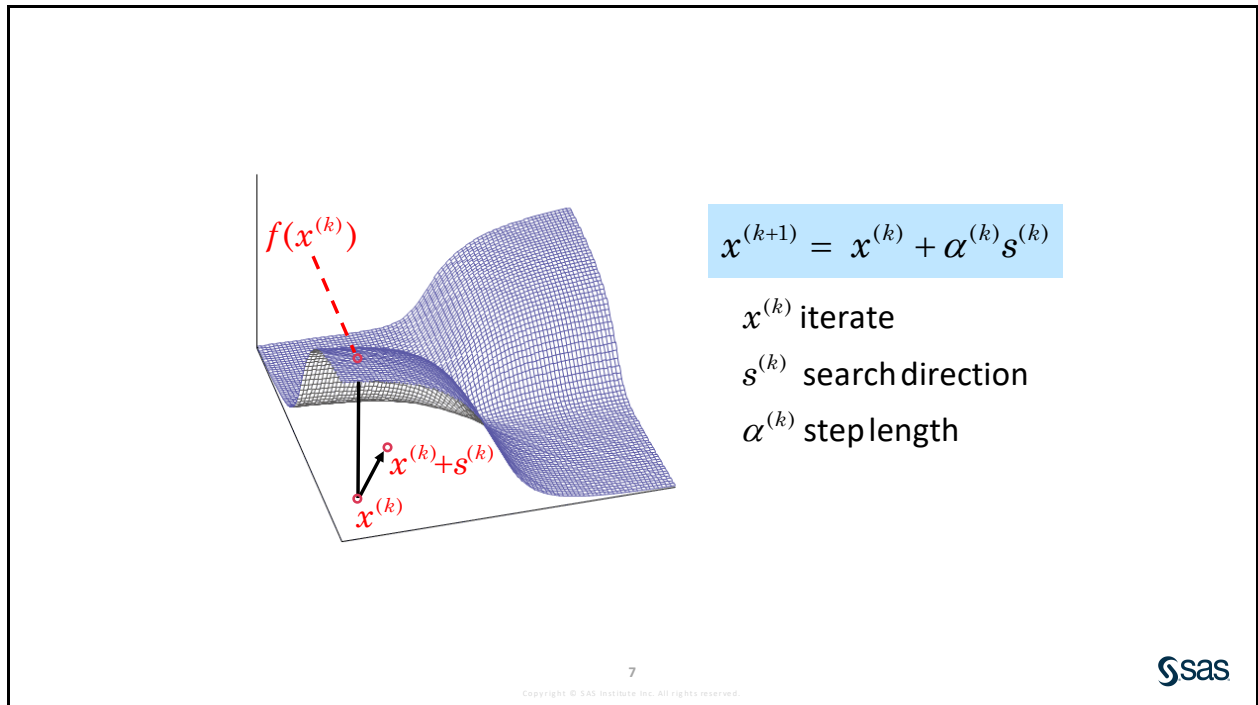
- One or more of the constraints is nonlinear.

NLPs can be classified according to their constraints. We will see examples of unconstrained and bound constrained NLPs in the next demonstration. Bound constrained NLPs can have either upper or lower bounds (or both) as shown in the notation. The portfolio optimization example later in this lesson shows an example of a nonlinear constraint.



The picture shows how we might start the solution a nonlinear minimization problem with two decision variables. The problem has a nonlinear objective, and the bound constraints mean that the values for the two decision variables reside in the nonnegative part of the plane.

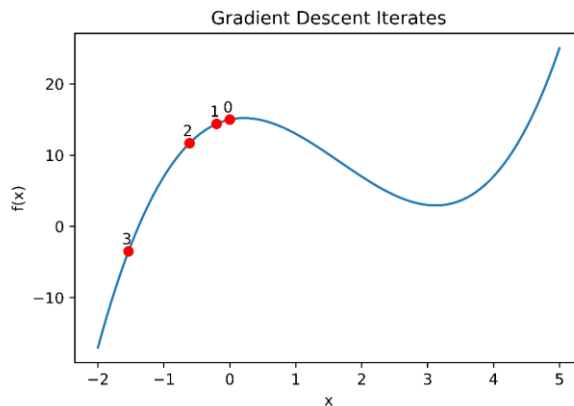
The idea is to iterate through different values for the decision variables until the lowest point on the objective function surface is found. Here, the decision variable values shown might be starting values that are chosen at iteration 1, so they would be the starting values for x_1 and the starting values for x_2 ($k = 0 \rightarrow (x_1^0, x_2^0)$). These correspond to some initial value for the objective function, that is $f(x^0)$. Most of the algorithms for solving nonlinear programming problems are iterative (see Lesson 1). Local information at some iterate x ($x^{(k)}$) is used to find the direction $s^{(k)}$, so essentially the objective function evaluated at those values of x ($f(x)$) is approximated by a quadratic ($x^{(k)}$).



Finding the fastest path downhill involves evaluating the slope or derivative of the objective function at the current iteration value for the decision variables. An updated iterate for decision variables proceeds from the current value along a path that is determined by the gradient and a step length. The NLP algorithms in PROC OPTMODEL use more information about the objective function than just the slope to determine the best search path.

Gradient Descent

$$x_{n+1} = x_n - \gamma \frac{df}{dx}(x_n)$$



$$x_0 = 0$$

$$f(x) = x^3 - 5x^2 + 2x + 15$$

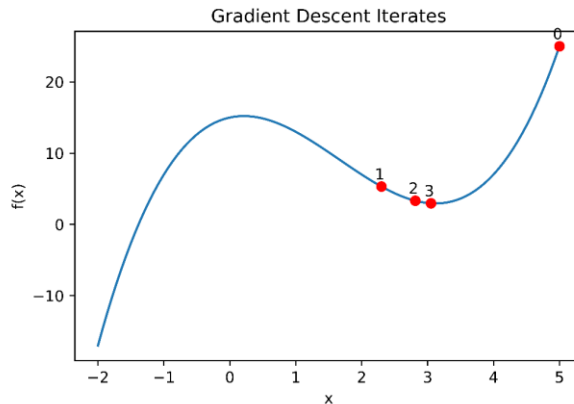
sas

Because the minimum or maximum of a function occurs when the derivative of the function is 0, we can follow the derivative up or down to find a local maximum or minimum. This method is generally called *gradient descent* for minimization problems, but for maximization problems, you would use *gradient ascent*. For this discussion, we restrict ourselves to the case of minimizing a function. The idea is that you are iteratively calculating points trying to get closer and closer to the local minimum. Starting with some initial point x_0 , you calculate a sequence of numbers in which the current iterate is a function of the previous value in the sequence. The idea is that you subtract a fraction of the derivative from the current point to find a new point closer to the local minimum. At some point, the derivative is very close to zero, and thus the iterates in the sequence are all very similar. This means that the algorithm has converged and the last iterate would be local minimum.

In this example, the function $f(x)$ is minimized starting at the initial point $x_0=0$. Because the derivative at 0 is negative, the sequence of points x_n is decreasing. In this case, the function is unbounded from below, and the gradient descent algorithm never converges. The points on the plot are labeled in order of iteration number, so x_0 is labeled with a 0.

Gradient Descent

$$x_{n+1} = x_n - \gamma \frac{df}{dx}(x_n)$$



$$x_0 = 5$$

$$f(x) = x^3 - 5x^2 + 2x + 15$$

sas

Now the same function $f(x)$ is minimized starting at the initial point $x_0=5$. Because the derivative at 5 is negative, the first iterate x_1 will have a value less than x_0 . The derivative at x_1 is positive, so x_2 will have a larger value than x_1 . In this case, the function has a local minimum at $x = 3.12$, whereas x_3 has a value 3.05. If we let the iteration continue, we find that by x_5 , we get the desired value 3.12. If we wanted more precision in the number, we would likely need more iterates.

Newton's Method

$$x_{n+1} = x_n + \gamma \Delta x$$

Second-order
Taylor expansion

$$f(x) = f(x_n + \Delta x) \approx f(x_n) + f'(x_n)\Delta x + \frac{1}{2} f''(x_n)\Delta x^2$$

Set the derivative
equal to 0

$$\frac{df}{dx} = 0 \Rightarrow \frac{d}{d\Delta x} [f(x_n) + f'(x_n)\Delta x + \frac{1}{2} f''(x_n)\Delta x^2] = 0$$

Solve for the step
size

$$f'(x_n) + f''(x_n)\Delta x = 0 \Rightarrow \Delta x = -\frac{f'(x_n)}{f''(x_n)}$$

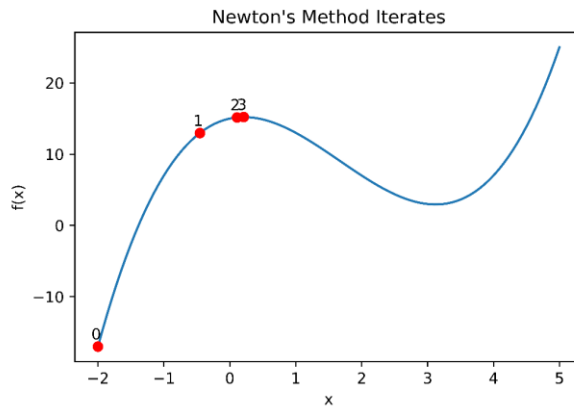
$$x_{n+1} = x_n - \gamma \frac{f'(x_n)}{f''(x_n)}$$



Gradient descent uses the first derivative to search for local minima, but Newton's method extends this idea to incorporate information from the second derivative into the iterative calculations. The idea is to perform a Taylor expansion of the function $f(x)$ that we want to optimize, and then truncate the expansion to keep only the second-order terms. We take the derivative of the second-order Taylor expansion and set it equal to zero. This is essentially looking for the local optimum using a local approximation, so we step toward this local optimum and recalculate the second-order Taylor expansion. This is very similar to gradient descent: We are following the derivative down (or toward the local optimum), but we choose our step size and direction based on the second derivative. Just like gradient descent, when the derivative approaches 0, the iterates stop changing, and the algorithm has converged.

Newton's Method

$$x_{n+1} = x_n - \gamma \frac{f'(x_n)}{f''(x_n)}$$

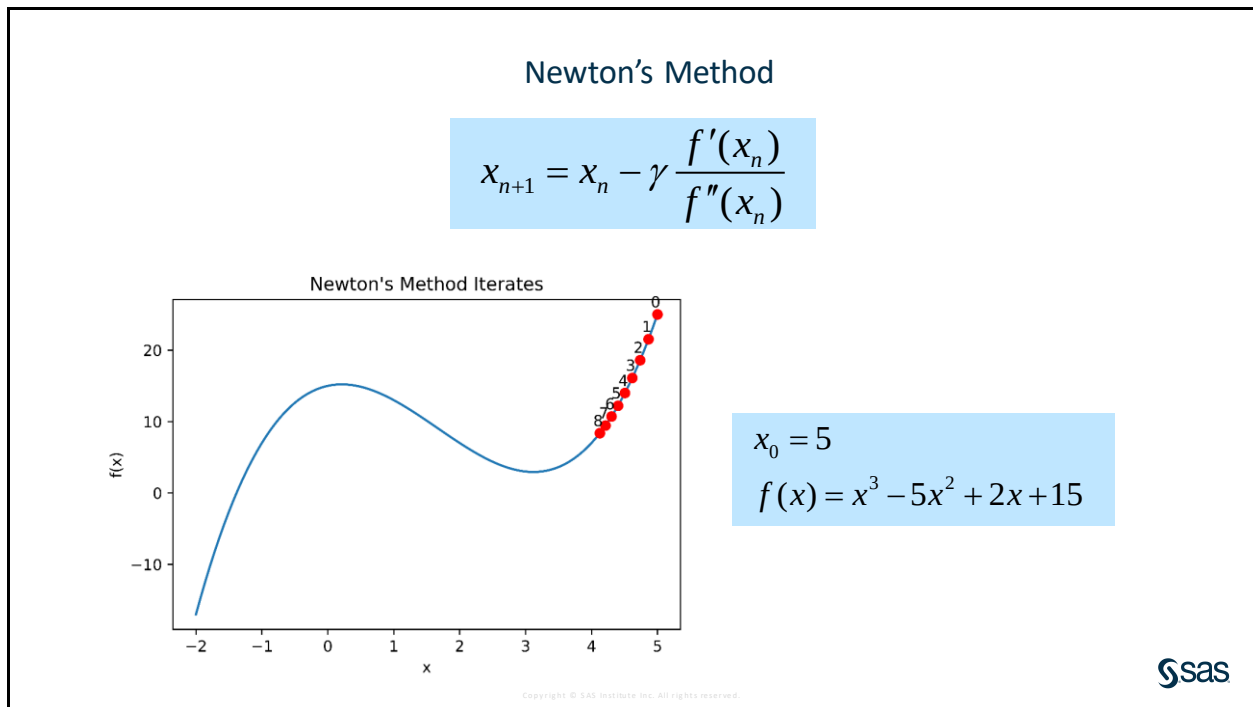


$$x_0 = -2$$

$$f(x) = x^3 - 5x^2 + 2x + 15$$

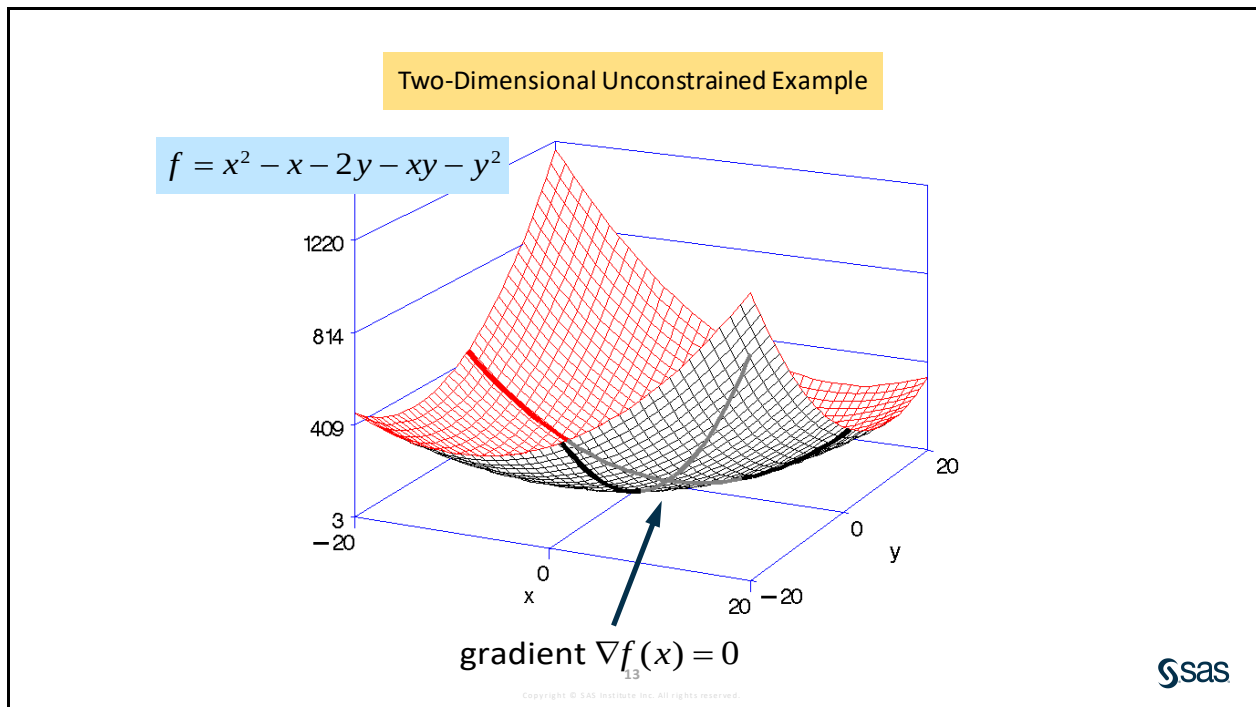
SAS

In this example, local optima are found in the function $f(x)$ starting at the initial point $x_0 = -2$. Newton's method finds local optima rather than minimizing or maximizing, so from the initial starting point, the iterates increase because the nearest local optimum is at $x = 0.21$, and it happens to be local maximum. The inclusion of information from the second derivative enables the algorithm to find local optima instead of looking for only minima or maxima. The points on the plot are labeled in order of iteration number, so x_0 is labeled with a 0.



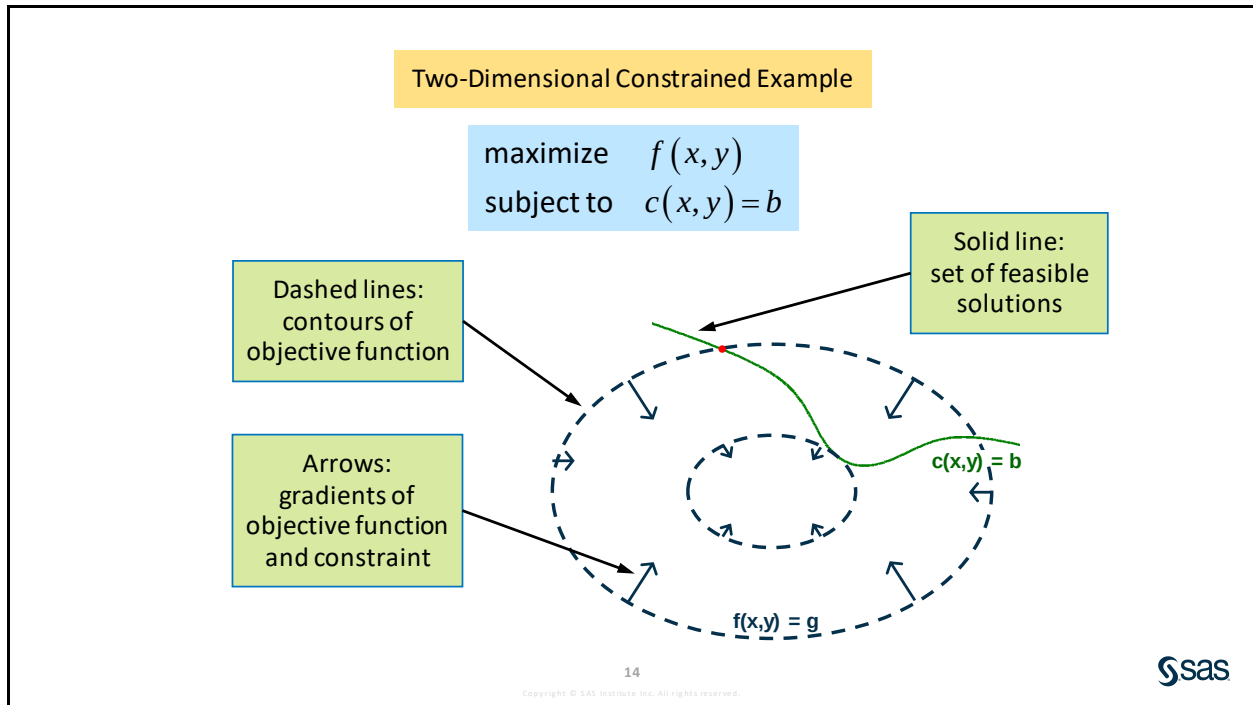
In this example, local optima are found in the function $f(x)$ starting at the initial point $x_0 = 5$. In this case, the iterates move toward the local minimum that we found earlier at $x = 3.12$. Notice that the iterates get closer together as the iteration number increases. This is because as we approach the local minimum, the derivative is shrinking (really the ratio of the derivative to the second derivative), so the steps get smaller. Evidently this algorithm has not yet converged.

The previous examples of gradient descent and Newton's method are demonstrated in a Jupyter Notebook located in on the virtual machine in **Documents** ⇒ **ORVY** ⇒ **GradientDescentAndNewtonsMethod.ipynb**.



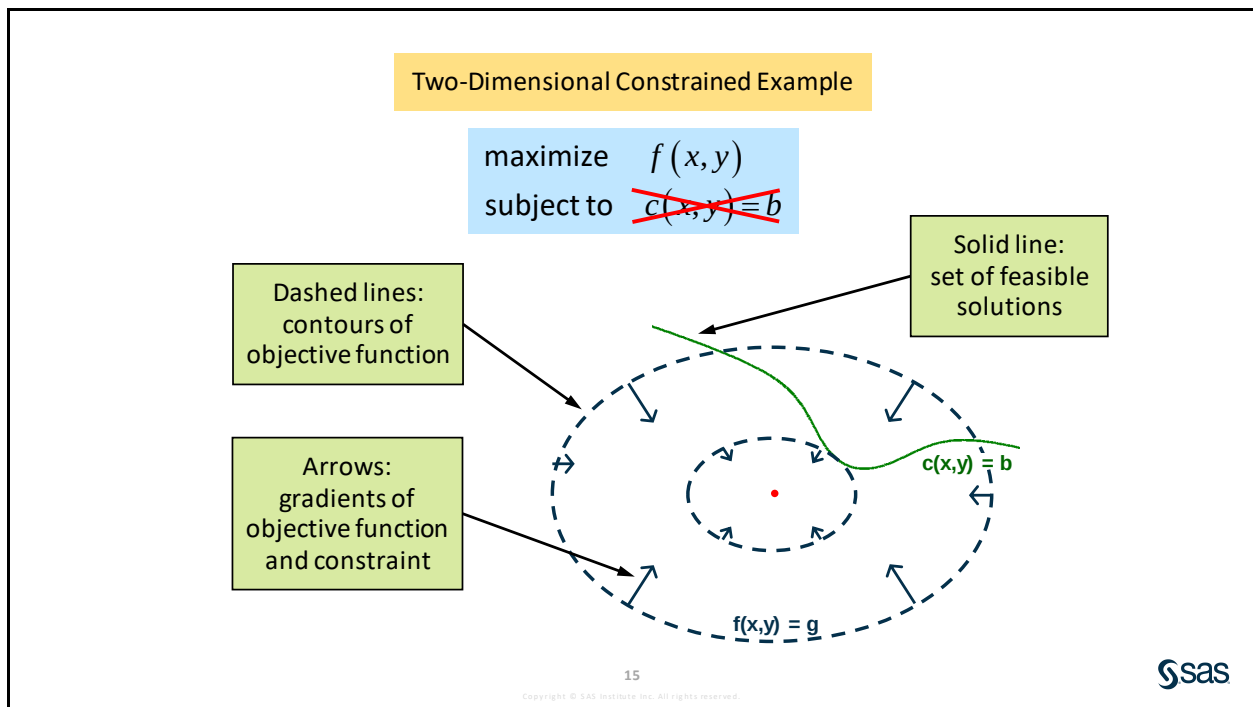
For an unconstrained problem, solvers search for the minimum or bottom of the surface. This is the point where the slope or gradient is zero.

What about constrained NLPs? The Lagrangian function that we described earlier provides the framework for solving constrained NLPs. The next examples show how the constraint multipliers or lambda values can modify the solution to the unconstrained problem.

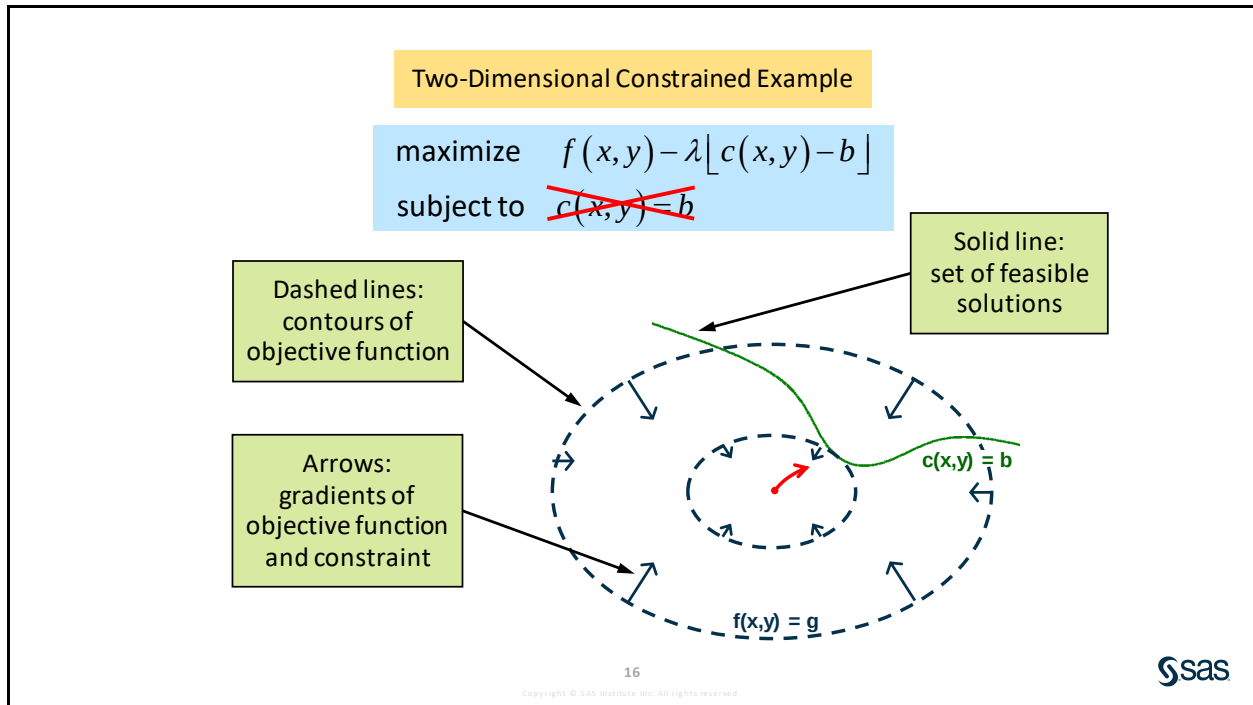


The feasible region in this two-dimensional constrained example lies on and above the green line generated by the constraint, $c(x,y) \geq b$. The arrows are pointing uphill, and their length is proportional to the steepness of the slope in that region of objective function. The red point is a feasible solution to the constrained problem, but it is not the optimal solution.

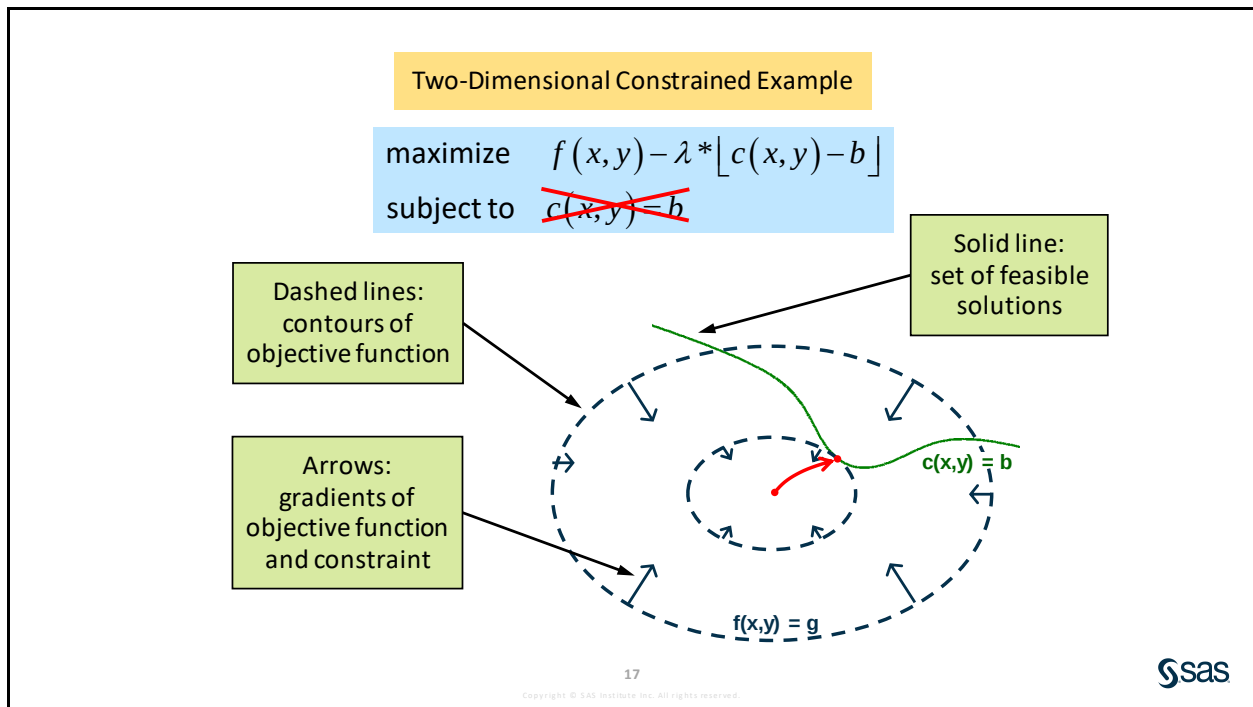
Where is the optimal solution to the constrained problem located?



To answer the question, let's start from the solution to the unconstrained problem. This is the red dot at the "top of the hill," but the value of $c(x, y)$ is less than b at that point, so it is not feasible. One approach to constrained optimization is to penalize those points where the constraint is violated by adding a multiple of the constraint violation to the objective function. What will this do to the optimal solution?



The Lagrangian for the NLP does not change from the form shown in the linear programming problem. The penalty drives the optimized value of the objective function toward the feasible boundary.



For some “magical” value of the penalty parameter, the penalized optimal solution is just feasible to the constraint. The constraint term in the Lagrangian penalizes points that do not satisfy the constraint.

Note: The asterisk indicates the magic of optimality and does not signify multiplication.

Optimality Conditions:

For multipliers $\lambda = [\lambda_1, \dots, \lambda_m]$, the *Lagrangian function*

$$L(x, \lambda) = f(x) - \sum_{i=1}^m \lambda_i [c_i(x) - b_i]$$

can be used to state optimality conditions for NLPs.

Necessary Conditions:

An optimal solution $\mathbf{x}^*$ must satisfy

$$\begin{aligned} \nabla L(x^*, \lambda^*) &= 0 \\ \lambda_i^* [c_i(x^*) - b_i] &= 0 \quad (i = 1, 2, \dots, m) \end{aligned}$$

for some *Lagrange multipliers* λ^* .

Optimality conditions for the Lagrangian function are derived as a function of both the decision variables, $\mathbf{x}$, and the λ . Those points where the partial derivative of the Lagrangian with respect to an individual λ_i is zero satisfy the corresponding constraint, $c_i(\mathbf{x})=b_i$. Recall that the optimality conditions result in a system of equations, and that the value of the penalty is found by solving this system of equations for the decision variables and the penalty.

The second necessary condition is also known as the *complementarity condition*. This condition ensures that the Lagrangian and objective have identical optimized values for the decision variables. Intuitively, the complementarity condition indicates that either the constraint is satisfied with equality (or *tight*) or the penalty term is zero. Penalties should be just enough to force the constraints to be satisfied. This condition is also known as *complimentary slackness*.

Optimality Conditions:

For multipliers $\lambda = [\lambda_1, \dots, \lambda_m]$, the *Lagrangian function*

$$L(x, \lambda) = f(x) - \sum_{i=1}^m \lambda_i [c_i(x) - b_i]$$

can be used to state optimality conditions for NLPs.

Necessary Conditions:

An optimal solution $\mathbf{x}^*$ must satisfy

$$\begin{aligned} \nabla L(x^*, \lambda^*) &= 0 \\ \lambda_i^* [c_i(x^*) - b_i] &= 0 \quad (i = 1, 2, \dots, m) \end{aligned}$$

for some *Lagrange multipliers* λ^* .

Inequality constraints: each λ_i must be nonnegative or nonpositive.

19

Copyright © SAS Institute Inc. All rights reserved.



For inequality constraints, each λ_i must be nonnegative or nonpositive, depending on whether the objective sense is *min* or *max*.



Minimizing the Banana Function Using PROC OPTMODEL

orvy03d01.sas

```
proc optmodel;
  var x >= 0, y <= 0;

  min f = (y-x**2)**2 + (1-x)**2;

  solve;

  print x x.dual y y.dual;

quit;
```

PROC OPTMODEL Output

The OPTMODEL Procedure

Problem Summary	
Objective Sense	Minimization
Objective Function	f
Objective Type	Nonlinear
Number of Variables	2
Bounded Above	1
Bounded Below	1
Bounded Below and Above	0
Free	0
Fixed	0
Number of Constraints	0

Solution Summary	
Solver	NLP
Algorithm	Interior Point
Objective Function	f
Solution Status	Optimal
Objective Value	0.2892740938
Optimality Error	5E-7
Infeasibility	0
Iterations	4
Presolve Time	0.00
Solution Time	0.00

x	x.DUAL	y	y.DUAL
0.58975	0.000001571	-9.6298E-07	-0.69562

The true Lagrange multipliers for the constraints $x \geq 0$ and $y \leq 0$ are 0 and -0.69562 .

Are the complementarity conditions satisfied for the solution $[x^*, y^*] = [0.58975, 0]$?

$$0(x^* - 0) = 0(0.58976) = 0$$

$$-0.69562(y^* - 0) = -0.69562(0) = 0$$

What is the Lagrangian function?

$$f - 0(x - 0) - (-0.69562)(y - 0) = f + 0.69562y$$

The necessary conditions imply that up to second-order terms,

$$f(\mathbf{x}^* + \mathbf{dx}) \approx f(\mathbf{x}^*) + \sum_{i=1}^m \lambda_i^* [c_i(\mathbf{x}^* + \mathbf{dx}) - b_i]$$

for any point $\mathbf{x}^*$ that satisfies the gradient and complementarity conditions. This helps explain the sign conditions on the Lagrange multipliers for inequality constraints.

End of Demonstration

$$\lambda_i^* = \frac{\text{Change in optimal objective}}{\text{Unit increase in constraint limit}}$$



Boundaries can now be curved.



21

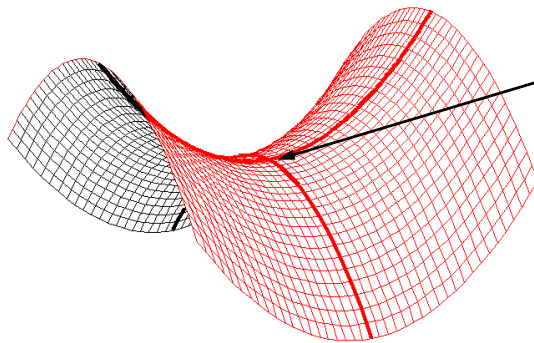
Copyright © SAS Institute Inc. All rights reserved.



Lagrange multipliers provide useful information and can be interpreted as shown. Exercise caution because boundaries can now be curved. That is, interpretations might be accurate only in a small neighborhood of the feasible region.

Second-Order Conditions: The Hessian

$$\nabla^2 f(x^*) \text{ or } \nabla^2 L(x^*, \lambda^*)$$



! unconstrained saddle point:
gradient $\nabla f(x^*) = 0$

22

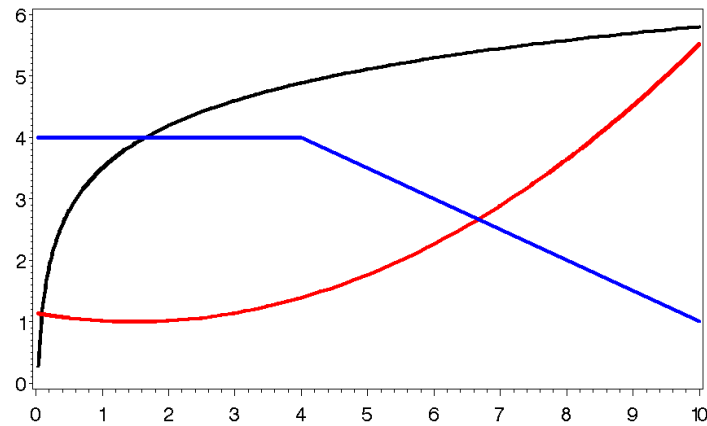
Copyright © SAS Institute Inc. All rights reserved.



Second-order conditions are necessary to avoid saddle points. They involve the Hessian

$\nabla^2 f(x^*)$ or $\nabla^2 L(x^*, \lambda^*)$ or second derivatives of the Lagrangian. Methods that search only for points where the gradient is zero can find non-optimal solutions, even for unconstrained problems. Necessary conditions for the optimality of minimum or maximum value solutions involve properties of the Hessian.

Requirements for Nonlinear Functions

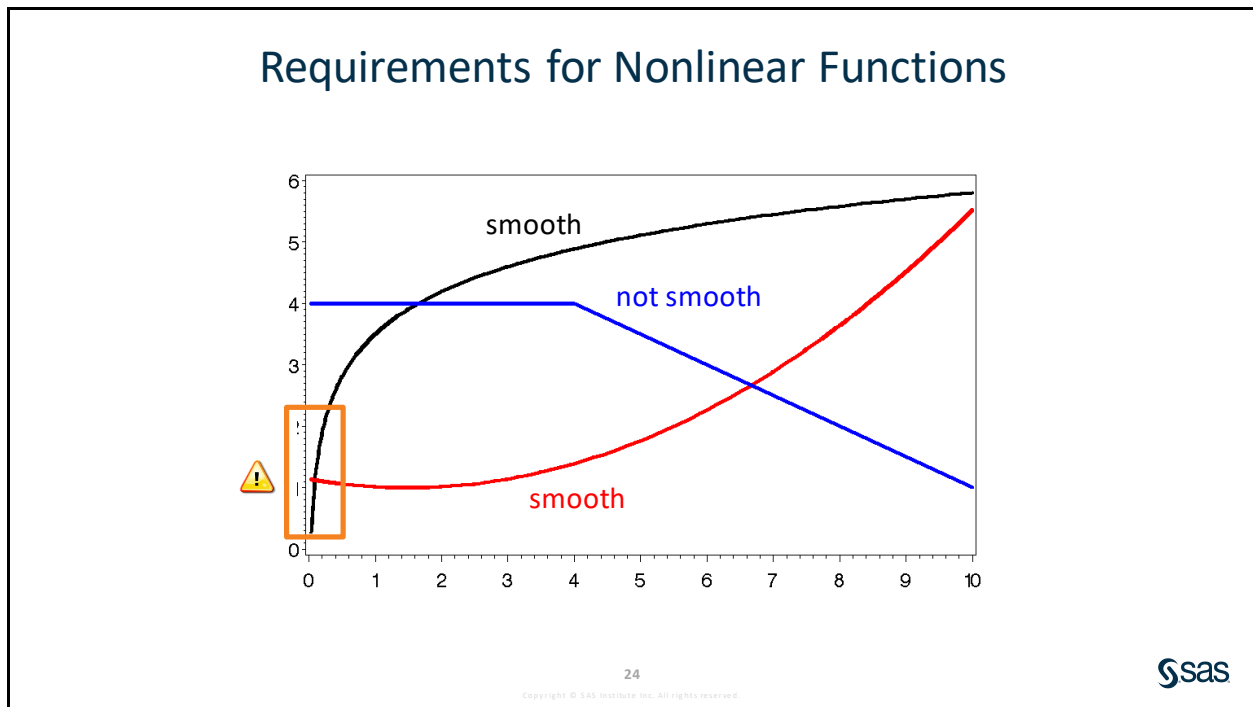


23

Copyright © SAS Institute Inc. All rights reserved.



The solvers approximate general nonlinear functions locally by quadratic functions, so in addition to being continuous, the functions should be **smooth** or **differentiable**.



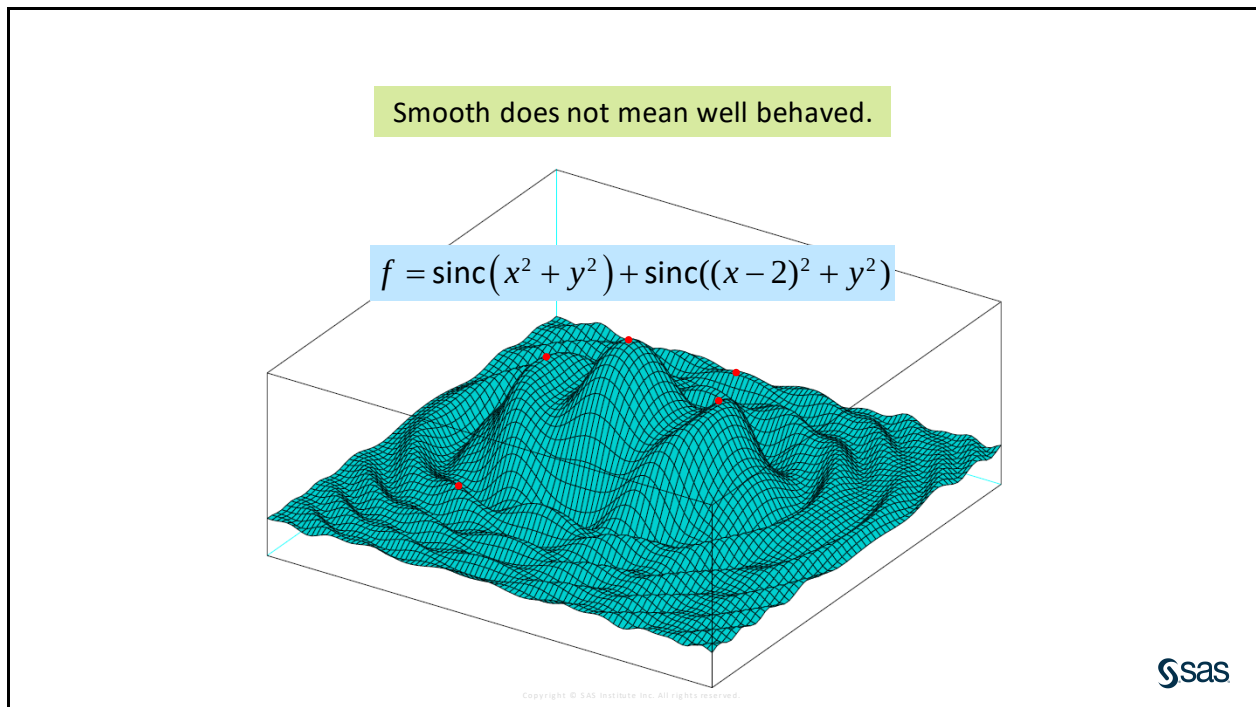
The black (upper curved) line represents a smooth function, but its initial steepness might be an issue. The NLP solvers will not appreciate functions that are poorly defined for some points in the feasible region.

Notes:

- The objective function $f(\mathbf{x})$ should be finite for all points in the feasible region, and in some cases, for all values of the decision variables that satisfy the bound constraints. (One of the functions above could present a problem.)
- The gradient $\nabla f(\mathbf{x})$ should be defined and continuous for all points in the feasible region, as should the Jacobian (the matrix whose rows are the gradients of the constraints).
- The solvers use or approximate the Hessian (the matrix of second partial derivatives of the objective function) and might require it to be continuous.

Computing the gradient takes more CPU time than computing the objective function value. Computing the Hessian matrix can take much more CPU time and memory than computing the gradient, especially when there are many decision variables.

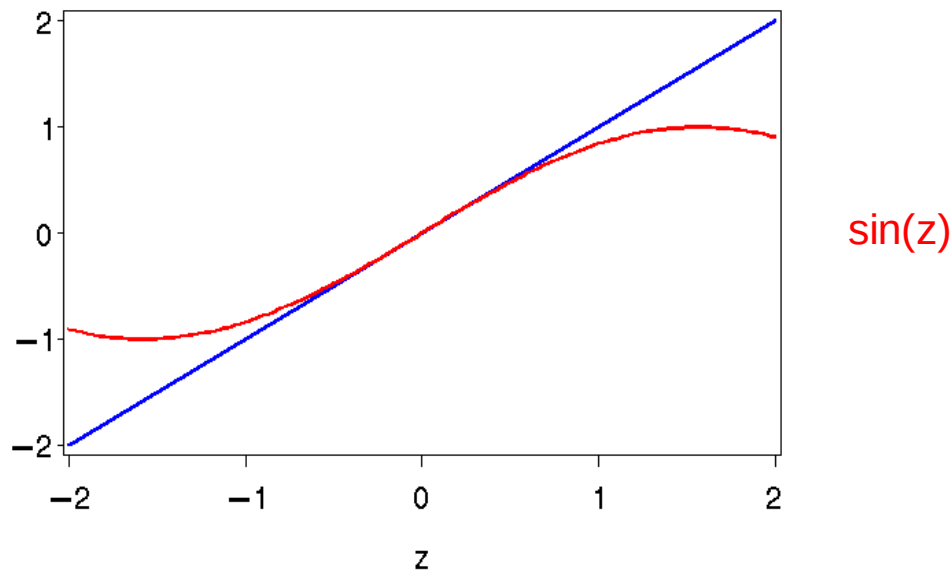
Generally speaking, optimization techniques that do not use the Hessian usually require more iterations than techniques that do use the Hessian. The former tend to be slower and less reliable. However, the techniques that use the Hessian can be prohibitively slow for larger problems.

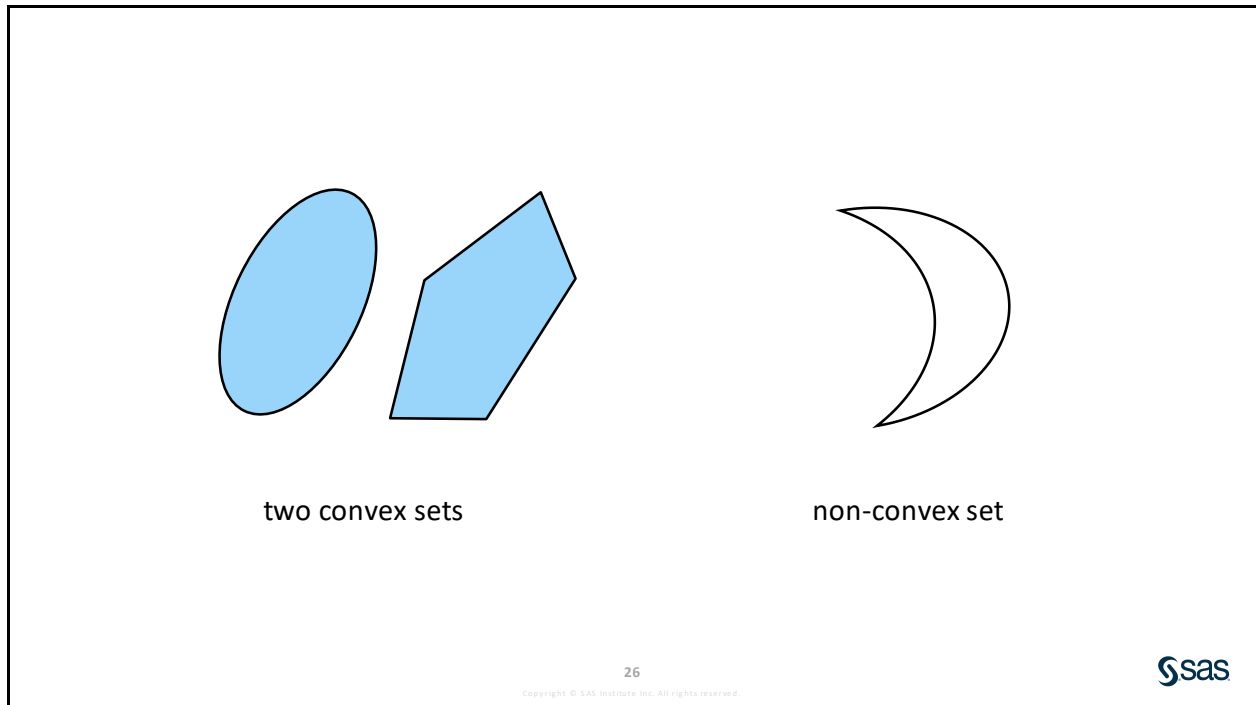


For some nonlinear objectives, there might be more than one maximum or minimum value. This can be a problem, because the optimality conditions can be satisfied at **local** optima.

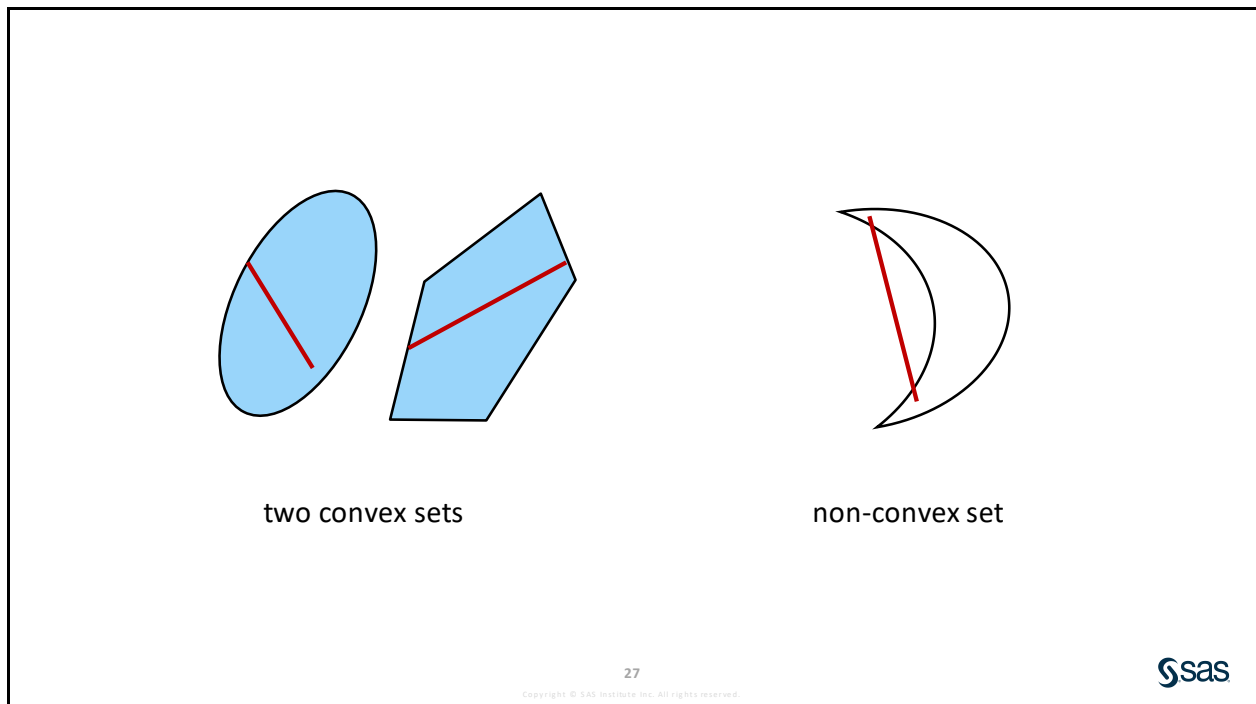
For this function, there is a single global optimal solution, or top of the highest hill, but at least 72 other local, optimal solutions. Where we start our search for a solution can determine which optima we arrive at.

This bound-constrained nonlinear programming problem has a single global maximum but at least 72 other local maxima. The sampling function is defined as $\text{sinc}(0)=1$ and $\text{sinc}(z) = \sin(z)/z$ for $z \neq 0$.





There are conditions that guarantee global optima. If the feasible region is **convex**, the local minimum of a **convex** function (or the local maximum of a **concave** function) is globally optimal.



You can visualize this property by drawing a line between any two points in a set. If all of the points on the line are also in the set, then the set is convex.

Notes:

A set of points is convex if it contains every line that connects two points in the set. In the example with the sinc functions, the feasible region is **convex** (the feasible region is a rectangle because the problem is bound constrained), but the objective function is not **concave**.

Here is a basic fact about convex feasible regions:

- Problems with linear constraints or inequalities $c_i(\mathbf{x}) \leq b_i$ whose body $c_i(\mathbf{x})$ is convex have convex feasible regions.

The ellipsoid on the left could be the feasible region of a problem with a single inequality constraint whose body is a convex quadratic function. The pentagon in the middle could be the feasible region of a problem with five linear inequalities. The crescent on the right could be the feasible region of a problem with two inequality constraints whose bodies are quadratic functions: one convex and the other concave.

Some basic facts about convex functions are as follows:

- $f(x)$ is convex if $f''(x) \geq 0$ (for example, $f(x) = \exp\{ax\}$ is convex).
- If $f(x)$ and $g(x)$ are convex, then so is the sum $f(x) + g(x)$.
- $\mathbf{x}^T \mathbf{P} \mathbf{x}$ is convex if the matrix $\mathbf{P}$ is positive semi-definite.
- Any norm $\|\mathbf{x}\|_p = (\sum_i x_i^p)^{1/p}$ for $p \geq 1$ is convex.
- $f(\mathbf{x})$ is convex if the Hessian is positive semi-definite.
- If $f(\mathbf{y})$ is convex, then so is $f(\mathbf{A}\mathbf{x} + \mathbf{b})$ as a function of $\mathbf{x}$.

Here is all that you ever need to know about concave functions:

- $f(\mathbf{x})$ is concave if and only if $-f(\mathbf{x})$ is convex.



Solving the Sinc Problem Using PROC OPTMODEL

orvy03d02.sas

```
proc optmodel;
  /* declare variables */
  var x >= -4 <= 6 init 0;
  var y >= -5 <= 5 init 0;

  /* declare objective */
  max SincQuad = if (x = 0 and y = 0) then
    1 + sin((x-2)**2+y**2)/((x-2)**2+y**2)
  else if (x = 2 and y = 0) then
    sin(x**2+y**2)/(x**2+y**2) + 1
  else sin(x**2+y**2)/(x**2+y**2) +
    sin((x-2)**2+y**2)/((x-2)**2+y**2);

  solve;

  print x x.dual y y.dual;

  /* alternate initial point */
  x = 0.275;
  y = 4.937;

  solve;

  print x x.dual y y.dual;

quit;
```

Notice that IF-THEN/ELSE statements are used to define the objective function in case $[x,y] = [0,0]$ or $[2,0]$, where the limiting value is used instead of the formula to prevent division by zero.

Using the analytic (limiting) formula yields $\text{sinc}'(0) = 0$ and $\text{sinc}''(0) = -1/3$. Replacing $\text{sinc}(z)$ by 1 at $z = 0$ should not adversely affect the NLP algorithms.

PROC OPTMODEL Output (omitting the second problem summary)

The OPTMODEL Procedure

Problem Summary	
Objective Sense	Maximization
Objective Function	SincQuad
Objective Type	Nonlinear
Number of Variables	2
Bounded Above	0
Bounded Below	0
Bounded Below and Above	2
Free	0
Fixed	0
Number of Constraints	0

Solution Summary	
Solver	NLP
Algorithm	Interior Point
Objective Function	SincQuad
Solution Status	Optimal
Objective Value	1.6829419696
Optimality Error	5E-7
Infeasibility	0
Iterations	3
Presolve Time	0.00
Solution Time	0.00

x	x.DUAL	y	y.DUAL
1	-6.6613E-16	1.8597E-20	-2.2403E-20

Solution Summary	
Solver	NLP
Algorithm	Interior Point
Objective Function	SincQuad
Solution Status	Optimal
Objective Value	0.0586579039
Optimality Error	5E-7
Infeasibility	0
Iterations	3
Presolve Time	0.00
Solution Time	0.00

x	x.DUAL	y	y.DUAL
1	-3.8416E-07	5	0.47508

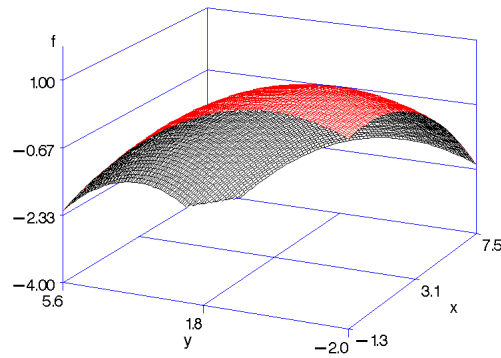
Notice the following:

- The second solution cannot be the global optimal solution (because $0.058658 < 1.682942$).
- The bound $y \leq 5$ is satisfied with equality.
- The corresponding Lagrange multiplier is positive.

End of Demonstration

3.01 Activity

Is the objective function $f(x, y) = 1 - 0.05x$ of the two-dimensional inequality constrained problem concave?

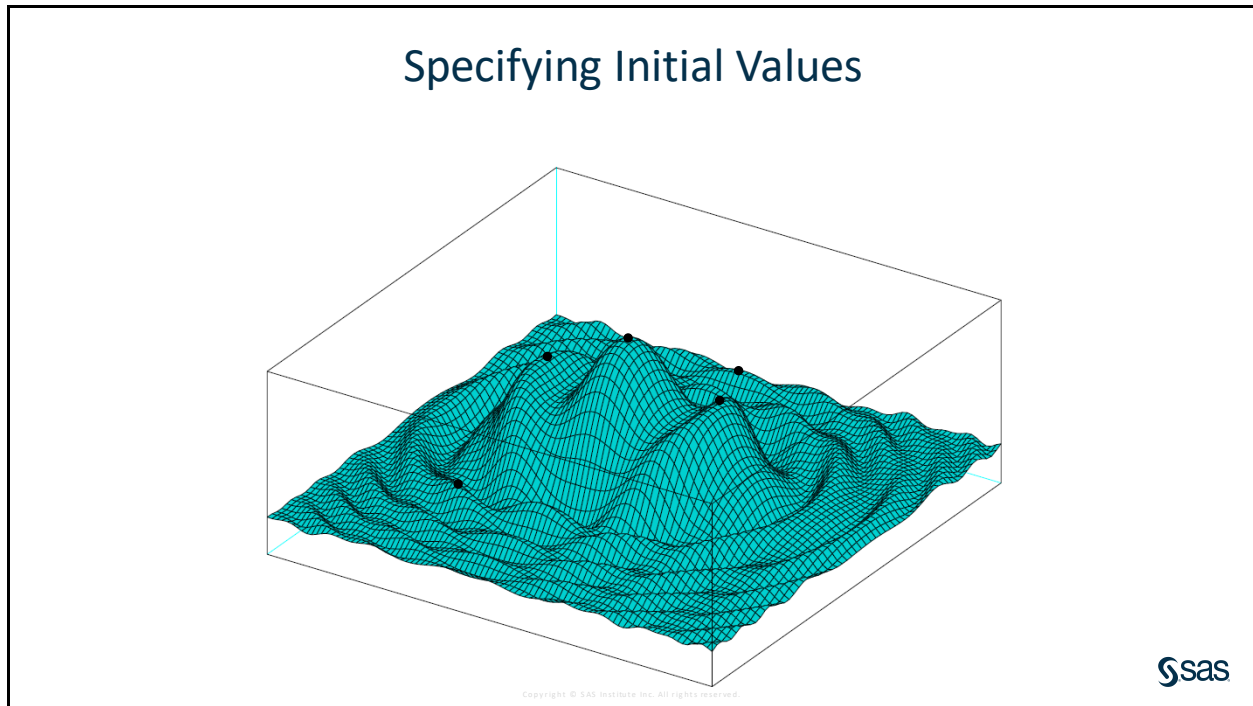


30

Copyright © 2021, SAS Institute Inc. All rights reserved.



MULTISTART Options



The initial values of the decision variables determine the local optima found by the NLP algorithms. One approach for thwarting problems that arise with multiple local optima is trying multiple starting values. PROC OPTMODEL has an enhanced MULTISTART option for problems with multiple local maxima or minima.

MULTISTART Solver Option



- The best solution is reported as the optimal solution.
- Use the `.msinit` suffix to access the starting point for the optimal solution.
- A separate multistart iteration log is produced.



33
Copyright © SAS Institute Inc. All rights reserved.

If the MULTISTART solver option is specified, the solver randomly samples the feasible region and generates a number of starting points. The points are separated both from each other and from known local optima. The MULTISTART option also takes advantage of multiple threads to execute optimization runs from multiple starting points in parallel.

Suboption	Description
MAXSTARTS=	Maximum number of starting points (default 100)
MAXTIME=	Upper limit in seconds of real time in multistart mode, checked after each completion of the local solver
BNDRANGE=	Maximum range of values for each decision variable, centered at the initial value (default 200)
DISTTOL=	Distance tolerance for distinct local optima (default 1e-6)
LOGLEVEL=	Amount of information in iteration log

Some of the MULTISTART suboptions are shown here. By default, a maximum of 100 starting values is tried. The BNDRANGE= suboption does not override the bounds on the decision variables.



Solving the Sinc Problem Using PROC OPTMODEL MULTISTART Options

orvy03d03.sas

```
proc optmodel;
  /* declare variables */
  var x >= -4 <= 6 ;
  var y >= -5 <= 5 ;

  /* declare objective */
  max SincQuad = if (x = 0 and y = 0) then
                    1 + sin((x-2)**2+y**2)/((x-2)**2+y**2)
  else if (x = 2 and y = 0) then
                    sin(x**2+y**2)/(x**2+y**2) + 1
  else sin(x**2+y**2)/(x**2+y**2) +
        sin((x-2)**2+y**2)/((x-2)**2+y**2) ;

  solve with nlp / multistart=(maxstarts=100) seed=12345;

  print x x.dual y y.dual;

quit;
```

The syntax above is similar to the syntax shown in the previous demonstration. Initial value options in the VARIABLE declaration statements are removed, and MULTISTART options are added to the SOLVE statement.

The optimum found by starting at initial values $x=0$ and $y=0$ in the previous demonstration appears to be the global optimum.

The OPTMODEL Procedure

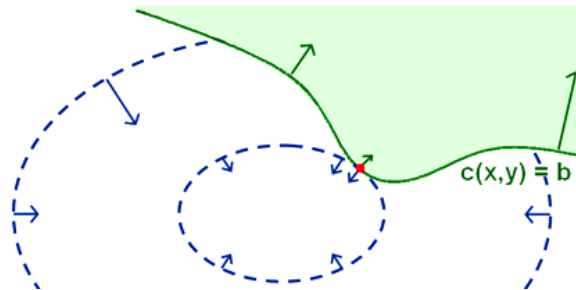
Solution Summary	
Solver	Multistart NLP
Algorithm	Interior Point
Objective Function	SincQuad
Solution Status	Optimal
Objective Value	1.6829419696
Number of Starts	36
Number of Sample Points	1600
Number of Distinct Optima	17
Random Seed Used	12345
Optimality Error	5E-7
Infeasibility	0
Presolve Time	0.00
Solution Time	0.04

x	x.DUAL	y	y.DUAL
1	2.3535E-11	-7.2777E-13	8.7672E-13

End of Demonstration

3.02 Activity

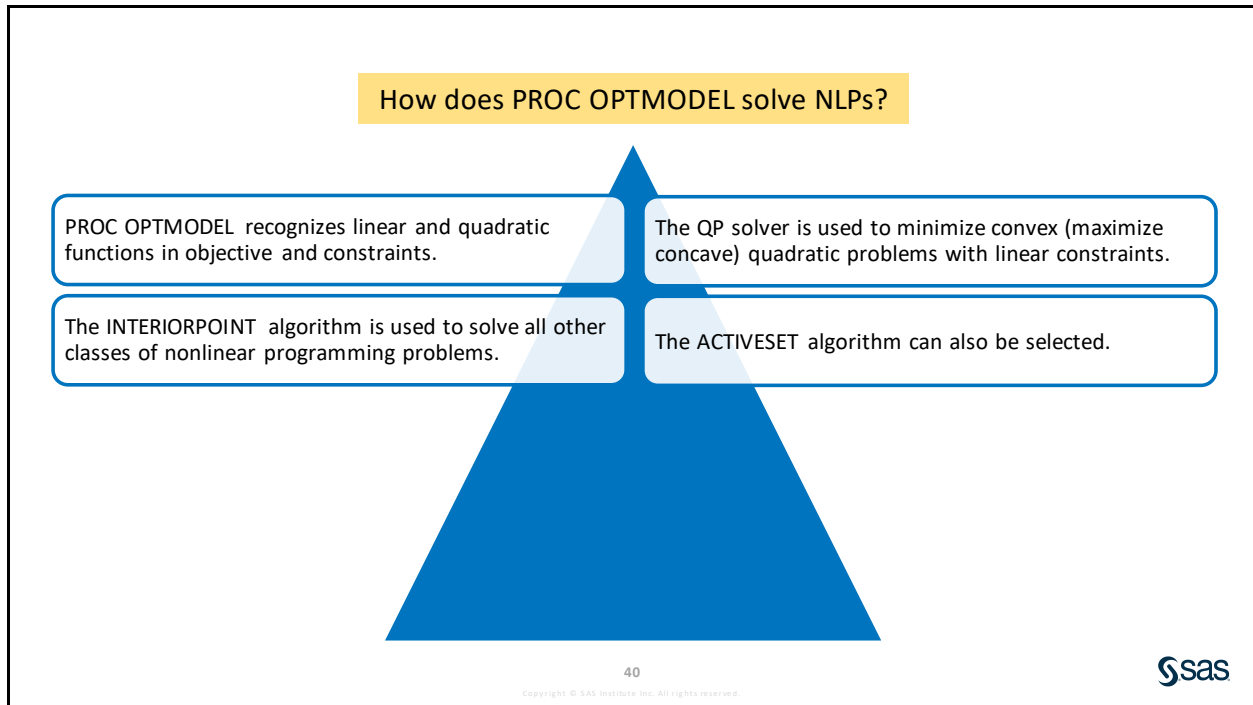
Is the feasible region of the two-dimensional inequality constrained problem convex?



3.2 Solving Nonlinear Programming Problems Using the OPTMODEL Procedure

Objectives

- Solve nonlinear programming problems using the OPTMODEL procedure.
- Interpret the information written to the SAS log during the solution of a nonlinear programming problem.

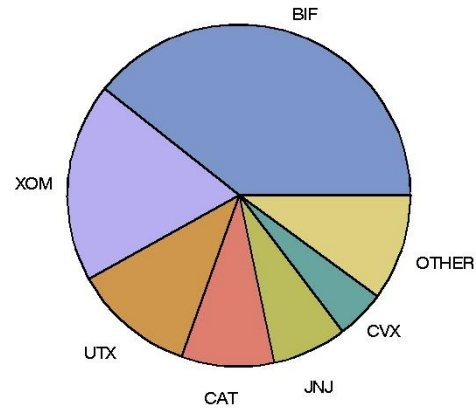


PROC OPTMODEL recognizes linear and quadratic functions in the objective and constraints, and the quadratic programming or QP solver is used under the listed conditions. The INTERIORPOINT algorithm is used to solve all other classes of NLPs by default. The QP solver can be called using SOLVE WITH QP. The INTERIORPOINT and ACTIVESET algorithms are called using SOLVE WITH NLP.

Example: Portfolio Optimization Problem

$$\begin{aligned}
 &\text{maximize} && \sum_{j=1}^n r_j x_j \\
 &\text{subject to} && \sum_{i=1}^n \sum_{j=1}^n x_i \sigma_{ij} x_j \leq R \\
 &&& \sum_{j=1}^n x_j = 1 \\
 &&& x_1 \geq 0, \dots, x_n \geq 0
 \end{aligned}$$

x_j	proportion of stock j
r_j	expected return of stock j
σ_{ij}	covariance between stocks i and j



45

Copyright © SAS Institute Inc. All rights reserved.

sas

The next demonstration formulates the portfolio optimization problem. The idea is to select a portfolio of stocks that maximizes expected return while not exceeding a specified risk threshold, R . Risk is measured by the variance of the portfolio, and this is a function of the variance of individual stocks in the portfolio and their covariances. The formula for calculating the portfolio variance is nonlinear. The decision variables are the proportion of each stock to hold in the portfolio.



Solving a Portfolio Optimization Problem Using PROC OPTMODEL

orvy03d04.sas

Part 1: Formulation Using Index Sets and Arrays

```
%let expected_risk = 200;
proc optmodel;
  /* declare sets and parameters */
  set ASSETS = /MMM AA AXP AIG T BAC BA CAT CVX C/;
  num return {ASSETS} =
    [9.85 10.90 7.32 1.31 5.74 4.58 11.15 16.21 11.92 7.37];
  num covariance {ASSETS, ASSETS} =
[427.77 318.95 170.78 143.20 90.28 105.25 203.89      252.32 108.30 200.50
 318.95 1443.56 291.85 183.88 226.06 219.65   302.50 489.91 242.84 408.71
 170.78 291.85 620.01  241.68 162.42 220.33 269.85 243.93 136.24 413.14
 143.20 183.88 241.68  806.94 136.94 144.73 187.12 192.30 127.08 294.13
  90.28 226.06 162.42  136.94 597.30 109.05 149.69 185.79 108.41 233.04
 105.25 219.65 220.33  144.73 109.05 570.09 125.58 180.80 122.89 383.32
 203.89 302.50 269.85  187.12 149.69 125.58 781.05 280.31 173.51 251.25
 252.32 489.91 243.93  192.30 185.79 180.80 280.31 838.37 220.19 331.84
 108.30 242.84 136.24  127.08 108.41 122.89 173.51 220.19 427.53 175.78
 200.50 408.71 413.14   294.13 233.04 383.32 251.25 331.84 175.78 1358.79]
;

  /* declare variables */
  var Prop {ASSETS} >= 0;

  /* declare constraints */
  con Portfolio: sum {j in ASSETS} Prop[j] = 1;

  con Variance: sum {i in ASSETS, j in ASSETS}
    covariance[i,j] * Prop[i] * Prop[j] <= &expected_risk;

  /* declare objective */
  max ExpectedReturn = sum {j in ASSETS} return[j] * Prop[j];

  solve ;

  print {j in ASSETS: Prop[j]>1e-4} Prop
        {j in ASSETS: Prop[j]>1e-4} return;

  print Variance.ub Variance.dual;

quit;
```


PROC OPTMODEL Output

The OPTMODEL Procedure

Problem Summary	
Objective Sense	Maximization
Objective Function	ExpectedReturn
Objective Type	Linear
Number of Variables	10
Bounded Above	0
Bounded Below	10
Bounded Below and Above	0
Free	0
Fixed	0
Number of Constraints	2
Linear LE ($\leq$)	0
Linear EQ ($=$)	1
Linear GE ($\geq$)	0
Linear Range	0
Nonlinear LE ($\leq$)	1
Nonlinear EQ ($=$)	0
Nonlinear GE ($\geq$)	0
Nonlinear Range	0

Solution Summary	
Solver	NLP
Algorithm	Interior Point
Objective Function	ExpectedReturn
Solution Status	Optimal
Objective Value	8.9805178971
Optimality Error	5E-7
Infeasibility	6.8182815E-8
Iterations	6
Presolve Time	0.00
Solution Time	0.01

[1]	Prop	return
AIG	0.030273	1.31
AXP	0.040525	7.32
BA	0.052205	11.15
BAC	0.129930	4.58
CAT	0.018218	16.21
CVX	0.301072	11.92
MMM	0.274334	9.85
T	0.153440	5.74

Variance.UB	Variance.DUAL
200	0.10913

The optimal portfolio contains only eight stocks, and the expected annual return is 8.98. Notice that the NLP solver takes advantage of multiple processors for evaluation.

Information that is put in the log indicates that the problem required six iterations (away from the initial values for the portfolio proportions) to converge to an optimum, and that the Interior Point algorithm is used.

```

OTE: Problem generation will use 4 threads.
NOTE: The problem has 10 variables (0 free, 0 fixed).
NOTE: The problem has 1 linear constraints (0 LE, 1 EQ, 0 GE, 0 range).
NOTE: The problem has 10 linear constraint coefficients.
NOTE: The problem has 1 nonlinear constraints (1 LE, 0 EQ, 0 GE, 0 range).
NOTE: The OPTMODEL presolver removed 0 variables, 0 linear constraints, and 0 nonlinear
constraints.
NOTE: Using analytic derivatives for objective.
NOTE: Using analytic derivatives for nonlinear constraints.
NOTE: Using 4 threads for nonlinear evaluation.
NOTE: The NLP solver is called.
NOTE: The Interior Point algorithm is used.
NOTE: Initial point was changed to be interior to bounds.
      Objective      Optimality
      Iter      Value      Infeasibility      Error
      0      0.86350000      0.90000000      0.98630409
      1      10.36744446      9.15491393      9.15491393
      2      9.79981567      12.74348115      12.74348115
      3      9.15519436      0.01377549      0.01377549
      4      8.98725627      0.04483831      0.04483831
      5      8.98051771      0.0000000735571      0.00002692
      6      8.98051790      0.0000000681828      0.0000005000000
NOTE: Optimal.
NOTE: Objective = 8.9805178971.

```

Part 2: Expanding the Instrument List and Reading in Data from In-Memory Tables

In this part of the demonstration, the number of candidate instruments is expanded to 35. Index sets and parameter arrays are declared and then populated using READ DATA statements. Note that the risk threshold has been reduced from 200 to 50.

```

/* Read in data with an expanded number of instruments for the
problem.
*/

libname casuser cas;

data casuser.covariance_matrix;
    set opt.covariance_matrix;
run;

data casuser.expected_returns;
    set opt.expected_returns;
run;

%let expected_risk = 50;
proc optmodel;
    /* declare sets and parameters */
    set <str> ASSETS;
    num return {ASSETS};
    num covariance {ASSETS, ASSETS};

    /* read data from SAS data sets */
    read data casuser.expected_returns into ASSETS=[Name] return;

    read data casuser.covariance_matrix into [Name]
        {j in ASSETS} <covariance[Name,j]=col(j)>;

    /* declare variables */
    var Prop {ASSETS} >= 0;

    /* declare constraints */
    con Portfolio: sum {j in ASSETS} Prop[j] = 1;

    con Variance: sum {i in ASSETS, j in ASSETS}
        covariance[i,j] * Prop[i] * Prop[j] <= &expected_risk;

    /* declare objective */
    max ExpectedReturn = sum {j in ASSETS} return[j] * Prop[j];

    solve;

    print {j in ASSETS: Prop[j]>1e-4} Prop
        {j in ASSETS: Prop[j]>1e-4} return;

```

```

print Variance.ub Variance.dual;

/* write data to SAS data sets */
create data pie_data from [Name]={j in ASSETS: Prop[j]>1e-4}
    Amount=Prop;

quit;

```

PROC OPTMODEL Output

The OPTMODEL Procedure

Problem Summary	
Objective Sense	Maximization
Objective Function	ExpectedReturn
Objective Type	Linear
Number of Variables	35
Bounded Above	0
Bounded Below	35
Bounded Below and Above	0
Free	0
Fixed	0
Number of Constraints	2
Linear LE (<=)	0
Linear EQ (=)	1
Linear GE (>=)	0
Linear Range	0
Nonlinear LE (<=)	1
Nonlinear EQ (=)	0
Nonlinear GE (>=)	0
Nonlinear Range	0

Solution Summary	
Solver	NLP
Algorithm	Interior Point
Objective Function	ExpectedReturn
Solution Status	Optimal
Objective Value	8.3459760344
Optimality Error	5E-7
Infeasibility	7.1128233E-8
Iterations	10
Presolve Time	0.00
Solution Time	0.02

[1]	Prop	return
BIF	0.51139384	4.3147
CAT	0.07007112	16.2121
CVX	0.03424566	11.9166
HPQ	0.02126078	11.4579
IBM	0.01606105	13.3466
JNJ	0.05482988	9.0100
MMM	0.00012661	9.8516
PG	0.00582762	7.2745
TM	0.01617485	10.6640
UTX	0.09108220	15.5724
WMT	0.02552440	9.8381
XOM	0.15328892	11.4612

Variance.UB	Variance.DUAL
50	0.045323

The optimal portfolio for this part of the demonstration contains 12 stocks, and the expected annual return is 8.35.

End of Demonstration

3.03 Activity

Predict (approximately) the expected return if the bound on the variance is decreased (by 10) to 190.

Objective Function	ExpectedReturn
Solution Status	Optimal
Objective Value	8.9805347586
Variance.	Variance.
UB	DUAL
200	0.10913

NLP Solver Iteration Log

LOGFREQ= k

Iteration
number

Objective
value

Infeasibility

Optimality
error

49

Copyright © SAS Institute Inc. All rights reserved.



In addition to the solution summary, the NLP algorithms (INTERIORPOINT and ACTIVESET) produce an iteration log that contains the following: iteration number, objective value, infeasibility, and optimality error. The LOGFREQ= k option controls the frequency of printing to the iteration log (every k iterations). For all NLP solvers, the default value is LOGFREQ=1.

Termination Criteria for the NLP Solver

Solver Option	Description
FEASTOL=	Tolerance for constraint violation at an optimal solution (default 1e-6)
OPTTOL=	Tolerance for Lagrangian gradient norm at an optimal solution (default 1e-6)
OBJLIMIT=	Upper limit on objective for detecting unboundedness (default 1e+20)
MAXITER=	Maximum number of iterations (default 5000)
MAXTIME=	Upper limit in seconds, checked at the end of each iteration (default infinity)

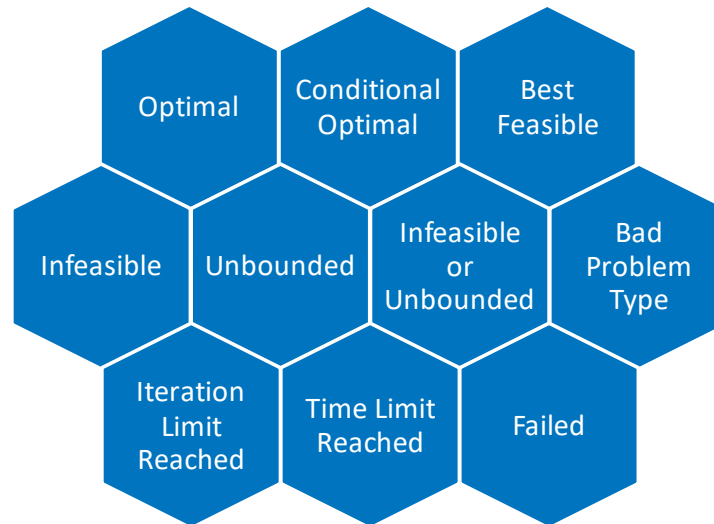
50

Copyright © SAS Institute Inc. All rights reserved.



Both tolerances must be met for the NLP solvers to terminate with an optimal solution.

The solution status on termination is one of the following:



51

Copyright © SAS Institute Inc. All rights reserved.



Conditional optimal means that the optimality of the solution cannot be proven, usually because one of the tolerances is not met. *Best feasible* means that the solver converged to a local optimum. However, a feasible solution that does not satisfy optimality conditions with a better objective value is found and returned as the solution. One possible reason for this is that the solver did not iterate long enough to satisfy optimality conditions at a new, feasible solution. Increasing the MAXITER option might be useful in this situation.

For more details about the status output generated after solving a nonlinear programming problem, see the following documentation:

https://go.documentation.sas.com/?cdclid=pgmsascdc&cdcVersion=9.4_3.5&docsetId=casmopt&docsetTarget=casmopt_nlp_solver_details16.htm&locale=en

Note that the **solution_status** output discussed here is not the only status output produced by the solver.



Practice

1. Finding the Best Angle to Throw a Ball Off a Cliff

Use PROC OPTMODEL to find the best angle at which to throw a ball off a cliff if you want to maximize the horizontal distance that the ball travels. The physics derivation of this problem is presented below, so you will be maximizing the distance **d** traveled by the ball, using the angle **θ** as a decision variable. Solve the following problem:

$$\begin{aligned} \max \quad d(\theta) &= \frac{v^2 \cos \theta \sin \theta}{g} + \frac{v \cos \theta}{g} \sqrt{v^2 \sin^2 \theta + 2gH} \\ \text{subject to} \quad &0 \leq \theta \leq \frac{\pi}{2} \end{aligned}$$

Use **v** = 40 meters/second, **H** = 20 meters, and **g** = 9.8 meters/second² as values for the numerical constants in the problem.

Note that the value of **θ** will, by default, be in radians, so convert the result that you get for **θ** to degrees by multiplying it by 180 and dividing by pi.

End of Practices

Physics Derivation of Solution for Finding the Best Angle to Throw a Ball Off a Cliff

You are standing on a cliff at a height H above the ground. You are throwing a ball off the cliff and want to throw it as far as possible away from the cliff before it hits the ground. You can throw the ball at any angle θ above the horizontal, but you can throw it with only a fixed magnitude velocity v . Assuming no air resistance and gravity g , at what angle θ should you throw the ball so that it lands as far from the cliff base as possible?

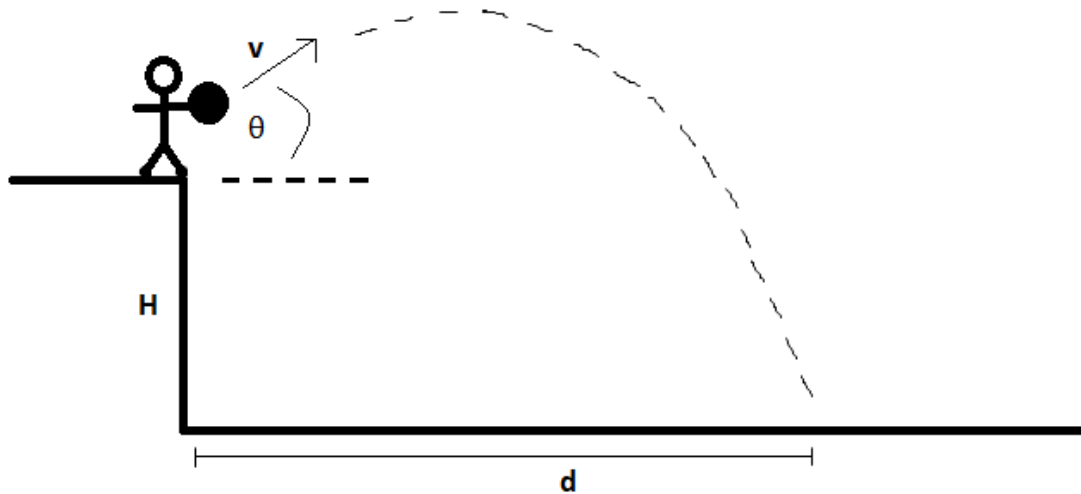


Figure 1: A diagram of the problem. In this case d will be determined as a function of the angle θ , velocity v , and height H .

Recall from kinematics the equation of motion for an object under constant acceleration is given as follows:

$$x(t) = x_0 + v_0 t + \frac{1}{2} a t^2$$

Here x_0 is the initial position of the object, v_0 is the initial velocity of the object, and a is the constant acceleration affecting the object.

For the problem that we are considering, we have two independent dimensions of motion, the horizontal direction $x(t)$ and the vertical direction $y(t)$.

$$x(t) = x_0 + v_{0x} t + \frac{1}{2} a_x t^2$$

$$y(t) = y_0 + v_{0y} t + \frac{1}{2} a_y t^2$$

There is no acceleration in the horizontal direction ($a_x = 0$), but the vertical direction has constant downward acceleration due to gravity, with magnitude g ($a_y = -g$). We will place the origin of our Cartesian coordinate system at the base of the cliff, so $x_0 = 0$ and $y_0 = H$ gives the initial position of the ball. The initial velocity of the ball is split between the x and y dimensions, with some

simple trigonometry revealing that $v_{0x} = v \cos \theta$ and $v_{0y} = v \sin \theta$. Putting all this together, we find the equations of motion for the ball:

$$\begin{aligned} x(t) &= v \cos(\theta)t \\ y(t) &= H + v \sin(\theta)t - \frac{1}{2}gt^2 \end{aligned}$$

These equations give the two-dimensional position of the ball at any time t after it has been thrown. At some point after being thrown, the ball will hit the ground and stop moving. Thus, there is a time T such that $y(T) = 0$, and because the ball stops, the horizontal distance that it has traveled at time T is the maximum distance that it will travel from the cliff base (that is, $x(T) = d$). Applying these conclusions to the equations of motion yield a system of equations that can be solved to eliminate the unknown time T :

$$\begin{aligned} d &= v \cos(\theta)T \\ 0 &= H + v \sin(\theta)T - \frac{1}{2}gT^2 \end{aligned}$$

First we solve the quadratic equation for the value of T as a function of H and θ , and then we plug into the equation for d to get an expression $d(\theta)$ that does not depend on the time T :

$$\begin{aligned} T^2 - \frac{2v \sin(\theta)}{g}T - \frac{2H}{g} &= 0 \\ T &= \frac{1}{2} \left(\frac{2v \sin(\theta)}{g} \pm \sqrt{\frac{4v^2 \sin^2(\theta)}{g^2} + \frac{8H}{g}} \right) \end{aligned}$$

In this analysis the ball was thrown at time $t = 0$, and the ball will hit the ground sometime after it was thrown. Thus, we want to select the positive root from the quadratic equation to ensure that $T > 0$. Plugging this result into the equation for d , we get the expression $d(\theta)$:

$$\begin{aligned} d(\theta) &= \frac{v \cos(\theta)}{2} \left(\frac{2v \sin(\theta)}{g} + \sqrt{\frac{4v^2 \sin^2(\theta)}{g^2} + \frac{8H}{g}} \right) \\ &= \frac{v^2 \cos \theta \sin \theta}{g} + \frac{v \cos \theta}{g} \sqrt{v^2 \sin^2 \theta + 2gH} \end{aligned}$$

We want to maximize the distance the ball travels. Thus, we want to maximize the function $d(\theta)$ with respect to θ . If we could control the velocity v , we would maximize d by maximizing v , but a more interesting problem is finding the best angle to maximize d . An angle of 0 would correspond to throwing the ball horizontally, and an angle of $\frac{\pi}{2}$ would correspond to throwing the ball straight up in the air. We do not want to throw the ball down or behind us. Thus, we restrict values of θ to satisfy the inequality: $0 \leq \theta \leq \frac{\pi}{2}$. We could take a derivative and solve the problem analytically, but we will use SAS Optimization to solve the following nonlinear optimization problem:

$$\begin{aligned} \max \quad & \frac{v^2 \cos \theta \sin \theta}{g} + \frac{v \cos \theta}{g} \sqrt{v^2 \sin^2 \theta + 2gH} \\ \text{subject to} \quad & 0 \leq \theta \leq \frac{\pi}{2} \end{aligned}$$

3.3 Solutions

Solutions to Practices

1. Finding the Best Angle to Throw a Ball Off a Cliff

We use the following SAS program. Note that we used numeric variables to save the constants v , H , and g . We could have hardcoded these values, but a more flexible solution uses variables.

```
proc optmodel;

    num v = 40; /* Magnitude of throwing velocity in meters/second
    */
    num h = 20; /* Height of the cliff in meters */
    num g = 9.8; /* Constant downward acceleration due to gravity
    in meters/second^2 */
    num pi=constant("pi");

    var Theta >= 0 <= pi/2;

    max D = v**2*sin(Theta)*cos(Theta)/g +
    v*cos(Theta)/g*sqrt(v**2*sin(Theta)**2+2*g*h);

    solve;

    print Theta;
    put 'Best angle in degrees is: ' (180*Theta/pi);

quit;
```

The PUT statement writes a line to the log telling us the best angle in degrees.

```
NOTE: Problem generation will use 4 threads.
NOTE: The problem has 1 variables (0 free, 0 fixed).
NOTE: The problem has 0 linear constraints (0 LE, 0 EQ, 0 GE, 0 range).
NOTE: The problem has 0 nonlinear constraints (0 LE, 0 EQ, 0 GE, 0 range).
NOTE: The OPTMODEL presolver removed 0 variables, 0 linear constraints, and 0 nonlinear constraints.
NOTE: Using analytic derivatives for objective.
NOTE: The NLP solver is called.
NOTE: The Interior Point algorithm is used.
NOTE: Initial point was changed to be interior to bounds.
      Objective      Optimality
      Iter      Value      Infeasibility      Error
      0      82.45719644      0      64.71493988
      1      178.10866422      0      0.54470266
      2      182.17068912      0      0.00500000
      3      182.17072330      0      0.00005000
      4      182.17072330      0      0.000005000000
NOTE: Optimal.
NOTE: Objective = 182.1707233.
88
89      print Theta;
90      put 'Best angle in degrees is: ' (180*Theta/pi);
Best angle in degrees is: 41.867376404
```

Running this program returns the following results:

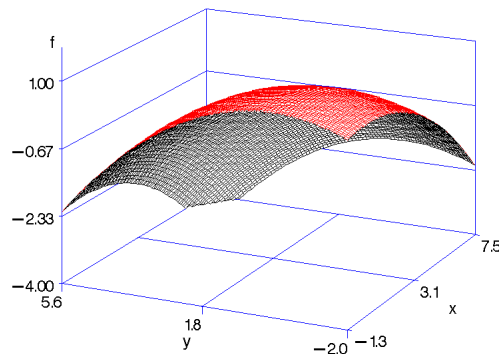
The OPTMODEL Procedure	
Solution Summary	
Solver	NLP
Algorithm	Interior Point
Objective Function	D
Solution Status	Optimal
Objective Value	182.1707233
Optimality Error	5E-7
Infeasibility	0
Iterations	4
Presolve Time	0.00
Solution Time	0.00
Theta	
0.73072	

End of Solutions

Solutions to Activities and Questions

3.01 Activity – Correct Answer

Is the objective function $f(x)$ of the two-dimensional inequality constrained problem concave?



Yes, the Hessian = $\text{diag}(-0.1, -0.2)$ is negative definite.

31

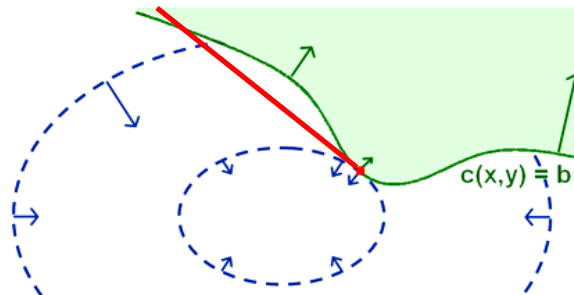
Copyright © SAS Institute Inc. All rights reserved.



The concavity property of the function can be derived from the Hessian. It can also be verified visually. Any points on a line connecting any two points on the surface lie below the surface of this function.

3.02 Activity – Correct Answer

Is the feasible region of the two-dimensional inequality constrained problem convex?



The feasible region is not convex. (See the line above.)

37

Copyright © SAS Institute Inc. All rights reserved.



The two-dimensional inequality constrained problem does **not** satisfy the conditions for a global optima, but it has no other local maxima.

3.03 Activity – Correct Answer

Predict (approximately) the expected return if the bound on the variance is decreased (by 10) to 190.

Objective Function	ExpectedReturn
Solution Status	Optimal
Objective Value	8.9805347586
Variance. UB	Variance. DUAL
200	0.10913

The predicted return is $8.98 - 10 \times 0.10903 = 7.9$.

Lesson 4 Integer and Mixed Integer Linear Programming

4.1	Introduction to Integer and Mixed Integer Linear Programming	4-3
	Demonstration: Solving a Mixed Integer Linear Programming Problem Using the Branch-and-Bound Algorithm	4-15
	Demonstration: Solving a Mixed Integer Linear Programming Problem Step-by-Step (Self-Study).....	4-17
	Practice.....	4-23
4.2	Solving Integer and Mixed Integer Linear Programming Problems Using PROC OPTMODEL.....	4-24
	Demonstration: Solving an Integer Linear Diet Problem Using PROC OPTMODEL.....	4-27
4.3	Solutions	4-35
	Solutions to Practices	4-35
	Solutions to Activities and Questions.....	4-37

4.1 Introduction to Integer and Mixed Integer Linear Programming

Objectives

- Describe, conceptually and geometrically, the formulation of integer and mixed integer linear programming problems.
- Illustrate the branch and bound algorithm for solving these problems.

3

Copyright © SAS Institute Inc. All rights reserved.



Mixed Integer Linear Programming Problems

$$\begin{array}{ll} \min \mid \max & f_1x_1 + \dots + f_nx_n \\ \text{subject to} & Ax \{ \leq, =, \geq \} b \\ & l_j \leq x_j \leq u_j \quad (j = 1, 2, \dots, n) \\ & x_j \text{ integer } (j \in S) \end{array}$$

4

Copyright © SAS Institute Inc. All rights reserved.



The specification of the general optimization problem for integer and mixed linear programming problems extends the linear programming problem that we saw in a previous lesson. Recall that the objective function is a linear combination of the decision variables, and each linear constraint can be either an inequality or an equation. The mixed integer linear programming problem, or MILP, adds the integer restriction on some subset of the decision variable values. That is, a mixed integer linear programming problem (MILP) has both continuous and integer variables.

Integer Linear Programming Problems

$$\begin{array}{ll} \min \mid \max & f_1x_1 + \dots + f_nx_n \\ \text{subject to} & Ax \{ \leq, =, \geq \} b \\ & l_j \leq x_j \leq u_j \quad (j = 1, 2, \dots, n) \\ & x_j \text{ integer} \quad (j = 1, 2, \dots, n) \end{array}$$

The difference between the MILP and an integer linear programming problem, or ILP, is that in an ILP, all of the variables are integers.

Branch-and-Bound Algorithm

```

maximize 6x + 5y
subject to 2x + 3y ≤ 7
           2x - y ≤ 2
           x ≥ 0, y ≥ 0
           x integer, y integer

```

6

Copyright © SAS Institute Inc. All rights reserved.



We are going to use an algorithm called branch-and-bound to solve integer constrained problems. The approach will be illustrated with this two-dimensional example. Both decision variables, x and y , are integer, so this is an ILP.

Branch-and-Bound Algorithm

```

maximize 6x + 5y
subject to 2x + 3y ≤ 7
           2x - y ≤ 2
           x ≥ 0, y ≥ 0
           x integer, y integer

```

linear constraints
and bounds

Feasible region of the linear
programming (LP) relaxation

7

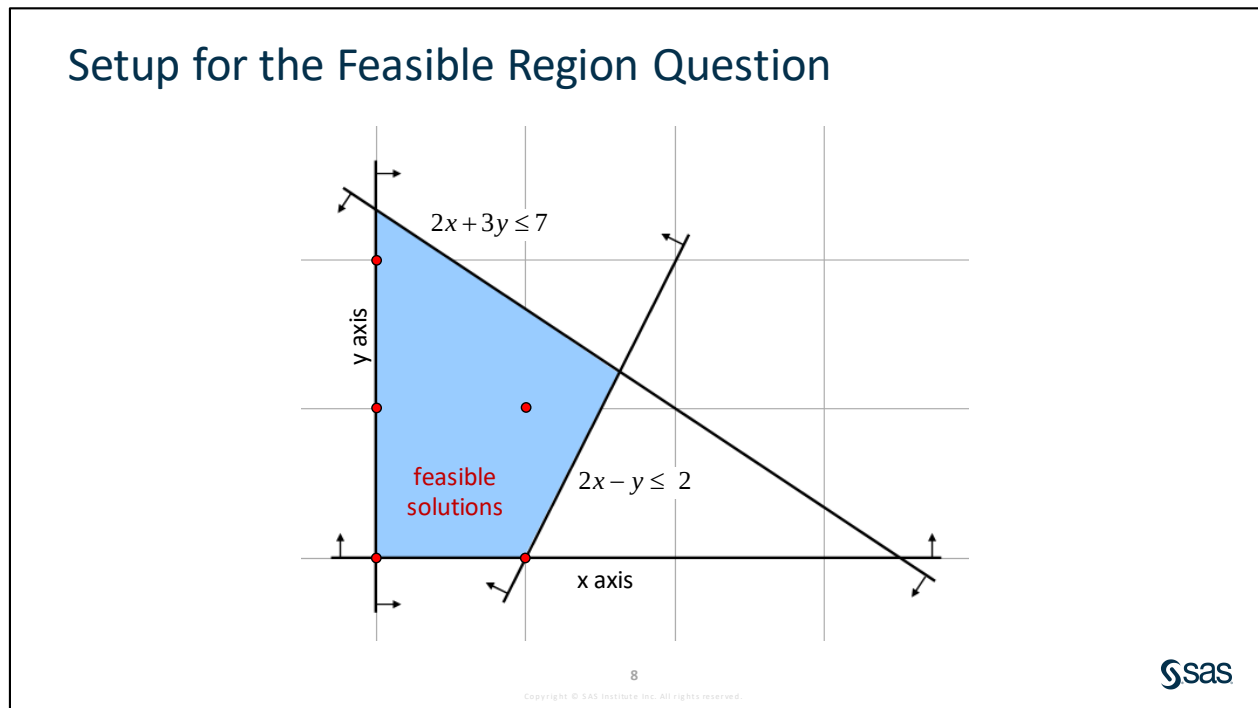
Copyright © SAS Institute Inc. All rights reserved.



The implementation of branch-and-bound begins by focusing only on the linear constraints and bounds, and the integer restrictions are ignored to start. The linear constraints and bounds determine a feasible region in two dimensions, and in this context, it is called the *linear programming relaxation*.

Note: The integer restricted variables are also referred to as *integral variables*.

In the figure below, the scale is 1 square = 1 unit.



The feasible solutions to the ILP are the five integer points $[0,0]$, $[1,0]$, $[0,1]$, $[1,1]$, and $[0,2]$. The shaded region is the feasible region of the LP relaxation. (The integrality of x and y is relaxed.) To solve the ILP, reduce the feasible region step by step until you find an integer solution.

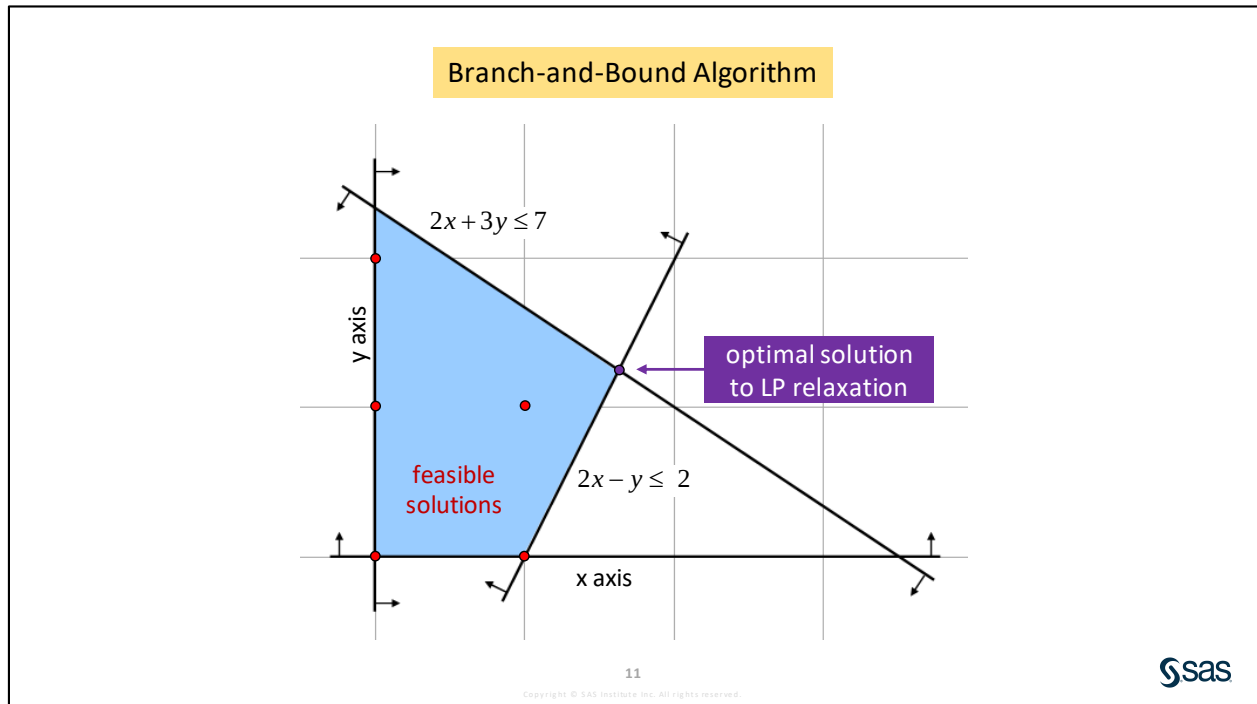
4.01 Multiple Choice Question

Which of the five feasible points is the optimal solution to the ILP with the objective to maximize $6x + 5y$?

- a. $[x,y] = [0,0]$
- b. $[x,y] = [1,0]$
- c. $[x,y] = [0,1]$
- d. $[x,y] = [1,1]$
- e. $[x,y] = [0,2]$

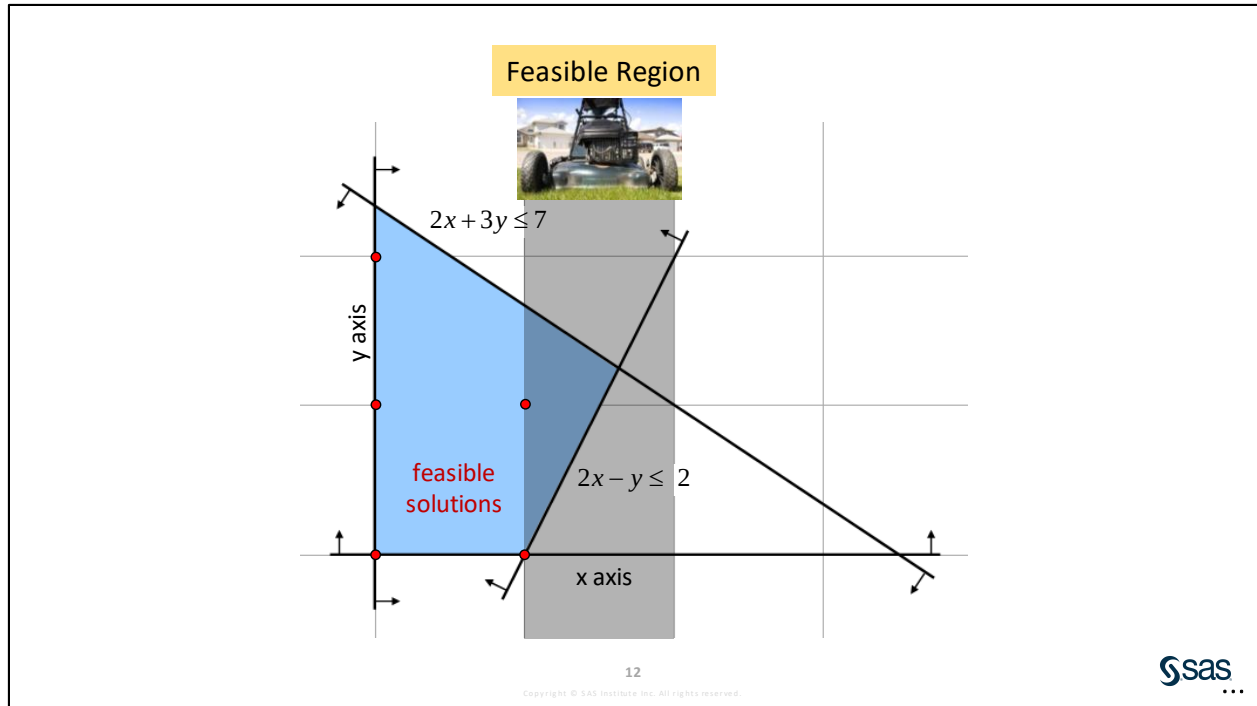
Branch-and-Bound Algorithm

The slides below illustrate how the branch-and-bound algorithm solves the two-dimensional integer-linear programming problem that was introduced above.

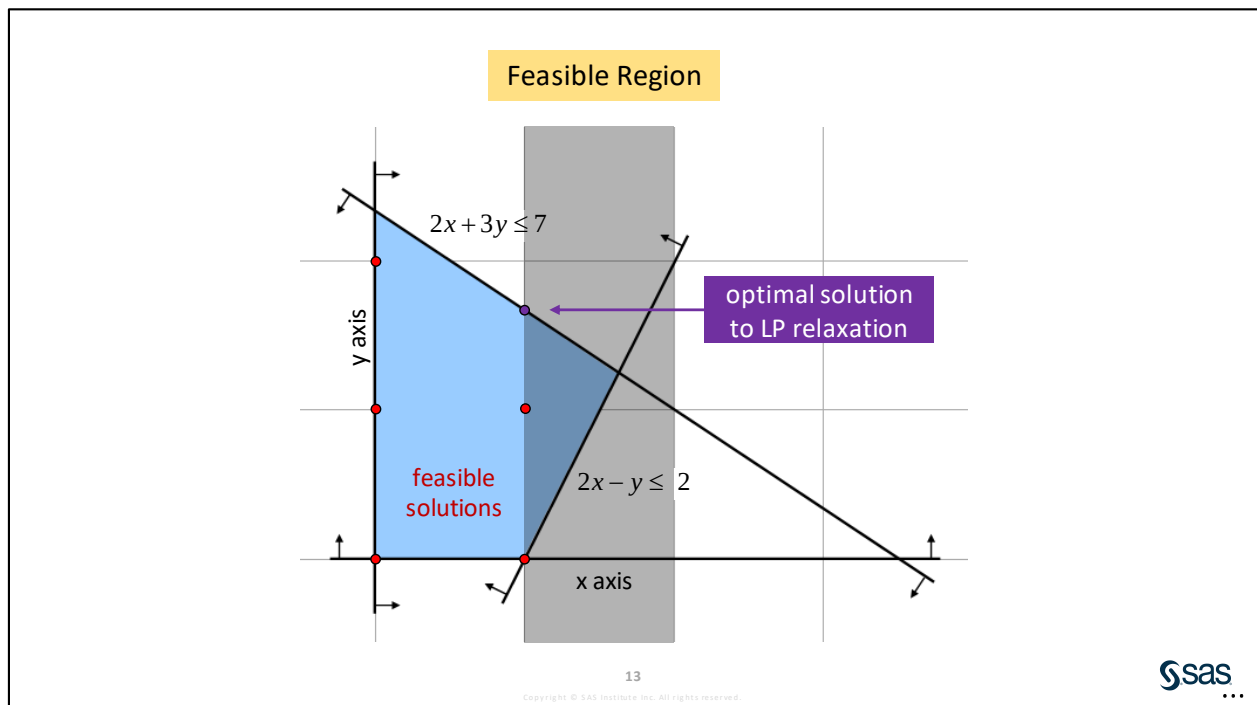


The optimal solution to the LP relaxation is $[x^*, y^*] = [1\frac{5}{8}, 1\frac{1}{4}]$. If both optimized values were integer valued, we could stop. The problem would be solved. In this case, neither decision variable has optimized values that are integers, so both are candidate branching variables.

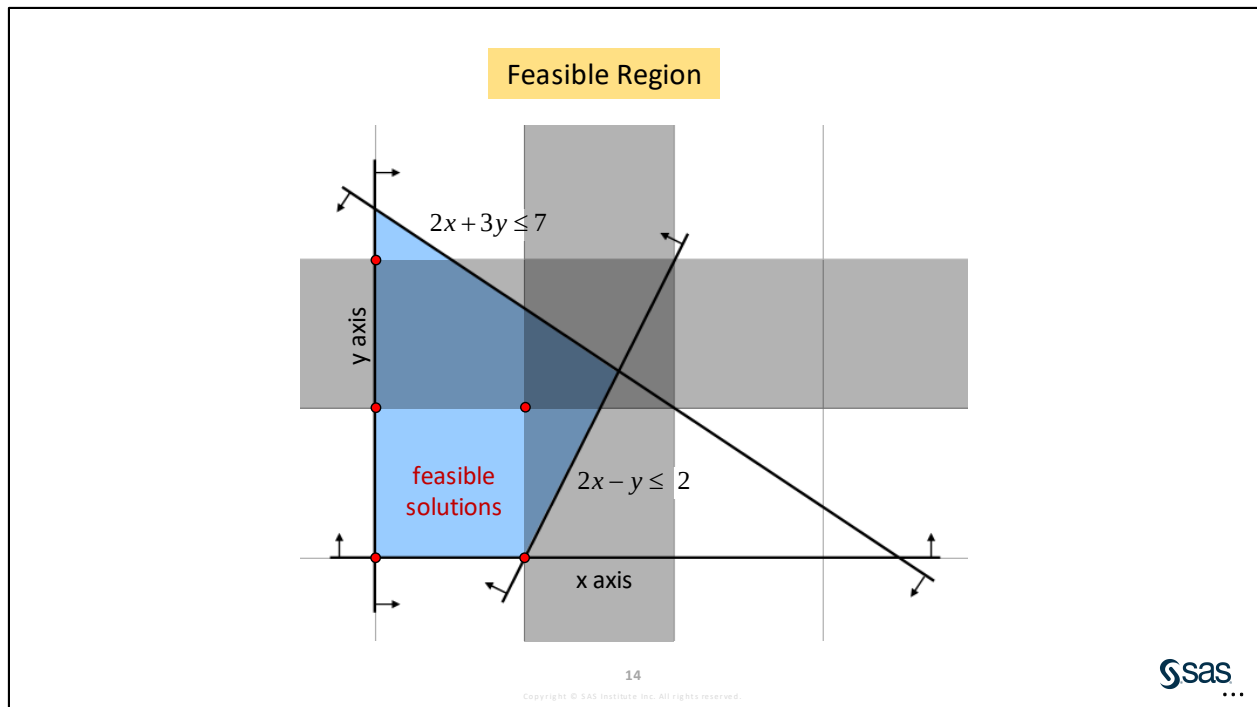
No solution to the ILP can have a fractional value for x , so you can remove the points with $1 < x < 2$ from the feasible region.



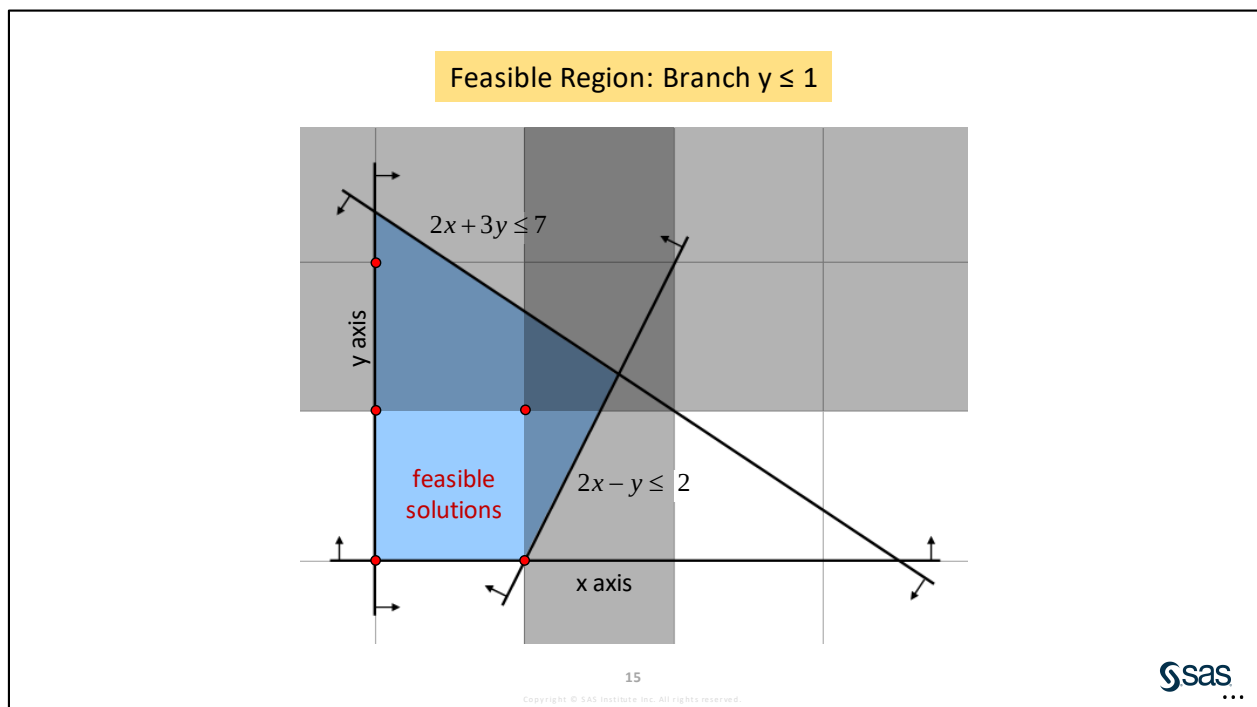
There are no points in the feasible region with $x \geq 2$, so add the bound $x \leq 1$ to the LP and reoptimize.



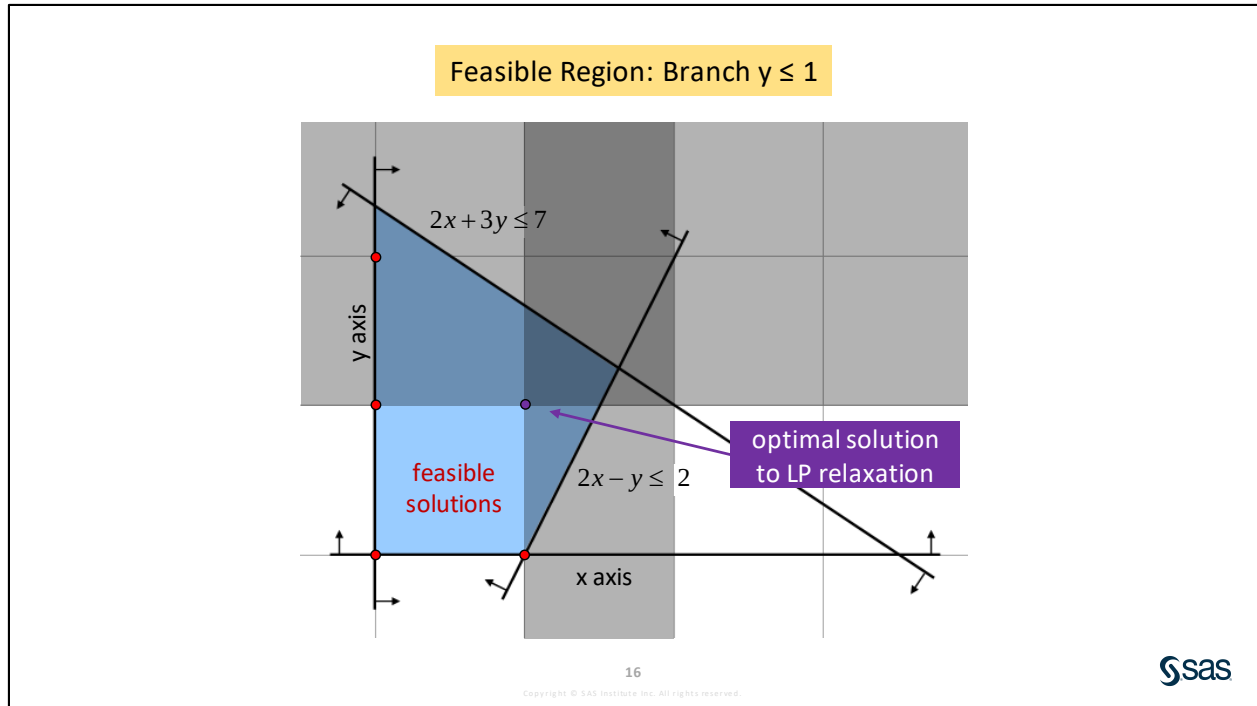
The optimal solution to the LP relaxation is $[x^*, y^*] = [1, 1\frac{2}{3}]$. Now, y becomes the branching variable, and we explore values where $y \leq 1$ and $y \geq 2$. Remove the points with $1 < y < 2$ from the feasible region.



The feasible region consists of two disconnected pieces. You *branch* on the alternatives $y \geq 2$ and $y \leq 1$.

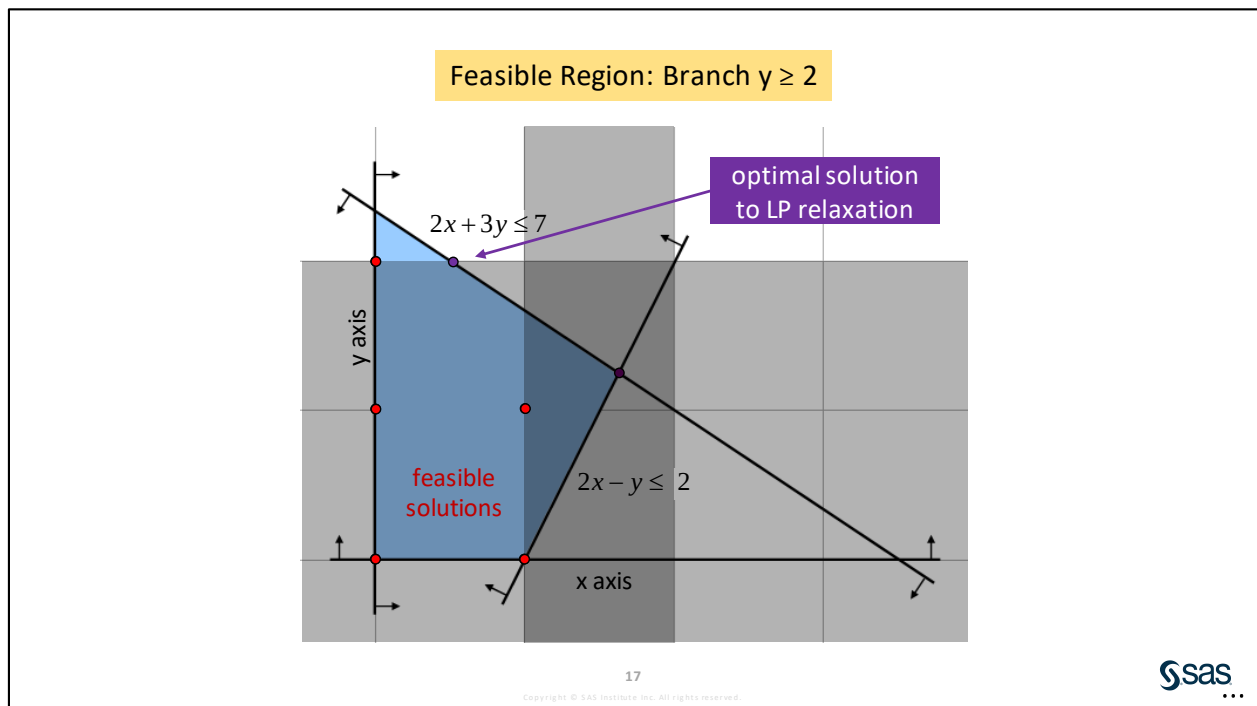


To explore the branch where $y \leq 1$, add the bound $y \leq 1$ to the LP and reoptimize.

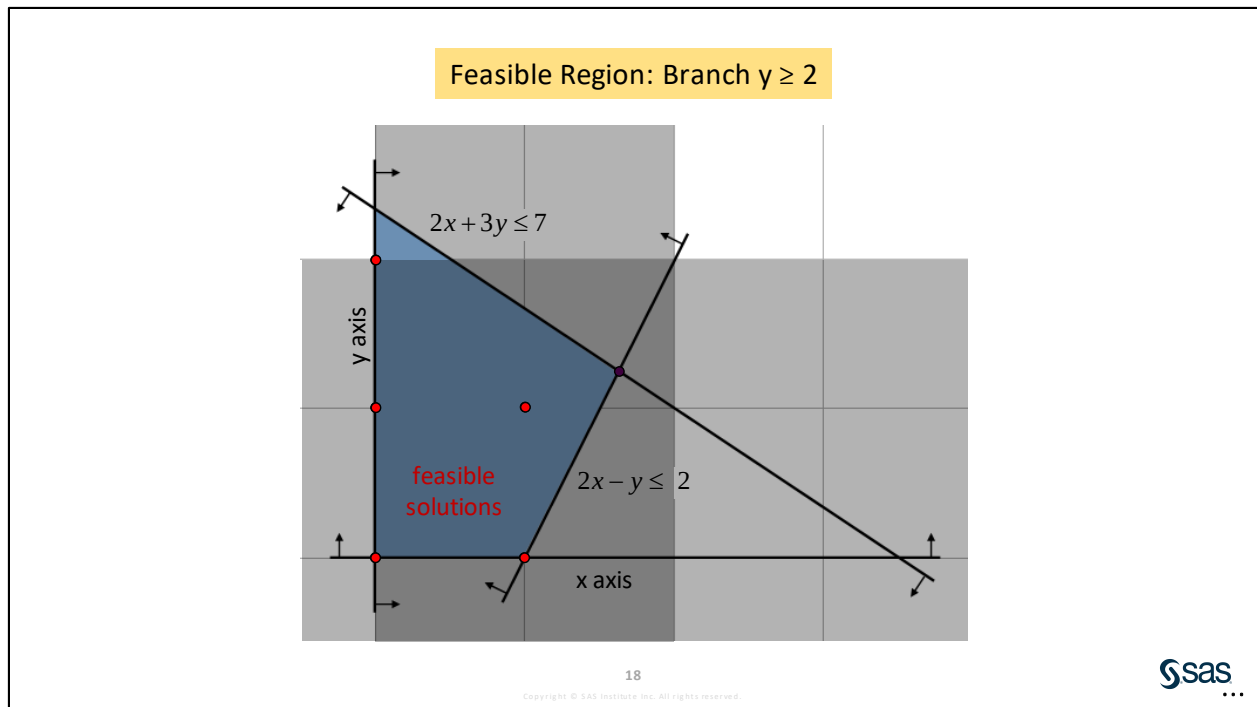


The optimal solution to the LP relaxation is the integral solution $[x^*, y^*] = [1, 1]$ with objective value 11. This objective value is known as **BestInteger**, and it is the best candidate solution so far.

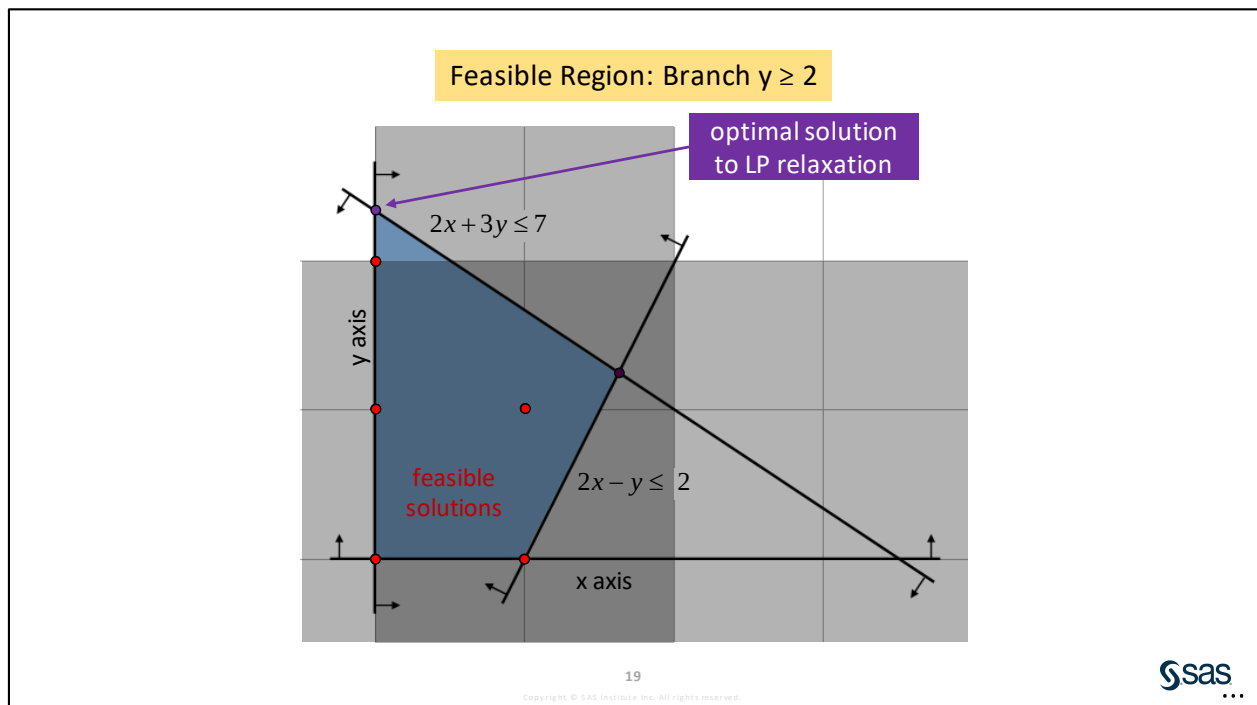
There might be a better solution in the other piece, so replace the bound $y \leq 1$ with $y \geq 2$ and reoptimize.



The optimal solution to the LP relaxation is $x=1/2, y=2$, which yields an objective value of 13. No solution to the ILP can have fractional values for x , so remove points with $0 < x < 1$ from the feasible region.



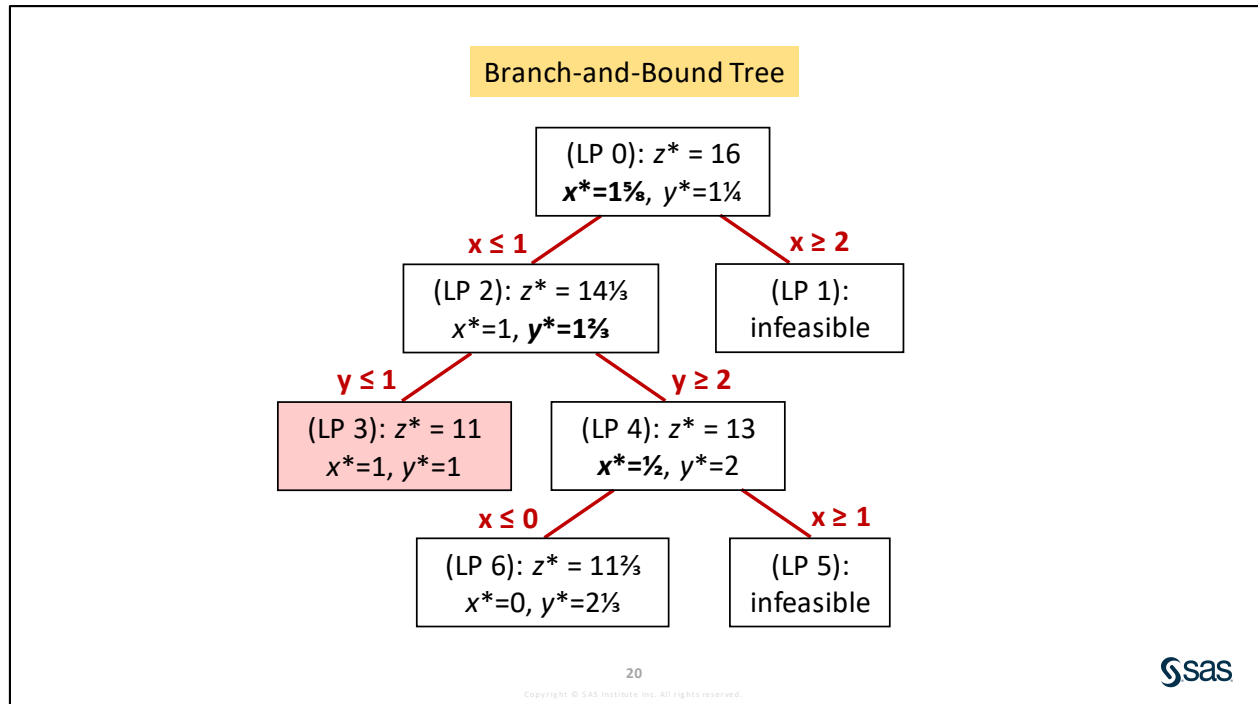
There are no points in the feasible region with $x \geq 1$, so add the bound $x \leq 0$ to the LP and reoptimize.



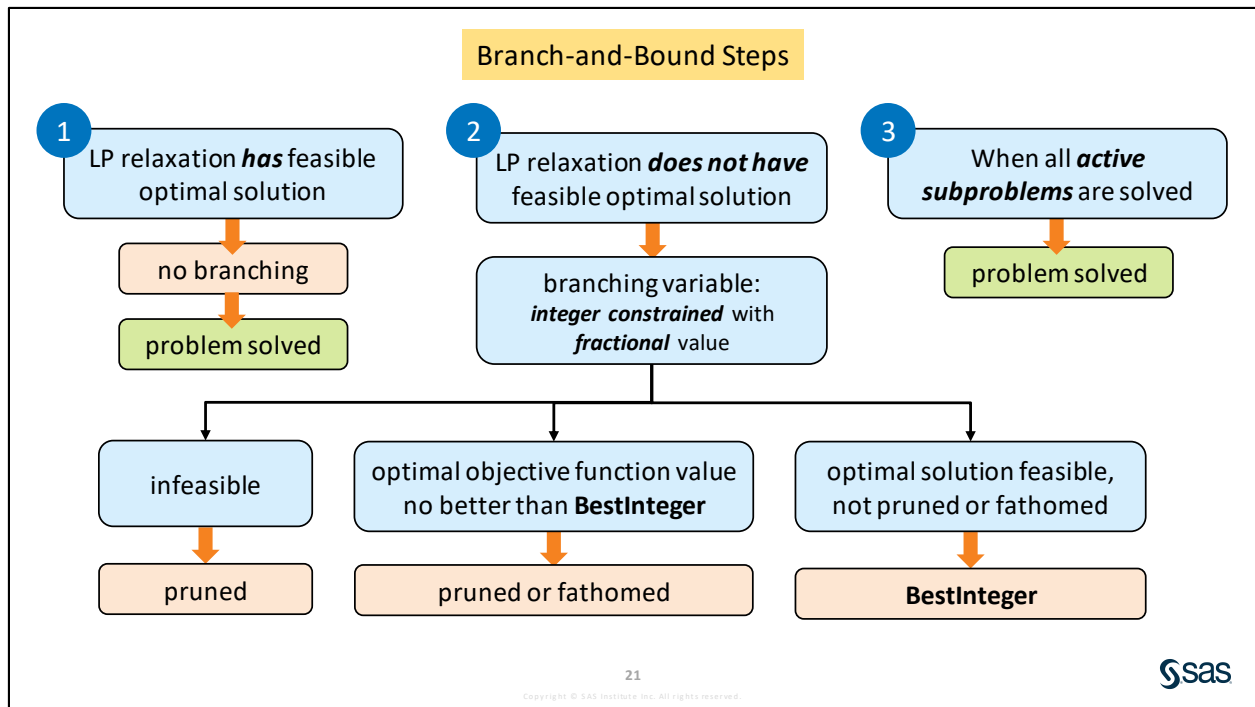
The optimal solution is $x=0, y=2 \frac{1}{3}$, which yields an objective value of $11 \frac{2}{3}$. The feasible integer solution in this branch does not have an objective value better than 11, which is **BestInteger**. The candidate feasible solution that we found earlier is in fact optimal, and the branch-and-bound algorithm terminates.

Note: This example might appear to over-complicate the search for a solution to the integer linear programming problem. Why not solve the problem by simply enumerating all of the feasible integer solutions? In normal applications, enumeration is, at best, a relatively inefficient way to find optimal integer solutions. For additional details, see https://janders.eecg.utoronto.ca/1387/readings/b_and_b.pdf.

Note: in the slides above, the red dots are the feasible integer solutions.



The branch-and bound steps to the solution of the two-dimensional ILP are summarized here. Note that we did not explore (LP1) or (LP5) because visual inspection established that the feasible regions were empty.



The application of the branch-and-bound algorithm proceeds in a stepwise way.

Step 1: If the LP relaxation of a subproblem has an optimal solution that is feasible to the MILP, then the problem is solved, and no branching is required.

Step 2: If the LP relaxation of a subproblem has no optimal solution that is feasible to the MILP, then a branching variable is chosen. The branching variable can be any integer constrained variable that has a fractional value in the optimal solution to the LP relaxation.

- A branch or subproblem that is infeasible can be pruned.
- A branch or subproblem whose optimal objective function value is no better than **BestInteger** can be pruned or fathomed.
- A branch whose optimal solution is feasible to the MILP, and is not pruned or fathomed, becomes the **BestInteger**.

Step 3: The MILP is solved when all active subproblems, or active nodes, of the branch-and-bound tree are solved.

Note: a subproblem whose optimal objective function value is no better than **BestInteger** (the objective value of a known feasible solution to the MILP) can be pruned (or fathomed). No branching is necessary, because adding constraints cannot improve the optimal objective function value of a problem.

Note: The term *fathom* is an analogy to the marine practice of using a sounding line to determine the depth of the water, which needs only to be greater than the draft of the ship (as a unit, 1 fathom = 6 feet). To fathom a node is to prune it, because its bound is no better than **BestInteger**.

4.02 Activity

$$\begin{aligned} &\text{maximize } 5x_1 + 4x_2 - y_1 - y_2 \\ &\text{subject to } 4x_1 + 6x_2 - 2y_1 \leq 11 \\ &\quad 2x_1 + x_2 - y_2 \leq 4 \\ &\quad x_1 \geq 0 \text{ integer, } x_2 \geq 0 \text{ integer} \\ &\quad 0 \leq y_1 \leq 5, 0 \leq y_2 \leq 5 \end{aligned}$$

The LP relaxation of the mixed integer linear programming problem has an optimal solution $x_1^*=4.5$, $x_2^*=0$, $y_1^*=3.5$, $y_2^*=5$ with objective value 14.

From this solution, which variable (or variables) could the branch-and-bound algorithm branch on?

22

Copyright © SAS Institute Inc. All rights reserved.



This mixed integer linear programming problem is solved using the branch-and-bound algorithm in the following demonstration.



Solving a Mixed Integer Linear Programming Problem Using the Branch-and-Bound Algorithm

orvy04d01.sas

The model from the activity is specified in the OPTMODEL procedure. Note that two of the four decision variables are integer constrained, and the integer restriction is part of a variable declaration statement in PROC OPTMODEL.

```
proc optmodel;
  /* declare variables */
  var x_1 >= 0 integer, x_2 >= 0 integer;
  var y_1 >= 0 <= 5, y_2 >= 0 <= 5;

  /* declare constraints */
  con 4*x_1 + 6*x_2 - 2*y_1 <= 11;
  con 2*x_1 + x_2 - y_2 <= 4;

  /* declare objective */
  max z = 5*x_1 + 4*x_2 - y_1 - y_2;

  solve;

  print x_1 x_2 y_1 y_2;

quit;
```

OPTMODEL Procedure Output

Problem Summary	
Objective Sense	Maximization
Objective Function	z
Objective Type	Linear
Number of Variables	4
Bounded Above	0
Bounded Below	2
Bounded Below and Above	2
Free	0
Fixed	0
Binary	0
Integer	2
Number of Constraints	2
Linear LE (<=)	2
Linear EQ (=)	0
Linear GE (>=)	0
Linear Range	0
Constraint Coefficients	6

Solution Summary	
Solver	MILP
Algorithm	Branch and Cut
Objective Function	z
Solution Status	Optimal
Objective Value	13.5
Relative Gap	0
Absolute Gap	0
Primal Infeasibility	0
Bound Infeasibility	0
Integer Infeasibility	0
Best Bound	13.5
Nodes	1
Solutions Found	5
Iterations	6
Presolve Time	0.00
Solution Time	0.00

x_1	x_2	y_1	y_2
4	0	2.5	4

PROC OPTMODEL automatically switches to the MILP solver and finds optimized values using the branch-and-cut algorithm.

Note that the relative gap, defined below, is the termination criteria for the MILP solver, and it is a function of the reported objective value and the best bound.

End of Demonstration



Solving a Mixed Integer Linear Programming Problem Step-by-Step (Self-Study)

orvy04d02.sas

This demonstration illustrates the steps in the solution to a MILP using a sequence of linear programming models. Following the branch-and-bound discussion, the starting point is the linear programming relaxation to the MILP solved above.

LP Relaxation

```
proc optmodel;
  /* declare variables */
  var x_1 >= 0, x_2 >= 0;
  var y_1 >= 0 <= 5, y_2 >= 0 <= 5;

  /* declare constraints */
  con 4*x_1 + 6*x_2 - 2*y_1 <= 11;
  con 2*x_1 + x_2 - y_2 <= 4;

  /* declare objective */
  max z = 5*x_1 + 4*x_2 - y_1 - y_2;

  solve;

  print x_1 x_2 y_1 y_2;

quit;
```

Keep in mind that the MILP constrains the decision variables x_1 and x_2 to integer values.

PROC OPTMODEL Output

The OPTMODEL Procedure

Solution Summary	
Solver	LP
Algorithm	Dual Simplex
Objective Function	z
Solution Status	Optimal
Objective Value	14
Primal Infeasibility	7.105427E-15
Dual Infeasibility	0
Bound Infeasibility	0
Iterations	3
Presolve Time	0.00
Solution Time	0.00

x_1	x_2	y_1	y_2
4.5	0	3.5	5

The only branching variable is x_1. (y_1 is continuous and x_2 is currently integer valued.) Two new active subproblems or nodes are created.

Branch on $x_1 \geq 5$.

The syntax below explores the node on the branch where $x_1 \geq 5$.

```
proc optmodel;
  /* declare variables */
  var x_1 >= 5, x_2 >= 0;
  var y_1 >= 0 <= 5, y_2 >= 0 <= 5;

  /* declare objective */
  max z = 5*x_1 + 4*x_2 - y_1 - y_2;

  /* declare constraints */
  con 4*x_1 + 6*x_2 - 2*y_1 <= 11;
  con 2*x_1 + x_2 - y_2 <= 4;

  solve;

  print x_1 x_2 y_1 y_2;

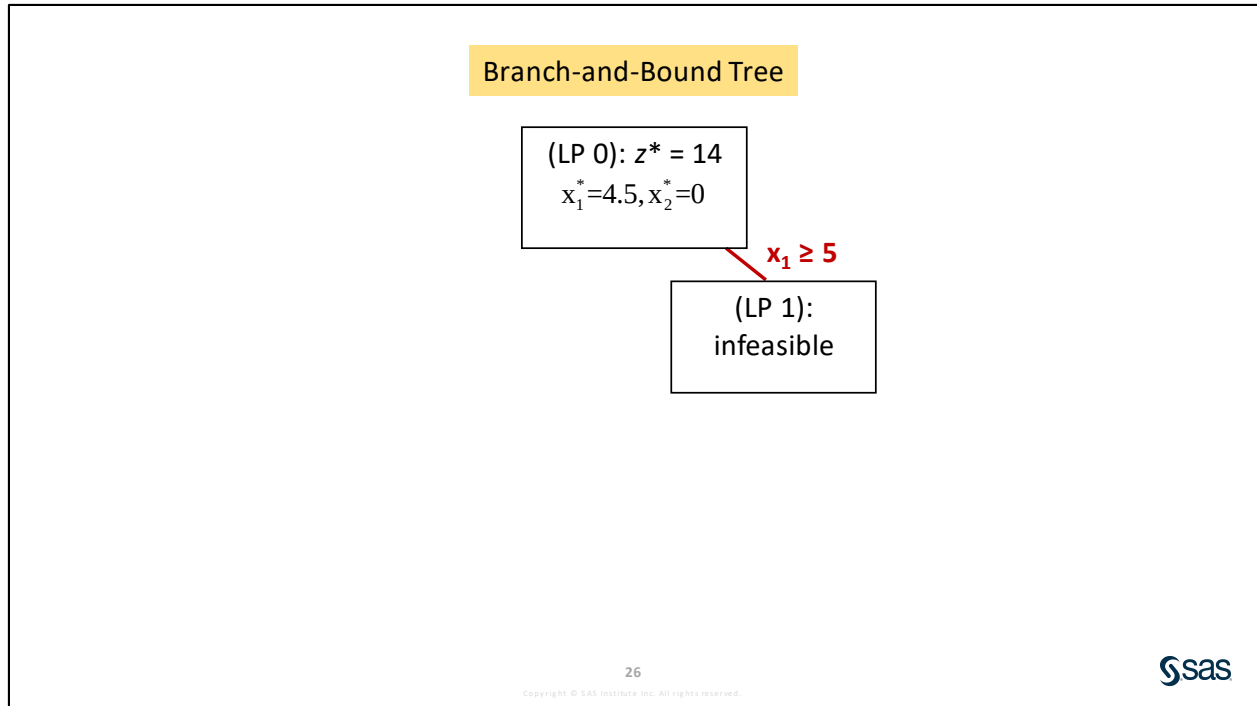
quit;
```

The formulation is infeasible, and this branch can be pruned.

The OPTMODEL Procedure

Solution Summary	
Solver	LP
Algorithm	Dual Simplex
Objective Function	z
Solution Status	Infeasible
Iterations	0
Presolve Time	0.00
Solution Time	0.00

x_1	x_2	y_1	y_2
0	0	0	0



Branch on x_1 ; $x_1 \leq 4$.

The syntax below explores the node on the branch where x_1 is less than or equal to 4.

```

proc optmodel;
  /* declare variables */
  var x_1 >= 0 <= 4, x_2 >= 0;
  var y_1 >= 0 <= 5, y_2 >= 0 <= 5;

  /* declare objective */
  max z = 5*x_1 + 4*x_2 - y_1 - y_2;

  /* declare constraints */
  con 4*x_1 + 6*x_2 - 2*y_1 <= 11;
  con 2*x_1 + x_2 - y_2 <= 4;

  solve;

  print x_1 x_2 y_1 y_2;

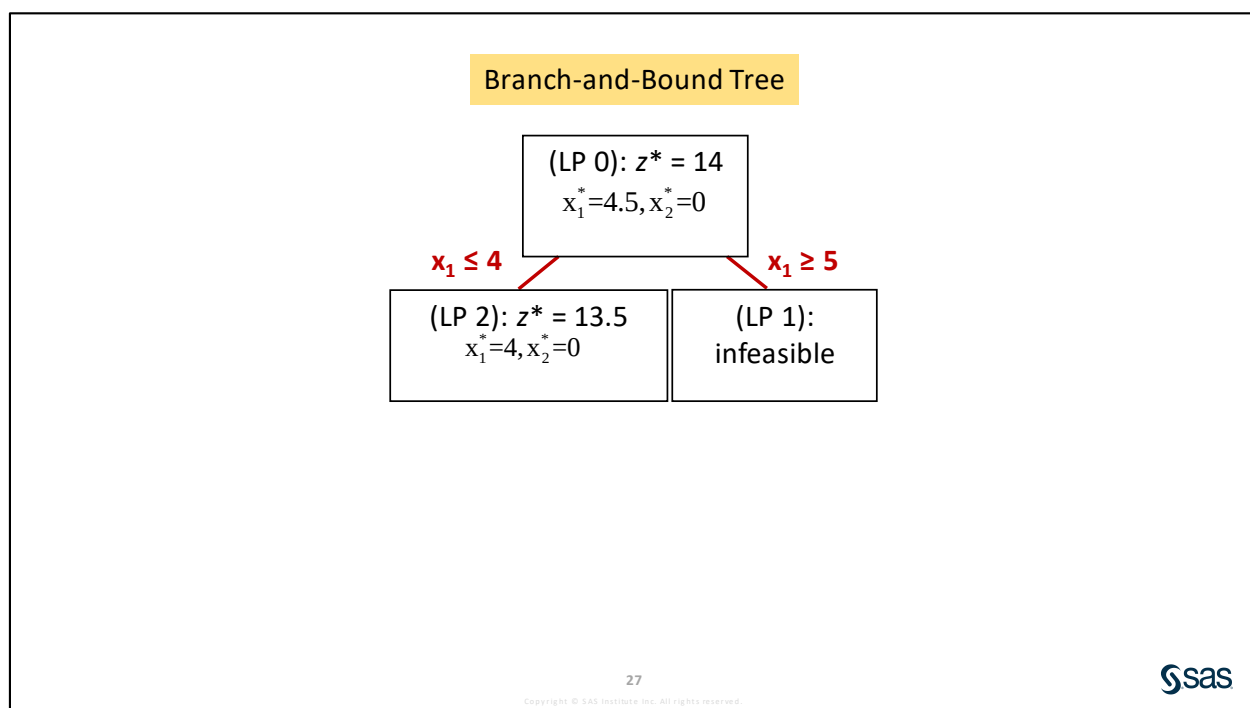
quit;
  
```

PROC OPTMODEL Output

Solution Summary	
Solver	LP
Algorithm	Dual Simplex
Objective Function	z
Solution Status	Optimal
Objective Value	13.5
Primal Infeasibility	0
Dual Infeasibility	0
Bound Infeasibility	0
Iterations	5
Presolve Time	0.00
Solution Time	0.00

x_1	x_2	y_1	y_2
4	0	2.5	4

The results are a feasible solution to the MILP, and the problem is solved.



The branch-and-bound tree illustrates a depth-first search strategy, exploring the right (ceiling) branch first. (The values of the continuous variables y_1 and y_2 are not shown in the branch-and-bound tree.)

Note: The order in which the nodes are explored can influence both the solution time and the optimal solution that is found by the branch-and-bound algorithm.

End of Demonstration



Practice

1. Solving the Furniture-Making Problem as an Integer Linear Programming Problem

Set up and solve the index sets and parameter arrays formulation of the furniture-making problem as an integer linear programming problem.

End of Practices

4.2 Solving Integer and Mixed Integer Linear Programming Problems Using PROC OPTMODEL

Objectives

- Solve integer and mixed integer linear programming problems using the OPTMODEL procedure.
- Interpret information written to the SAS log during the solution of an integer or mixed integer linear programming problem.

29

Copyright © SAS Institute Inc. All rights reserved.



Declaring Integer Variables in PROC OPTMODEL

```
var Floats >= 0 <= 2 integer;  
var AddCap {ARCS} >= 0 integer;  
var Switch {NODES} binary;
```

30

Copyright © SAS Institute Inc. All rights reserved.

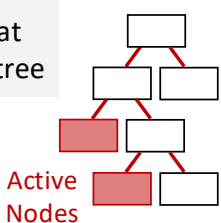


Decision variables can be declared to be either integer variables or binary (0-1) variables. Note that binary is just a special case of the integer restriction. The integer declaration and lower and upper bounds can be specified in any order in the VAR statement.

Termination Criteria for MILPs

$$\frac{|\text{BestInteger} - \text{BestBound}|}{|\text{BestBound}| + 10^{-10}} \leq 10^{-4}$$

BestInteger	Objective value of the best feasible solution found
BestBound	Most optimistic objective value at an active node of the branching tree



31
Copyright © SAS Institute Inc. All rights reserved.



BestInteger, as described in the illustration, is the best objective value associated with a candidate integer solution. You can think of it as the incumbent or baseline to beat as new branches are explored. **BestBound** is the most optimistic objective function value generated by all of the sub-problems currently under consideration.

For example, in a *maximization* problem:

- **BestInteger** is the largest objective value associated with a candidate integer solution (so far in the search). This value forms a lower bound on the solution. Objective values corresponding to other integer solutions must be larger than the **BestInteger** to avoid being eliminated.
- **BestBound** is the largest objective value of all the objective values of the leaf nodes under consideration.
- The relative objective *gap* is the difference between the **BestInteger** (lower bound) and the **BestBound** (upper bound). The MILP solver usually terminates when the relative objective gap falls below a certain value.

Note that branch-and-bound trees grow or split down from the initial LP relaxation.

Self-Study: Termination Criteria for the MILP Solver

Solver Option	Description
ABSOBJGAP=	Absolute difference between BestBound and BestInteger values (default 1e-6)
CUTOFF=	Cut off all nodes with a worse bound
MAXNODES=	Maximum number of nodes in the tree
MAXSOLS=	Maximum number of solutions found
MAXTIME=	Maximum real time (seconds) allowed
RELOBJGAP=	Relative difference (default 1e-4)
TARGET=	Target value for BestInteger

The default value for RELOBJGAP= guarantees optimality to within 0.01% relative error, which might not be realistic for large-scale business problems. The solution status might be reported as Optimal within Relative Gap (or Optimal within Absolute Gap) if the solver terminates by either of the gap criteria.



Solving an Integer Linear Diet Problem Using PROC OPTMODEL

orvy04d03.sas

Part 1

This demonstration finds a minimum cost daily diet from a menu that satisfies a given set of nutrient requirements bounds.

1. The formulation begins with the declaration of index sets.
 - **FOOD** is an index set of foods that might be available on a fast food restaurant menu.
 - **NUTRIENTS** is an index set that forms the basis for setting requirements.
 - **SAUCES** and **DRINKS** are index sets that will be used in ad hoc constraints to refine the diet.
 - **cost** and **content** are numeric arrays that provide the unit cost and nutrient content of menu items.

```
proc optmodel;
  set <str> FOOD = /Hamburger Cheeseburger 'Big Mac'
                  'Filet-O-Fish' 'Chicken McNuggets'
                  'Side Salad' 'Egg McMuffin' 'Barbeque Sauce'
                  'Sweet n Sour Sauce' Honey Milk Soda/;

  set <str> NUTRIENTS = /Cal Fat Salt Carbs Protein/;

  /* Declare sets for ad hoc constraints */
  set <str> SAUCES = /'Barbeque Sauce' 'Sweet n Sour Sauce'
                    Honey/;
  set <str> DRINKS = /Milk Soda/;

  /* Declare parameter arrays */
  num cost{FOOD} = [0.59 0.69 1.84 1.44 1.69 1.79 1.49 0 0 0
                    1.10 0.99];
  num content {NUTRIENTS, FOOD} =
  [255 305 500 370 270 30 280 50 60 45 110 205
   9 13 26 18 15 1 11 .5 .1 .1 10 .1
   490 725 890 730 580 35 710 340 190 .1 130 20
   30 30 42 38 17 4 28 12 14 12 12 55
   12 15 25 14 20 2 18 .1 .1 .1 9 .1]
  ;
```

The syntax below defines parameter arrays that contain nutrient upper and lower bounds. The VAR statement is used to declare the array of decision variables, **Diet** (indexed by **FOOD**).

Note: Menu choices are restricted to integer values.

```

num requirelb{NUTRIENTS} = [2000 10 100 350 50];
num requireub{NUTRIENTS} = [3500 200 3000 475 300];

/* declare decision variables and apply the integrality
   conditions */
var Diet{FOOD} >= 0 integer;

```

2. The constraints require the selected diet to satisfy the listed bounds on nutrient requirements, and they also restrict the quantities of sauces and soft drinks.

```

impvar Intake {i in NUTRIENTS} =
    sum{j in FOOD} content[i,j]*Diet[j];

/* constraint bounds on nutrient intake */
con Bounds{i in NUTRIENTS}:
    requirelb[i] <= Intake[i] <= requireub[i];

/* Limit sauces to two per day*/
con Saucelimit:
    sum{r in SAUCES} Diet[r] <=2;

/* Limit drinks to 3 per day */
con Drinklimit:
    sum{q in DRINKS} Diet[q] <= 3;

```

The following syntax declares the **Total Cost** objective function to be minimized. The final step prints the optimal diet.

```

min Total_Cost = sum{j in FOOD} Cost[j]*Diet[j];
expand;
solve;

print Diet;
run;

```

OPTMODEL Procedure Output**The OPTMODEL Procedure**

Solution Summary	
Solver	MILP
Algorithm	Branch and Cut
Objective Function	Total_Cost
Solution Status	Optimal
Objective Value	7.17
Relative Gap	0
Absolute Gap	0
Primal Infeasibility	0
Bound Infeasibility	0
Integer Infeasibility	0
Best Bound	7.17
Nodes	1
Solutions Found	4
Iterations	61
Presolve Time	0.00
Solution Time	0.01

The MILP solver finds an optimal solution in 61 iterations. The minimum daily cost for the optimal diet is \$7.17.

The diet that optimizes the objective is listed below.

[1]	Diet
Barbeque Sauce	0
Big Mac	1
Cheeseburger	0
Chicken McNuggets	0
Egg McMuffin	0
Filet-O-Fish	0
Hamburger	4
Honey	2
Milk	0
Side Salad	0
Soda	3
Sweet n Sour Sauce	0

Part 2

Expand the diet formulation, add menu items and nutrients, and read the values for index sets and parameter arrays from in-memory tables.

Note: In this demonstration, the tables have been preloaded into a caslib named **PUBLIC**. The caslib that you see in your demo programs or in class might be named differently.

Subsets of some in-memory tables that contain values to populate the index sets and parameter arrays for the formulation are shown below.

1. The **Nutr_req** table is shown below. Nutrient requirements have been modified from the requirements shown in Part 1.

Name	UOM	Min_RDA	Max_RDA
Cal	calories	2000	.
CalFat	calories	.	.
Fat	grams	.	100
SatFat	grams	.	30
Chol	milligrams	.	375
Sodium	milligrams	.	3000
Carbo	grams	350	375
Protein	grams	55	.
VitA	percent	100	.
VitC	percent	100	.
Calcium	percent	100	.
Iron	percent	100	.

Not every nutrient has both an upper and a lower bound. Initializing the **requirelb** and **requireub** parameter arrays when they are declared below is necessary in this formulation to avoid problems with missing values.

2. A sample of the **content_new** (below) table illustrates the “tall and skinny” format for input tables to the READ DATA statement.

	Nutrient	Menu_Item	Content
1	Cal	Hamburger	255
2	Cal	Cheeseburger	305
3	Cal	Quarter Pounder w/ Cheese	510
4	Cal	McLean Deluxe	320
5	Cal	McLean Deluxe w/ Cheese	370
6	Cal	Big Mac	500
7	Cal	Filet-O-Fish	370
8	Cal	McGrilled Chicken	400
9	Cal	McChicken Sandwich	470
10	Cal	Fries, small	220
11	Cal	Fries, large	400
12	Cal	Fries, super	620
...			
62	Cal	H-C Orange Drink (medium)	240
63	Cal	H-C Orange Drink (large)	350
64	CalFat	Hamburger	80
65	CalFat	Cheeseburger	120
66	CalFat	Quarter Pounder w/ Cheese	250
67	CalFat	McLean Deluxe	90
68	CalFat	McLean Deluxe w/ Cheese	125
69	CalFat	Big Mac	235
70	CalFat	Filet-O-Fish	160
71	CalFat	McGrilled Chicken	110
72	CalFat	McChicken Sandwich	225
73	CalFat	Fries, small	110
74	CalFat	Fries, large	200

Note: The “tall and skinny” format is an alternative to the “short and wide” format where, in this example, there would be an individual column for each nutrient. The “short and wide” format would require iterated read column syntax to populate the parameter array, **content** (declared below).

OPTMODEL Syntax

1. Index sets and parameter arrays are declared before being populated in READ DATA statements, as shown below.

```
proc optmodel;
  set <str> FOOD, NUTRIENTS;

  num cost{FOOD};
  num content {NUTRIENTS, FOOD};
  num requirelb{NUTRIENTS} init 0;
  num requireub{NUTRIENTS} init 10000;

  /* declare decision variables and apply the integrality
     conditions */
  var Diet{FOOD} >= 0 INTEGER;
```

Note: The parameter arrays **requirelb** and **requireub** are initialized to 0 and 10000. Why is this a useful thing to do?

2. The index set **NUTRIENTS** is populated from the **Name** variable in the **nutr_req** table. Here, **Name** is the read key column. Parameter arrays for nutrient bounds have names that are different from the corresponding variables (columns) that contain values, so names are mapped.

```
read data public.nutr_req nomiss into NUTRIENTS=[Name]
  requirelb=Min_RDA requireub=Max_RDA;
print requireub requirelb;
```

3. The index set **FOOD** is populated from a **Name** column in the **food_new** in-memory table.

```
read data public.food_new into FOOD=[Name] cost;
```

4. The parameter array **content** is declared to be indexed by both **NUTRIENTS** and **FOOD**. Below, **content** is populated by simply listing the columns in the **content_new** table that correspond to these index sets.

```
read data public.content_new into [Nutrient Menu_Item]
  content;
print content;
```

Note: As shown in previous demonstrations, the iterated read column is one approach to populating arrays with more than one index. An alternative is shown above. This syntax works because the **content_new** table has a “tall and skinny” format. (See above.) The nutrient contents are stacked on top of each other **by Nutrient** and **Menu_Item**.

5. Index sets for ad hoc constraints and the ad hoc constraints themselves are declared next.

```

set <str> SAUCES = /'Barbeque Sauce' 'Sweet 'N Sour Sauce'
                  'Hot Mustard Sauce' Honey/;
set <str> DRINKS = /'Vanilla Shake' 'Chocolate Shake'
                  'Strawberry Shake' '1% Lowfat Milk' 'Orange Juice'
                  'Coca-Cola (small)' 'Coca-Cola (medium)' 'Coca-
                  Cola (large)' 'Diet Coke (small)' 'Diet Coke
                  (medium)' 'Diet Coke (large)' 'Sprite (small)'
                  'Sprite (medium)' 'Sprite (large)'
                  'H-C Orange Drink (small)' 'H-C Orange Drink
                  (medium)' 'H-C Orange Drink (large)'/;

      /* Limit sauces to two per day*/
con Saucelimit:
    sum{r in SAUCES} Diet[r] <=2;

      /* Limit drinks to 3 per day */
con Drinklimit:
    sum{q in DRINKS} Diet[q] <= 3;

```

6. Nutrient bound constraints are declared.

```

impvar Intake {i in NUTRIENTS} =
    sum{j in FOOD} content[i,j]*Diet[j];

/* constraint bound on nutrient intake */
con Bounds{i in NUTRIENTS}:
    requirelb[i] <= Intake[i] <= requireub[i];

```

7. The objective function is specified below.

```

min Total_Cost = sum{j in FOOD} Cost[j]*Diet[j];
expand;
solve;

/* print non-zero number of FOOD items in the optimized Diet */
print {p in FOOD: Diet[p] > 0} Diet;

quit;

```


OPTMODEL Procedure Output

1. The output from the first PRINT statement shows the parameter arrays containing nutrient upper and lower bounds. Note that values used to initialize the arrays persist where there were missing values in the **Nutr_Req** table.

[1]	requireub	requirelb
Cal	10000	2000
CalFat	10000	0
Calcium	10000	100
Carbo	375	350
Chol	375	0
Fat	100	0
Iron	10000	100
Protein	10000	55
SatFat	30	0
Sodium	3000	0
VitA	10000	100
VitC	10000	100

2. Output from the second PRINT statement illustrates the parameter array **content**. A subset of content is shown below.

	1% Lowfat Milk	1000 Island Dressing	Apple Danish	Bacon Bits	Bacon, Egg & Cheese Biscuit	Barbeque Sauce	Big Mac	Bleu Cheese Dressing	Breakfast Burrito	Cheerios
Cal	110.0	225.0	360.0	15.0	440.0	50.0	500.0	250.0	280.0	80.0
CalFat	20.0	135.0	145.0	10.0	235.0	5.0	235.0	180.0	155.0	10.0
Calcium	30.0	0.0	6.0	0.0	20.0	0.0	25.0	0.0	10.0	2.0
Carbo	12.0	20.0	50.0	0.0	33.0	12.0	42.0	5.0	21.0	14.0
Chol	10.0	40.0	40.0	1.0	240.0	0.0	100.0	35.0	135.0	0.0
Fat	2.0	15.0	16.0	1.0	26.0	0.5	26.0	20.0	17.0	1.0
Iron	0.0	0.0	4.0	0.0	15.0	2.0	20.0	0.0	8.0	30.0
Protein	9.0	0.0	4.0	1.0	15.0	0.0	25.0	0.0	12.0	3.0
SatFat	1.5	5.0	5.0	0.5	8.0	0.0	9.0	5.0	4.0	0.0
Sodium	130.0	500.0	290.0	95.0	1215.0	340.0	890.0	750.0	580.0	210.0
VitA	10.0	0.0	8.0	0.0	10.0	4.0	6.0	0.0	10.0	15.0
VitC	4.0	0.0	0.0	0.0	0.0	4.0	2.0	0.0	10.0	15.0

3. The minimum cost diet has increased slightly in the formulation with the expanded menu and modified requirements to \$8.04.

Solution Summary	
Solver	MILP
Algorithm	Branch and Cut
Objective Function	Total_Cost
Solution Status	Optimal
Objective Value	8.04
Relative Gap	0
Absolute Gap	0
Primal Infeasibility	0
Bound Infeasibility	0
Integer Infeasibility	0
Best Bound	8.04
Nodes	115
Solutions Found	6
Iterations	1114
Presolve Time	0.01
Solution Time	0.26

4. The optimized diet has more variety (but no fries).

[1]	Diet
Cheerios	1
Chocolate Shake	2
Cinnamon Raisin Danish	1
French Reduced Cal Dressing	1
Hamburger	3
Honey	2
Orange Juice	1
Side Salad	1

End of Demonstration

4.3 Solutions

Solutions to Practices

1. Solving the Furniture-Making Problem as an Integer Linear Programming

```

proc optmodel;
  /* declare sets and parameters */
  set RESOURCES = /labor metal wood/;
  set PRODUCTS = /desks chairs bookcases bedframes/;
  num selling_price {PRODUCTS} = [94 79 125 109];
  num cost {RESOURCES} = [14 20 11];
  num availability {RESOURCES} = [225 117 420];
  num required {PRODUCTS, RESOURCES} =
    [2 1 3 1 1 3 3 1 4 2 1 4];

  /* declare variables */
  var NumProd {PRODUCTS} >= 0 INTEGER;

  impvar Revenue = sum {p in PRODUCTS}
    selling_price[p] * NumProd[p];

  impvar AmountUsed {r in RESOURCES} =
    sum {p in PRODUCTS} NumProd[p] * required[p,r];

  impvar TotalCost = sum {r in RESOURCES}
    cost[r] * AmountUsed[r];

  /* declare constraints */
  con Usage {r in RESOURCES}:
    AmountUsed[r] <= availability[r];

  /* declare objective */
  max NetProfit = Revenue - TotalCost;

  expand / var impvar;
  expand NetProfit;
  expand Usage;

  solve;

  print NumProd;
  print AmountUsed;

  quit;

```

The OPTMODEL Procedure

Solution Summary	
Solver	MILP
Algorithm	Branch and Cut
Objective Function	NetProfit
Solution Status	Optimal
Objective Value	1827
Relative Gap	0
Absolute Gap	0
Primal Infeasibility	0
Bound Infeasibility	0
Integer Infeasibility	1.065814E-14
Best Bound	1827
Nodes	1
Solutions Found	6
Iterations	6
Presolve Time	0.00
Solution Time	0.00

[1]	NumProd
bedframes	30
bookcases	39
chairs	48
desks	0

End of Solutions

Solutions to Activities and Questions

4.01 Multiple Choice Question – Correct Answer

Which of the five feasible points is the optimal solution to the ILP with the objective to maximize $6x + 5y$?

- a. $[x,y] = [0,0]$
- b. $[x,y] = [1,0]$
- c. $[x,y] = [0,1]$
- ☒ d. $[x,y] = [1,1]$
- e. $[x,y] = [0,2]$

10

Copyright © SAS Institute Inc. All rights reserved.



4.02 Activity – Correct Answer

The linear programming relaxation of the mixed integer linear programming problem

$$\begin{aligned}
 &\text{maximize } 5x_1 + 4x_2 - y_1 - y_2 \\
 &\text{subject to } 4x_1 + 6x_2 - 2y_1 \leq 11 \\
 &\quad 2x_1 + x_2 - y_2 \leq 4 \\
 &\quad x_1 \geq 0 \text{ integer, } x_2 \geq 0 \text{ integer} \\
 &\quad 0 \leq y_1 \leq 5, 0 \leq y_2 \leq 5
 \end{aligned}$$

has an optimal solution $x_1^*=4.5$, $x_2^*=0$, $y_1^*=3.5$, $y_2^*=5$ with objective value 14. From this solution, which variable (or variables) could the branch-and-bound algorithm branch on?

The only branching variable is x_1 (y_1 is not integer).

23

Copyright © SAS Institute Inc. All rights reserved.



4.03 Multiple Choice Question – Correct Answer

If you are solving an ILP/MILP for the first time, which of the following is a good strategy to use?

- ☒ a. a smaller value for RELOBJGAP
- ☐ b. the default value for RELOBJGAP
- ☐ c. a larger value for RELOBJGAP

Lesson 5 Open Source Interactivity

5.1	SAS Viya and Open Source Integration	5-3
5.2	SAS Viya Python APIs.....	5-11
	Demonstration: Executing a PROC OPTMODEL Program from Python Using CAS Actions	5-16
	Demonstration: Maximizing Distance Traveled by a Ball Thrown from a Cliff	5-22
	Demonstration: Ordinary Least Squares Optimization	5-26
	Practice.....	5-31
5.3	Solutions	5-35
	Solutions to Practices	5-35

5.1 SAS Viya and Open Source Integration

Objectives

- Introduce SAS Cloud Analytic Services (CAS) as a distributed environment.
- Overview CAS actions.

3

Copyright © SAS Institute Inc. All rights reserved.



SAS and Open Source Integration

Base SAS

- Execute open source code using system commands via the DATA step.

Interactive Matrix Language (IML)

- Submit R code within IML from SAS Code Editor.
- Pass data between R and SAS.

Enterprise Miner

- Execute R code from the open source integration node.
- Execute Python code using a system command.

SAS Viya

- Use open source software to take control of analytical tools.

4

Copyright © SAS Institute Inc. All rights reserved.



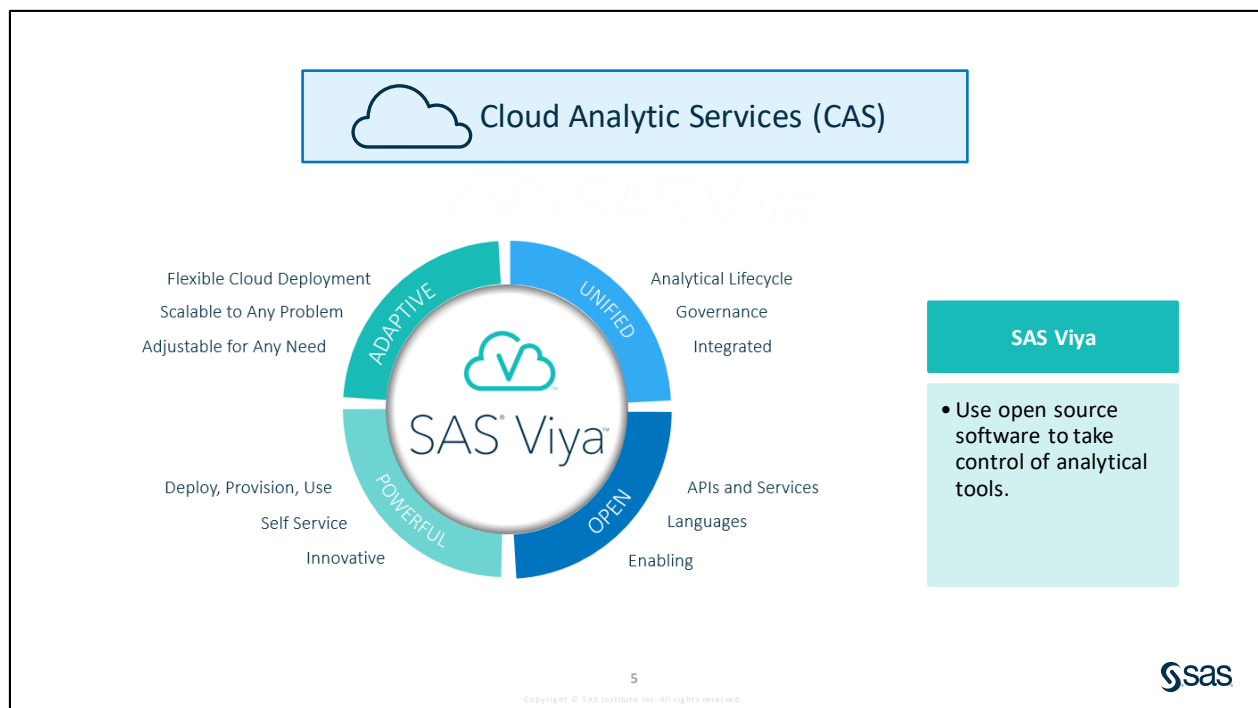
SAS views open source software as a complementary resource. For example, SAS has used SQL to query, merge, and manipulate data tables for years. R and Python are the newest open source tools that are used to create statistical models and graphics. You can assimilate R and Python into four different SAS products.

First, you can use a DATA step in Base SAS to submit a system command to execute a saved open source script. This effectively tells SAS to tell the operating system to run an R or Python script. Although this does enable you to execute an open source program from the SAS environment, it does not facilitate the passage of data between SAS and the open source software.

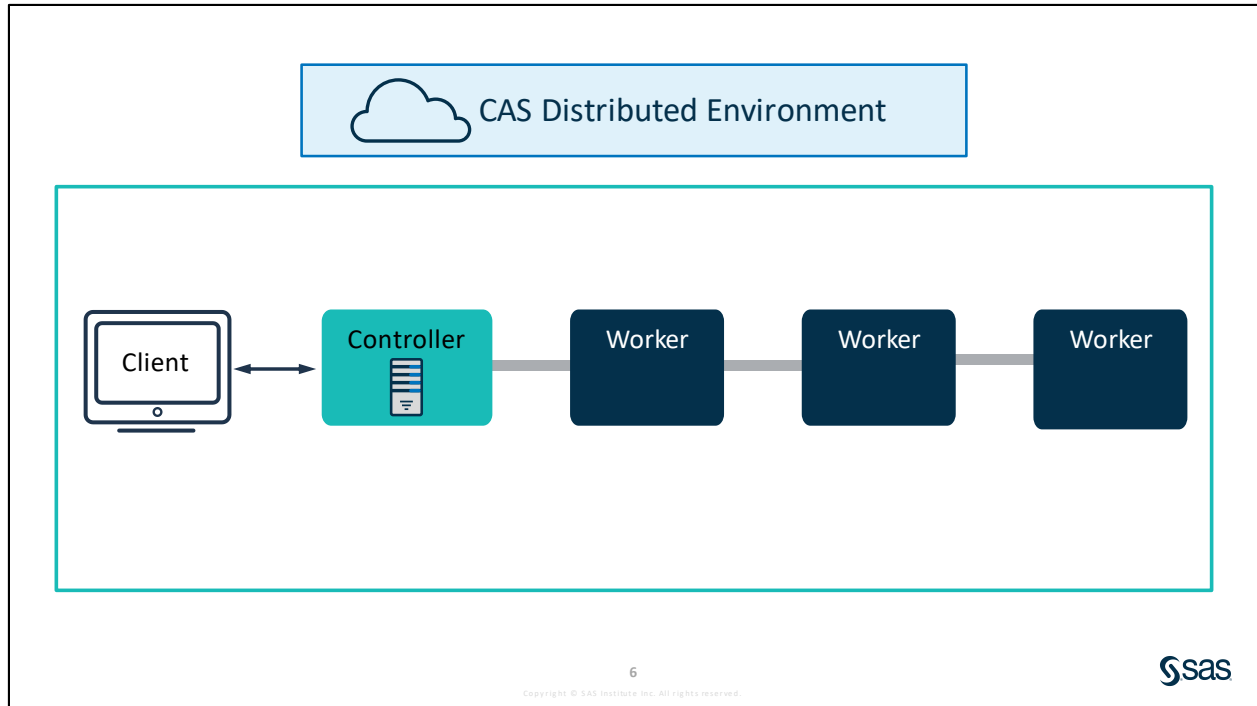
For R users, you can submit R code from the SAS code editor using the Interactive Matrix Language. The advantage of this method is that you are going from one matrix language to another, so you can seamlessly pass data tables and matrices between the two languages.

SAS Enterprise Miner enables you to execute open source software and pass data and results back and forth using the open source integration node. The open source models can then be compared side by side with the models developed in SAS.

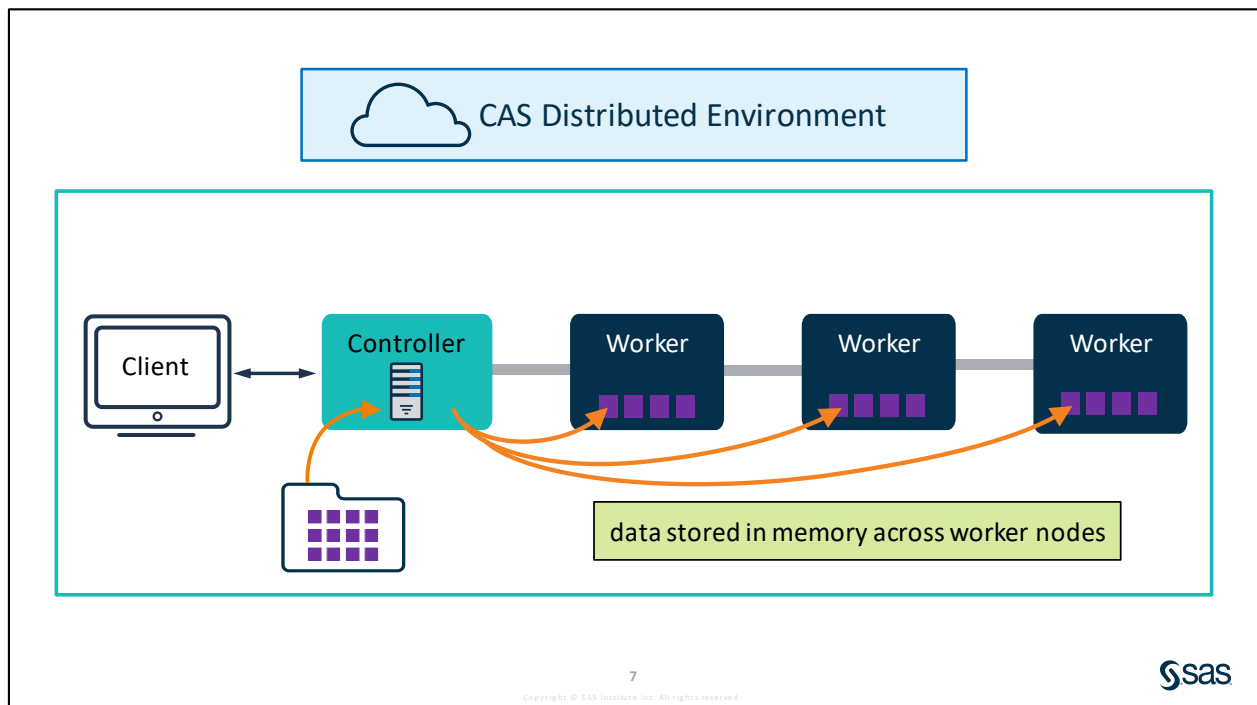
In Base SAS, the Interactive Matrix Language, and Enterprise Miner, you are using SAS to call R or Python. In SAS Viya (which is discussed shortly), the roles are reversed. From R or Python, you can use specially designed application programming interfaces, or *APIs*, to control the analytical tools available in SAS Viya. This enables you to program in the language that you are most comfortable with and still have access to the highly optimized analytical routines available with SAS Viya.



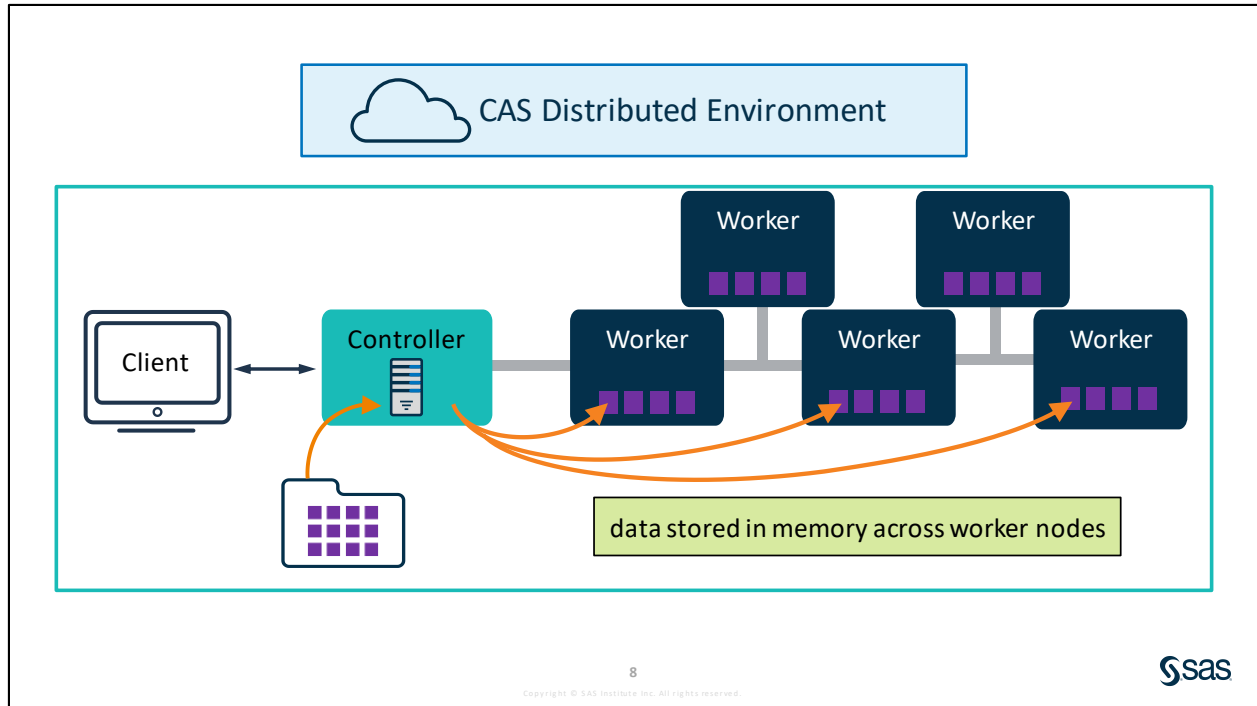
SAS Viya is a cloud-ready, in-memory analytics engine for fast and accurate analytical results. SAS Viya processes data and performs analytics using SAS Cloud Analytic Services, or CAS for short. The CAS environment is a distributed computing environment designed to store large data sets in memory for fast and efficient processing.



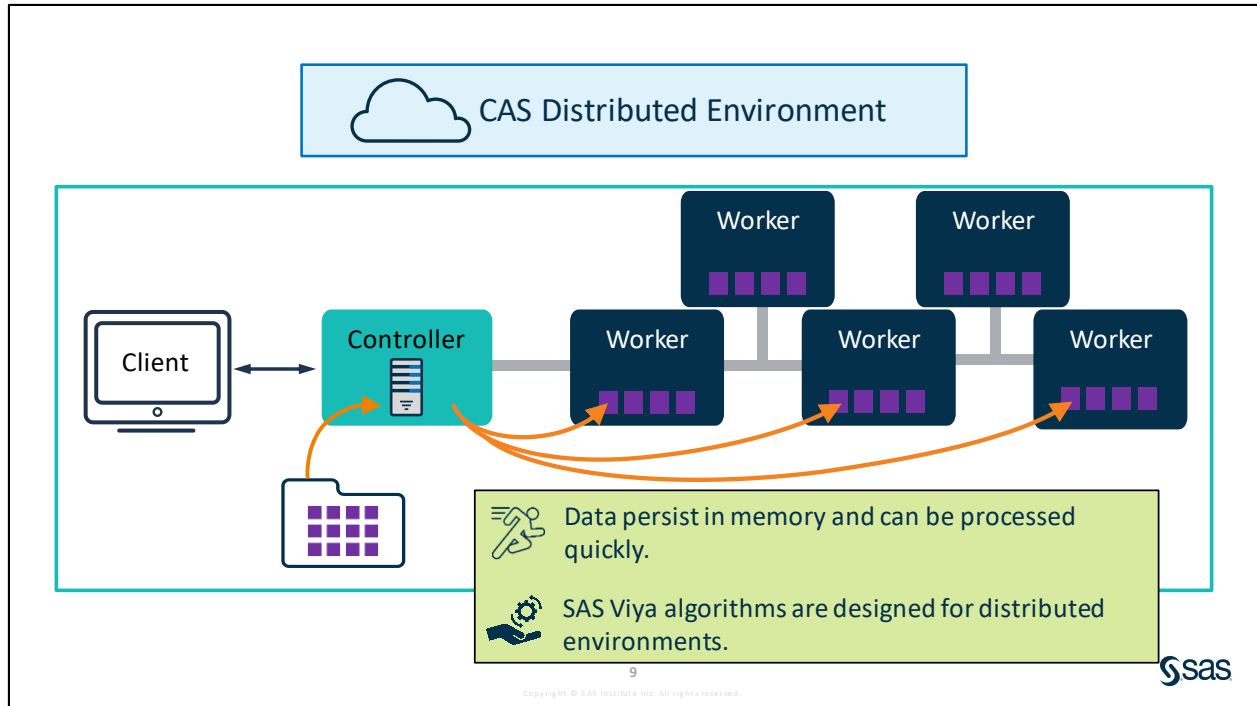
The CAS distributed environment consists of a single controller node, which is used to receive commands and schedule tasks, and a collection of worker nodes to perform data processing.



Data are stored in memory across the worker nodes, and each worker node performs processing on the data located locally before passing results back to the controller.



The CAS environment is flexible, enabling more worker nodes to be added if more in-memory space is needed, or if more processing power is required. CAS scales to your memory and processing needs.



Two major advantages of the CAS environment are speed and efficiency. Data can be processed quickly because they persist in memory. Thus, no time is wasted repeatedly reading and writing data to disk. The flexibility of the environment means that you can use even extremely large data sets to generate complicated analytical models, as long as enough worker nodes are added to CAS. The data processing algorithms in SAS Viya are designed so that analytical processing is distributed efficiently among the worker nodes for parallel processing. This means that you can spend your time and effort refining and building models, rather than worrying about how to manage data or parallelizing your algorithms for each analytical model of interest.

CAS Actions

- The tools used to interact with data on the CAS server
- Act as wrappers for parallel processing algorithms
- Load data, transform data, compute statistics, perform analytics, and create output

Open Source Functions $\equiv$ SAS Procedures $\equiv$ CAS Actions

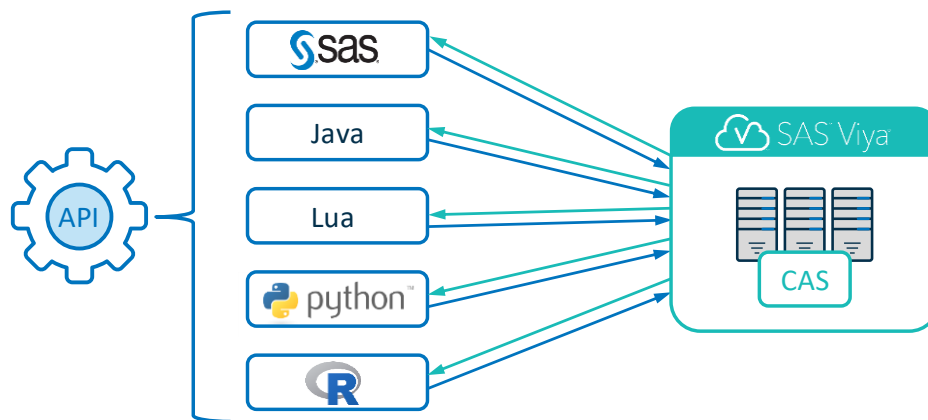
10

Copyright © SAS Institute Inc. All rights reserved.



We interact with the CAS server through CAS actions, which are individual processing tasks that we submit to the CAS server. They are essentially wrappers for parallel processing algorithms, and they can load data, transform data, compute statistics, perform analytics, and create output.

SAS Viya Is Open



11

Copyright © SAS Institute Inc. All rights reserved.



You can manage data and create analytical models using a variety of open source languages. You can use SAS Studio, Java, Lua, Python, or R to connect to CAS.

Open Source Development Interfaces



RStudio



Spyder

- R Studio and Spyder are integrated development environments (IDEs).



- Jupyter Notebook is a web application that is used to run several programming languages (R, Python, Julia, and so on).

12

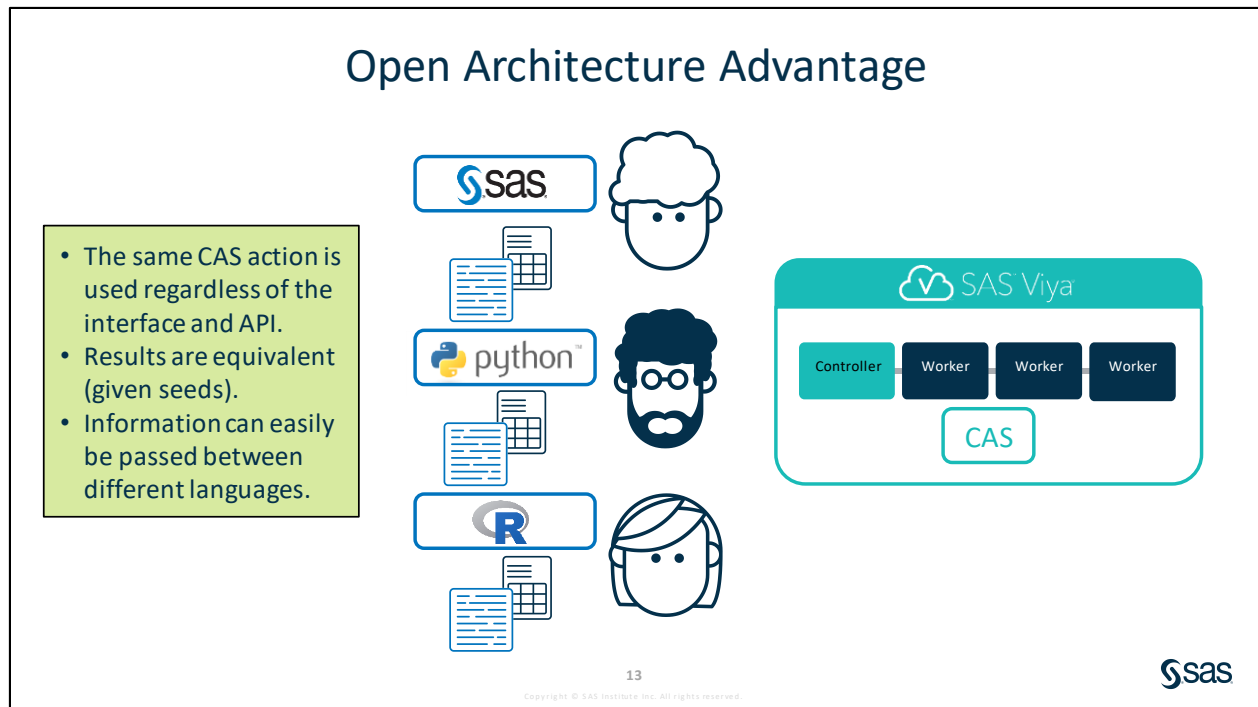
Copyright © SAS Institute Inc. All rights reserved.



Because CAS actions are submitted directly from R or Python, we can choose any development interface that we would like for submitting R or Python code.

RStudio is a popular R development interface, and Spyder is a similar interface for Python. These integrated development environments, or *IDEs*, make developing code easier by using features such as built-in graphics applications and interactive workspaces.

Jupyter Notebook is a web application for developing and organizing code and it is an extremely popular way to develop and present R or Python code. You can use these notebooks to write and submit code in either R or Python, and you can also add text and descriptions around blocks of code known as markdown. This is extremely useful for teaching and presenting results. Both the code and the results can be embedded in the notebook to provide additional details and avoid running the code in real time when presenting.



The open architecture of SAS Viya enables data and results to be passed easily between your favorite programming languages, speeding up development time.

If multiple people need to work on the same data, they can use different programming languages to do so while still interacting with the same data in memory on the CAS server. And no matter what language they choose, the CAS actions are the same. Therefore, if the same settings are chosen and the algorithm is seeded, the same results are created.

5.2 SAS Viya Python APIs

Objectives

- Overview open source interactivity with SAS Viya.
- Execute a PROC OPTMODEL program from Python using CAS actions.

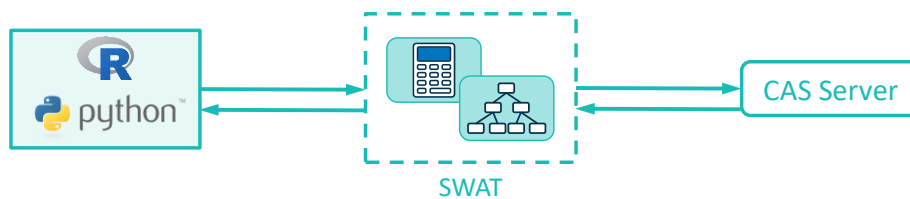
15

Copyright © SAS Institute Inc. All rights reserved.



Scripting Wrapper for Analytics Transfer (SWAT)

- The SWAT package enables you to interface with CAS from R or Python.
- You can write an R or Python program that connects to a CAS server, load data into CAS, analyze large in-memory data sets quickly and efficiently using CAS actions, and work with results of your analyses using familiar data preparation techniques in the open source language.

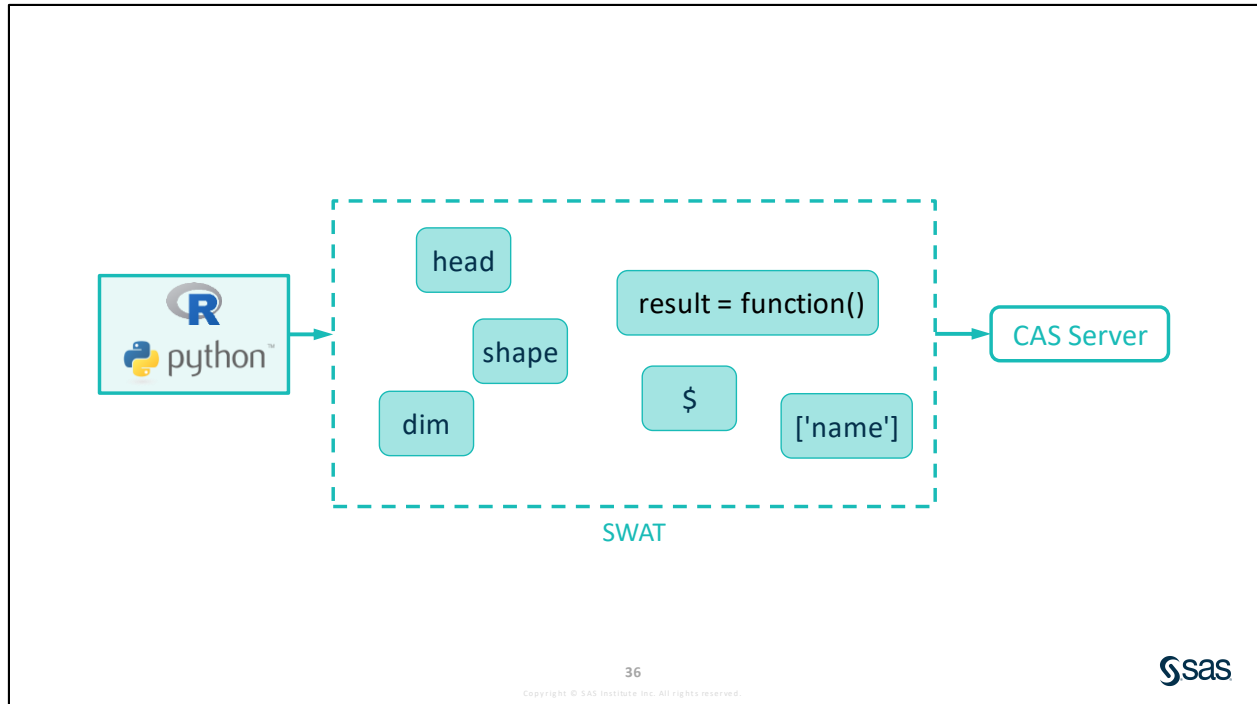


16

Copyright © SAS Institute Inc. All rights reserved.

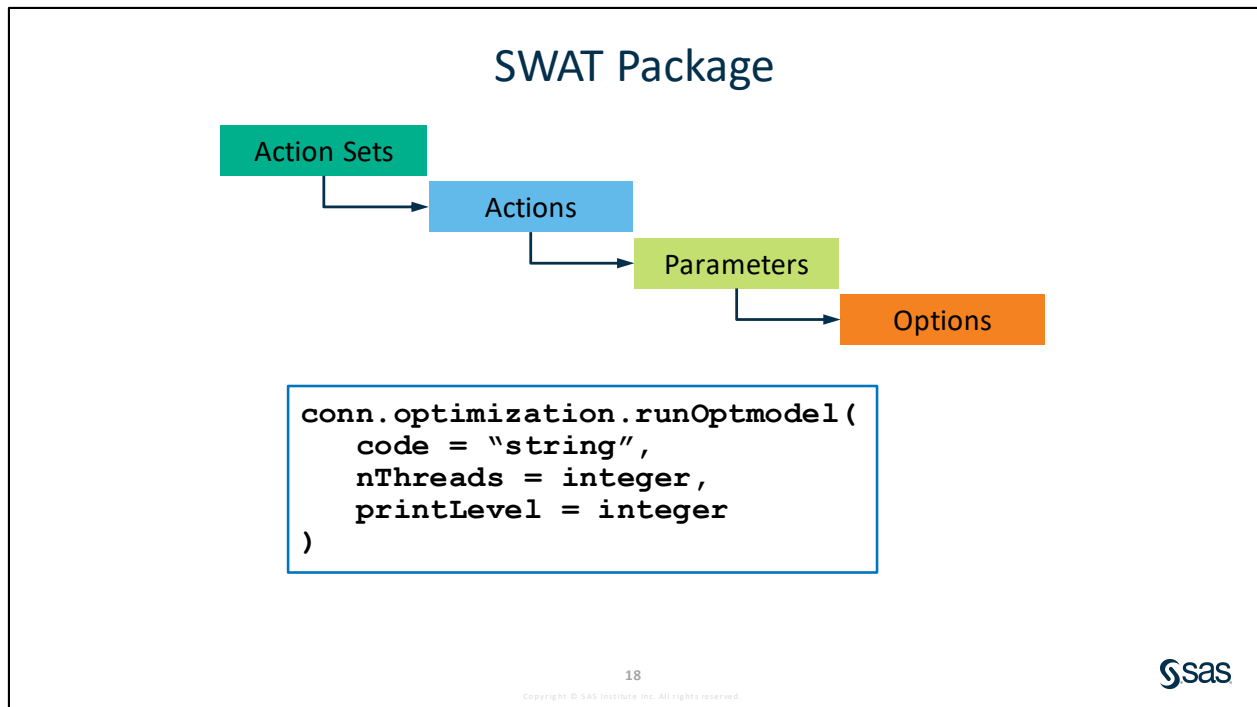


The SAS Scripting Wrapper for Analytics Transfer, or simply *SWAT*, is an open source package used by Python and R to submit CAS actions to the CAS server.

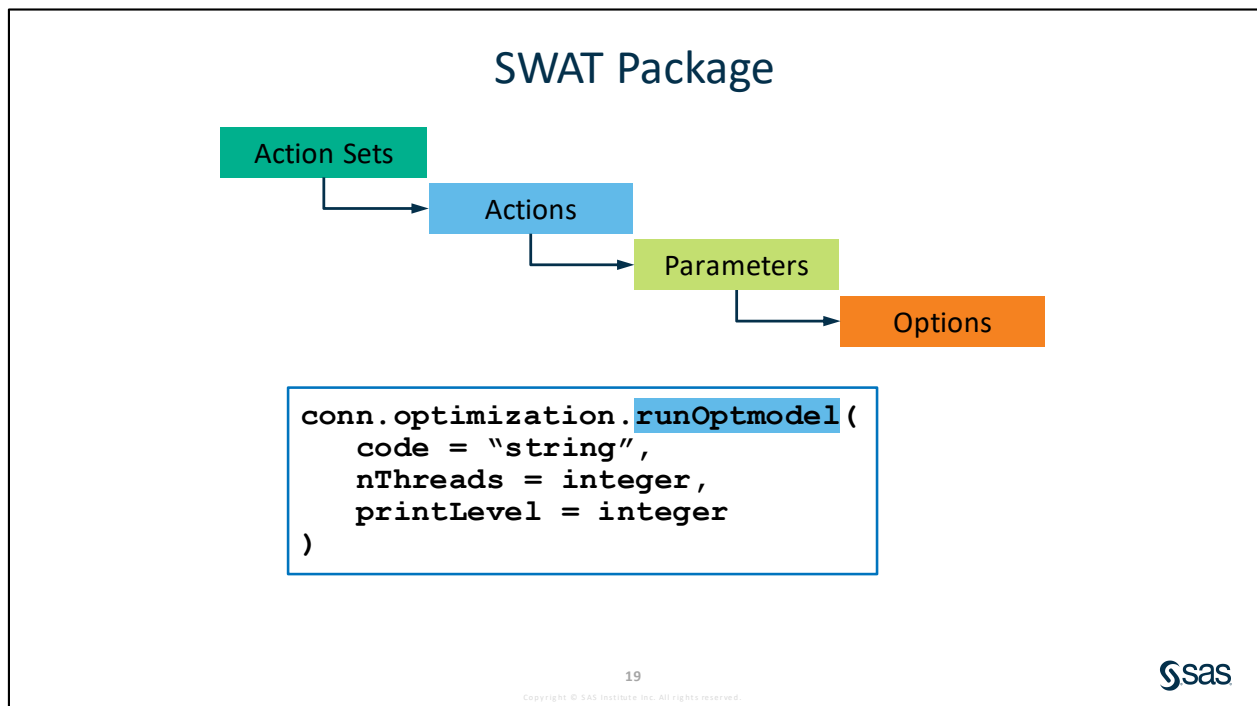


The SWAT package provides functionality and syntax that have a look and feel of open source code but just wrap up CAS actions. This enables you to use familiar syntax on the tables in memory on the CAS server.

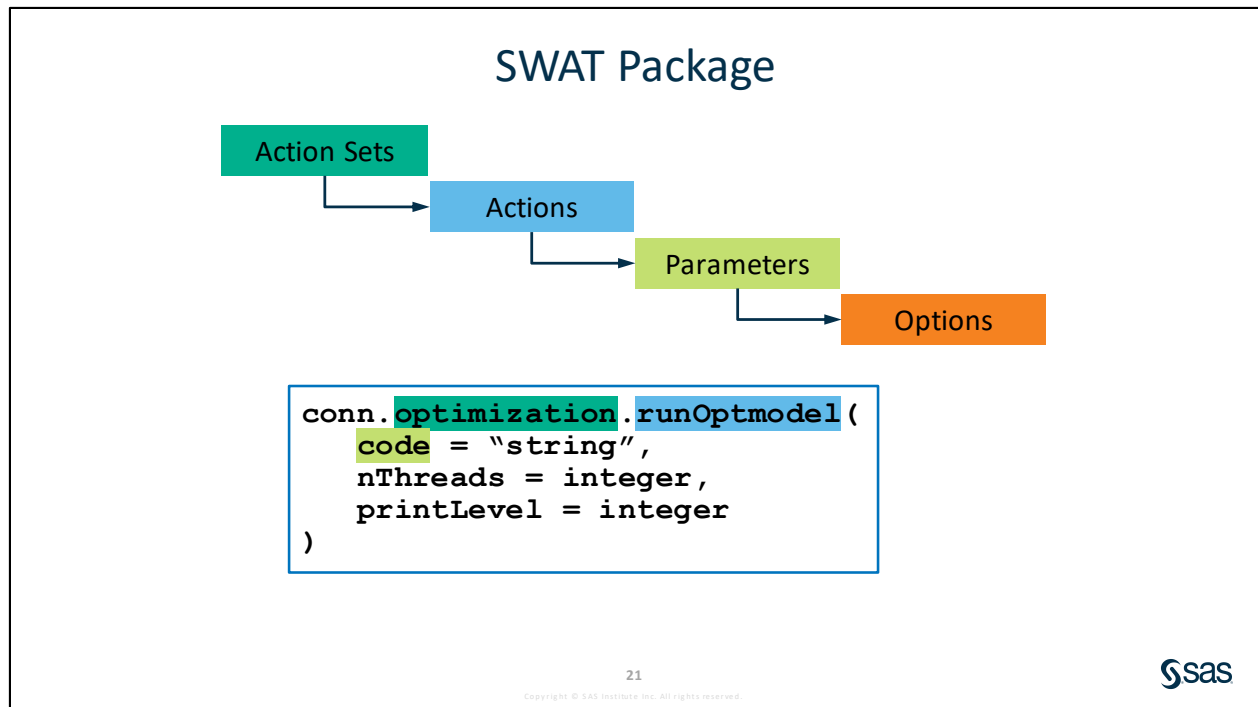
For example, you can apply the `head` function to a CAS table to view the first few rows of the table. Or you can use the `dim` function in R and the `shape` function in Python to view the dimension of the CAS data table. You can also save results similar to objects by setting results equal to a new variable, and you can pull fields from these results using the `$` sign operator in R or brackets in Python.



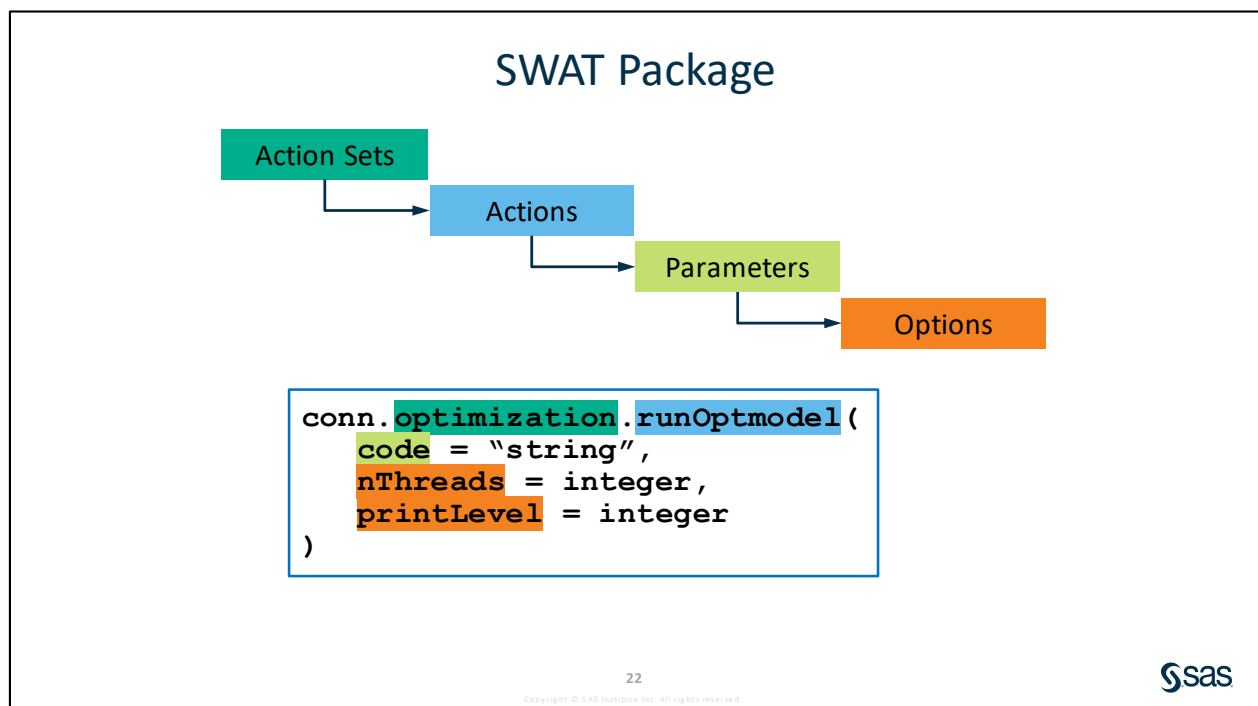
In addition to the SWAT functionality that looks like open source code, you can submit other CAS actions from R or Python as well. CAS actions are grouped into action sets based on a topic, and each action has a set of parameters and options to alter the algorithm as needed.



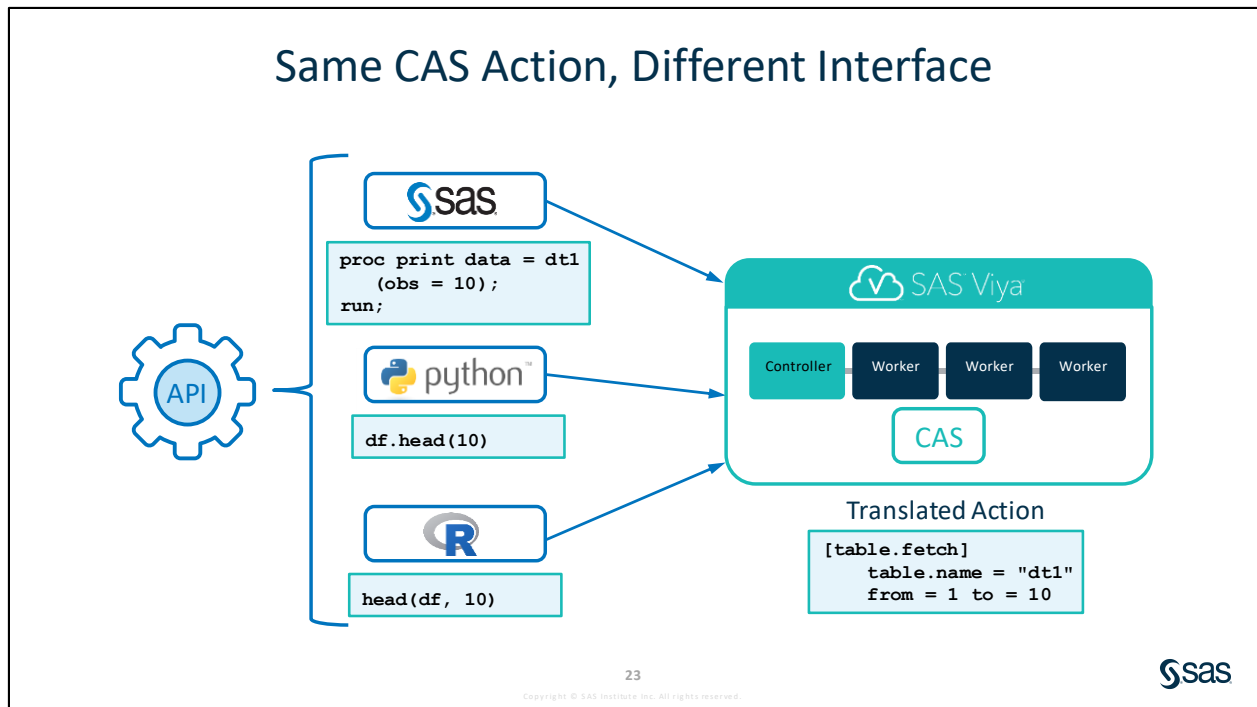
In this example, the `runOptmodel` action is a part of the optimization action set.



Parameters must be specified, whereas options have defaults that are used when a value is missing from the API call. For example, the parameter `code` is required in order to execute the action. The action needs to know what PROC OPTMODEL code to execute.



The `nThreads` argument is an option because if it is not specified, it defaults to the number of available cores on your machine. Each CAS action has its own parameters and options, although they are designed to be as similar as possible.



Regardless of the language in which the API call is issued, the underlying CAS action submitted to the CAS server is the same. For example, if you want to print the first 10 observations of your CAS table, you would use the PRINT procedure in SAS, the head method in Python, and the head function in R. In all cases, the command is translated into the same fetch action from the table action set and the first 10 rows of the CAS table are sent from the server to the client.



Executing a PROC OPTMODEL Program from Python Using CAS Actions

This demonstration illustrates submitting a PROC OPTMODEL program from a Python interface using the runOptmodel CAS action. In this case we reuse the program from the first demonstration in Lesson 3, minimizing the banana function with no constraints or bounds:

$$\text{minimize } (y - x^2)^2 + (1 - x)^2$$

The PROC OPTMODEL code is identical to the code that we submitted in Lesson 3, but this time we will submit the code from a Python interface.

1. Begin by opening Jupyter Notebook from the start menu on the virtual machine. Navigate to and open **Documents** ⇒ **ORVY** ⇒ **CASrunOptmodelPython.ipynb**.
2. Import the SWAT package.

```
import swat
```

3. Make a connection to the SAS Viya server.

```
conn = swat.CAS("http://server.demo.sas.com/cas-shared-default-http/", 8777, "student", "Metadata0", protocol="http")
```

4. Load the "Optimization" action set and use the runOptmodel() CAS action to submit the PROC OPTMODEL code.

```
conn.loadactionset("Optimization")

results = conn.runOptmodel(
    code="""
        var x, y;

        min f = (y-x**2)**2 + (1-x)**2;

        solve;

        print x y;
    """)
```

Note that the code was enclosed in triple quotation marks to make sure that any apostrophes or quotation marks inside the CODE statement are correctly interpreted by the software.

Results are stored in the local Python variable named results:

```
results
```

§ Print1.PrintTable

	x	y
0	1.0	1.0

§ Solve1.ProblemSummary

Problem Summary

	Name1	Label1	cValue1	nValue1
0	objectiveSense	Objective Sense	Minimization	NaN
1	objectiveName	Objective Function	f	NaN
2	objectiveType	Objective Type	Nonlinear	NaN
3				NaN
4	numVars	Number of Variables	2	2.0
5	numBoundedAbove	Bounded Above	0	0.0
6	numBoundedBelow	Bounded Below	0	0.0
7	numBounded	Bounded Below and Above	0	0.0
8	numFree	Free	2	2.0
9	numFixed	Fixed	0	0.0
10				NaN
11	numRows	Number of Constraints	0	0.0

§ Solve1.SolutionSummary

Solution Summary

	Name1	Label1	cValue1	nValue1
0	solver	Solver	NLP	NaN
1	algorithm	Algorithm	Interior Point Direct	NaN
2	objectiveName	Objective Function	f	NaN
3	solStatus	Solution Status	Optimal	NaN
4	objective	Objective Value	0	0.000000
5				NaN
6	optimalityError	Optimality Error	0	0.000000
7	infeasibility	Infeasibility	0	0.000000
8				NaN
9	iterations	Iterations	2	2.000000
10	presolveTime	Presolve Time	0.00	0.000054
11	solutionTime	Solution Time	0.00	0.000810

The major advantage to using the runOptmodel action from Python instead of just submitting your PROC OPTMODEL code in SAS Studio is the ability to consume the results of the optimization problem in Python without transferring data files between software interfaces.

The actual optimization algorithms are the same behind the scenes regardless of how we submit our SAS Optimization programs.

End of Demonstration

Note about the demonstration:

This CAS action could have been submitted from SAS Studio using the CAS procedure and would yield the same results. This is for users who want to use the CAS actions but do not program in an open source language like Python or R. The following program can be executed in SAS Studio to call the same CAS action as the previous demonstration:

```
cas;
proc cas;
  loadactionset "optimization";
  /* here is the text of our program, between source and
     endsource */
  source pgm;
  var x, y;

  min f = (y-x**2)**2 + (1-x)**2;

  solve;

  print x y;
  endsource;
  action runOptmodel result=r / code=pgm;
  /* These print statements are required to get the standard
     PROC OPTMODEL output information */
  print r["Solve1.ProblemSummary"];
  print r["Solve1.SolutionSummary"];
run;
quit;
```



SAS Software

Open Source from SAS Software

 Cary, North Carolina, USA
  <https://www.sas.com/>
 github@sas.com


python-dlpy

The SAS Deep Learning Python (DLPy) package provides the high-level Python APIs to deep learning methods in SAS Visual Data Mining and Machine Learning. It allows users to build deep learning models using friendly Keras-like APIs.

 Python
  Apache-2.0
  73
  129
  19
  1
 Updated yesterday

sasoptpy

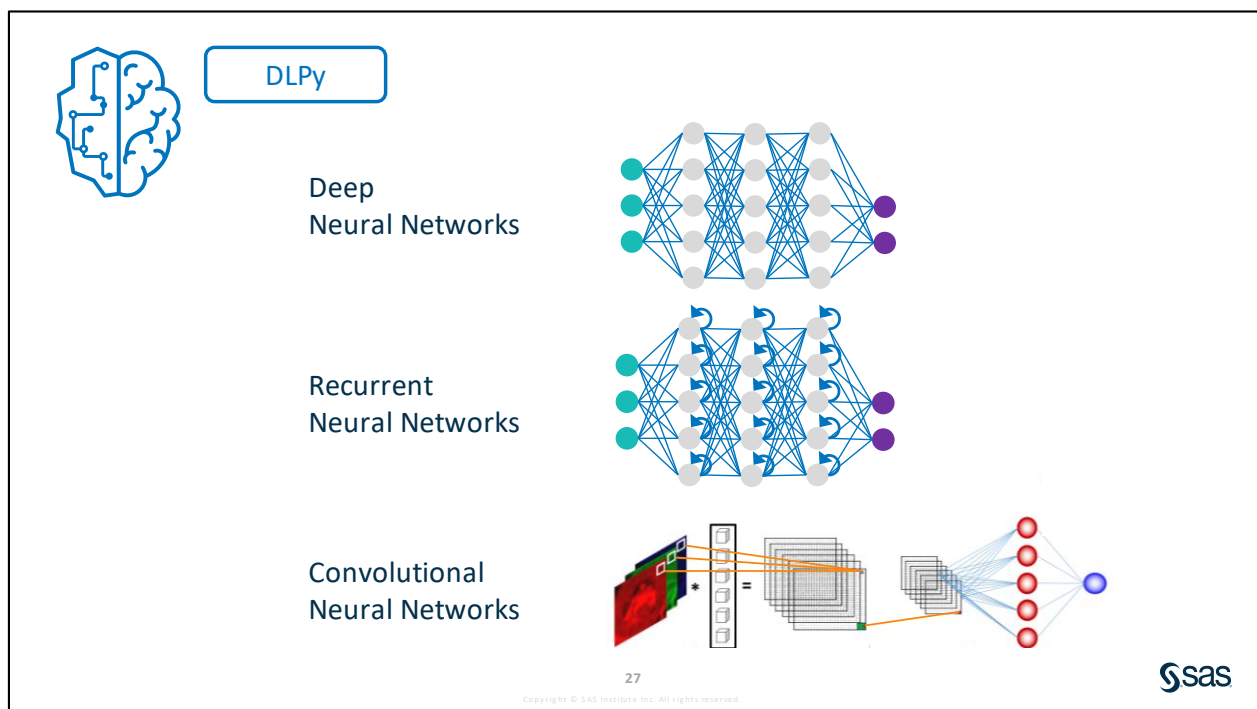
sasoptpy is a Python package providing a modeling interface for SAS Viya Optimization solvers.

 Python
  Apache-2.0
  5
  12
  0
  0
 Updated 23 hours ago

There are additional Python libraries for specific licenses of SAS Viya. For example, DLPy is a high-level Python library to access SAS deep learning features. It is designed to provide an efficient way to apply deep learning method to your data in order to build deep, recurrent, and convolutional neural networks with SAS Viya from the Python API (available at <https://github.com/sassoftware/python-dlpy>).

The sasoptpy python package provides a modeling interface to SAS Viya and the SAS operation research optimization solvers. It provides a quick way for users to deploy optimization models and solve them using CAS actions. All the optimization problems that you can fit in PROC OPTMODEL can also be optimized using sasoptpy from Python. However, from Python, you have access to native open source programming structures like dictionaries and data frames to define the optimization problem.

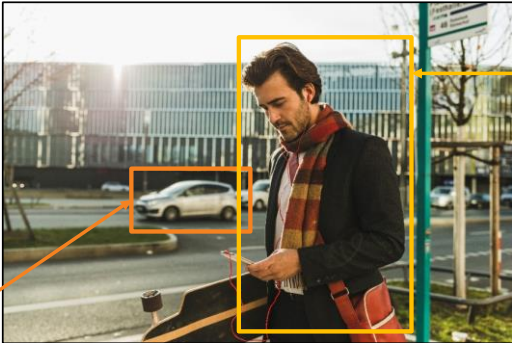
The sasoptpy github page (<https://github.com/sassoftware/sasoptpy>) provides examples and syntax documentation to learn how to use the Python library.



The SAS DLPy package provides a Python framework for submitting deep learning CAS actions. The DLPy API was designed to mimic the feel of Keras APIs so that Python users can build deep learning models in SAS Viya using familiar syntax. DLPy has APIs for submitting CAS actions to build a variety of complex deep learning models, with a focus on advanced applications of recurrent neural networks and convolutional neural networks. These models can be used for a range of tasks, including text classification and generation, sequence labeling, object detection, image segmentation, and time series analysis. To facilitate this broad range of data analysis tasks, DLPy has methods to read in and preprocess a variety of data types, including image data, text data, and even audio data.



DLPy



28

sas

Copyright © SAS Institute Inc. All rights reserved.

The package also includes several prebuilt neural network model architectures, like the Yolo model for object detection. These prebuilt models enable the user to quickly and easily create complex neural networks with many specialized layers without having to specify all the details. The complex architecture is predefined based on literature in the neural network field.

sasoptpy

Program.sas

```

1 proc optmodel;
2   /* declare variables */
3   var x >= 0, y >= 0;
4
5   /* declare objective */
6   max z = 12*x + 19*y;
7
8   /* declare constraints */
9   con x + 3*y <= 225;
10  con x + y <= 117;
11  con 3*x + 4*y <= 420;
12
13  solve;
14
15  print x y;
16
17  quit;
18

```

↔

```

In [1]: import swat
import sasoptpy as so

#Connect to CAS
conn = swat.CAS(hostname='host',port='8777')
#Create a model object
m = so.Model('ex', session=conn)
#Add decision variables
m.add_variables(['x','y'], vartype=so.CONT, lb=0)
#Set the objective function
m.set_objective(12*x+19*y, sense=so.MAX, name='z')
#Add constraints
m.add_constraint(x+3*y <= 225,name='c1')
m.add_constraint(x+y <= 117,name='c2')
m.add_constraint(3*x+4*y <= 420,name='c3')
#Solve
result = m.solve()
#Print results
print(so.get_solution_table(x,y))

```

29

sas

Copyright © SAS Institute Inc. All rights reserved.

The sasoptpy package provides a Python framework for building SAS optimization models. All of the concepts and ideas in PROC OPTMODEL are used in sasoptpy, but the syntax is written in an object-oriented way, so you create a model object and add properties like decision variables,

objective functions, and constraints. Behind the scenes, all of the Python code is used to create a PROC OPTMODEL program that is submitted for processing to the SAS Viya server. The results are brought back to Python in the form of a data frame, so the output of the optimization routine can be used in subsequent Python code. All the optimization problems you can solve in PROC OPTMODEL can also be optimized using sasoptpy from Python. However, Python gives you access to native open source programming structures like dictionaries and data frames to define the optimization problem. The goal of sasoptpy is to provide Python users easy access to the powerful optimization algorithms available in SAS Viya, while keeping a familiar syntax.

For a detailed example and more discussion about solving optimization problems in sasoptpy, see the SAS Global Forum paper titled “Optimization Modeling with Python and SAS Viya” available at <https://www.sas.com/content/dam/SAS/support/en/sas-global-forum-proceedings/2018/1814-2018.pdf>

The demonstrations that follow solve some simple optimization problems using sasoptpy. A useful way to practice with the software is to try to solve these same problems using PROC OPTMODEL. (Or, if you are familiar with Python, try to solve the problems introduced earlier using sasoptpy.)



Maximizing Distance Traveled by a Ball Thrown from a Cliff

This demo solves the same physics problem introduced in Lesson 3, but this time we use `sasoptpy` to solve the problem instead of PROC OPTMODEL. The derivation of the objective function and constraints is explained in Lesson 3.

1. Load the `swat`, `numpy`, and `sasoptpy` Python packages. To use mathematical functions in `sasoptpy`, load the `sasoptpy.math` submodule.

```
import swat
import numpy as np
import sasoptpy as so
import sasoptpy.math as sm
```

swat is an R package that enables you to interface with SAS Cloud Analytics Services (CAS), the in-memory server that is the centerpiece of SAS Viya. Using the SWAT package, you can write an R program that connects to a CAS server, analyze large in-memory data sets, and then work with the results of the data analysis using familiar data-preparation techniques in R.

numpy is a Python package for scientific computing. It supports functionality for multi-dimensional arrays and matrices, as well as high-level mathematical functions not contained in the base code.

sasoptpy is a Python package providing a modeling interface to SAS Viya and SAS/OR optimization solvers. It provides a quick way for users to deploy optimization models and solve them using CAS actions. It can handle linear, mixed integer linear, and nonlinear optimization problems. Users can benefit from native Python structures like dictionaries, tuples, lists, and Pandas data frames to define an optimization problem.

sasoptpy.math is a part of the `sasoptpy` package that contains mathematical functions that can be used in `sasoptpy` expressions. Python has a number of built-in functions, and `numpy` has its own mathematical functions, but `sasoptpy` can use only the functions available in `sasoptpy.math`. The native Python and `numpy` functions cannot be used in the `sasoptpy` constraints, objective functions, or implicit variables.

2. Create a connection to CAS with the appropriate server name, port, and log in credentials.

```
conn = swat.CAS("http://sasviya01.demo.sas.com/cas-shared-default-http/", 8777, "student", "Metadata0", protocol="http")
```

The arguments required for the CAS function to connect to the CAS server are dependent on your SAS Viya system. The arguments should be required from your IT professional.

3. Set numerical values for the constants in the physics problem.

```
#set the variables in the problem statement to numbers
v = 40 #40 m/s is a 90 mph fastball
H = 20 #20 m is a small cliff
g = 9.8 #gravity is 9.8 m/s^2
```

v is the magnitude of the velocity with which you throw the ball. In this problem, **v** is set to 40 meters per second, which is 89.48 miles per hour. This is about the speed baseball pitchers throw fastballs.

H is the height of the cliff. In this problem, **H** is set to 20 meters, which is a medium cliff or a medium height building. (For those who are more familiar with Imperial units, 20 meters is 65.62 feet.)

g is the downward acceleration objects on Earth experience due to the force of gravity. In this problem, g is approximated by 9.8 meters per second squared.

4. Use sasoptpy to maximize the distance traveled by the ball as function of the angle θ at which you throw ball.

```
phys_mod = so.Model('physics_demo', session=conn)
```

The `so.Model` method initializes a sasoptpy object. In addition to naming the object, we specify the CAS connection object in order to move data between the server and client seamlessly.

```
theta = phys_mod.add_variable(vartype=so.CONT, name='theta')
```

There is only one decision variable in this problem, the throwing angle θ . The `add_variable()` method is used to add the variable '**theta**' to the optimization model as a decision variable. This variable is continuous rather than binary or integer, so `vartype=` is set to `so.CONT`.

```
phys_mod.set_objective(v**2*sm.cos(theta)*sm.sin(theta)/g +  
v*sm.cos(theta)/g*sm.sqrt(v**2*sm.sin(theta)**2+2*g*H),  
sense=so.MAX, name='dist')
```

The `set_objective()` method is used to define the objective function for the optimization problem. In this case, the objective function is the horizontal distance (named '**dist**' here) traveled by the ball. Because the goal is to maximize this distance, the sense of the objective function is set to `so.MAX`. This objective function uses the mathematical functions sine, cosine, and square root, so those functions are pulled from the `sasoptpy.math` package, here imported as `sm`. Note that this definition would be invalid if the numpy math functions were used (`np.sin` instead of `sm.sin`).

```
phys_mod.add_constraint(theta <= np.pi/2)  
phys_mod.add_constraint(theta >= 0)
```

The `add_constraint()` method is used to define constraints for the decision variable '**theta**'. In this case, the angle must be greater than 0 radians and less than $\pi/2$ radians. In degrees, the angle must be between 0 and 90 degrees. In the context of the diagram, you want to throw the ball up and forward only. You do not want to throw the ball down (an angle less than 0), and you do not want to throw the ball behind you (an angle greater than $\pi/2$).

```
result = phys_mod.solve()
```

The `solve` method initiates the optimization of the model defined in the **phys_mod** object. When no additional solving options are specified, the software determines the best algorithm for solving the problem based on the objective function and the constraints. In this problem, the objective function is nonlinear (and it is not a quadratic nonlinear function, it has transcendental functions), so the solver uses either the Active Set or Interior Point optimization algorithm.

```
NOTE: Initialized model physics_demo.  
NOTE: Added action set 'optimization'.  
NOTE: Converting model physics_demo to OPTMODEL.  
NOTE: Submitting OPTMODEL code to CAS server.  
NOTE: Problem generation will use 16 threads.  
NOTE: The problem has 1 variables (1 free, 0 fixed).  
NOTE: The problem has 2 linear constraints (1 LE, 0 EQ, 1 GE, 0 range).  
NOTE: The problem has 2 linear constraint coefficients.  
NOTE: The problem has 0 nonlinear constraints (0 LE, 0 EQ, 0 GE, 0 range).  
NOTE: The OPTMODEL presolver removed 0 variables, 2 linear constraints, and 0 nonlinear  
constraints.  
NOTE: The OPTMODEL presolved problem has 1 variables, 0 linear constraints, and 0 nonlinear  
constraints.  
NOTE: The OPTMODEL presolver removed 2 linear constraint coefficients, leaving 0.  
NOTE: Using analytic derivatives for objective.
```

NOTE: The NLP solver is called.

NOTE: The Interior Point algorithm is used.

NOTE: Initial point was changed to be interior to bounds.

Objective		Optimality	
Iter	Value	Infeasibility	Error
0	58.18822844	0	64.01338668
1	169.70550194	0	0.00787882
2	169.70562748	0	0.00000500
3	169.70562748	0	0.0000000500000

NOTE: Optimal.

NOTE: Objective = 169.70562748.

NOTE: The CAS table 'primal' in caslib 'CASUSER(student)' has 1 rows and 6 columns.

NOTE: The CAS table 'dual' in caslib 'CASUSER(student)' has 2 rows and 4 columns.

NOTE: The CAS table 'problemSummary' in caslib 'CASUSER(student)' has 16 rows and 4 columns.

NOTE: The CAS table 'solutionSummary' in caslib 'CASUSER(student)' has 12 rows and 4 columns.

In this case, the Interior Point algorithm was used to solve the nonlinear programming problem. The optimization routine exited on the third iteration with an optimality error on the order of 10^{-8} . The objective value for the optimal solution is 169.7, which means that the maximum distance the ball will travel is 169.7 meters, or 556.8 feet.

```
print(so.get_solution_table(theta))
print('Maximum distance value:',phys_mod.get_objective_value())
print('The best angle
is:',round(so.get_solution_table(theta) ['theta'] [0]*180/np.pi,1) ,
'degrees')

#this just resets the variable name 'theta' so if you run it
again, you can use the same name
so.reset_globals()
```

The method `get_solution_table()` is used to get the solution for the decision variable `theta`. In this case, the optimal value for `theta` is 0.75597 radians. This angle is 43.3 degrees above the horizontal.

If there was no cliff ($H = 0$), the best angle to throw a ball to maximize distance is 45 degrees above the horizontal. The addition of the cliff means that the best angle is slightly less than 45 degrees. The taller the cliff, the lower the angle will become. In the limit of an infinitely tall cliff, the optimal angle would be 0 degrees.

```
theta
1
0.755969
Maximum distance value: 169.705627
The best angle is: 43.3 degrees
```

We can also get the standard PROC OPTMODEL output by applying the methods `get_problem_summary()` and `get_solution_summary()` to our model object.

```
#Show standard PROC OPTMODEL results
display(phys_mod.get_problem_summary())
display(phys_mod.get_solution_summary())
```

Problem Summary	
Label	Value
Objective Sense	Maximization
Objective Function	dist
Objective Type	Nonlinear
Number of Variables	1
Bounded Above	0
Bounded Below	0
Bounded Below and Above	0
Free	1
Fixed	0
Number of Constraints	2
Linear LE ($\leq$)	1
Linear EQ ($=$)	0
Linear GE ($\geq$)	1
Linear Range	0

Solution Summary	
Label	Value
Solver	NLP
Algorithm	Interior Point Direct
Objective Function	dist
Solution Status	Optimal
Objective Value	182.1707233
Optimality Error	9.0920214E-8
Infeasibility	0
Iterations	10
Presolve Time	0.00
Solution Time	0.00

End of Demonstration



Ordinary Least Squares Optimization

Multiple linear regression models the relationship between two or more predictor variables and a continuous response by fitting a linear model to the observed data. The model is formally stated as

$$y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + \dots + \beta_p x_{pi} + e_i$$

For $i = 1, \dots, n$, the response y_i , and p possible predictor variables. To estimate the populations parameters, $\beta_0, \beta_1, \beta_2, \dots, \beta_p$, we minimize the squared error by conducting ordinary least squares optimization.

$$\min_{\{\beta_j\}} \left[\sum_{i=1}^n e_i^2 = \sum_{i=1}^n \left(y_i - \sum_{j=1}^p \beta_j x_{ij} \right)^2 \right]$$

This demonstration shows how to use `sasoptpy` to find linear model parameters estimates via ordinary least squares optimization. This demonstration simulates regression data with two predictors (three total parameters including an intercept) and estimates the known parameters using `sasoptpy` to minimize the model error. The practice that follows uses real data to estimate unknown population parameters with the techniques learned in this demonstration.

Simulated Data

1. Load the `swat`, `numpy`, `pandas`, `matplotlib`, and `sasoptpy` Python packages.

```
import swat
import numpy as np
import pandas as pd
from matplotlib import pyplot as plt
%matplotlib inline
import sasoptpy as so
```

swat is an R package that enables you to interface with SAS Cloud Analytics Services (CAS), the in-memory server that is the centerpiece of SAS Viya. Using the SWAT package, you can write an R program that connects to a CAS server, analyze large in-memory data sets, and then work with the results of the data analysis using familiar data-preparation techniques in R.

numpy is a Python package for scientific computing. It supports functionality for multidimensional arrays and matrices, as well as high-level mathematical functions not contained in the base code.

pandas is a Python package that provides data structures that are designed to work with relational or labeled data such as data frames. It is a commonly used building block for data analysis in Python.

matplotlib is a 2-D plotting library that produces publication quality figures in a variety of formats and interactive environment across platforms. One of these plot types, PYPLOY, can be used to create histograms and scatter plot, among many others.

%matplotlib inline instructs Jupyter to print plots produced by matplotlib within the notebook, rather than to an external file or output destination.

sasoptpy is a Python package providing a modeling interface to SAS Viya and SAS/OR optimization solvers. It provides a quick way for users to deploy optimization models and solve them using CAS actions. It can handle linear, mixed integer linear, and nonlinear optimization

problems. Users can benefit from native Python structures like dictionaries, tuples, lists, and Pandas data frames to define an optimization problem.

2. Create a connection to CAS with the appropriate server name, port, and login credentials.

```
conn = swat.CAS("http://sasviya01.demo.sas.com/cas-shared-default-http/", 8777, "student", "Metadata0", protocol="http")
```

The arguments required for the CAS function to connect to the CAS server depend on your SAS Viya system. The arguments should be required from your IT professional.

3. Simulate 100 observations from a multiple linear regression model with an intercept and two predictors. The predictors and the errors should be normal with mean zero and standard deviation 0.1.

$$y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + e_i$$

$$i = 1, \dots, 100$$

$$\beta_0 = 2, \beta_1 = 3, \beta_2 = 4$$

$$x_{1i}, x_{2i}, e_i \sim N(\mu = 0, \sigma = 0.1)$$

```
np.random.seed(123)
mu, sigma, n = 0, 0.1, 100
x0 = pd.DataFrame( np.ones(n) )
x1 = pd.DataFrame( np.random.normal(mu, sigma, n) )
x2 = pd.DataFrame( np.random.normal(mu, sigma, n) )
err = pd.DataFrame( np.random.normal(mu, sigma, n) )

beta0, beta1, beta2 = 2, 3, 4
y = beta0*x0 + beta1*x1 + beta2*x2 + err

data = pd.concat([x0, x1, x2, y], axis=1)
data.columns = ['x0', 'x1', 'x2', 'y']
data.head()
```

The data is simulated using Numpy functionality and converted to a data frame with Pandas. Below shows the simulated data frame, which is 100 by 4. The variables x1 and x2 are the two predictors, and the variable y is the response. The variable x0 represents the intercept and is a column of ones.

	x0	x1	x2	y
0	1.0	-0.108563	0.064205	2.001464
1	1.0	0.099735	-0.197789	1.448238
2	1.0	0.028298	0.071226	2.589870
3	1.0	-0.150629	0.259830	2.656263
4	1.0	-0.057860	-0.002463	1.815939

4. Use sasoptpy to minimize the error of the linear model and compute estimates of the betas.

```
m = so.Model(name='least_squares', session=conn)
```

The `so.Model` method initializes a `sasoptpy` object. In addition to naming the object, we specify the CAS connection object in order to move data between the server and client seamlessly.

```
# Regression model: beta0 * x0 + beta1 * x1 + beta2 * x2
beta0 = m.add_variable(name='beta0')
beta1 = m.add_variable(name='beta1')
beta2 = m.add_variable(name='beta2')

x0 = data['x0']
x1 = data['x1']
x2 = data['x2']
y = data['y']
```

The `add_variable` method adds variables to the model that will be computed via the optimization routines. The `x0`, `x1`, `x2`, and `y` definitions are used to compute the error of the model.

```
error = m.add_implicit_variable((
    y[i] - (beta0*x0[i] + beta1*x1[i] + beta2*x2[i]) for i in
    data.index), name='error')
```

The `add_implicit_variable` method defines the error of the linear model and uses a for loop to define the error for all observations.

```
m.set_objective(so.quick_sum(error[i]**2 for i in data.index),
sense=so.MIN, name='total_error');
```

The `quick_sum` function is used to sum arguments of an expression and, in this case, sums the squared errors of the model. This function is faster for expressions compared to the native `sum` function in Python.

```
%%time
#Here we explicitly specify the method we want to use, it uses
the Nonlinear Programming method
m.solve(verbose=True, options={'with': 'nlp'})
print(m.get_solution())
```

The `solve` method initiates the optimization of the model defined in the `m` object. The `options` argument requests that we use nonlinear programming to optimize the function. For this model, we could have also used quadratic.

The `get_solution` function prints the model and optimization definition, as well as the optimization history and final estimates for the variables optimized.

```
NOTE: Initialized model least_squares.
NOTE: Added action set 'optimization'.
NOTE: Converting model model_1 to OPTMODEL.
var var_1;
var var_2;
var var_3;

solve with nlp;
print _var_.name _var_.lb _var_.ub _var_ _var_.rc;
print _con_.name _con_.body _con_.dual;

NOTE: Submitting OPTMODEL code to CAS server.
NOTE: Problem generation will use 16 threads.
NOTE: The problem has 3 variables (3 free, 0 fixed).
NOTE: The problem has 0 linear constraints (0 LE, 0 EQ, 0 GE, 0 range).
```

NOTE: The problem has 0 nonlinear constraints (0 LE, 0 EQ, 0 GE, 0 range).
 NOTE: The OPTMODEL presolver removed 0 variables, 0 linear constraints, and 0 nonlinear constraints.

NOTE: Using analytic derivatives for objective.

NOTE: Using 3 threads for nonlinear evaluation.

NOTE: The NLP solver is called.

NOTE: The Interior Point algorithm is used.

	Objective		Optimality
Iter	Value	Infeasibility	Error
0	423.05997354	0	199.08053662
1	394.50717437	0	192.22153909
2	340.40341561	0	178.50484844
3	244.20105868	0	151.07748353
4	99.79726682	0	65.62779682
5	0.93594417	0	2.7435552332E-13

NOTE: Optimal.

NOTE: Objective = 0.9359441691.

NOTE: The CAS table 'primal' in caslib 'CASUSER(student)' has 3 rows and 6 columns.

NOTE: The CAS table 'dual' in caslib 'CASUSER(student)' has 0 rows and 4 columns.

NOTE: The CAS table 'problemSummary' in caslib 'CASUSER(student)' has 12 rows and 4 columns.

NOTE: The CAS table 'solutionSummary' in caslib 'CASUSER(student)' has 12 rows and 4 columns.

	var	lb	ub	value	rc
0	var_1	-1.797693e+308	1.797693e+308	1.990671	1.378009e-12
1	var_2	-1.797693e+308	1.797693e+308	2.948576	5.427013e-13
2	var_3	-1.797693e+308	1.797693e+308	4.022712	-9.647821e-13

The optimization routine exited on the sixth iteration with the optimality error near zero. The estimates for the population parameters are nearly equivalent to the known values used to simulate the model.

$$\beta_0 = 2, \beta_1 = 3, \beta_2 = 4$$

$$\hat{\beta}_0 = 1.99, \hat{\beta}_1 = 2.95, \hat{\beta}_2 = 4.02$$

We also could have solved using the default method:

```
%%time
#Here we let the solver choose the best method, it uses the
Quadratic Programming method (the objective function is quadratic)
m.solve()
print(m.get_solution())
```

```

NOTE: Added action set 'optimization'.
NOTE: Converting model least_squares to OPTMODEL.
NOTE: Submitting OPTMODEL code to CAS server.
NOTE: Problem generation will use 16 threads.
NOTE: The problem has 3 variables (3 free, 0 fixed).
NOTE: The problem has 0 linear constraints (0 LE, 0 EQ, 0 GE, 0 range).
NOTE: The problem has 0 nonlinear constraints (0 LE, 0 EQ, 0 GE, 0 range).
NOTE: The OPTMODEL presolver removed 0 variables, 0 linear constraints, and 0 nonlinear constraints.
NOTE: Using 3 threads for nonlinear evaluation.
NOTE: The QP presolver value AUTOMATIC is applied.
NOTE: The QP presolver removed 0 variables and 0 constraints.
NOTE: The QP presolver removed 0 constraint coefficients.
NOTE: The presolved problem has 3 variables, 0 constraints, and 0 constraint coefficients.
NOTE: The QP solver is called.
NOTE: The Interior Point algorithm is used.
NOTE: The deterministic parallel mode is enabled.
NOTE: The Interior Point algorithm is using up to 16 threads.

```

	Iter	Complement	Duality Gap	Primal Infeas	Bound Infeas	Dual Infeas	Time
	0	0.0000E+00	5.3737E-16	0.0000E+00	0.0000E+00	4.4035E-17	0

```

NOTE: Optimal.
NOTE: Objective = 0.9359441691.
NOTE: The Interior Point solve time is 0.00 seconds.
NOTE: The CAS table 'primal' in caslib 'CASUSER(student)' has 3 rows and 6 columns.
NOTE: The CAS table 'dual' in caslib 'CASUSER(student)' has 0 rows and 4 columns.
NOTE: The CAS table 'problemSummary' in caslib 'CASUSER(student)' has 17 rows and 4 columns.
NOTE: The CAS table 'solutionSummary' in caslib 'CASUSER(student)' has 15 rows and 4 columns.

```

	var	lb	ub	value	rc
0	beta0	-1.797693e+308	1.797693e+308	1.990671	4.662937e-15
1	beta1	-1.797693e+308	1.797693e+308	2.948576	1.831868e-15
2	beta2	-1.797693e+308	1.797693e+308	4.022712	4.215378e-15

End of Demonstration



Practice

1. Ordinary Least Squares Optimization Using Real Data

The Ames Housing data contains information about 300 homes sold in Ames, Iowa, from 2006 to 2010. The table below displays nine potential inputs for a linear model and the response, **Sale Price**. Use these data to fit a multiple linear regression model with two predictors, similar to the first part of the demonstration. In this scenario, you do not know the actual population parameters. Use sasoptpy to estimate those values.

NAME	MODEL ROLE	DESCRIPTION
Lot_Area	Input	Lot area in square feet
House_Style	Input	Style of dwelling
Heating_QC	Input	Heating quality and condition
Central_Air	Input	Presence of central air
Gr_Liv_Area	Input	Above ground living area in square feet
Fireplaces	Input	Number of fireplaces
Garage_Area	Input	Size of garage in square feet
SalePrice	Target	Sale Price in dollars
Age_Sold	Input	Age of house when sold in years
Season_Sold	Input	Season when house sold

Some starter code is provided to help load and explore the data:

Load the swat, numpy, pandas, matplotlib, and sasoptpy Python packages.

```
import swat
import numpy as np
import pandas as pd
from matplotlib import pyplot as plt
%matplotlib inline
import sasoptpy as so
```

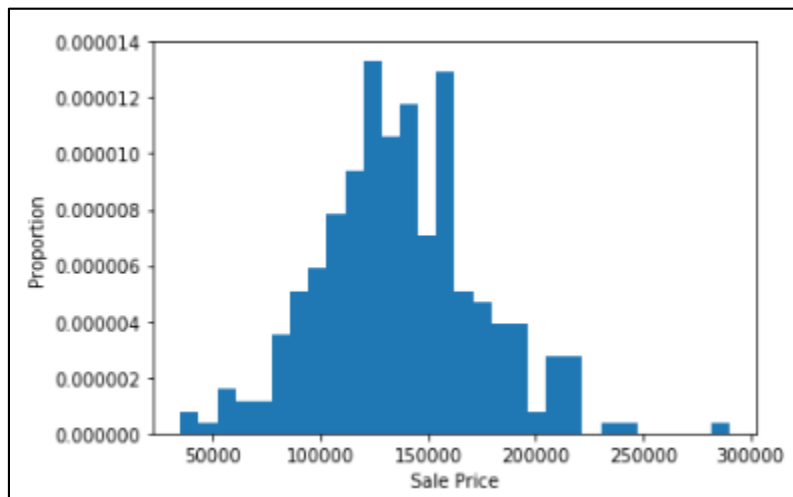
Load the Ames housing data set into a Pandas data frame and print the head of the table.

```
Ames = pd.read_csv("D:/Workshop/Winsas/VOSI/Ames_Housing.csv")
Ames.head()
```

	Lot_Area	House_Style	Heating_QC	Central_Air	Gr_Liv_Area	Fireplaces	Garage_Area	SalePrice	Age_Sold	Season_Sold
0	4920	1Story	Ex	Y	1338	0	582	213500	9	2
1	5005	1Story	Ex	Y	1280	0	506	191500	18	1
2	10500	1Story	TA	Y	864	1	0	115000	39	2
3	3203	1Story	Ex	Y	1145	0	437	160000	4	1
4	7750	SLvl	Ex	Y	1430	1	400	180000	10	2

Create a histogram of the response to view its shape.

```
plt.hist(Ames['SalePrice'], normed=True, bins=30)
plt.ylabel('Proportion')
plt.xlabel('Sale Price')
```



Create a correlation matrix of the ten variables in the data set to choose two predictors for a regression model.

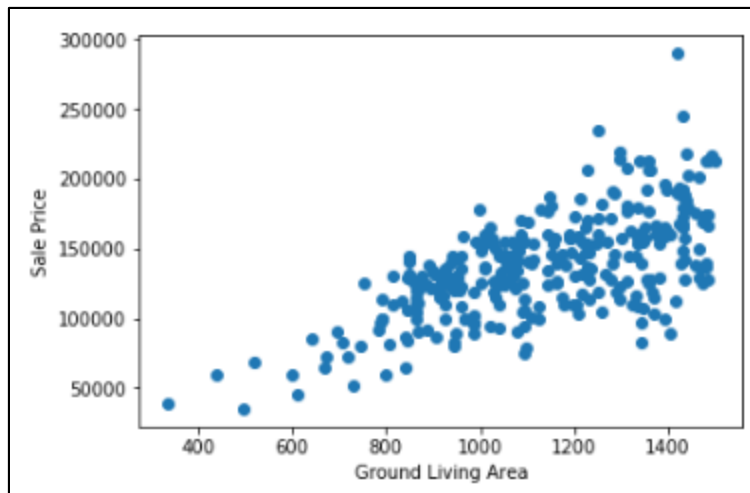
```
Ames.corr().round(2)['SalePrice']
```

```
Lot_Area      0.25
Gr_Liv_Area   0.65
Fireplaces    0.34
Garage_Area   0.58
SalePrice     1.00
Age_Sold     -0.62
Season_Sold    0.11
```

In this case, the **Gr_Liv_Area** variable has the highest positive correlation with **SalePrice**, and **Age_Sold** has the largest negative correlation. We will use these two variables jointly to build a regression model.

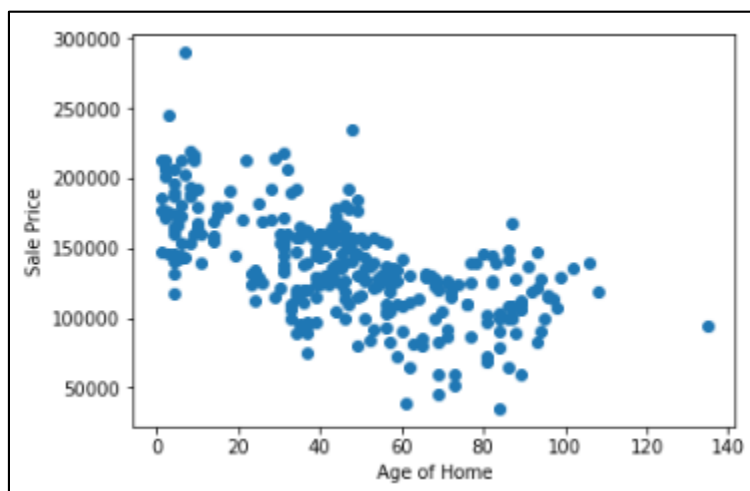
Create a scatter plot of **Sale Price** by **Ground Living Area**.

```
plt.scatter(Ames['Gr_Liv_Area'], Ames['SalePrice'])
plt.xlabel('Ground Living Area')
plt.ylabel('Sale Price')
```



Create a scatter plot of **Sale Price** by **Age of Home**.

```
plt.scatter(Ames['Age_Sold'], Ames['SalePrice'])
plt.xlabel('Age of Home')
plt.ylabel('Sale Price')
```



Import the LinearRegression function from the scikit-learn Python library and initialize the model object. Then use the fit method to fit the linear regression model and view the parameter estimates.

```
from sklearn.linear_model import LinearRegression
mymod = LinearRegression()

features = Ames[['Gr_Liv_Area', 'Age_Sold']]
target = Ames[['SalePrice']]
mymod.fit(features, target)
print('beta0:', mymod.intercept_[0].round(2), '    beta1:',
      mymod.coef_[0][0].round(2), '    beta2:',
      mymod.coef_[0][1].round(2))
```

```
beta0: 68498.36    beta1: 89.33    beta2: -696.9
```

These estimates for β_0 , β_1 , and β_2 were found using the linear regression method in scikit-learn. This method tries to minimize the sum of squares between the true value of the target and the predicted value (the output of the regression model). Using sasoptpy, you should find near equivalent estimates for the model, given that the optimization problem is correctly specified.

Find the values of the three betas that minimize the sum of squared errors using similar functionality as the previous example.

End of Practices

5.3 Solutions

Solutions to Practices

1. Ordinary Least Squares Optimization Using Real Data

Find the values of the three betas that minimize the sum of squared errors for predicting **Sale_Price** in the Ames Housing data set.

```
m = so.Model(name='least_squares', session=conn)

# Regression model: beta0 * x0 + beta1 * x1 + beta2 * x2
beta0 = m.add_variable(name='beta0')
beta1 = m.add_variable(name='beta1')
beta2 = m.add_variable(name='beta2')

Ames['int'] = pd.DataFrame( np.ones(Ames.shape[0]) )
x0 = Ames['int']
x1 = Ames['Gr_Liv_Area']
x2 = Ames['Age_Sold']
y = Ames['SalePrice']

error = m.add_implicit_variable((
    y[i] - (beta0*x0[i] + beta1*x1[i] + beta2*x2[i]) for i in
    Ames.index), name='error')

m.set_objective(so.quick_sum(error[i]**2 for i in Ames.index),
    sense=so.MIN, name='total_error')

m.solve(verbose=True)

print(m.get_solution())
```

```
NOTE: Initialized model least_squares.
NOTE: Added action set 'optimization'.
NOTE: Converting model model_1 to OPTMODEL.
var var_1;
var var_2;
var var_3;

solve with nlp;
print _var_.name _var_.lb _var_.ub _var_ _var_.rc;
print _con_.name _con_.body _con_.dual;

NOTE: Submitting OPTMODEL code to CAS server.
NOTE: Problem generation will use 16 threads.
NOTE: The problem has 3 variables (3 free, 0 fixed).
NOTE: The problem has 0 linear constraints (0 LE, 0 EQ, 0 GE, 0 range).
NOTE: The problem has 0 nonlinear constraints (0 LE, 0 EQ, 0 GE, 0 range).
NOTE: The OPTMODEL presolver removed 0 variables, 0 linear constraints, and 0 nonlinear
constraints.
```

NOTE: Using analytic derivatives for objective.
 NOTE: Using 3 threads for nonlinear evaluation.
 NOTE: The NLP solver is called.
 NOTE: The Interior Point algorithm is used.

Iter	Objective Value	Infeasibility	Optimality Error
0	6097150205016	0	48353796895
1	6097144745164	0	48353773966
2	6097133825467	0	48353728108
3	6097111986107	0	48353636392
4	6097068307518	0	48353452959
5	6096980950869	0	48353086094
6	6096806239685	0	37382914905
7	6096456825770	0	23030104612
8	6095758031751	0	13026433842
9	6094360578961	0	6970069231
10	6091566214369	0	3611051853
11	6085979649130	0	1838096765
12	6074815174338	0	926726981
13	6052520846765	0	464614223
14	6008070673793	0	231924410
15	5919724208978	0	115167854
16	5832116163631	0	76190329
17	5659114907204	0	44984584
18	5489066004526	0	31606197
19	5157821512237	0	19439648
20	4530714244210	0	10472775
21	3417706983279	0	4837528
22	1751952068611	0	1651189
23	809370174981	0	561701.04990896
24	441387088606	0	160437.11384200
25	278262049154	0	61933.15892987
26	180694036642	0	27461.98110344
27	138591995062	0	0.000000145478

NOTE: Optimal.
 NOTE: Objective = 138591995062.
 NOTE: The CAS table 'primal' in caslib 'CASUSER(student)' has 3 rows and 6 columns.
 NOTE: The CAS table 'dual' in caslib 'CASUSER(student)' has 0 rows and 4 columns.
 NOTE: The CAS table 'problemSummary' in caslib 'CASUSER(student)' has 12 rows and 4 columns.
 NOTE: The CAS table 'solutionSummary' in caslib 'CASUSER(student)' has 12 rows and 4 columns.

	var	lb	ub	value	rc
0	var_1	-1.797693e+308	1.797693e+308	68498.362105	0.000001
1	var_2	-1.797693e+308	1.797693e+308	89.326565	-0.000997
2	var_3	-1.797693e+308	1.797693e+308	-696.904306	-0.000047

End the CAS session. (This is optional because, by default, the session times out after 60 seconds.)

```
conn.session.endSession()
```

End of Solutions

Appendix A A Note about Distributed Processing in the OPTMODEL Procedure

A.1	Distributed Processing in the OPTMODEL Procedure	A-3
-----	--	-----

A.1 Distributed Processing in the OPTMODEL Procedure

This appendix contains a note for students that will run the OPTMODEL procedure on a distributed or massively parallel processing (MPP) installation of Viya OPTIMIZATION.

Part 1

The server that you used in this course is a symmetric multiprocessing or SMP installation of Viya. In the course demonstrations that have loaded tables, the necessary connections to CAS were created using LIBNAME and DATA statements. For example, in addition to creating the library reference, **casuser**, the syntax below automatically generates and connects to a session named CASAUTO.

```
libname casuser cas;
```

Data are loaded into **casuser** using a subsequent DATA step (see below), but there are no workers to distribute it to.

```

82          libname casuser cas;

NOTE: The session CASAUTO connected successfully to Cloud Analytic Services
sasserwer.demo.sas.com using port 5570. The UUID is

      88c631e3-dd80-9a4a-a38b-52dd13591906. The user is sasdemo and the active caslib is
CASUSER(sasdemo).

NOTE: The SAS option SESSREF was updated with the value CASAUTO.

NOTE: The SAS macro _SESSREF_ was updated with the value CASAUTO.

NOTE: The session is using 0 workers.

NOTE: Libref CASUSER was successfully assigned as follows:

      Engine:          CAS

      Physical Name: 88c631e3-dd80-9a4a-a38b-52dd13591906

```

```

data casuser.class;

    set opt.class;

run;

```

```

%studio_hide_wrapper;

82          data casuser.class;
83          set sashelp.class;
84          run;

NOTE: There were 19 observations read from the data set SASHELP.CLASS.
NOTE: The data set CASUSER.CLASS has 19 observations and 5 variables.
NOTE: DATA statement used (Total process time):
      real time          0.01 seconds
      cpu time           0.00 seconds

```

```
proc optmodel;
    /* declare variables */
    set <str> STUDENTS;
    ...
```

Note: That said, PROC OPTMODEL is multi-threaded and takes advantage of multiple cores on a single machine to run processes in parallel.

Part 2

Students that will work on distributed, or *MPP*, installations of SAS Viya OPTIMIZATION with multiple available workers might find the syntax below useful.

```
cas mysess;
options casnworkers=n;
```

- The CAS statement creates a new session named MYSESS.
- The CASNWORKERS= option in the OPTIONS statement enables users to allocate available workers for their session. See below for details.

```
proc optmodel sessref=mysess;
    /* declare variables */
    set <str> STUDENTS;
    ...
```

- Referencing the CAS session created above using the SESSREF= option is necessary to run PROC OPTMODEL in MPP mode.

Note: Some optimization problems are more suited for distributed computing than others. The software automatically detects whether it can speed computation by distributing work across the nodes. If the problem is not suitable for distributed computing, the solver executes in single-machine mode, even if you specify the CAS session using the SESSREF= option. For example, a generic nonlinear programming problem can execute in single-machine mode, but if you specify the MULTISTART options for the NLP solver, the problem executes in distributed mode, using the number of workers that you specified with the CASNWORKERS= option. Here is an example.

```
88 solve with nlp;
NOTE: Problem generation will use 4 threads.
NOTE: The problem has 1 variables (0 free, 0 fixed).
NOTE: The problem has 0 linear constraints (0 LE, 0 EQ, 0 GE, 0 range).
NOTE: The problem has 0 nonlinear constraints (0 LE, 0 EQ, 0 GE, 0 range).
NOTE: The OPTMODEL presolver removed 0 variables, 0 linear constraints, and 0 nonlinear constraints.
NOTE: The SOLVE statement is executing in the distributed computing environment in single-machine mode.
```

Adding MULTISTART= causes the worker nodes to be used.

```
88 solve with nlp/multistart=(maxstarts=5);
NOTE: Problem generation will use 4 threads.
NOTE: The problem has 1 variables (0 free, 0 fixed).
NOTE: The problem has 0 linear constraints (0 LE, 0 EQ, 0 GE, 0 range).
NOTE: The problem has 0 nonlinear constraints (0 LE, 0 EQ, 0 GE, 0 range).
NOTE: The OPTMODEL presolver removed 0 variables, 0 linear constraints, and 0 nonlinear constraints.
NOTE: The SOLVE statement is executing in the distributed computing environment with 27 worker nodes.
```

Syntax

CASNWORKERS= ALL | *number*

Syntax Description

ALL

specifies to use all of the worker nodes.

number

specifies the number of worker nodes to use.

In SMP mode, *number* is always 0, whether you set CASNWORKERS= to 0 or 1.

In MPP mode, specify *number*=0 to use the maximum number of worker nodes that are available. You can set CASNWORKERS= to a number that is less than or equal to the maximum number of worker nodes.

Range 0–5000

Example

```
options casnworkers=10;
```


Appendix B References

B.1	References	B-3
-----	------------------	-----

B.1 References

Lesson 2

The furniture-making problem is from Chvátal, V. 1980. *Linear Programming*. New York: W.H. Freeman and Company.

Other references:

Banker, R. D., A. Charnes, and W. W. Cooper. 1984. "Some Models for Estimating Technical and Scale Inefficiencies in Data Envelopment Analysis." *Management Science* 30(9): 1078–1092.

Charnes, A., W. W. Cooper, and E. Rhodes. 1978. "Measuring the efficiency of decision making units." *European Journal of Operational Research* 6: 429–444.

Seiford, L. M. and J. Zhu. 1999. "Profitability and Marketability of the Top 55 US Commercial Banks." *Management Science* 45(9): 1270–1288.

Zhu, Joe. 2002. *Quantitative Models for Performance Evaluation and Benchmarking*. New York: Springer.

Lesson 3

The covariance matrix for the portfolio optimization problem is generated using the shrinkage estimator from Ledoit, O. and M. Wolf. 2004. "Honey, I Shrunk the Covariance Matrix." *Journal of Portfolio Management* 30(4):110–119.

